

VG101 KNOWLEDGE BASE

Videogames as Their Own Medium

A complete text on videogame design

Marty Scott · marty64.net

Contents

PART I The Framework

- 1 Core Framework
- 2 Gesture
- 3 The 4 A's
- 4 Aesthetic Heritage
- 5 Permissions

PART II Foundations

- 6 Foundations
- 7 What Makes a Videogame
- 8 The Weekly Rhythm: Play → Name → Make → Reflect
- 9 Player Psychology
- 10 The Design Lens

PART III Narrative Design

- 11 Narrative Design
- 12 Environmental Storytelling
- 13 Branching & Consequence
- 14 Dialogue Systems
- 15 Ludonarrative Harmony

PART IV Level Design

- 16 Level Design
- 17 Spatial Communication
- 18 Pacing & Flow
- 19 Guidance Without Hand-Holding
- 20 Verticality & Sight Lines

PART V Systems Design

- 21 Systems Design
- 22 Economy & Resources
- 23 Progression Systems
- 24 Balance Philosophy
- 25 Emergent Behavior

PART VI Pedagogy

- 26 Pedagogy
- 27 Play → Name → Make → Reflect
- 28 Accessibility as Craft
- 29 Debugging as Literacy
- 30 Code as Material
- 31 Why Unity
- 32 The Final Project

PART VII Case Studies

- 33 Case Studies
- 34 Celeste: The Dash
- 35 Celeste: Assist Mode
- 36 Dark Souls: The Bonfire
- 37 Hades: God Mode
- 38 Portal: Level One
- 39 Undertale: Genocide Route

PART VIII Code Bank

- 40 Code Bank
- 41 Input Setup
- 42 Basic Jump
- 43 Variable Jump

44	Double Jump
45	Coyote Time
46	Input Buffer
47	Dash
48	Squash & Stretch
49	Screen Shake
50	Hitstop
51	Knockback

APPENDIX Reference

52	Glossary
53	References

PART I

The Framework

The core lens of VG101: Gesture, the 4 A's, Aesthetic Heritage, and Permissions.

Core Framework

The foundational concepts of VG101: Gesture, the 4 A's, Aesthetic Heritage, and Permissions.

Gesture

The surface area of contact between player and videogame. Not a single action, not a fixed size. The sum of everything happening when a videogame is being played. It can be a micro-moment or an entire videogame.

The 4 A's

What compose a Gesture, inseparable when played, separable when analyzed:

- **Action:** What the videogame has the player do
- **Art:** Art direction, lighting, staging, camera grammar, composition
- **Arc:** The amount of time
- **Atmosphere:** The feeling and vibe experienced

Aesthetic Heritage

Where the choices in each A come from. The lineage. What the videogame inherits, quotes, or transforms from other videogames and other media. All four A's are aesthetic choices with lineages.

Permissions

The limits and boundaries of the experience. What the system allows, requires, and forbids, intentionally or not. Discovered through the 4 A's as we play.

Gesture

The surface area of contact between player and videogame system.



Journey - A single *Gesture* spanning an entire videogame, every moment oriented toward the mountain.
Courtesy of thatgamecompany / Sony Interactive Entertainment

What Gesture Is

A Gesture is not necessarily a single action like you may initially imagine. It could be a singular moment or action, but we can also investigate gestures that are a series of actions, a period of play, a second of something silly or spooky occurring, or an entire arc of a character for hundreds of hours. A gesture is just to identify a unit of investigation. A boundary box to the most often elusive experience that is playing videogames.

So a Gesture can be a micro-moment: the split-second of pressing jump and feeling the character move from the ground. Or it can be the entire videogame: the full story plot and mechanics through a hundred hour RPG.

Do not conflate Gesture with "action" or "mechanic." Gesture contains these things but is not reducible to them. Gesture is the whole experience at whatever scale the Gesture is being experienced/performed.

The 4 A's

A Gesture is composed of four elements, experienced as one thing:

- **Action:** What the videogame has the player do in that Gesture, including the Action of nothing at all!
- **Art:** Art direction, lighting, level design, camera grammar, music, sound design, etc.
- **Arc:** The amount of time. A moment, a minute, an entire videogame's length.
- **Atmosphere:** The feeling and ambiance experienced with the videogame

When playing a videogame we do not aim to parse out all of these separate portions to process them. When analyzing, we attempt to identify how each portion contributes to that singular Gesture being looked at.

Analyzing Gesture

When you analyze a Gesture, ask:

- What is the **Action**? What are the things I can do (or not do)?

- What is the **Art**? What am I seeing and hearing? What traditions or ideas is the videogame drawing from?
- What is the **Arc**? How long does this last? What is its shape over time?
- What is the **Atmosphere**? What does it feel like? What vibe does it create?

Then ask: Where does each of these come from? What is the Aesthetic Heritage?

Exercise

Play any videogame for 10 minutes. Pause. Identify one Gesture you just experienced. Write down each of the 4 A's. Don't worry about being "right." The goal is to practice seeing.

Why Start Here

Gesture is the entry point to VG101 because it centers player experience rather than designer abstractions. Most frameworks start with mechanics, systems, or rules. These are useful, but we feel like they can miss what players actually feel.

By starting with "what did you experience?" rather than "what are the rules/permissions/affordances?", students develop the habit of attending to feel before structure. This prevents the common trap of designing systems before they are needed, or creating something seemingly logical but ends up being vestigial or feeling not right.

Teaching Sequence

Introduce Gesture before the 4 A's. Let students sit with the holistic concept before giving them analytical tools.

1. Play a short videogame together (5-10 min)
2. Ask: "Describe one moment that stuck with you"
3. Ask: "What do you think made up that moment? What were its parts?"
4. Introduce "Gesture" as a name for that moment
5. Only then introduce the 4 A's as a way to analyze Gesture

Common Misconceptions

"Gesture = one button press"

Students often want to equate Gesture with a single input. Bring up early that it can be a multitude of things dependent on the situation: Gesture is about experience, not input. A Gesture can span seconds, hours, days, even years.

"The 4 A's are separate things"

Students may try to analyze each A in isolation. In such a situation emphasize that the 4 A's are experienced as one thing. Analysis separates what is unified in play.

Assessment

For the Gesture concept, assess through discussion. Look for: Can the student identify a Gesture? Can they articulate what makes it a Gesture rather than just "a mechanic" or "an action"?

Why "Gesture"

The term "gesture" draws from many sources of inspiration, one of the most prominent is Andre Bazin's film theory, where he identified gesture as an important unit and facet of painting, film, acting, and other creative forms. Gesture is a meaningful unit that communicates through form rather than (or in addition to) explicit content.

Videogame Gesture extends Bazin's insight: it's not just what's in the frame, but what the player does within and in response to that frame. It is the surface area of contact between the system and the player's action. There are even Gesture readings in which occurrences from outside a videogame's rendered space can affect the Gesture.

Relation to Other Frameworks

MDA (Mechanics, Dynamics, Aesthetics): MDA moves from designer-facing (mechanics) to player-facing (aesthetics). Gesture inverts this: we start with player experience and work backward to understand what produced it.

Game Feel (Steve Swink): Swink's work on game feel is a major inspiration in Gesture, and its particular bridge of aesthetic and mechanic is of great use. However, Gesture extends beyond the integration of game feel. Gestures also encompass longer arcs of play and atmospheric qualities as well as narrative experience in both its implicated and explicit uses.

Common Questions

Can there be nested gestures? (A gesture in a gesture?)

Yes, of course - you can multiply your analysis however you find useful and informative. But please, do not over-complicate things. Gestures and their interrelationships are essential to the reading of a videogame, however perceiving gestures inside of gestures is most commonly counter-productive.

Can Gestures fail?

Yes and no! There will always be a gap between the intended affect and materialized affect in any creative form.

Is an unintentional experience still a Gesture?

Absolutely! The experience a player has is the only experience they have at that moment. The systems and what contact looks like with the player to that player is the experience of the gesture.

Related

- [The 4 A's](#)
- [Aesthetic Heritage](#)
- [Permissions](#)

The 4 A's

Action, Art, Arc, Atmosphere. The four components of a Gesture.



Shadow of the Colossus - All four A's visible at once. The action of climbing, the art of scale, the arc of the battle, the atmosphere of awe.

Courtesy of Team Ico / Sony Interactive Entertainment

How the 4 A's Work

When playing, the 4 A's are inseparable. You experience them as one thing. When analyzing, you pull them apart to understand how each contributes to the whole.

Action

What the videogame has the player do.

This includes direct interaction: pressing buttons, moving a character, making choices. But it also includes absence of interaction: waiting, watching, being still. A cutscene is an Action (the action of watching). A pause is an Action (the action of not acting).

Action is not just "verbs" or "mechanics." It is whatever the videogame asks of the player, whether that asking is explicit or implicit, active or passive, incidental or intentional.

Art

Art direction, lighting, staging, camera grammar, composition, and sound design.

Art is not just "how it looks." It is the full vocabulary of visual and audio craft: art direction and style, lighting and color, staging and blocking, camera grammar (angles, movement, cuts), composition (what's in frame, what's emphasized), sound design and music.

Every Art choice has an Aesthetic Heritage. A Dutch angle comes from somewhere. A specific retro synth sound comes from somewhere. Understanding where these choices come from can help you understand why they work (or don't).

Arc

The amount of time. The shape of the experience over duration.

An Arc can be a micro-moment: the 0.2 seconds of a jump's apex. Or it can be a long moment: the 10-minute drive through the desert. Or it can be the entire videogame: the 40-hour journey from start to credits.

Arc is not the same as " pacing ," though pacing is part of it. Arc is the temporal container of the Gesture . It answers: how long does this last, and what shape does it take?

A Gesture could be nested inside another Gesture. The Arc of a single jump exists inside the Arc of a level, which exists inside the Arc of the whole videogame.

Atmosphere

The feeling and vibe experienced from the videogame.

Atmosphere is what emerges when Action, Art, and Arc combine. It is the emotional texture of the experience. The mood. The tone.

Atmosphere is not generally something designers add on top. It is what players feel as a result of the aggregate of the experience. But designers can and do craft toward specific Atmospheres by shaping the other three A's.

Words for Atmosphere: tense, melancholy, cozy, uncanny, triumphant, lonely, frantic, peaceful. These are not prescriptions. They are observations of what emerges.

Exercise

Pick a videogame you know well. Identify one Gesture (any scale). Write one sentence for each A:

- **Action:** What is the videogame having me do?
- **Art:** What do I see and hear? What traditions is it drawing from?
- **Arc:** How long does this last? What's its shape?
- **Atmosphere:** What does it feel like?

Teaching the 4 A's

Introduce the 4 A's only after students have grasped [Gesture](#) as a holistic concept. The danger is students treating the 4 A's as a checklist rather than lenses for understanding.

Order of Introduction

Action first. It's the most intuitive: "What are you doing?"

Art second. Students are often visual-first, so this often connects quickly. However, astute observations about influences and aesthetic heritage of art history, music history, and film can go a long way to enriching perception.

Atmosphere third. Once they can identify Action and Art, ask "What does it feel like?" This can often be assisted by the synthesis of the first two.

Arc last. Time is the hardest to see. Students need practice before they can attend to temporal shape of a videogame.

Common Pitfalls

"Action = button press"

Students reduce Action to input. Push them: What about waiting? What about choosing not to act? What about the action of reading text? Watching a cutscene?

"Art = graphics quality"

Students may conflate Art with fidelity or polish. Redirect: a pixel art videogame has Art choices just as much as a AAA title. "What traditions is it drawing from?" is the real question here.

"Atmosphere is just Art"

Students may collapse Atmosphere into Art. Distinguish: Art is what you see/hear; Atmosphere is what you *feel*. Same Art can produce different Atmospheres with different Actions.

Assessment

Have students analyze the same videogame moment. Compare their 4 A's breakdowns. Differences reveal interpretation, not error. Discuss why different students have preference to different A's.

THEORY

- deeper grounding

Why Four?

The 4 A's are not exhaustive. They are a heuristic: four lenses that together cover most of what matters in a [Gesture](#). Other frameworks slice differently.

MDA uses three (Mechanics, Dynamics, Aesthetics). Swink's *Game Feel* focuses on real-time control. The 4 A's emphasize that Action and Art are equally important, and that time (Arc) and feeling (Atmosphere) deserve explicit attention.

Why "Art" and Not "Aesthetics"?

We use "Art" for the visual/audio component and reserve "Aesthetic" for [Aesthetic Heritage](#), which refers to videogames as an aesthetic medium where all 4 A's have lineages.

This avoids confusion: "Art" is one of the four components; "Aesthetic Heritage" is the concept that all four have traditions they inherit from.

The Problem of Separation

The 4 A's are analytically distinct but experientially unified. This creates a pedagogical tension: we teach students to separate what play unifies.

The solution is to always return to [Gesture](#). The 4 A's are tools for understanding Gesture, not replacements for it. Analysis serves experience, not the other way around.

Related

- [Gesture](#)
- [Aesthetic Heritage](#)
- [Permissions](#)

Aesthetic Heritage

Where the choices in each of the 4 A's come from. The lineage.



Elden Ring - Sites of Grace inherit from Dark Souls' bonfires, Aesthetic Heritage in action.
Courtesy of FromSoftware / Bandai Namco

What Aesthetic Heritage Is

Every choice in a videogame comes from somewhere. A jump arc, a color palette, a camera angle, a musical sting, a pacing structure. None of these are invented from nothing.

Aesthetic Heritage is the term for tracing where these choices come from. What the videogame inherits, quotes, or transforms from:

- Other videogames
- Cinema
- Music
- Visual art
- Theatre
- Dance
- Literature
- Architecture
- Any other medium or tradition

Why "Aesthetic" Heritage

Videogames are an aesthetic medium. All four A's (Action, Art, Arc, Atmosphere) are aesthetic choices with lineages.

This is not limited to visuals. A jump mechanic has an aesthetic heritage. A difficulty curve has an aesthetic heritage. A control scheme has an aesthetic heritage. "Aesthetic" here means the full craft of the medium, not just the visual.

Inheritance, Quotation, Transformation

Videogames relate to their aesthetic heritage in three main ways:

Inheritance

Using a solution because it works. The double-jump exists in your platformer because double-jumps have proven useful in platformers. You inherit the convention.

Quotation

Deliberately referencing a source. The camera work in your horror videogame quotes the camera work in a specific horror film. Players who recognize the reference experience an additional layer of meaning.

Transformation

Taking something and making it new. The bonfire in *Dark Souls* transforms the checkpoint into something with narrative weight. The aesthetic heritage is visible, but the result is distinctly its own.

Exercise

Pick one element from a videogame you know (a jump, a camera angle, a musical cue, a UI choice). Trace its aesthetic heritage:

- Where have you seen this before? In other videogames? In film? In music?
- Is this inheritance (convention), quotation (reference), or transformation (something new)?
- What does knowing the aesthetic heritage add to your understanding?

Why Teach Aesthetic Heritage

Students often think they are inventing from scratch. Aesthetic heritage hopefully assists them in seeing that many solutions we can use exist in many creative traditions. The intention is for this to be freeing, not limiting. For many challenges in videogame development you can learn faster by studying how others solved similar problems in seemingly unrelated fields rather than struggling without direction.

Aesthetic Heritage Is Not Trivia

Try avoiding turning aesthetic heritage into "videogame trivia." The goal is not to memorize release dates or pieces of obscure media. The goal is to develop the habit of asking: "Where does this come from? What traditions is it pulling from?"

Teaching Sequence

1. Start with obvious quotations (a videogame that explicitly references a film)
2. Move to conventions (why do platformers have double-jumps?)
3. Explore cross-media aesthetic heritage (how did cinema influence videogame cameras?)
4. End with transformations (how did *Dark Souls* change the checkpoint?)

Common Pitfalls

"Aesthetic Heritage = copying"

Students may comment something along the lines of this tool is reinforcing derivative interpretation. We are not fundamentalists, but we were trained to study and respect the pivotal works of our field. It informs us of the places it's been and the ways it's been used in the past. Not knowing the history of a field is a treacherous and dubious thing to do. Things we do or say in the space have a much wilder breadth of implications if not respected. And be wary of claims or wants of "originality" - all things stand on the shoulders of their predecessors. Even the most original works exist inside tradition, technique, and context.

"I need to know everything"

Students may feel challenged or intimidated by the vastness of aesthetic heritage. Remind them: you don't need encyclopedic knowledge. We all have different interests and entry points into videogames, whatever that may be for you, it is a valuable voice to be heard in videogame analysis. What matters most is developing the habit of asking "where does this come from?"

Assessment

Have students present an "aesthetic heritage map" for one element of a videogame they want to make or analyze. Looking for: Can they identify at least two sources? Can they articulate what they're inheriting, quoting, or transforming?

Aesthetic Heritage and Medium Specificity

Videogames inherit from many different traditions and media but should not be reduced to any of them. Cinema contributed camera grammar. Music contributed rhythm. Theatre contributed stage and lighting. But videogames transform these inheritances through action and interactivity.

The question is: what does this inheritance inform us of when in the context of the 4 A's?

Aesthetic Heritage and Meaning

Aesthetic heritage is not just craft knowledge. It's also meaning. When a videogame quotes a film, players who recognize the quote experience a richer message. When a videogame inherits a convention, it carries the accumulated flair.

This creates layers of literacy. Some players see the reference and others do not. This is not so much a problem to solve but another layer of communication.

Aesthetic Heritage and Accessibility

Aesthetic heritage can be exclusionary. If a videogame assumes players know 40 years of platformer conventions, it may alienate newcomers. Part of [accessibility as craft](#) is deciding which aesthetic heritage to assume and which to teach.

Related

- [Gesture](#)
- [The 4 A's](#)
- [Permissions](#)

Permissions

The boundaries of interface, contact, system, and mechanic. What the videogame allows, requires, and forbids.



The Stanley Parable - *What's allowed, what's required, what's forbidden, and what happens when you test the boundaries.*

Courtesy of Galactic Cafe

What Permissions Are

Permissions are the boundaries of the experience. They define what's allowed, what's required, and what's forbidden within the videogame's systems.

Every videogame has Permissions, whether the designers intended them or not. Some are explicit (you cannot go through this wall). Some are implicit (you would never think to try). Some are designed. Some are accidents. All of them shape the experience.

Three Types of Permissions

Allowed

What the videogame lets you do. The possibility space. Can you jump? Can you talk to NPCs? Can you save anywhere?

Required

What the videogame makes you do. The demands of progression. Must you defeat the boss? Must you watch the cutscene? Must you wait for the timer?

Forbidden

What the videogame prevents you from doing. The boundaries. Cannot harm children NPCs. Cannot skip the tutorial. Cannot return to the previous area.

Discovering Permissions

We often don't know the Permissions preemptively. We discover them through play, through the [4 A's](#). It's like reaching out and feeling for feedback in the dark.

The Action shows us what we can do. The Art signals what might be possible. The Arc reveals what's required to progress. The Atmosphere hints at what kind of experience this is. Through all of these, we come to understand the limits of the experience, the Permissions.

Sometimes we know the Permissions before playing: from genre conventions, from marketing, from other players. But even then, the explicit boundaries are felt out through the

Gesture itself.

Permissions and Meaning

Permissions carry meaning. What a videogame allows, requires, and forbids communicates values, priorities, and worldview.

A videogame that forbids killing communicates something different than one where it is essential for progression. A videogame that allows broad nonlinear exploration communicates different Permissions than one of linear level progression. These aren't neutral technical decisions. They are intentional for one of these affects or another.

Exercise

Pick a videogame. List:

- 3 things the videogame **allows** (but doesn't require)
- 3 things the videogame **requires**
- 3 things the videogame **forbids**

For each: How did you discover this Permission? Through what Action, Art, Arc, or Atmosphere?

Why Teach Permissions

Students tend to focus on what is being created (features, content) but don't directly address what they are avoiding or removing from the preemptively expected experience of what they are creating, not acknowledging the constraining in the space. To our mind this is tantamount to ignoring negative space as a compositional tool. In this, Permissions help to establish the frame of the videogame's staging and intentionality. You are defining the space of possibilities.

Teaching Sequence

1. Start with "Forbidden." It's often invisible but powerful. Ask: what can't you do in your videogame?
2. Move to "Required." What must you do to progress? What has no alternative?
3. End with "Allowed." This is the space between the previous two, the Required and the Forbidden, this is the space of play and agency.

The "Permissions Remix" Exercise

Have students take a videogame and change one Permission:

- What if *Mario* couldn't jump? (Forbidden something that was Allowed)
- What if *Dark Souls* let you pause? (Allowed something that was Forbidden)
- What if *Tetris* didn't require pieces to fall? (Removed a Requirement)

Discuss: How does this single change affect the whole experience?

Common Pitfalls

"More Allowed = better"

Students may assume more player freedom is always more fun/good. Generally this is not true and, like in any creative space, only constraints create meaning. A videogame that allows everything requires nothing and often feels vapid.

"Required = bad"

Students may feel Requirements are limitations to avoid. But Requirements are how videogames create shared experiences. Without them, there's no guarantee any two players experience the same thing.

Assessment

Have students write a "Permissions document" for a videogame they're designing. Look for: Do they articulate all three types? Do they explain why each Permission exists?

THEORY

- deeper grounding

Permissions and Agency

Agency exists in the space between Required and Forbidden. If everything is Required, there's no agency. If nothing is Forbidden, agency may feel meaningless (there are no stakes to making one decision over another).

The art of Permission design is balancing constraints: enough Required and Forbidden to give choices weight, enough Allowed to give players room to express themselves.

Permissions and the 4 A's

Permissions are not separate from the Gesture. They are discovered through it. The 4 A's are how we come to understand what's allowed, required, and forbidden.

This means Permissions are not purely abstract rules. They are felt, encountered, tested. A Permission you never bump against might as well not exist. A Permission you discover through play becomes part of the experience.

Related

- Gesture
- The 4 A's
- Aesthetic Heritage

PART II

Foundations

What a videogame is, the weekly rhythm of the course, player psychology, and the design lens.

Foundations

Baseline concepts for thinking about videogames before diving into the framework.

What Makes a Videogame

Videogames are a medium of action. They are played, not just watched or read. They are neither just products nor just art. They are both and neither.

The Weekly Rhythm

Play → Name → Make → Reflect. The cycle that drives learning in VG101. Experience first, vocabulary second, making third, understanding last.

Player Psychology

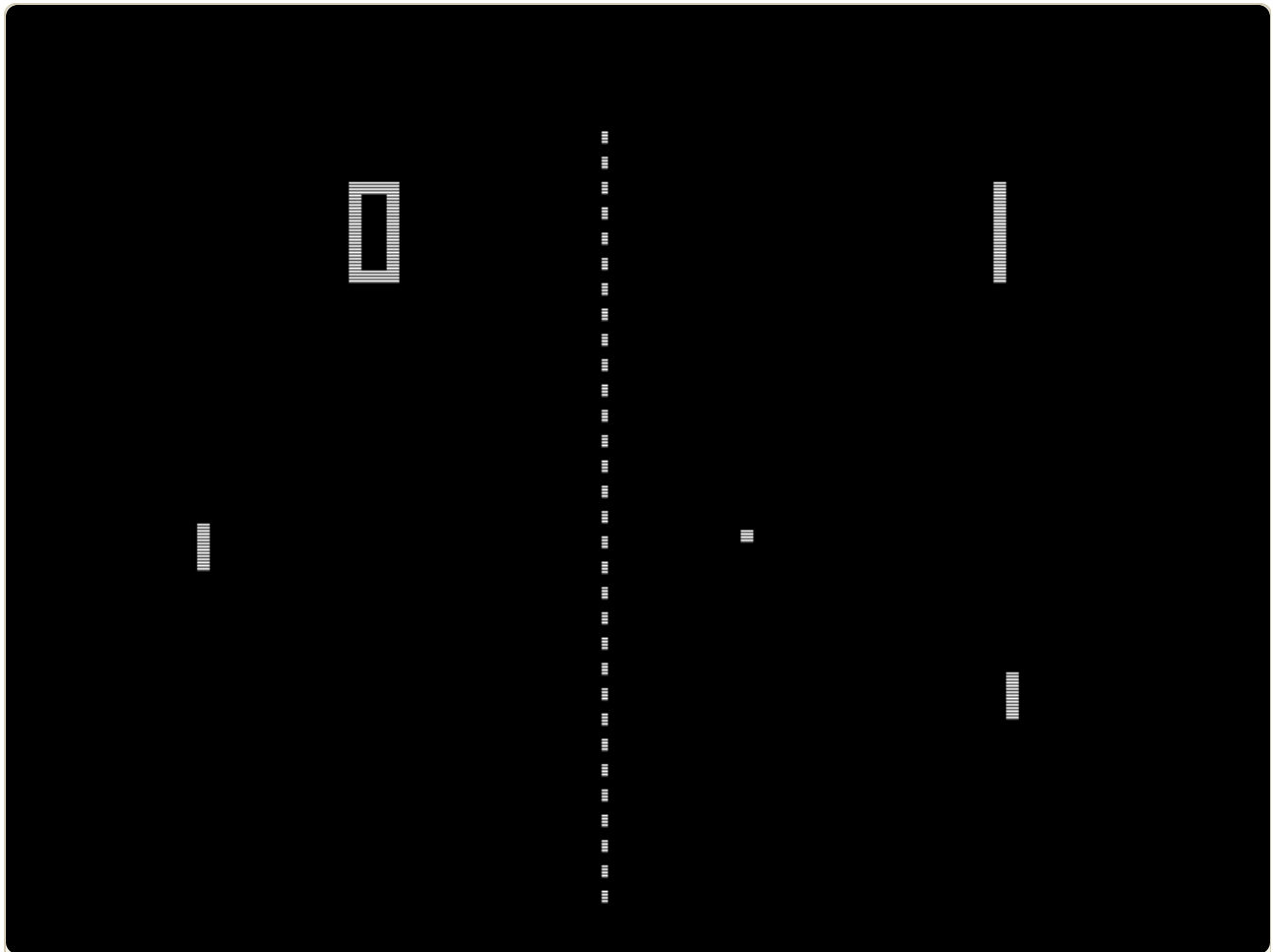
How players think, feel, and respond. Flow state, motivation, the Tetris effect, feedback loops, human random vs. computer random, player fair vs. computer fair.

The Design Lens

A way of seeing videogames as they are unto themselves. This framework respects what came before while offering a lens focused on experience, heritage, and craft.

What Makes a Videogame

Videogames are a medium of action. They are played, not just watched or read.



Pong - *Where the medium begins. Simple rules, immediate action, emergent experience.*
Courtesy of Atari

Videogames Are Their Own Medium

Videogames borrow from cinema, music, literature, theatre, visual art. But they are not reducible to any of these. What makes videogames distinct is that the player acts.

A film asks you to watch. A book asks you to read. A videogame asks you to **do**.

This doing can be pressing buttons, making choices, moving through a space, or even choosing to be still. But the player is always implicated in what happens. The videogame cannot exist without having a direct context to the player.

Not Just Product, Not Just Art

Videogames are often framed as products (entertainment, consumer goods) or as art (expressive works, cultural artifacts). Both framings capture something true.

But videogames are neither just products nor just art. They are both and neither. The creations that take full advantage of the medium understand this. They don't pick one framing and ignore the other. They embrace the full complexity of what videogames can be.

What We Study

In VG101, we study videogames as they are. We analyze the [Gesture](#), the surface area of contact between player and videogame. We trace [Aesthetic Heritage](#), where design choices come from. We examine [Permissions](#), what the system allows and forbids.

We do this not to reduce videogames to formulas, but to develop vocabulary for what we experience. The goal is to play with more awareness, analyze with more precision, and create with more intention. Just as the atomic unit of a book is a sentence, or of a movie a scene, so too does a videogame contain an atomic parcel of explicit, concrete beats.

Exercise: Medium Comparison

Pick a videogame you've played recently. Ask:

- What does this videogame ask me to DO that a film couldn't?

- What experience emerges from my doing, not just my watching?
- How would this be fundamentally different if I weren't playing?

CRAFT

- for educators

Why Start Here

Students arrive with assumptions about what videogames are - often from marketing, reviews, or casual conversation. This page establishes the foundation: videogames are a medium of action, distinct from other media.

The product-vs-art framing helps students see that both perspectives have value, and neither is complete. This prevents the "videogames are just entertainment" dismissal AND the "videogames are high art" overclaim.

Common Misconceptions

"Videogames are interactive movies":

This privileges the cinematic elements over the action. Even story-heavy videogames like visual novels require the player to act (advance text, make choices). The action is the medium, not an add-on.

"The story is what matters":

Story can matter deeply. But the story is delivered through action - the player's and the videogame's. A videogame's story is inseparable from its [Gestures](#).

Medium Specificity

The argument that videogames are their own medium draws on medium specificity - the idea that each medium has properties unique to it, and the best works in that medium leverage those properties.

For videogames, the unique property is player action. Not interactivity as an abstract concept, but the concrete reality that the player does things, and those doings constitute the experience.

Unresolved Questions

- Where do idle videogames, walking simulators, and other minimal-action videogames fit? Are they edge cases or do they reveal something about action we're missing?
- Is "action" the right word? Or does it privilege physical doing over cognitive/emotional engagement?

Related

- [Gesture](#) - the surface area of contact
- [Player Psychology](#) - how players think and respond
- [The Design Lens](#) - our framework for seeing videogames

The Weekly Rhythm: Play → Name → Make → Reflect

This is how learning happens in VG101. Every concept, every week, follows the same cycle. It's not optional structure - it's the method.

The Four Phases

Play

Experience videogames with intention. Not just "playing for fun" - playing to feel, to notice, to gather raw material for naming. Play is research.

Name

Develop vocabulary for what you felt. If you can't name it, you can't discuss it, design it, or improve it. Naming makes the invisible visible.

Make

Create something that embodies the concept. Your idea only becomes a videogame when you commit to constraints: input, timing, state, feedback. Making reveals what you actually understand.

Reflect

Articulate what you learned. What worked? What didn't? What do you understand now that you didn't before? Reflection closes the loop and prepares you for the next cycle.

Why This Order?

The conventional order - theory first, application second - doesn't work for videogame design.

The problem with theory-first: students learn words without referents. "Ludonarrative dissonance" means nothing until you've *felt* the dissonance.

The Play → Name → Make → Reflect sequence is just to enforce the proceduralization of the experience over the common more passive and unexamined play we often go through when playing a videogame.

Play: Active Engagement

"Play" isn't just passing time with videogames. It's active, attentive engagement.

- Playing with a question in mind ("What makes this jump feel good?")

- Journaling while or after playing
- Playing something unfamiliar (not just comfort videogames)
- Playing critically (noticing choices, not just reacting)

Name: Vocabulary That Fits

Naming should feel like relief, not burden.

If we've shown students *Celeste* and had them write about the dash, then we introduce "gesture" - they should think "oh, that's what that was!"

Make: Constrained Creating

Making should be scoped and focused, not open-ended.

"Make a videogame" is paralyzing.

"Recreate the *Celeste* dash using the dash scaffold in 30 minutes" is actionable.

The constraint is the gift. Freedom overwhelms; constraint enables.

Reflect: Closing the Loop

Reflection is where learning consolidates. Without it, students do but don't understand.

Good reflection prompts:

- "What surprised you in the making process?"
- "How is what you made different from what you planned?"
- "What would you do differently?"
- "How does this change how you see the original videogame?"

Teaching the Rhythm

Students trained in lecture-first education sometimes resist playing before reading. They want to "know the answer" before engaging. The rhythm requires unlearning that expectation.

Assessment Alignment

If the rhythm is play → name → make → reflect, assessment should match. Assess reflection and making, not vocabulary recall.

Common Pitfalls

"Play" becomes passive consumption:

Without journaling prompts or questions, "play" can become just playing. Give students something to attend to.

"Name" becomes lecture:

Naming works best as dialogue, not delivery. Ask students what they noticed, then provide vocabulary for what they're describing.

"Make" becomes overwhelming:

Scope aggressively. A 30-minute constraint is better than a week-long project. Small wins build confidence.

"Reflect" gets skipped:

When time is short, reflection goes first. Resist this. Reflection is where learning consolidates. Build it into class time.

Theoretical Background

John Dewey: Learning by doing. Experience precedes abstraction. The philosophical backbone of this approach.

Donald Schön: *The Reflective Practitioner*. Reflection-in-action and reflection-on-action. Professionals learn by reflecting on what they do.

Kolb's Experiential Learning Cycle: Concrete experience → reflective observation → abstract conceptualization → active experimentation.

Studio pedagogy: Art, architecture, design schools all use variations of this rhythm. Make, critique, reflect, iterate.

What This Rhythm Resists

The lecture-exam model. Videogame design isn't learned by absorbing information. It's learned by making things and understanding what you made.

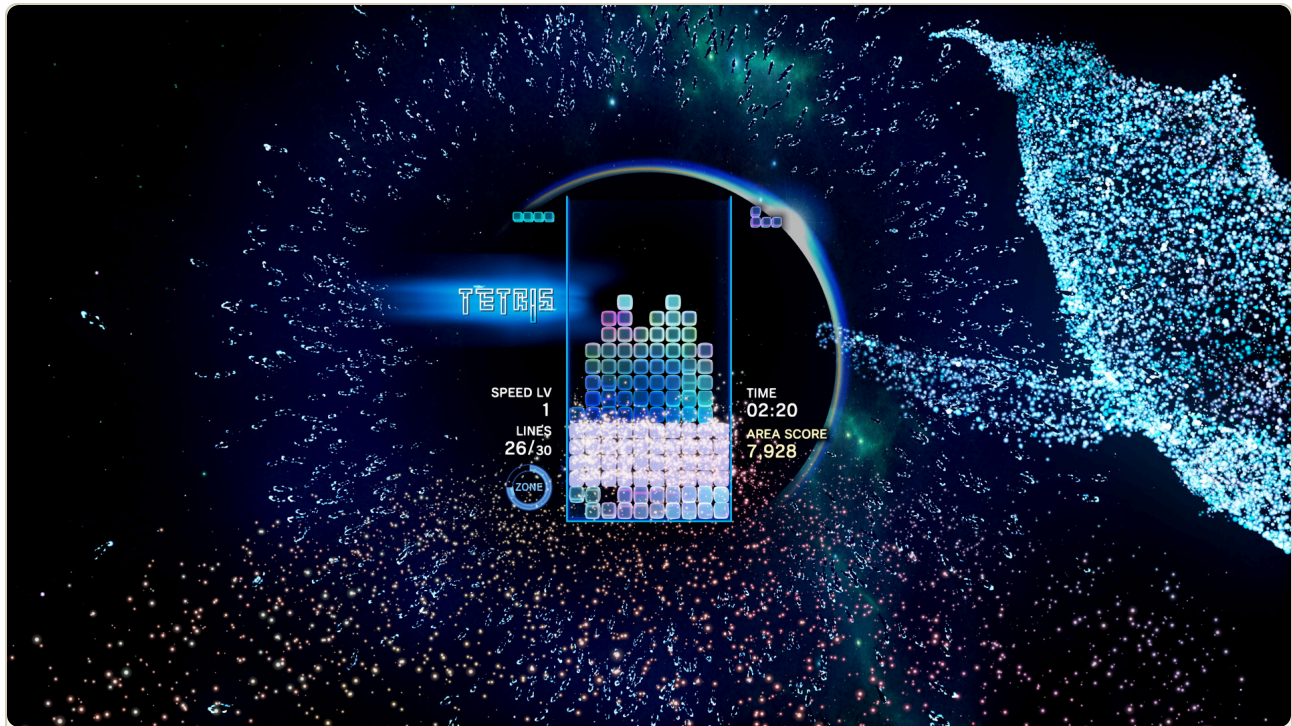
The "naturalistic" fallacy. Just playing doesn't teach design. Unstructured play is valuable, but it's not education. The structure (naming, focused making, reflection) is essential.

Related

- [Gesture](#) - First "name" in the sequence
- [Play → Name → Make → Reflect \(Pedagogy\)](#) - Full teaching details
- [Code Bank](#) - Tools for the "make" phase

Player Psychology

Understanding how players think, feel, and respond to videogames.



Tetris Effect - *Flow state made visible, the patterns persist long after you stop playing.*

Courtesy of The Tetris Company

Flow State

Flow is a state of complete absorption where challenge matches skill. Mihaly Csikszentmihalyi identified this state across many activities, but videogames are particularly good at inducing it.

When challenge exceeds skill, players feel anxiety. When skill exceeds challenge, players feel boredom. Flow lives in the channel between.

Videogames can dynamically adjust difficulty, pacing, and feedback to keep players in flow. This is part of why they are so engaging.

Motivation

Players are motivated by different things:

- **Intrinsic motivation:** Playing because the activity itself is rewarding. The joy of movement, discovery, mastery.
- **Extrinsic motivation:** Playing for external rewards. Achievements, unlocks, leaderboards, social recognition.

Neither is better. Most players experience both. But understanding which motivations your videogame serves helps you design more intentionally.

The Tetris Effect

When you play a videogame intensely, it can bleed into your non-play life. You see Tetris shapes when you close your eyes. You think about optimal routes while walking. You dream about the videogame.

This phenomenon, named after Tetris, shows how deeply videogames can embed themselves in cognition. It's a testament to how effectively videogames train patterns of thought and perception.

Feedback Loops

Feedback loops occur when the output of a system becomes input to that system. Videogames are full of them.

Positive Feedback Loops

Amplify change. The rich get richer. A player who is winning gains advantages that help them win more. This creates snowballing, runaway leaders, decisive endings.

Negative Feedback Loops

Dampen change. Pull toward equilibrium. A player who is losing gains advantages (rubber-banding). This keeps competition close, extends play, can feel unfair.

Neither is inherently good or bad. The question is what experience you want to create.

Human Random vs. Computer Random

Humans are bad at perceiving true randomness. A truly random sequence often looks "streaky" to humans, who expect more alternation.

Computer random is mathematically fair. But it can feel unfair to players. Three critical hits in a row is possible and will happen eventually. When it happens to players, they feel cheated.

Many videogames use weighted randomness that feels more fair to humans while sacrificing mathematical purity. This is a design choice, not a bug.

Player Fair vs. Computer Fair

Related: what feels fair to the player is not always what is mathematically fair.

Coyote time (letting players jump slightly after leaving a platform) is not "fair" in a strict sense. The player is jumping from midair. But it feels fair because it matches player intention.

Input buffering (remembering inputs for a grace period) is not "fair" either. But it makes the videogame feel responsive rather than demanding frame-perfect precision.

Good videogame design often prioritizes player-fair over computer-fair. The goal is an experience that feels right, not one that is mathematically pure.

Exercise: Identify the Loops

Pick a competitive videogame (Burnout, Mario Kart, any fighting videogame). Identify:

- One positive feedback loop (winners get stronger)
- One negative feedback loop (losers get help)
- How these loops shape the experience

Teaching Flow

Flow is often taught abstractly (the "flow channel" diagram). Make it concrete: have students play Tetris at increasing speeds and articulate when they feel anxious, bored, or absorbed.

The goal is recognizing flow as a felt state, not just a theoretical concept.

The Fairness Conversation

Students often think "fair" means "mathematically equal." Coyote time and input buffering are excellent examples to show that fairness is about matching player expectation, not mathematical purity.

The deeper lesson: design serves experience, not abstract correctness.

Feedback Loops in the Classroom

Students may initially think positive feedback loops are "good" and negative ones are "bad." Use examples:

- **Burnout Paradise:** Takedowns give boost (positive loop), but crashed players respawn near the pack (negative loop)
- **Mario Kart:** Blue shells and item distribution favor trailing players (negative loop)

Neither is inherently better. Each creates different experiences.

Common Misconceptions

"Flow means the videogame is too easy":

Flow requires challenge. Too easy is boredom, not flow. The key is matching challenge to skill, not removing challenge.

"Extrinsic motivation is manipulative":

It can be. But it's also natural - we like achievements, recognition, progress markers. The question is whether rewards enhance or replace the core experience.

Key Influences

Mihaly Csikszentmihalyi: *Flow: The Psychology of Optimal Experience* (1990). The foundational work on flow state, applied across many domains including videogames.

Raph Koster: *A Theory of Fun for Game Design* (2004). Connects learning and pattern recognition to player enjoyment.

Self-Determination Theory (Deci & Ryan): Identifies autonomy, competence, and relatedness as core psychological needs. Widely applied to videogame motivation research.

The Tetris Effect in Research

The Tetris effect is real and documented. Studies show that intensive videogame play creates mental imagery, dreams, and perceptual changes related to the videogame. This is sometimes called "game transfer phenomena."

It raises interesting questions about what videogames do to cognition - not in a moral panic sense, but in terms of how deeply they shape perception and thought.

What We're Taking

The practical tools: flow as a design target, feedback loops as structural elements, player-fair as a design principle. These are immediately applicable to making and analyzing.

What We're Resisting

Over-psychologizing: Player psychology is useful, but videogames are not just psychological manipulation machines. The framework matters more than the psychology.

Gamification discourse: Applying these concepts to "gamify" non-videogame contexts often misses what makes videogames work. Points and badges are not the insight.

Unresolved Questions

- How do cultural differences affect these psychological concepts? Is flow universal?

- Where is the line between engaging design and exploitative design?
- Does understanding these mechanisms make us more or less susceptible to them?

Related

- [What Makes a Videogame](#) - the medium we're studying
- [The Design Lens](#) - our framework for analysis
- [Gesture](#) - where psychology meets design

The Design Lens

A way of seeing videogames as they are, not cherry-picking their purpose.



Breath of the Wild - *Seeing the whole, the design lens applied to an open world.*

Courtesy of Nintendo

What a Lens Does

A lens focuses attention. It brings some things into clarity while letting others blur into background.

Every framework is a lens. MDA (Mechanics, Dynamics, Aesthetics) is a lens. Formal analysis is a lens. Player experience models are lenses. The 4 A's and Gesture are a lens.

No lens shows everything. The question is what a particular lens helps you see.

What This Lens Helps You See

The VG101 lens focuses on:

- **Experience over abstraction.** We start with what the player feels, not with systems diagrams.
- **Heritage over invention.** We trace where choices come from, not pretend they emerged from nothing.
- **Craft over theory.** We develop practical vocabulary for making and analyzing, not just categorizing.
- **Wholeness over dissection.** We study the Gesture as a whole before pulling it apart into components.

Using Multiple Lenses

You don't have to pick one lens and ignore all others. Different lenses serve different purposes.

If you're balancing an economy, MDA might help. If you're studying player behavior, psychology frameworks might help. If you're analyzing narrative, literary theory might help.

The VG101 lens is particularly useful for:

- Analyzing why a moment feels the way it does
- Understanding where a design choice comes from

- Developing vocabulary to talk about videogame craft
- Creating intentional experiences rather than accidental ones

The Goal

The goal is not to become a theorist. The goal is to become someone who can play a videogame and understand why it feels the way it does. And then make something that feels intentional.

Theory serves practice. Analysis serves creation. The lens is a tool, not an end.

Exercise: Lens Comparison

Take a single videogame moment (e.g., landing a jump in Celeste). Analyze it through:

- **MDA:** What mechanics create this dynamic? What aesthetic results?
- **VG101:** What is the Gesture? What are the 4 A's? What heritage is present?

What does each lens reveal? What does each miss?

Respecting What Came Before

This framework is not about erasing the work of other videogame designers, theorists, or scholars. All of that knowledge is respected and in play.

Academic perspectives matter. Social perspectives matter. Entertainment perspectives matter. Product perspectives matter. We are not here to dismiss any of these.

We are here to see videogames as they are unto themselves. To analyze them without cherry-picking which framing serves our argument. To take them seriously as a medium with its own properties, its own traditions, its own craft.

Why Another Framework

Students may ask: "Why not just use MDA?" or "Why do we need new vocabulary?"

The answer: existing frameworks are valuable but often privilege certain perspectives. MDA privileges system thinking. Formal analysis privileges structure. Player experience models privilege psychology.

The VG101 framework privileges the felt experience and its heritage. This is a different entry point, not a replacement.

Teaching Lens-Switching

Students should learn to switch between lenses fluently. An exercise: analyze the same videogame moment three ways (MDA, player psychology, VG101). What does each reveal?

The goal is flexibility, not orthodoxy. No single lens is "correct."

Common Challenges

"This is too abstract":

Ground everything in specific videogame moments. Never discuss Gesture without pointing to a concrete Gesture. Theory without examples is empty.

"I just want to make, not analyze":

Analysis serves making. You can't intentionally create a feeling you can't name. The vocabulary is a tool for creation, not a replacement for it.

Why "Lens"

The lens metaphor comes from Jesse Schell's *The Art of Game Design*, which presents 100+ lenses for examining videogames. Each lens asks different questions.

We use "lens" to emphasize that frameworks are tools, not truths. You look through them; you don't worship them.

Phenomenology and Experience

The emphasis on felt experience connects to phenomenological traditions in philosophy - starting from how things appear to consciousness rather than from abstract structures.

Brendan Keogh's work applies this to videogames explicitly. We start with the body playing the videogame, not with the system that the body interacts with.

What We're Taking

The insight that frameworks are lenses, not truths. The permission to develop vocabulary suited to what we're studying. The commitment to experience as the starting point.

What We're Resisting

Framework wars: "MDA is wrong, VG101 is right." No. Different lenses for different purposes.

Theory for theory's sake: The vocabulary serves making. If it doesn't help you create, it's not doing its job.

Unresolved Questions

- Can we ever see a videogame "as it is" or only through lenses? Is there a lens-free view?
- How do we know when a framework is useful vs. when it's imposing structure that isn't there?

- What's the relationship between analyzing videogames and making them? Do they require the same skills?

Related

- [What Makes a Videogame](#) - what we're looking at
- [Player Psychology](#) - another lens on players
- [Gesture](#) - the core concept of this lens
- [Aesthetic Heritage](#) - tracing where choices come from

PART III

Narrative Design

Telling stories in a medium that is played, not read or watched.

Narrative Design

Storytelling through and alongside videogame systems. Not just cutscenes and dialogue, but meaning that emerges from moment to moment interaction as well as system tension.

Entries

Entry	What it covers
Environmental Storytelling	Narrative embedded in space and objects - story discovered, not told
Branching & Consequence	Player choice and its long-term effects on narrative
Dialogue Systems	Conversation as interface, gameplay, and character
Ludonarrative Harmony	When mechanics and story align - or deliberately clash

Core Insight

The narrative experience in videogames isn't just "the story." It's how the videogames' story relates to play.

The best videogame narratives use the medium to create not just meaning from what you are shown and told, but also emerges from what you *do*. A cutscene can tell you the villain is powerful, but having you fail against them makes you experience it.

By Framework Concept

Concept	Narrative Application
<u>Gesture</u>	Narrative moments as complete experiences (Action + Art + Arc + Atmosphere)
<u>The 4 A's</u>	Arc is particularly central - story is shaped by duration
<u>Aesthetic Heritage</u>	Where your narrative techniques come from (film, literature, theatre, other videogames)
<u>Permissions</u>	What the player is allowed to do shapes what stories are possible

Teaching Sequence

Recommended order for introducing narrative concepts:

1. **Environmental Storytelling** - Most accessible. Students can analyze without playing.
2. **Ludonarrative Harmony** - Introduces the core tension of videogame narrative.
3. **Branching & Consequence** - Requires understanding of systems and scope.
4. **Dialogue Systems** - Most technical. Benefits from prior concepts.

Common Misconceptions

"Narrative = Writing"

Students often conflate narrative design with scriptwriting. Clarify: narrative design is about *structure* and *integration with systems*. Writing is one tool among many.

"More story = better"

Some videogames are harmed by too much explicit narrative. Help students see that restraint and implication are also narrative choices.

"Ludonarrative dissonance is always bad"

Dissonance can be intentional. *Spec Ops: The Line* weaponizes dissonance. The question is whether it's designed or accidental.

Assessment Approaches

- Have students map a videogame's narrative structure without describing the plot
- Ask students to identify the [Aesthetic Heritage](#) of a videogame's narrative techniques
- Design a narrative moment using only environmental storytelling (no text, no dialogue)

Narrative vs. Story

In narratology, "story" (fabula) is the chronological sequence of events. "Narrative" (syuzhet) is how those events are presented.

Videogames add a third layer: the player's construction of meaning through interaction. This makes videogame narrative fundamentally different from film or literature.

The Embedded vs. Emergent Distinction

Embedded narrative is authored content: cutscenes, dialogue, scripted events. Emergent narrative arises from systems: the stories players tell about what happened in their playthrough.

Most videogames mix both. The question is the ratio and how they interact.

Related Sections

- [Permissions](#) - what you're allowed to do shapes what stories are possible
- [Gesture](#) - the whole experience, including narrative
- [Level Design](#) - narrative often lives in space
- [Undertale: Genocide Route](#) - narrative through permissions

Environmental Storytelling

Narrative embedded in the world itself - not told through cutscenes or dialogue, but discovered through exploration, observation, and inference.



Gone Home - Every object placed with intent, narrative embedded in the environment itself.

Courtesy of Fullbright

What Environmental Storytelling Is

A skeleton clutching a note. A child's room in an abandoned house. Scorch marks on a wall. The environment tells you what happened without *telling* you.

Videogames have a unique capacity for spatial narrative. Unlike film, which controls exactly what you see, videogames let players **move through** story.

This creates:

- **Discovery:** The player finds the story rather than receiving it
- **Agency:** The player chooses where to look, what to investigate
- **Immersion:** Narrative feels embedded in the world, not layered on top
- **Interpretation:** The player fills gaps, becomes co-author

Environmental storytelling respects player intelligence. It shows rather than tells - and sometimes doesn't even show, just implies.

Techniques

Found Objects: Items that imply history: a stuffed toy in a war zone, a wedding ring by a grave, a locked diary. Objects need context - a toy alone is just a toy. A toy next to small bones tells a story.

Spatial Arrangement: How objects relate in space: a chair facing a window, a table set for two with one plate untouched, a bed in a closet. Think about *before* and *after*. What was the scene before something happened?

Environmental Damage: What violence or time has done: bullet holes, bloodstains, ceilings, overgrown gardens. Damage tells you about forces. What could make this mark?

Contrast: Juxtaposition that creates meaning: a pristine room in a destroyed building, a child's drawing of violence, a celebration banner in a morgue. Contrast implies *break* - something changed.

Absence: What's missing: an empty picture frame, a gap in a row of trophies, a name scratched off a list. Absence requires establishment - you need to show what *should* be there.

Levels of Environmental Narrative

Level	What it tells	Example
World history	What happened before the player	<i>Fallout</i> : the bombs fell, society
Location history	What happened in this place	This office was barricaded; someone lived here alone
Character traces	Who was here, what were they like	Love letters suggest a relationship
Recent events	What just happened	Warm coffee suggests someone left moments ago
Player impact	What the player caused	Returning to see consequences of your earlier choice

The richest environments layer multiple levels - the world's history, the location's story, the individual's life.

The Discovery Arc

Environmental storytelling has pacing, even without scripted events:

1. **Initial confusion:** Player enters a space, sees elements, doesn't yet understand
2. **Accumulating details:** Each discovered element adds context
3. **Moment of understanding:** The pieces click; the story crystallizes
4. **Emotional response:** Player feels the implications

This arc requires **restraint**. If you explain everything immediately, there's no discovery. If you explain nothing, there's only confusion.

Common Mistakes

- **Over-signposting:** Highlighting every environmental detail ("Press X to examine") trains players not to look.
- **Under-signposting:** Making crucial story elements invisible or missable without any guidance.
- **Inconsistency:** Environmental logic that breaks (food that doesn't decay, bodies that don't decompose) unless deliberately stylized.
- **Audio log dependency:** Using environmental storytelling as an excuse for audio logs everywhere. Audio logs are exposition wearing exploration's clothes.
- **Dead ends:** Environmental stories that have no resolution or relevance. A mysterious setup needs at least partial payoff.

Exercise: Scene Forensics

Take an image of an environmental storytelling scene from *Gone Home*, *What Remains of Edith Finch*, or *Dark Souls*. Ask:

1. What's the story here?
2. What specific elements tell you that?
3. What's left ambiguous?
4. How would you break this if you changed one element?

Why Teach This

Environmental storytelling bridges **Narrative Design** and **Level Design** - it's where story becomes spatial.

Students often think of narrative as cutscenes or dialogue. This concept expands their understanding of how videogames can tell stories through their unique capacity for spatial exploration.

Environmental vs. Expository

Environmental	Expository
Implied	Stated
Discovered	Delivered
Gaps require inference	Complete information
Player interprets	Author interprets for player
Easy to miss	Hard to miss

Neither is superior. Expository narrative has its place. But environmental storytelling does something expository can't: it makes the player *detective*.

Assessment

Exercise: Crime Scene Design

Students create a small environment (one room) that tells a story with zero text or dialogue. Classmates must "read" the scene and describe what happened.

Debrief: What was communicated? What was missed? Where was interpretation different from intention?

Key concept: Environmental storytelling is **design**, not decoration. Every element is a choice that either contributes to the story or detracts from it.

Key Influences

Henry Jenkins - "Game Design as Narrative Architecture." The foundational essay. Jenkins argues videogames are less like traditional stories and more like designed spaces that enable storytelling.

Don Carson - Disney Imagineer who articulated environmental storytelling for theme park design. Videogames inherited heavily from this tradition.

Marie-Laure Ryan - Narrative theory applied to digital media. The distinction between "narrative" and "narrativity" - some videogames have embedded narrative, others have emergent narrativity.

What We're Taking

The spatial turn. Videogames tell stories *through* space in ways other media can't. Environmental storytelling isn't a workaround for lacking cutscenes - it's a native videogame capacity.

What We're Resisting

The "games must have narrative" push: Not all videogames need environmental storytelling. *Tetris* isn't narratively impoverished; it's doing something else.

Environmental fetishism: The idea that environmental = good, expository = bad. Sometimes you need to *tell* the player something. The audio log is mocked because it's overused, not because it's inherently bad.

Unresolved Questions

- How much authorial control should there be? *Dark Souls* is notoriously ambiguous - is that meaningful mystery or insufficiently designed? Different players have different answers.
- Can environmental storytelling work for character interiority? It's good at *what happened* but harder for *how it felt*. How do you environmentally tell an emotional

story?

- What's the relationship between environmental storytelling and genre? Horror uses it extensively; does it work for comedy?

Related

- [Spatial Communication](#) - how spaces communicate non-narratively
- [Ludonarrative Harmony](#) - when environment and mechanics align
- [Aesthetic Heritage](#) - where visual choices come from

Glossary Terms

- [Diegetic](#) - story elements that exist within the videogame world
- [Found Narrative](#) - story discovered rather than delivered

Branching & Consequence

The technique of writing multiple passes of an event - usually a conversation or action - with the goal of creating more tailored player expression and more varied playthroughs.



The Witcher 3 - *Branching that ripples forward, choices with delayed, meaningful consequences.*

Courtesy of CD Projekt Red

Branching vs. Consequence

Branching is when player choices create different paths. **Consequence** is when those choices matter - when the path you took shapes what happens later.

Every choice-based videogame grapples with these: How much do choices diverge? How long do consequences echo? And crucially: how do you create the *feeling* of consequence when you can't actually build infinite permutations?

Meaningful Choice vs. Illusion of Choice

Meaningful choice has an actual effect on the player's character, world, or story. **Illusion of choice** ensures the same narrative occurs no matter what your moment-to-moment choices are.

Neither is inherently bad. Illusion of choice can let players express themselves without ballooning scope. But players resent it when they feel deceived - when a choice *seems* significant but leads nowhere.

What Makes Consequence Feel Real

A consequence feels real when the player believes their decisions put them where they are - *even if they actually didn't*. This is the designer's advantage: perception matters more than structure.

Techniques for felt consequence:

- **Acknowledgment:** Have the videogame recognize what the player did. An NPC mentions your earlier choice. The world shows a scar from your decision.
- **Delayed Callback:** Reference an early choice late in the videogame. The longer the delay, the more impactful.
- **Character Reaction:** Have characters respond emotionally, not just informationally. Disappointment. Gratitude. Betrayal.
- **Absence:** Remove something. A character who dies is *gone*. A path not taken is closed.

Branching Structures

Linear with Flavor: The story goes one place, but dialogue and details change based on choices.

Foldback (Bottleneck): Paths diverge, then converge at key points before diverging again. (Mass Effect)

Parallel Paths: Genuinely different routes that occasionally touch. (Silent Hill 2)

Branching Tree: Exponential divergence. Expensive, rare. (Some visual novels)

Types of Consequence

Type	When it appears	Player experience	Cost
Immediate	Right after choice	"That affected something"	Low
Delayed	Hours later	"My earlier choice mattered"	Medium
Cumulative	Based on pattern	"Who I've been matters"	Medium
Cosmetic	Different details, same outcome	"I customized my experience"	Low
Structural	Different content entirely	"This is a different game"	High

Key insight: Players often can't tell the difference between structural and cosmetic consequences on a single playthrough. This is your ally.

Exercise: Study Existing Videogames

Play through key sequences from:

- *Fable 2* - Good/evil system with cosmetic and structural consequences
- *Tales from the Borderlands* - Telltale's strongest work on felt consequence
- *The Walking Dead* - Classic example of illusion of choice done well (and sometimes poorly)

- *Life is Strange* (2015) - The rewind mechanic lets players see immediate consequences before committing, but delayed consequences still accumulate through the butterfly effect system. Study how the final choice recontextualizes every earlier decision.

For each: What choices did you make? What consequences followed? How much did the content actually diverge?

Why Teach This

Students often think branching narrative means creating massive branching conversations and choices. But that approach is exponential in scope - a videogame with 10 meaningful binary choices has 1,024 possible endings. That's unshippable.

Teaching branching and consequence properly prevents the "scope trap": designing systems that are theoretically interesting but practically impossible to execute. We must find smarter, more refined solutions.

Common Misconceptions

"Every choice should have entirely bespoke experiences"

Students want to make every choice lead to completely unique content. This is admirable but unsustainable. The skill is creating *felt* uniqueness with *actual* modularity.

"More branches = better narrative"

Some of the best narrative videogames (*Brothers: A Tale of Two Sons*) are almost entirely linear. Branching is a tool, not a virtue.

Assessment

Test understanding through analysis questions on different types of narrative branching:

- What are the strengths and shortcomings of foldback structure?
- When is illusion of choice appropriate? When is it harmful?
- Map a sequence from a published videogame: perceived branches vs. actual branches

Teaching Exercise: Minimal Viable Branch

Students design a 3-choice sequence where each choice has a consequence, but total content is limited to 5 scenes. Forces creative use of callbacks, acknowledgments, and cosmetic variation.

Warning: Make students *count their nodes* before building. Scope awareness is the lesson.

Genre Expectations

What players expect from branching depends heavily on genre. RPGs expect a lot of meaningful choices. Roguelikes expect high variation but accept strong modularity and repetition. Linear action videogames expect almost none.

Study the choices your genre makes. What do players of this genre expect? What conventions can you inherit, and where can you transform them?

Authored vs. Emergent Consequence

Authored	Emergent
Designer crafted this outcome	Outcome emerged from systems
Predictable, polished	Surprising, rough edges
Limited permutations	Potentially infinite permutations
Story-shaped	Anecdote-shaped

Authored: Mass Effect's loyalty missions affecting the ending

Emergent: Dwarf Fortress's fortress collapsing from a butterfly effect of decisions

Both are valuable. Authored consequence feels intentional; emergent consequence feels personal.

Key References

- **Sam Barlow** - *Her Story, Immortality*. Non-linear discovery rather than branching choices.
- **Emily Short** - Interactive fiction pioneer. Quality-based narrative as alternative to simple branching.
- **Chris Gardiner** - Failbetter Games. "Quality-based narrative" uses accumulating stats rather than branch points.

Unresolved Questions

- How do we balance player agency with authored meaning? If the player can choose anything, how does the author say anything?
- Is consequence even possible in a save/reload paradigm? Can choices "really" matter when the player can undo them?
- What's the relationship between branching and replayability? Some videogames are better experienced once.

Related

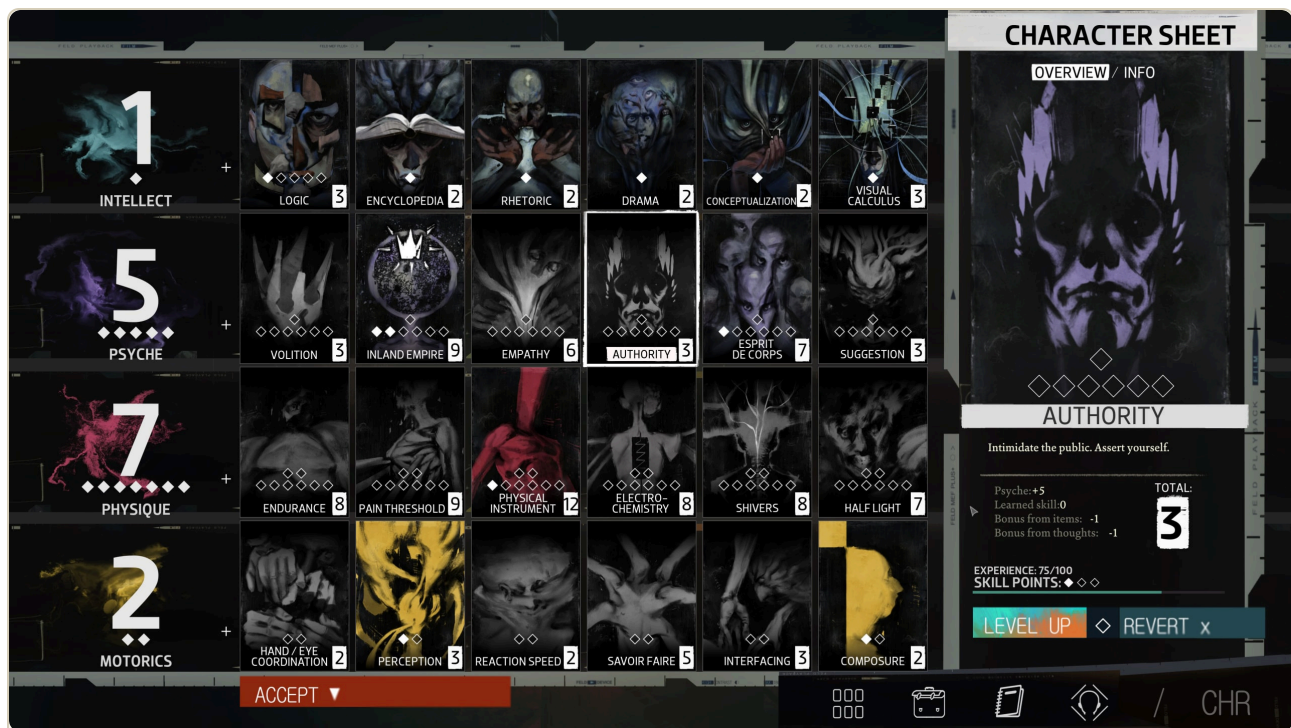
- [Permissions](#) - choices are permissions granted or withheld
- [Dialogue Systems](#) - where many branches happen
- [Ludonarrative Harmony](#) - when story and mechanics align

Glossary Terms

- [Foldback Structure](#) - branches that reconverge
- [False Choice](#) - choices without real consequence
- [Quality-Based Narrative](#) - tracking player history as stats

Dialogue Systems

Any system that supplies conversation or interaction in your videogame's world. The need to create systems that support communication and guidance is paramount.



Disco Elysium - Dialogue as system, your own mind becomes the most complex NPC.

Courtesy of ZA/UM

What Dialogue Systems Are

A dialogue system is the interface and logic that governs conversation in a videogame. It determines how players talk to characters, how characters respond, and how conversations flow.

Dialogue systems are where narrative meets interface design. The *structure* of conversation shapes what kinds of stories you can tell.

Core Components

Triggering: How do conversations start?

- Automatic (enter area, story triggers)
- Player-initiated (talk button near NPC)
- Contextual (interrupt after certain actions)

Display: How is dialogue presented?

- Text boxes
- Voiced audio with subtitles
- Speech bubbles
- Full-screen conversational UI

Player Input: How does the player participate?

- Advance text (passive)
- Select from options
- Type responses (parser)
- Timed choices
- No input (cinematic)

Flow Control: How does conversation progress?

- Linear (one path)

- Branching (multiple paths)
- Hub-and-spoke (return to menu of topics)
- State-based (available options depend on flags)

Common Structures

Simple Linear: Player advances through fixed text. No choices. Use for exposition, atmospheric flavor, world-building.

Menu Selection: Player chooses from a list of topics/responses. Use for information gathering, controlled exploration.

Branching Tree: Choices lead to different dialogue paths that may reconverge or diverge. Use for roleplaying, consequence, character expression.

Hub and Spoke: Keep returning to a central menu after each topic is explored. Use for investigation, lore, optional depth.

The Option Problem

What the player sees vs. what the character says is a design choice:

Approach	Example	Pro	Con
Full preview	"I'll help you find your daughter"	Player knows exactly what they're choosing	Long, can break flow
Paraphrase	"[Agree to help]"	Clean, brief	Player may disagree with delivery
Tone tag	"[Sympathetic]"	Expresses character without spoiling	Ambiguous
First words	"I'll help you..."	Natural, like real conversation	Can be misleading

Mass Effect used a tone-based wheel that sometimes delivered lines players didn't expect.

Disco Elysium shows full text. Neither is "correct" - they serve different experiences.

Conversation as Gameplay

Dialogue can be more than information delivery - it can be **play**.

- *Ace Attorney*: Present evidence during testimony, catch contradictions
- *Disco Elysium*: Skill checks embedded in dialogue, your thoughts argue with each other
- *Hades*: Brief, contextual exchanges that deepen over dozens of runs

When dialogue is gameplay, it needs stakes (what can I win or lose?), player agency (what decisions matter?), and feedback (how do I know what worked?).

Exercise: Dialogue Autopsy

Take a dialogue sequence from a published videogame. Map:

1. What's the structure? (Linear, branching, hub?)
2. How is text displayed?
3. What player input is required?
4. How are options presented?

Why Teach This

Dialogue systems are where writing meets UI/UX. Students often think about dialogue as pure writing, ignoring that the *structure* of conversation constrains what stories can be told.

The right approach depends entirely on design goals and [Aesthetic Heritage](#) of the genre. A visual novel, an action RPG, and a puzzle videogame all need fundamentally different dialogue systems.

Common Misconceptions

"More options = better dialogue"

Students often want to add more choices without considering scope or whether choices matter. A few meaningful choices beat many meaningless ones.

"Voice acting is always better"

Voice acting is expensive and constrains iteration. Text-only dialogue allows for more content, faster changes, and player projection onto characters.

Assessment

Ask students to:

- Map the dialogue structure of an existing videogame
- Redesign a linear dialogue sequence as branching (and vice versa)
- Design a dialogue encounter where conversation IS the gameplay

Historical Lineage

Infocom - Parser-based interactive fiction that pioneered computer conversation. "Ask about" and "tell about" as primary verbs.

BioWare - The dialogue wheel, romance options, companion relationships. Defined modern RPG conversation expectations.

ZA/UM (Disco Elysium) - Pushed dialogue as gameplay. Your skills are characters in the conversation. Showed that text-heavy videogames can succeed commercially and critically.

Current Questions

LLMs and procedural dialogue: How will large language models change dialogue systems? Potentially infinite variation, but loss of authorial control. The tradeoff between authored precision and generative flexibility is unresolved.

Pacing: Is there a "natural" dialogue pacing? How long should text stay on screen? How fast should it scroll? These are UX questions with narrative implications.

Related

- [Branching & Consequence](#) - how dialogue choices create story paths
- [Permissions](#) - dialogue as permission system
- [Aesthetic Heritage](#) - genre determines approach

Glossary Terms

- [Dialogue Tree](#) - branching conversation structure
- [Barks](#) - short contextual voice lines

- Hub and Spoke - menu-based conversation structure

Ludonarrative Harmony

*When what you do (ludo) and what you're told (narrative) reinforce each other.
The mechanics and the story say the same thing.*



Brothers: A Tale of Two Sons - Mechanics and story perfectly aligned, the controls ARE the narrative.
Courtesy of Starbreeze Studios / 505 Games

Harmony vs. Dissonance

Ludonarrative harmony is when mechanics and narrative point the same direction. What you do *is* what the story is about.

Ludonarrative dissonance is when they contradict: the story says one thing, the gameplay says another.

The Classic Dissonance

Uncharted's Nathan Drake is a likable everyman in cutscenes. In gameplay, he kills hundreds of people without hesitation or consequence.

The story says: charming adventurer.

The gameplay says: mass murderer.

This contradiction is ludonarrative dissonance. The two channels of meaning are out of sync.

Harmony in Practice

Videogame	How mechanics reinforce narrative
<i>Journey</i>	Cooperation mechanics support theme of connection
<i>Brothers: A Tale of Two Sons</i>	Control scheme embodies the relationship
<i>Papers, Please</i>	Bureaucratic mechanics create complicity
<i>Celeste</i>	Difficulty mechanics mirror struggle with mental health
<i>Shadow of the Colossus</i>	Mounting and stabbing creates ambivalence about killing

In each case, the *feeling* of playing matches the *meaning* of the story.

Three Relationships

Harmony: Mechanics and narrative point the same direction. *Journey*: You walk toward a mountain. You meet strangers. You help each other. The mechanics ARE the theme.

Dissonance: Mechanics and narrative contradict. *BioShock*: "A man chooses, a slave obeys" - but you have no choice but to follow the objective markers.

Productive Tension: The contradiction is the point. *Spec Ops: The Line*: You play a shooter, but the videogame wants you to feel sick about shooting. The dissonance indicts the genre.

Achieving Harmony

Start with verbs: What will the player *do* most? That's what your videogame is about, regardless of your narrative intentions. Design story to match.

Audit your mechanics: For each mechanic, ask: what does this *say*? A regenerating health system says "damage is temporary." A permanent death system says "death is final." Do those statements match your story?

Use mechanics for theme: Don't just tell a story about loss - make the player lose something. Don't just tell a story about connection - make the player depend on someone.

Accept constraints: If your videogame is a shooter, it's about shooting. You can critique shooting (Spec Ops), but you can't pretend you're not making a videogame about shooting.

Exercise: Dissonance Hunt

Play a videogame and identify every instance of ludonarrative dissonance you can find. Then classify each: accidental, harmful, or productive?

Why Teach This

The core provocation: "Your mechanics are your story. If they contradict your cutscenes, your mechanics win."

Students often want to add narrative to justify mechanics they've already designed. This concept pushes them to ask whether the mechanics *should* be what they are.

Common Misconceptions

"All dissonance is bad"

Dissonance is only a problem when it's accidental - when the designer didn't notice the contradiction. (Well, a problem for the videogame designer's artistic intent; for players it's just a new [gesture!](#)) Dissonance can be a very meaningful and persuasive tool when it's intentional - it doesn't even have to be a contradiction that makes you uncomfortable to be dissonance, though that is always cool.

The biggest example always gone to is *BioShock* (from Clint Hocking's 2007 blog post that coined the term as a derogatory). The narrative plot is about free will and control, but the videogame is fairly rigid about things in the end.

Another famous example is *Uncharted*. When applying the idea of ludonarrative dissonance, dashing Nathan Drake is a qualified genocidal force of nature - a player will murder (in self-defense?) 585 to 1,130 people (depending on play style) in his first outing, *Uncharted: Drake's Fortune*.

It can also be the gap between cutscene and play scene. *Halo* had a player experience design theory of BYOG (Bring Your Own Gun) - whatever gun you had equipped in play, Bungie wanted it to be in the cutscenes. They wanted you to experience "choices matter" in a very different context than we usually experience.

"Perfect harmony is always the goal"

Tension, contradiction, and discomfort are valid artistic choices. *Mario* rescues Peach through murdering ~143 Goombas in *Super Mario Bros.*; no one has ever cared. Context

matters.

The "Fun" Problem

Sometimes mechanics are fun but thematically wrong. *Uncharted*'s shooting is satisfying - that's the problem. If it weren't fun, the dissonance wouldn't exist.

Approaches:

- **Lean into fun:** Accept that your videogame is partly power fantasy. Don't pretend otherwise in cutscenes.
- **Make the theme fun:** Design mechanics where the thematic action IS the satisfying one. *Portal*'s puzzle-solving is both fun and thematically coherent.
- **Make fun uncomfortable:** Design mechanics that are satisfying and then make you feel bad about that satisfaction (*Spec Ops*, *Undertale*).
- **Reduce fun dissonance:** If violence is thematically weighty, make combat difficult, costly, or rare.

Assessment

Exercise: Harmony Redesign

Take a videogame with clear dissonance. Redesign EITHER the narrative to match the mechanics OR the mechanics to match the narrative. What's lost in each direction?

Origin of the Term

"Ludonarrative dissonance" was coined by Clint Hocking in a 2007 blog post about *BioShock*. It became one of the most-used (and sometimes overused) terms in videogame criticism.

Hocking's original critique remains sharp: *BioShock* asks you to question blind obedience while the gameplay requires blind obedience to objective markers.

Key Influences

Clint Hocking - The original formulation.

Ian Bogost - Procedural rhetoric provides the theoretical backing: videogames make arguments through mechanics.

What We're Taking

The core insight that mechanics *mean something*, and that meaning should be in dialogue with explicit narrative. This is basic but often ignored.

What We're Resisting

Dissonance policing: The idea that all dissonance is bad. Sometimes it's fine. Context matters more than consistency.

The harmony fetish: The idea that perfect harmony is always the goal. Tension, contradiction, and discomfort are valid artistic choices.

Unresolved Questions

- How much dissonance can players tolerate before it breaks immersion?
- Is dissonance a Western critical concern? Do players from different traditions parse the ludo/narrative relationship differently?

- Can systemic videogames (*Dwarf Fortress*, *Rimworld*) have ludonarrative dissonance if they have no authored narrative?

Related

- [Gesture](#) - the whole experience, including this tension
- [Permissions](#) - what you're allowed to do shapes what the story can be
- [Branching & Consequence](#) - when choices create meaning

Glossary Terms

- [Ludonarrative Dissonance](#) - when mechanics and story contradict
- [Procedural Rhetoric](#) - arguments made through rules

PART IV

Level Design

Shaping space so players understand where to go and how to feel getting there.

Level Design

Spatial communication, pacing, and world-building through environment. Every room, corridor, and vista is saying something.

Entries

Entry	What it covers
<u>Spatial Communication</u>	How space talks to the player - landmarks, framing, contrast
<u>Pacing & Flow</u>	Rhythm of intensity over time and space
<u>Guidance Without Hand-Holding</u>	Teaching through design, not tutorials or text
<u>Verticality & Sight Lines</u>	Height, visibility, and "weenies" as design tools

Core Insight

Level design is **communication design**. Every room, corridor, and vista is saying something. The question is whether it says what you intend.

Good level design teaches without tutorializing, guides without hand-holding, and creates pacing through spatial arrangement.

By Framework Concept

Concept	Level Design Application
Gesture	Each space creates a Gesture - the experience of being there and moving through
The 4 A's	Art (visuals, lighting), Arc (duration of traversal), Atmosphere (mood of spaces)
Aesthetic Heritage	Where your spatial design vocabulary comes from (architecture, cinema, theme parks, other videogames)
Permissions	Space defines what's allowed - walls are Permissions

Teaching Sequence

Recommended order for introducing level design concepts:

1. **Spatial Communication** - Foundation. How does space convey meaning?
2. **Guidance Without Hand-Holding** - Practical application of communication.
3. **Verticality & Sight Lines** - Specific techniques for guiding attention.
4. **Pacing & Flow** - Requires understanding of the above to discuss rhythm.

Common Misconceptions

"Level design = environment art"

Students conflate visual design with spatial design. Clarify: level design is about *player movement and experience*. Environment art supports this but isn't the same discipline.

"More detail = better"

Visual noise can harm navigation. Teach students to use contrast and restraint. The most important thing in a room should be the most visually distinct.

"Players will explore everywhere"

Players often miss content that isn't on the critical path. If something matters, design ensures players encounter it - or accept that it's optional.

Analysis Exercise

Have students map the first 10 minutes of a videogame:

- Draw a top-down sketch of the spaces (doesn't need to be accurate)
- Mark where the videogame guided their attention
- Mark where they got confused or lost
- Identify the techniques used (lighting, landmarks, geometry)

Creation Exercise

Design a level that teaches a mechanic without any text:

- The mechanic must be used to progress
- No tutorials, prompts, or written instructions
- Space itself must teach the mechanic

Architecture and Level Design

Level design borrows heavily from architectural theory - circulation, wayfinding, spatial sequence. But architecture serves inhabitants; level design serves players. The goals differ.

Architects design for repeated use over years. Level designers often design for single traversal. This changes everything about how we think about space.

The Theme Park Heritage

Disney Imagineering contributed key concepts to level design: the "weenie" (a distant landmark that draws you forward), forced perspective, and environmental storytelling.

Theme parks and videogames share a concern: how do you guide visitors through a space while maintaining the illusion of freedom? Level design inherits this problem.

Film and Spatial Composition

Cinematography provides vocabulary for spatial composition: framing, leading lines, depth staging. Level designers are often implicitly cinematographers, composing what the player sees.

The difference: players control the camera (in most videogames). Level design must work from many angles, not just the one the director chose.

The Procedural Question

Procedural generation challenges traditional level design. If space is generated, can it still communicate?

The answer is yes, but differently. Procedural levels communicate through systemic patterns, not authored moments. This is its own discipline.

Related Sections

- [Gesture](#) - how levels shape the player's experience
- [Environmental Storytelling](#) - narrative in space
- [Portal: Level One](#) - implicit teaching through spatial constraint
- [Case Studies](#) - spatial design masterclasses

Spatial Communication

Spatial communication is how level design talks to the player - guiding attention, suggesting paths, establishing importance, all without words.

Space isn't neutral. Every room, corridor, vista, and dead end is saying something. The question is whether it's saying what you intended.



Half-Life 2 - *Spatial communication through landmarks, the Citadel orients every moment of play.*
Courtesy of Valve Corporation

Space as Language

Level design is communication design. You're trying to tell the player:

- Where they are
- Where they can go
- Where they should go
- What matters
- What's dangerous
- What's safe

You do this through the arrangement of space and the objects within it.

The Vocabulary of Space

Framing

What you put in the player's line of sight. A doorway frames what's beyond. A window frames a destination. A narrow passage focuses attention.

Contrast

Difference draws the eye. A red door in a gray room. A lit area in darkness. An open space after tight corridors.

Scale

Size indicates importance. A huge door suggests something significant behind it. A tiny crawlspace suggests secret.

Repetition

Patterns establish expectations. Repeated columns create rhythm. Breaking the pattern draws attention.

Landmarks

Distinctive, memorable elements that help orientation. The Citadel in *Half-Life 2*. The volcano in *Breath of the Wild*.

Lines

Leading lines - paths, edges, beams of light - direct the eye. Railway tracks, rivers, rows of pillars.

The Critical Path

The **critical path** is the route through the videogame. Spatial communication should make it findable without making it compulsory.

Techniques:

- Light the critical path more brightly
- Make it wider or more inviting
- Place it in the player's natural eye line
- Use enemies or pickups as breadcrumbs

Danger: If the critical path is too obvious, the world feels like a corridor. If it's too hidden, players get lost.

The art is making the critical path feel *discovered* rather than *prescribed*.

Negative Space

What you *don't* put in a space matters.

- Empty rooms feel ominous or contemplative
- Missing objects (gaps in a lineup, empty pedestals) suggest something was taken
- Blocked paths show where you *can't* go, defining where you can

Teaching Through Space

The best tutorials are levels that teach without text.

World 1-1 of Super Mario Bros. teaches:

- Run right (empty left, enemies from right)
- Hit blocks (first block has a mushroom)
- Avoid enemies (Goomba in your path)

- Jump (first pit requires it)

No text. Just spatial arrangement.

Principle: Put players in situations where the correct action is the natural action.

Orientation Systems

Players need to know where they are. Spatial communication supports this through:

Technique	How it works
Distinct zones	Each area looks different (ice level, fire level)
Visible landmarks	You can always see the mountain
Sightlines	You can see where you came from, where you're going
Logical geography	The world makes spatial sense
Environmental cues	Moss grows on north sides, water flows downhill

Disorientation is sometimes the goal (horror, maze puzzles). But it should be chosen, not accidental.

See Also

- [The 4 A's](#) - Art component includes spatial composition
- [Guidance Without Hand-Holding](#) - adjacent concept
- [Environmental Storytelling](#) - space as narrative

Glossary Connections

- [Critical Path](#) - the main route through a level
- [Weenie](#) - Disney term for a visible landmark that draws you forward
- [Leading Lines](#) - visual elements that direct the eye

Teaching Notes

Spatial Communication is the bridge between **Level Design** and **Feedback & Readability**.
Space is a feedback system.

Exercise: Screenshot Analysis

Students screenshot 5 moments where they knew where to go without being told. Analyze: what spatial cues did the designer use?

Then screenshot 5 moments where they were lost. What cues were missing or misleading?

Exercise: Paper Prototype

Design a level on paper (top-down). Identify the critical path. Then mark every spatial cue that will guide the player along it. Have someone else trace what they think the path is.

Where they diverge = communication failure.

Key Insight

Players don't read your intentions. They read your spaces.

Common Misconceptions

- **"Good design means players never get lost."** Sometimes disorientation is the goal. Horror, maze puzzles, and exploration videogames use confusion intentionally.
- **"More landmarks = better navigation."** Too many landmarks compete for attention and become noise. The right number is fewer than you think.
- **"The critical path should be obvious."** Too obvious feels like a corridor. The goal is discoverable, not obvious.

Assessment Approaches

- **Blind playtest:** Watch someone navigate a level with no instruction. Where do they go? Where do they hesitate?
- **Heat mapping:** Track player paths through spaces. Do they match intended flow?
- **Eye tracking:** What do players look at? Does it match what you're trying to communicate?

Theoretical Background

Christopher Alexander - A Pattern Language

Architecture as communication. Rooms shape behavior. Videogames inherited heavily from Alexander's work on how spatial patterns affect human experience.

Rudolf Arnheim - Art and Visual Perception

How humans parse visual space. The principles of visual weight, balance, and tension apply directly to level design.

Valve's Developer Commentaries

The *Half-Life* and *Portal* teams are masters of spatial communication. Their GDC talks and in-game commentary explain their techniques in detail.

What This Framework Takes

Level design is visual communication design. You're designing how eyes move, how attention is directed, how space is read.

Unresolved Questions

- **Camera perspective:** How do different camera perspectives change spatial communication? First-person, third-person, top-down each have different vocabularies.
- **Cultural variation:** Do players from different visual cultures read space differently? Does architectural tradition affect how players navigate?
- **Accessibility:** How do players with visual impairments navigate spatial communication? What alternatives exist to visual cues?

Pacing & Flow

Pacing is the rhythm of intensity over time - when the videogame pushes, when it relaxes, how those alternate.

Good pacing isn't constant intensity. It's contrast. The quiet moment makes the loud one louder. The rest makes the exertion meaningful.



Super Mario Bros. - *The most studied pacing in videogame history, each screen builds on the last.*
Courtesy of Nintendo

The Intensity Curve

Every videogame session has an **intensity curve** - how engaged/stressed/active the player is over time.

```

Intensity
|
|      ,-,      ,--,      ,----,
|      /  \    /      \  /      \
|      /      \-/      \ \      \--- <- Finale/Resolution
|      /
|/ <- Opening hook
+----- Time
  
```

The pattern: hook, build, release, build higher, release, climax, resolution.

Components of Pacing

Tension

What creates stakes, pressure, uncertainty.

- Combat
- Time pressure
- Resource scarcity
- Unknown threat
- Decision weight

Release

What relieves tension, allows recovery.

- Safe zones
- Story beats
- Rewards

- Humor
- Beauty

Rhythm

The alternation pattern. Fast-slow-fast. Push-rest-push.

Burnout Paradise understood this: the intensity of racing followed by free cruising creates satisfying oscillation. *Tetris* has natural rhythm in its escalation and momentary relief when clearing lines. Even a musical solo follows this pattern - tension builds, then resolves.

Macro vs. Micro Pacing

Scale	What it affects	Example
Macro	Hours, entire videogame	Act structure, chapter pacing
Meso	Minutes to hour	Level pacing, mission structure
Micro	Seconds to minutes	Combat encounters, puzzle flow

Good pacing exists at all three scales. A well-paced combat encounter in a poorly-paced level still feels wrong.

Pacing Tools

Gates

Points where the player can't proceed until something is done. Bosses, puzzles, story triggers. Gates punctuate.

Escalation

Gradually increasing intensity. More enemies, faster pace, harder challenges. Creates the build.

Respite

Deliberate low-intensity moments. Walking sections, safe rooms, campfire scenes. Creates the release.

Climax

Maximum intensity. Everything converges. The boss fight, the final chase, the revelation.

Denouement

Post-climax cool-down. The credits walk, the epilogue. Prevents emotional whiplash.

Player-Controlled vs. Designer-Controlled Pacing

Some videogames give pacing to the player:

- Open-world videogames (you decide when to do the main quest)
- Sandbox videogames (no imposed rhythm)
- Roguelikes (player sets their own pace through runs)

Some videogames control pacing tightly:

- Linear action videogames (this fight, then this cutscene, then this exploration)
- Horror videogames (tension carefully managed)
- Narrative videogames (story beats are sequenced)

Neither is superior. But know which you're doing.

Pacing Killers

Grind: Repetitive, low-meaning activity that pads time without building intensity.

Cutscene overload: Too much non-interactive time breaks the player's agency rhythm.

Difficulty spikes: Sudden, unexpected intensity without buildup.

Backtracking: Returning through low-content areas deflates momentum.

Information dumps: Pausing action for lore breaks the flow.

These aren't always bad - but they disrupt pacing. Use deliberately.

Flow State and Pacing

Flow (Csikszentmihalyi) is the state of complete absorption. Pacing supports or disrupts flow.

- **Too much tension without release:** Player can't reach flow (too anxious)
- **Too little tension:** Player can't reach flow (too bored)
- **Poor rhythm:** Flow keeps breaking (interrupted)

The goal isn't constant flow - it's appropriate oscillation between flow, tension, and rest.

See Also

- [Player Psychology](#) - flow state and engagement
- [The 4 A's](#) - Arc as the time component of pacing
- [Spatial Communication](#) - spatial pacing (corridor vs. arena)

Glossary Connections

- [Flow](#) - optimal engagement state
- [Intensity Curve](#) - pacing over time
- [Beat](#) - single pacing unit

Teaching Notes

Pacing is best understood through comparison. Have students map intensity curves for different videogames and see the different shapes.

Exercise: Intensity Mapping

Play a videogame for 30 minutes. Every 2 minutes, rate your intensity (1-10). Plot the graph. Where are peaks? Valleys? What caused them?

Compare graphs across different videogames and genres.

Exercise: Pacing Autopsy

Identify a moment where pacing felt *wrong* - dragging, rushed, jarring. Diagnose:

- What was the intensity just before?
- What was the intensity during?
- What transition was missing?

Key Insight

Pacing isn't about constant excitement. It's about purposeful contrast.

Common Misconceptions

- **"More action = better pacing."** Constant action becomes monotonous. The release makes the tension matter.
- **"Slow sections are boring."** Slow sections are necessary. Without valleys, peaks aren't peaks.
- **"Pacing is just for linear videogames."** Open-world videogames have pacing too - it's just emergent rather than authored.

Assessment Approaches

- **Intensity graph:** Have students create intensity graphs for their own levels or videogames.
- **Playtest observation:** Watch where players seem engaged vs. disengaged. Does it match intended pacing?
- **Comparative analysis:** Compare pacing across different videogames in the same genre.

Theoretical Background

Three-Act Structure

Screenwriting's tension model. Setup, confrontation, resolution. Videogames inherit and adapt this structure, though the interactive nature creates different possibilities.

Music Theory

Dynamics (loud/soft), tempo (fast/slow), phrasing. Videogames are temporal experiences; musical concepts apply directly to understanding pacing.

Csikszentmihalyi - Flow

Flow theory describes the optimal experience that pacing should support. The balance between challenge and skill creates engagement.

What This Framework Takes

The centrality of contrast. Intensity without relief is exhausting, not exciting. Rest without tension is boring, not peaceful.

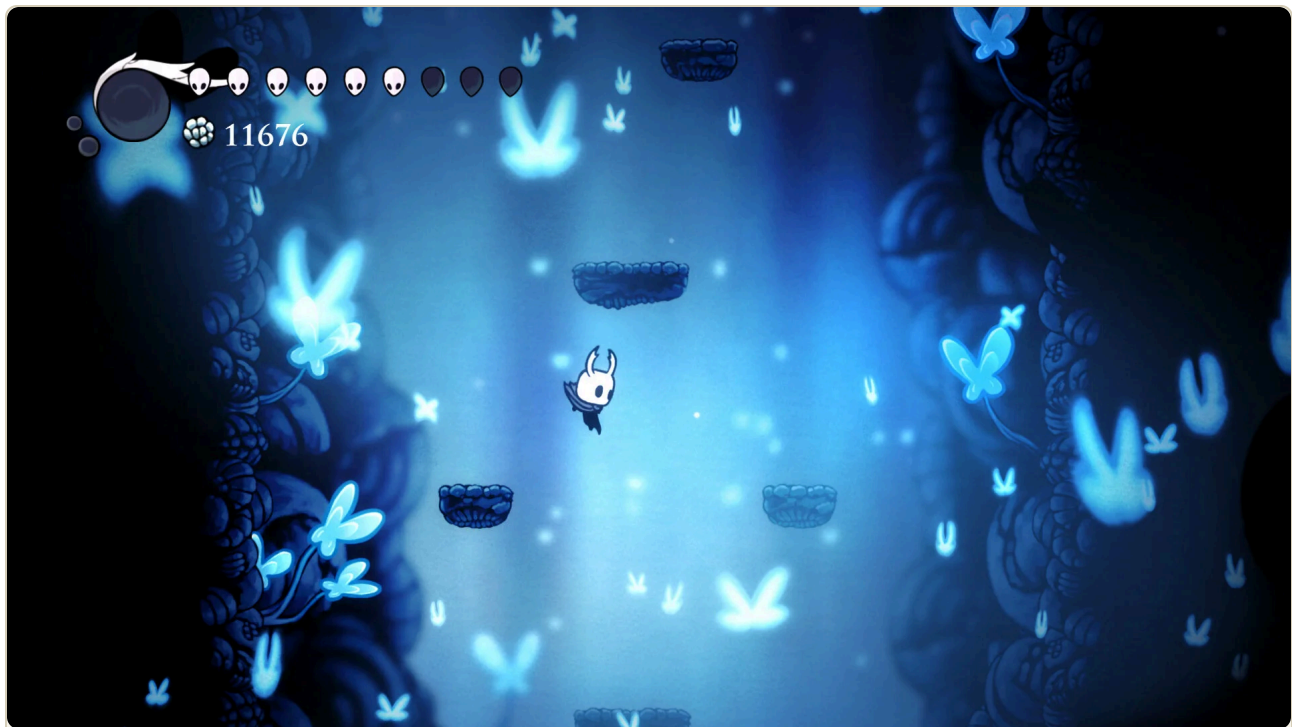
Unresolved Questions

- **Open-world pacing:** How do open-world videogames pace when players control sequence? Is emergent pacing as satisfying as authored pacing?
- **Natural rhythms:** Is there a "natural" human rhythm that videogames should match? Circadian? Breathing? Heartbeat?
- **Accessibility:** Different players have different fatigue thresholds. How do we pace inclusively? Should difficulty options affect pacing, not just challenge?

Guidance Without Hand-Holding

The best videogames teach you everything you need without making you feel taught. You discover; you're not told. You figure it out; you're not shown the answer.

This is **guidance without hand-holding**: creating conditions for learning without removing the satisfaction of learning.



Hollow Knight - *Guidance through atmosphere and contrast, no arrows, no waypoints.*
Courtesy of Team Cherry

The Hand-Holding Problem

Hand-holding damages the player experience:

- **Removes discovery:** If the videogame tells you where the secret is, it's not a secret
- **Undercuts mastery:** If the solution is given, solving it isn't an achievement
- **Breaks immersion:** Tutorial prompts remind you you're playing a videogame
- **Insults intelligence:** "Press A to jump" to a player who has jumped in 100 videogames

But insufficient guidance damages it too:

- **Frustration:** Players stuck without knowing why
- **Abandonment:** Players quit rather than guess
- **Missed content:** Players don't know features exist

The goal is guidance that doesn't *feel* like guidance.

Principles

Teach Through Play

The best tutorial is a level designed so the correct action is the natural action.

Portal: First room has one exit. The portal is already there. You walk through it. Tutorial complete.

Establish Then Test

Introduce a concept in a safe environment, then test it with stakes.

Mega Man: First encounter with a new enemy is usually in a safe spot. Later encounters are over pits.

Progressive Disclosure

Don't teach everything at once. Introduce complexity as the player demonstrates readiness.

Zelda: Breath of the Wild: Great Plateau teaches core systems before releasing you to the world.

Failure as Teaching

Let players fail in low-stakes ways that teach the lesson.

Dark Souls: You're supposed to die to the first boss. The death teaches that death is part of the videogame.

Consistent Rules

If the player understands the rules, they can reason from them. Consistent rules reduce hand-holding needs.

Baba Is You: Rules are explicitly stated on screen. Everything follows from them.

Techniques

Environmental Cues

Use space, light, color, and objects to suggest correct action. (See: [Spatial Communication](#))

Gated Learning

Put a skill-gate where the player can't proceed until they've demonstrated understanding.

Observational Learning

Show NPCs or enemies doing the thing before the player must do it.

Safe Experimentation Zones

Spaces where the player can try things without consequence.

Just-in-Time Information

Provide information at the moment of need, not before.

Optional Hints

Hints that the player can access if stuck, but aren't forced on them. Preserves player agency.

The Tooltip Problem

Tooltips, button prompts, and tutorial pop-ups work - but at a cost:

Pro	Con
Clear, unambiguous	Breaks immersion
Accessible	Can feel patronizing
Localization-friendly	Clutters screen
Doesn't require level design for teaching	Doesn't use level design for teaching

The best videogames minimize explicit instruction. But when necessary, make it:

- Skippable for experienced players
- Contextual (appears when relevant)
- Brief (don't overexplain)
- Diegetic if possible (character speech, in-world signs)

Onboarding vs. Ongoing Teaching

Onboarding: Teaching new players the basics. First hour. Can be more explicit.

Ongoing: Teaching new systems mid-game. Should feel like discovery.

The tolerance for hand-holding decreases as the videogame progresses. Early prompts are expected; mid-game prompts are intrusive.

Reading Player Skill

Videogames can adapt guidance to player skill:

- **Death count:** Many deaths = add hints
- **Time stuck:** Long time in one area = offer help
- **Explicit setting:** "Show hints" option in menu
- **Behavioral analysis:** Player isn't using a mechanic = prompt about it

Resident Evil 4 adjusts difficulty based on performance, invisibly. *Celeste* offers explicit assist mode. Both respect the player.

See Also

- [Spatial Communication](#) - using space to teach
- [Gesture](#) - how actions communicate meaning
- [Debugging as Literacy](#) - learning through failure

Glossary Connections

- [Gating](#) - blocking progress until skill is demonstrated
- [Diegetic UI](#) - information that exists within the videogame world
- [Progressive Disclosure](#) - revealing complexity gradually

Teaching Notes

This topic is deeply connected to **accessibility** - guidance is an accessibility issue. Players with different backgrounds, abilities, and available time need different guidance.

Exercise: Blind Playtest

Have someone unfamiliar with the videogame play your prototype with no instruction. Watch silently. Where do they struggle? What did they miss?

Then: redesign to teach those things without adding explicit instruction.

Exercise: De-Tutorializing

Take a videogame with heavy tutorials. Redesign the first 10 minutes to teach the same things through level design, not text. What has to change?

Key Insight

Every tutorial prompt is a design failure - something that could have been communicated through play but wasn't. (This is purist and sometimes impractical, but it's a useful frame.)

Common Misconceptions

- **"Players will figure it out."** Some will, many won't. The question is whether the failure is theirs or yours.
- **"Tutorials are always bad."** Explicit instruction has its place, especially for accessibility. The goal is minimizing unnecessary prompts, not eliminating all guidance.
- **"Hard videogames don't need tutorials."** *Dark Souls* teaches constantly - just through death rather than text. Teaching and difficulty are orthogonal.

Assessment Approaches

- **Blind playtest:** Watch new players without intervening. Document where they struggle.

- **Tutorial audit:** List every explicit instruction in a prototype. For each, ask: could this be taught through design?
- **Completion rates:** Track where players abandon the videogame. Correlate with tutorial density.

Theoretical Background

Shigeru Miyamoto

Nintendo's design philosophy: players should figure things out. *Mario* and *Zelda* are masterclasses in implicit teaching.

Fumito Ueda

Ico and *Shadow of the Colossus* famously stripped out tutorials. Trust the player.

Keita Takahashi

Katamari Damacy teaches through scale and tactile feedback. You understand the size you need by feeling it, not reading it.

What This Framework Resists

The "accessibility means more prompts" assumption. Accessibility can mean better level design, not more tooltips. The most accessible teaching is the teaching that happens through play.

The "hardcore = no guidance" assumption. *Dark Souls* is famously hard but it *does* teach. It just teaches through death rather than text.

Unresolved Questions

- **Accessibility tension:** How do we balance teaching through design with accessibility needs for players who genuinely need explicit instruction?
- **Cultural variation:** Are players from different gaming traditions more or less tolerant of explicit guidance?
- **Genre expectations:** Some genres (puzzle games, strategy) may require more explicit information than others. Where's the line?

Verticality & Sight Lines

Verticality is the use of height - how videogames exploit the up-down axis for gameplay, navigation, and meaning.

Sight lines are what you can see from where you are - how visibility is controlled to guide attention, create anticipation, and manage information.

Together, they're fundamental to 3D level design.



Dark Souls - *Verticality as orientation, seeing where you've been from where you are.*

Courtesy of FromSoftware / Bandai Namco

Why Verticality Matters

The vertical axis does things the horizontal can't:

Power Dynamics

High ground = advantage in most videogames. Looking down on enemies vs. looking up at threats creates different feelings.

Discovery

Climbing to see more. The vista from the peak. Dropping into the unknown.

Navigation

"I need to get up there" is clearer than "I need to get over there somewhere."

Variety

Horizontal-only spaces get monotonous. Verticality creates interest.

Types of Verticality

Type	Player experience	Example
Ascent	Climbing, progress, revelation	Hiking up in <i>Celeste</i> , tower climbs
Descent	Delving, danger, commitment	<i>Dark Souls</i> descent to Blighttown
Layered	Multiple levels to navigate	<i>Dishonored</i> 's multi-story buildings
Hub	Central vertical space connecting areas	<i>Metroid Prime</i> 's vertical shafts
Arena	Combat space with height variation	<i>Halo</i> 's multi-level arenas

Designing Verticality

Establish the Vertical Goal

Show players where they need to go vertically. A tower in the distance. A light above. A pit below.

Make Height Readable

Distinct visual treatment for different elevations. Players should know what level they're on.

Provide Landmarks

In vertical spaces, horizontal landmarks aren't enough. Distinctive elements at different heights.

Consider Camera

Some camera angles handle verticality poorly. Top-down struggles with layered spaces. First-person handles it naturally.

Design for Movement

How does the player traverse vertically? Jumping? Climbing? Elevators? Flight? The traversal mechanics shape what verticality means.

Sight Lines

Sight lines are what the player can see from any given position.

Control of sight lines = control of information = control of experience.

Long Sight Lines

You can see far. Creates:

- Anticipation (I can see what's coming)
- Goals (I can see where I'm going)
- Vulnerability (enemies can see me too)
- Scale (the world feels big)

Blocked Sight Lines

You can't see around the corner. Creates:

- Mystery (what's there?)
- Tension (threat could be anywhere)
- Surprise (reveal around the bend)
- Intimacy (the world feels close)

Controlled Reveals

The sight line opens at a specific moment. Creates:

- Vista moments (you emerge and see the landscape)
- Dramatic reveals (the boss appears)
- Teaching moments (you see the solution before you can reach it)

Sight Line Techniques

The Weenie

Disney Imagineering term: a tall, visible landmark that draws you forward. *Breath of the Wild's* towers. The Citadel in *Half-Life 2*.

The Denied View

You can almost see something, but not quite. A window that shows only a glimpse. Creates desire.

The Frame

Architecture that frames a view. A doorway, a window, an arch. Focuses attention on what's framed.

The Turn

Corners and bends that block sight. Every corner is a potential reveal.

The Overlook

A position where you can see an area you'll later traverse. Creates anticipation and aids navigation.

Verticality + Sight Lines

The most powerful level design combines them:

- **Climb to overlook:** Ascend to gain information about the space below
- **Descend into unknown:** Sight lines blocked as you go deeper
- **Spotted from above:** High enemies see you before you see them
- **The vista moment:** Emerging from a confined space to see the vertical world spread before you

Dark Souls' Firelink Shrine is a masterclass: you can see places you'll visit dozens of hours later. The sight lines create the world before you traverse it.

See Also

- [Spatial Communication](#) - how space communicates
- [Pacing & Flow](#) - verticality affects pacing
- [Environmental Storytelling](#) - sight lines as narrative control

Glossary Connections

- [Weenie](#) - visible landmark drawing player forward
- [Vista](#) - dramatic revealing view
- [Sight Line](#) - what's visible from a position

Teaching Notes

Verticality is often undertaught. Students default to flat level design because it's easier. Push them into 3D thinking.

Exercise: Vertical Diagram

Design a level on paper using ONLY a side-view cross-section. No top-down allowed. How does this change what you think about?

Exercise: Sight Line Control

In a 3D engine, build a simple space. Then modify it to:

1. Maximize sight lines (player can see everything)
2. Minimize sight lines (player can see almost nothing)
3. Control one specific reveal moment

What does each feel like to traverse?

Key Insight

Sight lines are information design. What the player knows and when they know it is determined by what they can see.

Common Misconceptions

- **"More vertical = more complex = better."** Some videogames benefit from flatness. 2D videogames have their own vocabulary. Verticality is a tool, not a goal.
- **"Players will always climb."** Without clear motivation, players may ignore vertical paths. The goal needs to be visible or implied.
- **"Long sight lines are always good."** Mystery requires blocked sight lines. Horror requires limited visibility. Match the sight line strategy to the experience.

Assessment Approaches

- **Cross-section analysis:** Have students draw side-view cross-sections of their levels. Does interesting vertical design emerge?
- **Sight line audit:** Screenshot every major viewpoint in a level. What can the player see? What is hidden? Is it intentional?
- **Vertical navigation test:** Time how long it takes players to find vertical paths. If they struggle, the visual communication has failed.

Theoretical Background

Disney Imagineering

The "weenie" concept. Theme park design as precedent for videogame level design. Visible landmarks that draw visitors forward through space.

Architecture Theory

Spatial sequence, procession, the journey through a building. Videogames inherit this vocabulary. How we move through space is designed.

Film

Cinematography controls sight lines; level design gives control to the player (or pretends to). The tension between authored sight lines and player-controlled cameras.

What This Framework Takes

The information-control lens. Sight lines aren't just aesthetics - they're how you manage what the player knows.

What This Framework Resists

The "more vertical = more complex = better" fallacy. Some videogames benefit from flatness. 2D videogames have their own vocabulary. Verticality is a tool, not a goal.

Unresolved Questions

- **Accessibility:** How do accessibility needs affect verticality? Traversal challenges can exclude players. Are there ways to make vertical spaces accessible without removing the verticality?
- **VR:** VR changes everything about sight lines (you can look anywhere). How does VR level design differ from traditional 3D? What new principles emerge?
- **2D vs. 3D:** How do these concepts translate to 2D spaces? What's the vocabulary of sight lines in side-scrollers or top-down videogames?

PART V

Systems Design

Economies, progression, balance, and the behaviors that emerge from rules.

Systems Design

Economies, progression, balance, and emergence - the interconnected rules that create depth and replayability.

Entries

Entry	What it covers
Economy & Resources	How value flows through videogames - sources, sinks, and flow
Progression Systems	Tracking and displaying player advancement
Balance Philosophy	What "balance" means and how to achieve it
Emergent Behavior	Complex patterns from simple rules

Core Insight

Systems design is about **relationships between parts**, not individual mechanics.

A resource matters because of what creates it, what consumes it, and what it enables. A progression system matters because of what it rewards and what it gates. Balance matters relative to the experience you're creating.

By Framework Concept

Concept	Systems Application
Gesture	Systems shape the feel of playing - scarcity vs abundance, power vs vulnerability
The 4 A's	Arc is central - systems unfold over time and create rhythms
Aesthetic Heritage	Where your systems vocabulary comes from (board games, economics, simulations, other videogames)
Permissions	Systems are Permissions made mathematical - rules about what's allowed, required, and forbidden

Teaching Sequence

Recommended order for introducing systems concepts:

1. **Economy & Resources** - Concrete and visual. Students can diagram it.
2. **Progression Systems** - Familiar from every RPG. Easy to analyze.
3. **Balance Philosophy** - Requires understanding of context and player experience.
4. **Emergent Behavior** - Most abstract. Requires understanding of systemic interaction.

Common Misconceptions

"More systems = more depth"

Students often add systems without purpose. Complexity isn't depth. Every system should serve the core experience.

"Balance = equality"

Balance doesn't mean everything is equal. It means the experience works as intended. Asymmetric balance is often more interesting than symmetric.

"Emergence just happens"

Emergence is designed, not accidental. You create the conditions for emergence through careful system interaction. Random systems produce noise, not emergence.

Analysis Exercise

Map a videogame's economy:

- Identify all resources (currency, items, time, attention)
- Draw sources (where resources come from)
- Draw sinks (where resources go)
- Identify feedback loops (positive and negative)
- Find the pressure points - where does the system create tension?

Creation Exercise

Design a two-resource economy:

- Resources must be interdependent (you need A to get B, B to get A)
- Include at least one meaningful choice
- Playtest: does it create interesting decisions or degenerate into one strategy?

THEORY

- deeper grounding

Systems Thinking Origins

Systems design draws from cybernetics, economics, and ecological modeling. Videogames are simulations, and simulation inherits these disciplines.

The key insight: behavior emerges from structure. Change the relationships, change the behavior. Individual components matter less than their connections.

Feedback Loops and Player Psychology

Positive feedback loops (success breeds success) create momentum and drama. Negative feedback loops (rubber banding) create tension and catch-up.

The balance between these loops shapes the emotional arc of play. See [Player Psychology](#) for more.

The Simulation Gap

All systems are simplifications. The gap between your simulation and reality (or player expectation) creates the design space.

This gap isn't a flaw - it's where videogames live. The question is whether your simplifications serve your design intent.

Related Sections

- [Player Psychology](#) - feedback loops are the engine of systems behavior
- [Permissions](#) - systems shape what's possible
- [Case Studies](#) - systems in action

Economy & Resources

*An **economy** is any system where resources flow - created, spent, exchanged, lost. Most videogames have economies, even if they don't look like money. Understanding economy design is understanding how **value** moves through your videogame.*



Diablo II - Economy as expression, every trade is a design decision.

Courtesy of Blizzard Entertainment

Resources

A **resource** is anything the player has, wants, or manages:

Type	Examples
Currencies	Gold, coins, credits, gems
Consumables	Health potions, ammo, food
Durables	Weapons, armor, tools
Stats	Health, mana, stamina, experience
Time	Real time, in-game time, cooldowns
Attention	What the player can track and manage
Space	Inventory slots, building area
Information	Map knowledge, enemy patterns

If the player cares about having more or less of it, it's a resource.

The Four Flows

Resources do four things:

Sources - Where resources come from. Faucets.

- Enemy drops
- Mission rewards
- Time-based generation
- Purchase/crafting

Sinks - Where resources go. Drains.

- Spending on items

- Consumable use
- Decay/maintenance costs
- Death penalties

Converters - One resource becomes another.

- Crafting (materials → items)
- Training (gold → stats)
- Trading (one currency → another)

Storage - Where resources wait.

- Inventory
- Banks
- Stockpiles

Healthy economies have **balanced flows** - sources and sinks in rough equilibrium over time.

Economic Health

Inflation: Too many sources, too few sinks. Resources pile up. Value collapses. Nothing feels precious.

Signs: Players have more gold than they can spend. Shops become irrelevant.

Deflation: Too many sinks, too few sources. Resources are scarce. Progress stalls. Players feel punished.

Signs: Players can never afford anything. Hoarding becomes mandatory.

Dead Currencies: A resource with no meaningful use. Takes up space, adds no decisions.

Signs: You collect something but never spend it. "What is this even for?"

Optimal Path: One strategy dominates. Choice becomes illusion.

Signs: Players always buy the same thing. Other options are "traps."

Designing Economies

Start with Verbs: What do you want players to *do*? The economy should reward those actions.

Create Decisions: Resources should force interesting choices. "Save or spend?" "This upgrade or that?"

Tie to Progression: Resource flow should track player progress. Early scarcity, late abundance (or vice versa, strategically).

Playtest Relentlessly: Economies are emergent. You won't know the real balance until players optimize.

Soft vs. Hard Currencies

Hard currency: Difficult to get, premium, often monetized. Real-money purchases, rare drops.

Soft currency: Easy to get, common, earned through play. Gold, experience.

In F2P design, the relationship between hard and soft currencies is the business model. Handle with ethical care.

Single-Resource vs. Multi-Resource

Single-resource economies are simpler:

- One thing to track
- Clear value
- Limited depth

Multi-resource economies add complexity:

- Trade-offs between resources
- Conversion decisions
- More possible strategies
- Harder to balance

More isn't always better. Some videogames thrive on elegant single-resource economies (*Into the Breach*: just managing unit positioning).

Closed vs. Open Economies

Closed: Fixed resource pool. What exists is what exists. Zero-sum.

Example: Chess - pieces don't regenerate.

Open: Resources generated over time. Not zero-sum.

Example: Most RPGs - gold keeps dropping.

Multiplayer consideration: Open economies in multiplayer create inflation over server lifetime. Closed economies create scarcity competition.

Videogames to Study

- *Universal Paperclips* - Economy as the entire game. Watch resources transform and compound.
- *Animal Crossing* - Soft economy with real-time gates. Turnip market as emergent trading system.
- *The Sims* - Multi-resource economy (money, needs, relationships, time). Competing sinks.
- *Eve Online* - Player-driven economy at massive scale. Real inflation, real market crashes.

Exercise: Economy Mapping

Map the economy of any videogame:

- What are the resources?
- What are the sources for each?
- What are the sinks?
- What are the converters?
- Are flows balanced?

Diagram it. Then play more and see if your map was accurate.

Why Teach This

Economy design is where videogame design meets systems thinking. Students often start by adding currencies without understanding flows. This leads to inflation, dead currencies, and economies that don't serve gameplay.

Teaching economy mapping before economy design prevents these mistakes. We often use board videogames to teach this before moving to digital - the systems are more visible.

Common Misconceptions

"More currencies = deeper economy"

Students add currencies because they seem "RPG-like." But currencies without meaningful sinks become clutter. Teach subtraction before addition.

"Balance means equal numbers"

Economy balance isn't about making all resources equivalent. It's about creating flows that support the desired experience.

Assessment

- Can students identify sources/sinks/converters in an existing videogame?
- Can students predict what happens when a flow is disrupted?
- Can students design an economy that creates a specific experience (scarcity, abundance, trade-offs)?

Teaching Exercise: Economy Breaking

Take a working economy. Introduce one change (double a source, halve a sink, add a converter). What happens? Play it forward.

This builds intuition for systemic effects.

Key insight: Economies are feedback systems. Change one part, the whole system responds.

THEORY

- deeper grounding

Key References

Joris Dormans - *Game Mechanics: Advanced Videogame Design*. The Machinations framework for diagramming economies. Essential reading.

Raph Koster - Writings on virtual economies. The insight that player behavior in economies is economically rational, even in videogames.

Real economics - Videogames are simulations of economic behavior. Supply and demand, marginal utility, inflation - these concepts apply directly.

The Systems Lens

Economies aren't designed by tuning individual numbers - they're designed by understanding flows. A single number change can cascade through the entire system.

Ethical Considerations

Exploitative economy design - Economies designed to frustrate players into spending money. This is skilled work, but ethically troubled. The same techniques that create satisfying economies can be weaponized.

Unresolved Questions

- How do we design economies that are satisfying but not exploitable?
- What's the relationship between videogame economies and real economic intuitions? Do videogames teach economic thinking?
- How do we balance player expression with economic coherence in multiplayer?

Related

- [Player Psychology](#) - economies are loop systems; why resource gathering feels good
- [Progression Systems](#) - economies drive progression
- [Balance Philosophy](#) - balancing economies

Glossary Terms

- [Source](#) - where resources come from
- [Sink](#) - where resources go
- [Inflation](#) - too much resource generation
- [Soft Currency](#) - easily obtained in-game currency

Progression Systems

*A **progression system** is any structure that tracks and displays player advancement. Levels, unlocks, skill trees, gear scores - all the ways videogames say "you're getting somewhere." Progression systems answer the question: **how** does the player know they're advancing?*



Final Fantasy X - Progression made spatial, every node a visible choice.

Courtesy of Square Enix

Why Progression Matters

Extrinsic Motivation: Visible progress toward goals. The XP bar filling. The new ability unlocking.

Pacing Control: Gates content behind progression. Ensures players don't access everything immediately.

Investment Creation: The more you've leveled up, the more you have to lose by quitting.

Skill Scaffolding: New abilities introduce new complexity. Progression can match mechanical depth to player readiness.

Power Fantasy: Numbers go up. Character gets stronger. Feels good.

Types of Progression

Character Progression: The avatar improves: stats, abilities, gear.

Example: RPG leveling, skill trees, equipment upgrades

Player Progression: The human improves: skill, knowledge, mastery.

Example: Getting better at Dark Souls. No character stats changed - you changed.

World Progression: The videogame world changes: areas unlock, story advances, state evolves.

Example: Metroidvania unlocks, NPC relationships, world state

Meta Progression: Unlocks that persist across runs/sessions.

Example: Roguelike permanent upgrades, account-wide unlocks

Most videogames combine multiple types.

Progression Structures

Linear: Everyone follows the same path. Level 1 → 2 → 3...

Pro: Easy to balance. Con: No player expression.

Branching: Choose your path through a tree.

Pro: Player agency, replayability. Con: Balance complexity.

Open: Any ability accessible (if you have currency/points).

Pro: Maximum freedom. Con: Trap options, analysis paralysis.

Gated: Progression unlocked by achievements, not just time/XP.

Pro: Ties advancement to accomplishment. Con: Can feel arbitrary.

The Leveling Curve

How fast does the player progress? The **leveling curve** determines feel:

Linear Leveling: Same XP for each level. Predictable, but can feel grindy late-game.

Exponential Leveling: Each level takes more XP. Early levels fly by; late levels slow down. Creates extended endgame.

Logarithmic Leveling: Early levels are slow; later levels come faster. Rare, but used to extend early game.

Soft Cap / Hard Cap: Progression slows dramatically (soft) or stops entirely (hard) at a certain point.

The curve should match your pacing goals. Fast early = hook players. Slow late = extend engagement.

Progression vs. Power

Progression = visible advancement

Power = actual capability increase

These aren't the same:

Scenario	Experience
High Progression, Low Power	Level numbers go up, but enemies scale. You're not stronger, just "higher level."
Low Progression, High Power	Few visible markers, but you feel much more capable. <i>Dark Souls</i> : same "level," but you learned the boss.
Matched	Progression markers correspond to actual power increase. Traditional RPGs.
Mismatched	Feels bad when you level up but don't feel stronger, or feel stronger but don't see it reflected.

Skill Trees & Build Diversity

Skill trees create **build diversity** - different players develop different characters.

Good Skill Trees:

- Meaningful choices at each node
- Multiple viable builds
- Clear identity for each path
- Synergies to discover

Bad Skill Trees:

- One obviously optimal path
- Nodes that are never worth taking ("trap" options)
- Too many small incremental bonuses ("+1% damage")
- Choices that can't be understood until late-game

The test: do different players actually choose differently? Do those choices feel meaningfully distinct?

Platform Differences

Progression philosophy varies significantly by platform and genre:

Mobile: Often engagement-focused. Session-length progression. Daily rewards. Real-money shortcuts. Designed around interruption.

Console/PC Single-player: Experience-focused. Progression tied to authored content. Respects player time (ideally). Designed around immersion.

Crafting/Survival: Economy compounding as progression. *Valheim*, *Satisfactory* - your infrastructure IS your progression. Building capability compounds: better tools enable better resources enable better tools.

These aren't moral judgments - they're different design goals. But know which paradigm you're working in.

Prestige Systems

Prestige = resetting progress to start again, usually with some persistent bonus.

Examples: Call of Duty prestige, roguelike meta-progression, New Game+

Why it works:

- Extends content without new content
- Provides long-term goals
- Lets players re-experience early videogame with mastery
- Creates visible status markers

Danger: Can feel like treadmill without meaning.

Exercise: Progression Audit

Play a videogame for 2 hours. Log every progression marker:

- What leveled up?
- What unlocked?
- How did you feel at each marker?
- Did progression match power?

Why Teach This

Progression systems are where videogame design meets psychology research. This is where the "manipulation" critique is strongest - and where ethical design matters most.

Students need to understand both the craft (how to create satisfying progression) and the ethics (when progression becomes exploitation).

Common Misconceptions

"More progression = better videogame"

Some students add leveling systems to everything. But progression without purpose is noise. A platformer doesn't need XP. A puzzle videogame doesn't need skill trees.

"Players want to feel powerful"

Some players do. Others want challenge. Progression that trivializes content can ruin experiences for mastery-oriented players.

Key Discussion

What's the difference between a satisfying progression system and an exploitative one? Both use the same techniques. The difference is *whose interest is served*.

Assessment

- Can students identify progression types in existing videogames?
- Can students design a skill tree with genuine build diversity?
- Can students articulate when progression serves the player vs. the business model?

Teaching Exercise: Design a Skill Tree

Design a skill tree with 3 branches and 12 total nodes. Ensure at least 4 viable builds with distinct playstyles. Then have classmates evaluate - is it actually balanced?

THEORY

- deeper grounding

Key References

Nick Yee - Early research on MMORPG motivation. Achievement, social, immersion motivations map to progression system design.

Sebastian Deterding - Gamification research. The critique of progression-as-manipulation.

Amy Jo Kim - Videogame thinking for product design. Progression as engagement loop.

Progression as Communication

Progression is a communication system. It tells players where they are and where they're going. Good progression is honest communication.

Ethical Considerations

Progression as addiction machine: Skinner box critiques apply. When progression systems are designed to maximize time-in-game rather than player satisfaction, something has gone wrong.

The "numbers go up" design default: Not every videogame needs visible progression. Some of the best videogames have none. Progression is a choice, not a requirement.

Unresolved Questions

- Where's the line between satisfying progression and exploitative engagement?
- How do we design progression that respects player time?
- Is character progression necessary if player progression is rich?

Related

- [Economy & Resources](#) - progression often tied to resource systems
- [Player Psychology](#) - why progression feels good; progression as positive feedback
- [Balance Philosophy](#) - balancing progression curves

Glossary Terms

- [Skill Tree](#) - branching ability unlocks
- [Meta Progression](#) - persistent cross-session advancement
- [Level Scaling](#) - world adjusting to player level

Balance Philosophy

Balance is one of the most misunderstood concepts in videogame design. It doesn't mean "everything is equal." It means "everything is right for the experience you're creating." Balance is a philosophy, not a formula.



StarCraft - Asymmetric balance, three factions each powerful in different ways.

Courtesy of Blizzard Entertainment

What Does "Balanced" Mean?

Balance has multiple, sometimes conflicting meanings:

Fairness: No player/option has unfair advantage. "Balanced" = "not broken."

Variety: Multiple strategies are viable. "Balanced" = "diverse meta."

Intentional Hierarchy: Some things are supposed to be better. "Balanced" = "power curve is correct."

Felt Rightness: It *feels* fair, even if numbers aren't equal. "Balanced" = "satisfying."

A videogame can be balanced in one sense and imbalanced in another. The first question: balanced *for what*?

Balance Is Context-Dependent

Game Type	What "balance" means
Competitive	No dominant strategy; skill decides
Cooperative	Each role contributes; no one carries alone
Single-player	Challenge matches player progression; fair difficulty
Asymmetric	Different sides have different strengths; overall match is even
Casual	Feels good; specifics don't matter much

Chess needs different balance than *Mario Kart*. *League of Legends* needs different balance than *Stardew Valley*.

Perfect Balance Is Impossible (And Maybe Undesirable)

Reasons perfect balance fails:

Complexity: Any system complex enough to be interesting is too complex to perfectly balance analytically.

Player Diversity: Different players have different skills. "Balanced for pros" does not equal "balanced for casuals."

Meta Evolution: Players discover new strategies. What's balanced today is broken tomorrow.

Discovery Value: Finding what's strong is part of the fun. A perfectly flat meta is boring.

The goal isn't perfect balance. It's *good enough* balance - where imbalances don't ruin the experience.

The Balance Process

1. Set Goals: What experience are you balancing for? "Competitive fairness" vs. "power fantasy" vs. "strategic diversity."

2. Identify Pain Points: What's actually breaking? Not everything needs tuning. Tune what hurts the experience.

3. Change One Thing: Single variable changes. If you change multiple things, you won't know which fixed (or broke) it.

4. Playtest: Theory doesn't catch everything. Watch real players. Especially skilled ones.

5. Iterate: Balance is never done. Especially in live games. Expect ongoing tuning.

Balance Tools

Numbers: Stats, damage, cooldowns, costs. The obvious levers.

Access: When you can get something. A powerful item is less dominant if it comes late.

Counters: Rock-paper-scissors. Strength A beats B beats C beats A. Nothing dominates.

Opportunity Cost: Choosing X means not choosing Y. If X is strong but costs more, that's balance.

Skill Requirement: Powerful things that are hard to use are self-balancing. Skill floors and ceilings.

Information: Knowledge as balance. A strategy that works only if opponents don't know about it.

Imbalance as Design

Some imbalance is intentional:

Power Curve: Early videogame is weak; late videogame is strong. That's the point.

Asymmetric Design: Different factions/classes *should* feel different. Equal is boring.

Discovery: If everything is equally viable, there's nothing to discover. Some imbalance creates exploration.

Narrative: Story might want you to feel weak, then strong. "Balance" undermines the arc.

The question: is this imbalance *serving the design* or *breaking it*?

The Metagame

Metagame = the strategies players converge on given current balance.

A "healthy meta" has:

- Multiple viable strategies
- Counters to dominant strategies
- Room for innovation
- No single dominant "solved" approach

An "unhealthy meta" has:

- One or few dominant strategies
- High barrier to competing
- Stagnation
- Player frustration/exodus

Balance patches reshape the meta. But heavy-handed patching alienates players who invested in now-nerfed strategies.

Balance vs. Feel

Numbers can be balanced but feel wrong.

A 50% win rate option that feels terrible to play is "balanced" but bad.

Feel matters more than math. Players experience feel, not numbers. If it feels broken, it's broken - even if the spreadsheet says otherwise.

Exercise: Balance Autopsy

Find a videogame with a notorious balance problem (any competitive videogame will do).

Research:

- What was broken?
- How did players discover it?
- How did designers respond?
- Was the fix successful?

Why Teach This

Balance is often taught as math. We teach it as philosophy first, math second. Students who understand why balance matters make better decisions than students who know formulas.

The Provocation

"Perfect balance is not the goal. Good enough balance that serves the experience is."

This challenges students who think balance means spreadsheet perfection.

Common Misconceptions

"Balance = equality"

Students often think balanced means everything is the same power level. But asymmetric balance, power curves, and intentional hierarchy are all valid.

"Math will solve it"

Spreadsheets can't capture feel, strategy, or player psychology. Playtesting is irreplaceable.

Assessment

- Can students articulate what "balance" means for different game types?
- Can students identify whether an imbalance serves or breaks a design?
- Can students propose balance changes and predict their systemic effects?

Teaching Exercise: Asymmetric Balance

Design a 2-player asymmetric videogame where players have completely different abilities. Playtest for fairness. What makes it *feel* fair even when capabilities differ?

Key insight: Balance is about player experience, not spreadsheets.

Key References

David Sirlin - *Playing to Win*. The competitive player's perspective on balance. What it means for a videogame to be "solvable."

Extra Credits - Their series on balance is accessible and solid.

Frank Lantz - Writings on elegance and depth. Sometimes imbalance creates depth.

The "Balance for What" Question

Balance isn't an objective property - it's relative to goals. The same videogame might be balanced for casual play and unbalanced for tournament play.

Resisting Common Approaches

The nerf-hammer approach: Constantly adjusting based on complaints. Sometimes players are wrong. Sometimes the meta needs time to develop.

The "math will save us" fantasy: Spreadsheets can't capture feel, strategy, or player psychology. Playtesting is irreplaceable.

Unresolved Questions

- How do you balance for different skill levels simultaneously?
- When is imbalance acceptable? When does it become unacceptable?
- How do you know when to trust player complaints vs. when to wait for the meta to evolve?

Related

- Player Psychology - feedback loops often cause balance problems; why balance *feels* the way it does
- Economy & Resources - economic balance
- Emergent Behavior - emergence complicates balance

Glossary Terms

- Metagame - dominant strategies in competitive play
- Power Creep - gradual strengthening over updates
- Rock Paper Scissors - balance through counters
- Skill Floor - minimum skill to use something effectively

Emergent Behavior

Emergence is when simple rules combine to produce complex, unexpected behaviors. It's when the videogame surprises its own designer. Emergent behavior is why Dwarf Fortress creates tragedies no author wrote, why Breath of the Wild enables solutions no designer anticipated, and why multiplayer videogames generate stories that single-player videogames can't.



Dwarf Fortress - Emergence from complexity, no single rule creates the story.

Courtesy of Bay 12 Games

What Is Emergence?

Emergence = macro-level patterns arising from micro-level rules.

The rules of chess are simple: how pieces move, capture, check. The strategies that emerge - openings, defenses, endgames - are not written in the rules. They *emerge* from players interacting with rules over time.

Emergence means the videogame has more possibility than you explicitly designed.

Why Designers Love Emergence

Content Multiplication: Designers create rules. Players create content by combining those rules.

Breath of the Wild: Physics + fire + wind + enemies = infinite videos of creative kills.

Replayability: Emergent systems aren't solved. Each playthrough produces different patterns.

Player Authorship: Players feel like co-creators. The story that emerges is *their* story.

Surprise: Even the designer discovers new behaviors. The videogame is bigger than its creator.

Why Emergence Is Hard

Unpredictability: You can't fully predict what will emerge. Playtesting reveals, but can't cover everything.

Exploits: Players find emergent behaviors that break the videogame. "Emergent" can mean "broken."

Balance Difficulty: Emergence + balance = nightmare. Systems interact in untestable combinations.

Narrative Dilution: Authored meaning requires control. Emergence cedes control. Hard to have both.

Designing for Emergence

You can't design emergent behaviors directly - you design the **conditions** for emergence.

Combinable Elements: Systems that interact with each other. Fire spreads. Water conducts electricity. Enemies have physics.

Consistent Rules: Rules that always apply. If fire burns wood, it burns ALL wood - barrels, bridges, weapons.

Player-Readable Interactions: Players can predict (roughly) how combinations will work. Transparency enables experimentation.

Forgiving Failure: If experimenting is punished, players don't experiment. Emergence needs safety to explore.

Sandbox Spaces: Areas where players can test combinations without stakes.

Emergence vs. Randomness

Emergence isn't randomness.

Emergence	Randomness
Deterministic rules → unpredictable outcomes	Non-deterministic events
Player understands why it happened	Player doesn't control
Replayable (same inputs → same outputs)	Variable (same inputs → different outputs)
Surprising but explicable	Surprising and arbitrary

Emergence feels like *discovery*. Randomness feels like *luck* (or bad luck).

Emergent Narrative

Emergent narrative = stories that arise from systems rather than scripts.

Dwarf Fortress generates stories: a dwarf loses their spouse, falls into despair, drinks too much, stumbles into the forge, starts a fire, destroys the fort.

No author wrote that. The systems produced it.

What Enables Emergent Narrative:

- Agents with goals and states (characters who want things)
- Systems that create consequences (actions affect world state)
- Player interpretation (humans find stories in patterns)

Limits of Emergent Narrative:

- Rarely has dramatic structure (no rising action, climax)
- Can feel arbitrary or meaningless
- Lacks authored emotional arc
- Requires player investment to "see" the story

Emergent narrative supplements, but rarely replaces, authored narrative.

Case Studies

Dwarf Fortress: Maximum emergence. Simple AI rules for dwarves, complex interactions with environment, no authored story. Players project narrative onto simulated events.

Noita: Physics-based emergence. Every pixel is simulated. Fire spreads, liquids flow, explosions chain. The videogame is constantly surprising itself. Death comes from interactions you couldn't predict but understand in retrospect.

Breath of the Wild: Constrained emergence. Physics systems interact predictably. Players discover combinations. Authored story exists alongside emergent play.

Hitman: Curated emergence. Systems enable many solutions, but levels are designed to suggest specific emergent moments.

Chess: Pure emergence. No narrative, just strategy arising from rules.

Exercise: Emergence Observation

Play a systemic videogame (*Breath of the Wild*, *Noita*, *Divinity: Original Sin*). Document three emergent moments:

- What happened?
- What rules combined to produce it?
- Could the designer have predicted this?

Why Teach This

Emergence is where systems design becomes exciting - and where it gets hard to predict. Students need to understand both the promise (content multiplication, player authorship) and the challenge (exploits, balance nightmares).

Common Misconceptions

"Emergence = good design"

Students sometimes treat emergence as inherently positive. But emergence can produce broken games, meaningless noise, or exploitative behaviors. Emergence is a tool, not a virtue.

"Add more systems = more emergence"

Emergence requires systems that *interact*. Parallel systems that don't touch each other don't create emergence - they create complexity without depth.

Assessment

- Can students distinguish emergence from randomness?
- Can students identify the conditions that enable emergence in a videogame?
- Can students predict potential emergent behaviors (and potential exploits) from a rule set?

Teaching Exercise: Emergent System Design

Design a simple system (5-7 rules) that you believe will produce emergent behavior. Then test with classmates. Did expected emergence appear? What surprised you?

Key insight: You design rules, not emergence. Emergence is a gift from the rules to the players.

Key References

Steven Johnson - *Emergence*. Accessible introduction to emergence as a phenomenon.

Jesper Juul - *Half-Real*. The relationship between rules and fiction; emergence as a game-specific concept.

Warren Spector / Deus Ex - Immersive sim design philosophy: authored spaces, emergent solutions.

The Humility of Emergence

Emergence requires letting go. You can't control everything. Good design creates conditions and then steps back. This is philosophically challenging for designers who want to author specific experiences.

The "Emergence Is Always Good" Myth

Emergence can produce broken videogames, meaningless noise, or exploitative behaviors. Emergence is a tool, not a virtue. The question isn't "does this videogame have emergence?" but "does this emergence serve the experience?"

Unresolved Questions

- How much emergence is enough? Too little = linear; too much = chaos.
- Can emergence and authored meaning coexist? Or are they in tension?
- Whose story is an emergent narrative? Designer's? Player's? System's?

Related

- [Player Psychology](#) - feedback loops drive emergence

- [Balance Philosophy](#) - emergence complicates balance
- [Permissions](#) - emergence emerges from what's allowed

Glossary Terms

- [Emergence](#) - complex from simple
- [Immersive Sim](#) - genre built around emergence
- [Systemic Design](#) - designing for interaction

PART VI

Pedagogy

How VG101 teaches: play, name, make, reflect.
Accessibility, debugging, and code as material.

Pedagogy

Meta-content about teaching and learning videogame design.

Pedagogy Entries

Entry	What it covers
Play → Name → Make → Reflect	The weekly rhythm of VG101
Why Unity	The engine as workshop for constraints
The Final Project	Make one action feel undeniable
Debugging as Literacy	Reading and responding to systems
Accessibility as Craft	Inclusive design as design quality
Code as Material	Design through implementation

Core Insight

Videogame design is learned by **making**, not by absorbing theory. The pedagogical approach here is: experience first, vocabulary second, making third, understanding fourth. The Play → Name → Make → Reflect cycle.

Code isn't a prerequisite; it's a material. Debugging isn't remediation; it's literacy. Accessibility isn't compliance; it's craft.

VG101 Structure

The full VG101 course structure lives in the course materials, but the rough shape:

Weeks	Focus	Core Concept
1-4	Core Framework	Gesture , Aesthetic Heritage , Permissions
5-8	Feel & Communication	Feedback, Readability, The 30-Minute Action
9-12	Meaning & Synthesis	Ludonarrative, Procedural Rhetoric, Final Project

Each week follows Play → Name → Make → Reflect.

CRAFT

- for educators

For Educators

These entries explain *how* we teach the Framework concepts, not just *what* they are. If you're adapting this material for your own teaching, the Pedagogy section tells you the reasoning behind the sequence.

Why This Section Exists

Most videogame design resources present content as if teaching it is straightforward: here's the knowledge, now students know it. But teaching videogame design is its own craft.

The Pedagogy section makes explicit what's often implicit: why concepts are sequenced the way they are, what common misconceptions arise, how to assess design thinking rather than rote recall.

Using These Pages

Each Pedagogy page has:

- **Practice:** The approach itself - what students do
- **Craft:** Teaching notes - how to implement it
- **Theory:** Theoretical grounding - why it works

The Craft register on Pedagogy pages is meta-pedagogy: teaching teachers how to teach the approach.

Assessment Philosophy

VG101 assesses reflection and making, not vocabulary recall. If the rhythm is play → name → make → reflect, assessment should match. Students demonstrate understanding by making things and articulating what they learned, not by defining terms on a test.

Theoretical Foundations

The VG101 pedagogical approach draws from several traditions:

Experiential Learning (John Dewey, David Kolb): Learning by doing. Experience precedes abstraction. The philosophical backbone of the Play → Name → Make → Reflect cycle.

Studio Pedagogy: Art, architecture, and design schools use variations of make-critique-reflect-iterate. VG101 adapts this for videogame design.

Constructionism (Seymour Papert): Learning happens through making things. The Code Bank and making exercises reflect this.

What We're Resisting

The lecture-exam model: Videogame design isn't learned by absorbing information. It's learned by making things and understanding what you made.

Theory-first approaches: Students learning words without referents. "Ludonarrative dissonance" means nothing until you've felt the dissonance.

The naturalistic fallacy: Just playing doesn't teach design. Unstructured play is valuable, but it's not education. Structure (naming, focused making, reflection) is essential.

Unresolved Questions

- How does this approach scale? Studio pedagogy is labor-intensive. How do you run it with 100 students?
- Does every concept benefit from the full cycle, or are some better taught directly?
- How do we assess process-oriented learning without distorting it?

Related

- [Play → Name → Make → Reflect](#) - the core weekly rhythm
- [Why Unity](#) - the engine as design workshop
- [The Final Project](#) - make one action feel undeniable
- [Debugging as Literacy](#) - failure as learning
- [Accessibility as Craft](#) - inclusive design
- [Code as Material](#) - design through implementation

Play → Name → Make → Reflect

*The weekly rhythm of VG101 - and our core pedagogical method. Learning videogame design isn't about reading theory first. It's about **feeling, then naming, then making, then understanding.***

VG101 Classroom

Students at computers playing videogames, notebooks open beside them

Play with intention, then name what you felt.

By Marty Scott

The Sequence

1. Play

Experience videogames firsthand. Not analyzing yet - just playing. Letting the videogame affect you.

What students do: Play assigned videogames with journaling prompts. Notice what they feel. Don't explain yet.

2. Name

Give vocabulary to the experience. "That feeling when you dash" becomes "the dash gesture." "Where did that mechanic come from?" becomes "aesthetic heritage."

What students do: Learn concepts that name what they've already felt. Discuss. Make connections.

3. Make

Create something that embodies the concept. Not just talk about gestures - build one. Use the Code Bank scaffolds.

What students do: Small, focused making exercises. Recreation, modification, original work.

4. Reflect

Step back and understand. What did you make? Why did it work or not? How does this change how you see videogames?

What students do: Written reflection, critique sessions, connecting to larger ideas.

Then repeat. Each week, a new cycle.

Why This Order?

The conventional order - theory first, application second - doesn't work for videogame design.

Conventional	Play → Name → Make → Reflect
Read about jumping	Jump in a videogame, feel it
Learn "game feel" vocabulary	Give name to what you felt
Try to implement jump	Build a jump that matches the feeling
Grade the implementation	Reflect on what you learned

The problem with theory-first: students learn words without referents. "Ludonarrative dissonance" means nothing until you've *felt* the dissonance.

The Play → Name → Make → Reflect sequence ensures that **vocabulary attaches to felt experience**, and that **understanding emerges from making**.

Play: What Counts?

"Play" isn't just passing time with videogames. It's **active, attentive engagement**.

Good playing:

- Playing with a question in mind ("What makes this jump feel good?")
- Journaling while or after playing
- Playing something unfamiliar (not just comfort videogames)
- Playing critically (noticing choices, not just reacting)

Less useful playing:

- Mindless grinding
- Checking phone while playing
- Only playing what you already know

The assignment matters: what question are students taking into play?

Name: Vocabulary That Fits

Naming should feel like relief, not burden.

If we've shown students *Celeste* and had them write about the dash, then we introduce "gesture" - they should think "oh, that's what that was!"

If they're confused by "gesture," we introduced it too early. They needed more play.

Naming principles:

- Name what they've experienced
- Use terms that help, not impress
- Let naming emerge from discussion, not just lecture
- Connect new names to old names

Make: Constrained Creating

Making should be **scoped and focused**, not open-ended.

"Make a videogame" is paralyzing.

"Recreate the *Celeste* dash using the dash scaffold in 30 minutes" is actionable.

Good making prompts:

- Clear scope (one gesture, one system, one level)
- Available tools (Code Bank scaffolds, not raw engine)
- Time limit (forces prioritization)
- Evaluable criteria (what does "done" mean?)

The constraint is the gift. Freedom overwhelms; constraint enables.

Reflect: Closing the Loop

Reflection is where learning consolidates. Without it, students do but don't understand.

Reflection prompts:

- "What surprised you in the making process?"
- "How is what you made different from what you planned?"
- "What would you do differently?"

- "How does this change how you see the original videogame?"

Written reflection is valuable, but so is critique: presenting work and receiving feedback.

The Rhythm Across VG101

Week	Play	Name	Make	Reflect
2	<i>Celeste, Katamari</i>	Gesture	Recreate a gesture	What is a gesture?
3	Trace gestures backward	Aesthetic Heritage	Heritage Translation	What's inherited?
4	Permission-heavy videogames	Permissions	Permission Audit	What does this allow?
5	Feel-focused videogames	Feedback & Readability	30-Minute Action	What makes feel?
...	...	...	...	...

Each week cycles through the sequence with different content.

Meta-Pedagogy

This entry is *about* pedagogy, so the "craft" section is meta.

Why We Designed This Rhythm

We came from music education, where the sequence is obvious: listen, name (theory), practice, reflect. You don't teach music theory before students have heard music. Videogames should work the same.

Challenges

Students trained in lecture-first education sometimes resist playing before reading. They want to "know the answer" before engaging. The rhythm requires unlearning that expectation.

Assessment Alignment

If the rhythm is play → name → make → reflect, assessment should match. We assess reflection and making, not vocabulary recall.

The Role of Failure

"Make" often involves failure. That's intentional. Reflection on failure is some of the best learning. (See: [Debugging as Literacy](#))

Common Pitfalls

"Play" becomes passive consumption:

Without journaling prompts or questions, "play" can become just playing. The assignment structure matters. Give students something to attend to.

"Name" becomes lecture:

Naming works best as dialogue, not delivery. Ask students what they noticed, then provide vocabulary for what they're describing.

"Make" becomes overwhelming:

Scope aggressively. A 30-minute constraint is better than a week-long project. Small wins build confidence.

"Reflect" gets skipped:

When time is short, reflection goes first. Resist this. Reflection is where learning consolidates. Build it into class time, not just homework.

Theoretical Background

Key Influences

John Dewey: Learning by doing. Experience precedes abstraction. The philosophical backbone of this approach.

Donald Schön: *The Reflective Practitioner*. Reflection-in-action and reflection-on-action. The idea that professionals learn by reflecting on what they do.

Kolb's Experiential Learning Cycle: Concrete experience → reflective observation → abstract conceptualization → active experimentation. Our rhythm maps loosely to Kolb.

Studio pedagogy: Art, architecture, design schools all use variations of this rhythm. Make, critique, reflect, iterate.

What We're Resisting

The lecture-exam model. Videogame design isn't learned by absorbing information. It's learned by making things and understanding what you made.

The "naturalistic" fallacy. Just playing doesn't teach design. Unstructured play is valuable, but it's not education. The structure (naming, focused making, reflection) is essential.

Unresolved Questions

- How do you assess the "play" and "name" phases? Can they be graded without distorting them?
- Does this rhythm work for all students? Some may need more structure, more theory, more time.
- How does this scale? Studio pedagogy is labor-intensive. How do you run this with 100 students?

Related

- [Gesture](#) - Week 2 concept, first "name" in the sequence
- [Aesthetic Heritage](#) - Week 3 concept
- [Debugging as Literacy](#) - failure as learning
- [Code Bank](#) - tools for the "make" phase

Accessibility as Craft

*Accessibility isn't compliance. It isn't a checklist at the end of development. It's **craft** - a design skill that makes videogames playable by more people without compromising vision. Treating accessibility as craft changes everything: when you consider it, how you implement it, and what you think it means.*



The Last of Us Part II - Over 60 accessibility options, the industry benchmark for inclusive design.

Courtesy of Naughty Dog / Sony Interactive Entertainment

The Compliance View

Conventional accessibility work looks like:

- Subtitles
- Colorblind modes
- Remappable controls
- Done at the end

This is necessary. It's not sufficient.

The compliance view treats accessibility as **adding features for disabled players**. It's additive, late, often underfunded.

The Craft View

The craft view sees accessibility as **design quality**. Accessible videogames are well-designed videogames.

- Clear visual hierarchy → good design AND readable for low vision
- Responsive controls → good feel AND usable for motor impairments
- Multiple difficulty options → respects player time AND enables more players
- Clear audio cues → good feedback AND playable without sight

Good design *is* accessible design. They're not separate goals.

Why "Craft" Not "Feature"?

A **feature** is bolted on. It has a scope, a budget, a checkbox.

A **craft** is woven through. It's a way of thinking, not a deliverable.

Calling accessibility a craft means:

- It's considered from the start
- It's everyone's responsibility (not one specialist's)

- It's ongoing (not one release milestone)
- It's skilled work (not just implementation)

Core Accessibility Domains

Domain	What it addresses	Craft questions
Visual	Low vision, blindness, colorblindness	Is the screen readable at a glance? Does color alone convey information?
Auditory	Deafness, hard of hearing	Is audio information redundantly presented? Are subtitles good?
Motor	Limited mobility, precision, fatigue	Are controls flexible? Are time-sensitive moments adjustable?
Cognitive	Learning differences, attention, memory	Is information clearly presented? Can pace be controlled?

Every player exists on spectrums in all four domains. "Disabled" isn't a binary.

Accessibility as Design Constraint

Constraints drive creativity. Accessibility constraints are design constraints.

"This videogame must be playable with one hand" isn't a limitation - it's a prompt. What designs emerge from that constraint?

- *Celeste's* assist mode emerged from thinking about who gets to finish videogames
- *The Last of Us Part II's* accessibility options were designed alongside core mechanics
- *Hades's* "God Mode" respects player time without removing challenge

These aren't compromises. They're design innovations.

Common Objections (And Responses)

"Accessibility undermines artistic vision."

Does difficulty selection undermine vision? Does subtitles undermine vision? Accessibility is player options, not mandate. The core experience is still there.

"Our audience doesn't need it."

Your audience includes disabled players. 20-25% of people have some form of disability. You just don't see them because they can't play your videogame.

"We don't have the budget."

Accessibility built from the start costs less than accessibility retrofitted. The cheapest accessibility is considered design.

"Some videogames are meant to be hard."

Hard *for whom*? A timing-based videogame is harder for someone with motor impairments. Is that the challenge you intended? *Celeste* is hard AND accessible.

Practical Accessibility Craft

Visual

- Don't rely on color alone (shape, pattern, position as redundant cues)
- Scalable text
- High contrast option
- Screen reader support for menus

Auditory

- Subtitles (good ones: speaker identification, sound effects)
- Visual cues for important sounds
- Adjustable audio channels

Motor

- Remappable controls
- Toggle vs. hold options
- Adjustable timing windows

- One-handed control schemes

Cognitive

- Clear objectives
- Adjustable speed/pace
- Save anywhere
- Minimize memory load

Accessibility in VG101

We integrate accessibility throughout the course, not as a single topic:

- **Week 5 (Feedback & Readability):** Feedback channels as accessibility - if all feedback is audio, deaf players are excluded
- **Week 7 (Feel Experiment):** Adjust timing variables. How does changing them change who can play?
- **Case studies:** Analyze how videogames handle accessibility

Key message: Accessibility isn't extra. It's part of craft.

Teaching Notes

Accessibility can feel like a checklist or like a moral imperative. We try to frame it as *interesting design work*.

Exercise: Accessibility Audit

Audit a videogame's accessibility features. What's there? What's missing? What would it take to improve? How would those improvements affect the core design?

Exercise: Constraint Design

Design a videogame concept around an accessibility constraint: playable with no audio, playable with one button, playable with no text. What emerges?

Guest Speakers

We bring in disabled players and accessibility consultants. Students hear directly from people affected.

Danger: Performative Accessibility

Accessibility can become performative ("we care about disabled people") without substance. Push for *specific* craft work. What exactly are you doing? How does it help? Who did you talk to?

Connection to [Permissions](#)

Accessibility is deeply connected to the [Permissions](#) framework concept. What a videogame allows, requires, and forbids determines who can play it. Accessibility expands Permissions to include more players.

Theoretical Background

Key Influences

Ian Hamilton: Accessibility specialist. His work on *The Last of Us Part II* is a reference point.

Cherry Thompson / AbleGamers: Advocacy and practical guidance.

Universal Design: Architecture's "design for all" principle. Videogames inherit this.

The Curb Cut Effect

Accessibility features benefit everyone. Curb cuts help wheelchairs, but also strollers, luggage, bikes. Subtitles help deaf players, but also players in noisy environments or learning a language.

Accessibility is good design.

What We're Resisting

The charity model: "Let's help the disabled." Accessibility isn't charity. It's design quality.

The minimum viable compliance: Checking boxes isn't craft.

Unresolved Questions

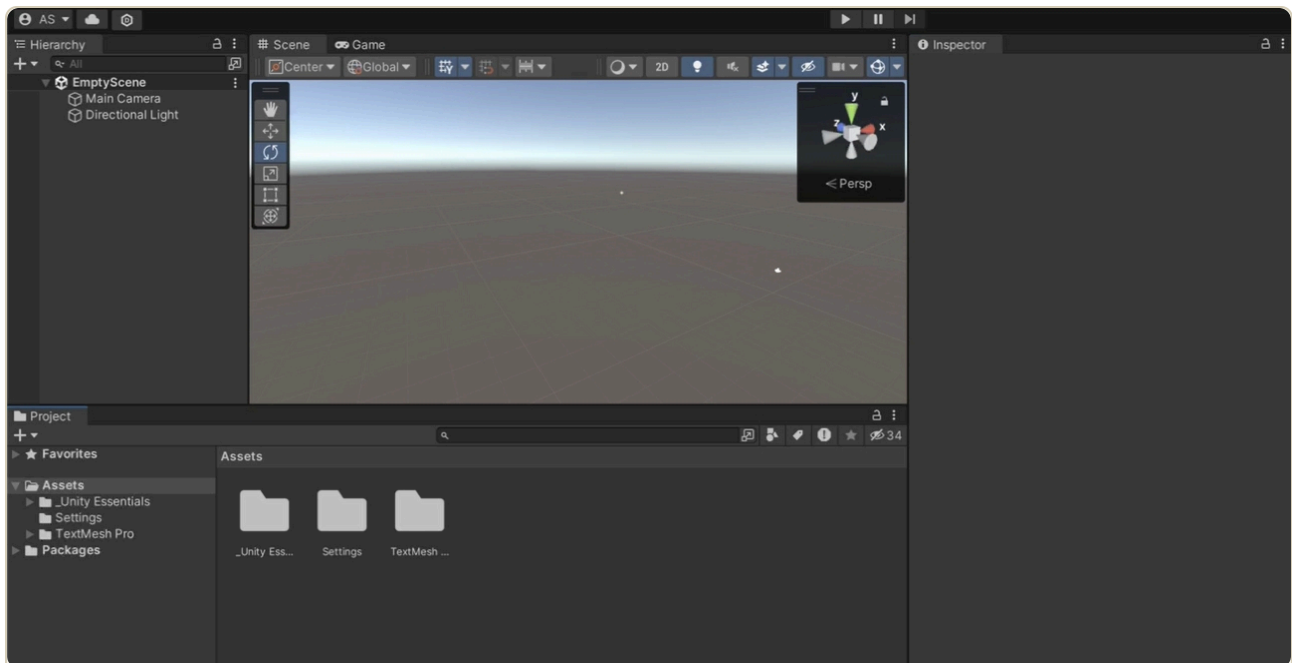
- Where's the line between accessibility and core design? Some argue certain experiences require certain abilities. We're skeptical, but the debate exists.
- How do we handle accessibility in competitive videogames? If accessibility options change difficulty, what's fair in competitive contexts?
- Resource reality: accessibility is skilled work. How do small teams do it well?

Related

- [Permissions](#) - who gets to play
- [Play → Name → Make → Reflect](#) - where accessibility fits in the cycle
- [Case Studies](#) - accessibility analysis

Debugging as Literacy

Debugging isn't just fixing code. It's a mode of thinking - a literacy in the root sense: the ability to read, understand, and respond to systems. Treating debugging as literacy changes how we teach code in videogame design.



Unity - Reading errors is reading design, debugging builds understanding.

By Unity Technologies

The Conventional View

In CS education, debugging is remediation. You wrote a bug; now fix it. Debugging is failure recovery.

This framing makes debugging shameful. Students hide their bugs. They feel dumb when code doesn't work.

The Literacy View

Debugging is **reading**. When code breaks, you read what it's actually doing (vs. what you intended). That reading skill transfers: you can read code you didn't write, read systems you didn't design, read behavior you didn't expect.

Debugging is **inquiry**. "Why doesn't this work?" is a research question. You form hypotheses, test them, revise. This is scientific thinking applied to systems.

Debugging is **craft**. Every craft has failure. Potters have pots. Writers have bad drafts. Debugging is normal, not shameful.

Why This Matters for Videogame Design

Videogame designers who can't debug are dependent on engineers. They can't prototype. They can't tweak feel. They can't understand why their design isn't working.

Videogame designers who *can* debug have:

- **Prototyping independence:** Build rough versions without waiting for engineers
- **Feel intuition:** Tweak values, see results, iterate fast
- **System literacy:** Understand how videogame systems actually work
- **Communication ability:** Talk to engineers in their language

You don't need to be a great programmer. You need to be a competent debugger.

Teaching Debugging

Normalize Failure

Code not working isn't failure - it's the normal state. Nobody writes perfect code first try. Professional developers spend most of their time debugging.

Read Before Fix

Before changing anything, understand what's happening. Read the error message. Read the code. Articulate (in words) what you think is wrong.

One Change at a Time

Change one thing, test, repeat. If you change five things and it works, you don't know which change fixed it.

Print Statements Are Your Friend

The simplest debugging tool: print the value. See what's actually happening, not what you think is happening.

Rubber Duck Debugging

Explain the code to someone (or something - a rubber duck). The act of articulating often reveals the bug.

Use the Error Message

Error messages tell you something. Read them. They're not random noise.

The Code Bank and Debugging

The Code Bank scaffolds are designed for **reading and modification**, not just execution.

When a scaffold doesn't produce the feel you want:

1. **Read** the scaffold code
2. **Identify** which variable controls the aspect you want to change
3. **Hypothesize** what changing it will do
4. **Test** by changing and running
5. **Iterate** until it feels right

This is debugging as design process.

Debugging Without Coding

The debugging mindset applies beyond code:

Debugging a design: Something doesn't work. Why? Form hypothesis. Test. Revise.

Debugging feedback: Playtesters report frustration. Where? Why? What's the root cause?

Debugging communication: Message isn't landing. What's being misunderstood? Where's the gap?

The skill transfers. Learning to debug code teaches you to debug everything.

Teaching Notes

We introduce debugging explicitly in **Week 2**, the first time students use the Code Bank. We talk about debugging *before* they encounter bugs, to normalize it.

Key Move

When students get stuck, don't give the answer. Guide them through debugging:

- "What's the error message say?"
- "What do you *think* is happening?"
- "How could you test that?"

This is slower. It's also where learning happens.

Exercise: Intentional Bugs

Give students code with intentional bugs. Have them debug without seeing the working version. Then discuss: how did you find it? What strategies did you use?

Exercise: Explain the Code

Students present someone else's Code Bank scaffold to the class. Forces reading and articulating unfamiliar code.

Assessment

We assess debugging process, not just outcome. "How did you approach the problem?" matters as much as "did you fix it?"

Common Challenges

Students want to be told the answer:

Resist. The learning is in the process. Give hints, not solutions. Ask questions that lead them to discover the bug themselves.

Students make random changes hoping something works:

Stop them. Require articulation before action. "Before you change anything, tell me what you think is wrong and why this change will fix it."

Students are ashamed of bugs:

Share your own bugs. Show professional code with bugs. Make it clear that bugs are normal, not a sign of incompetence.

Theoretical Background

Key Influences

Seymour Papert: *Mindstorms*. The idea that programming teaches thinking. Debugging is the thinking.

Brian Cantwell Smith: *On the Origin of Objects*. Computational thinking as a distinct mode of understanding.

Annette Vee: *Coding Literacy*. The argument for coding as a form of literacy, not just a technical skill.

What We're Taking

The literacy frame fundamentally changes the pedagogical stance. If debugging is literacy, it's a core skill, not a remedial one.

What We're Resisting

The "debugging is for engineers" view. Designers debug too. Just differently.

The shame around failure. CS culture often makes bugs shameful. That's counterproductive.

Unresolved Questions

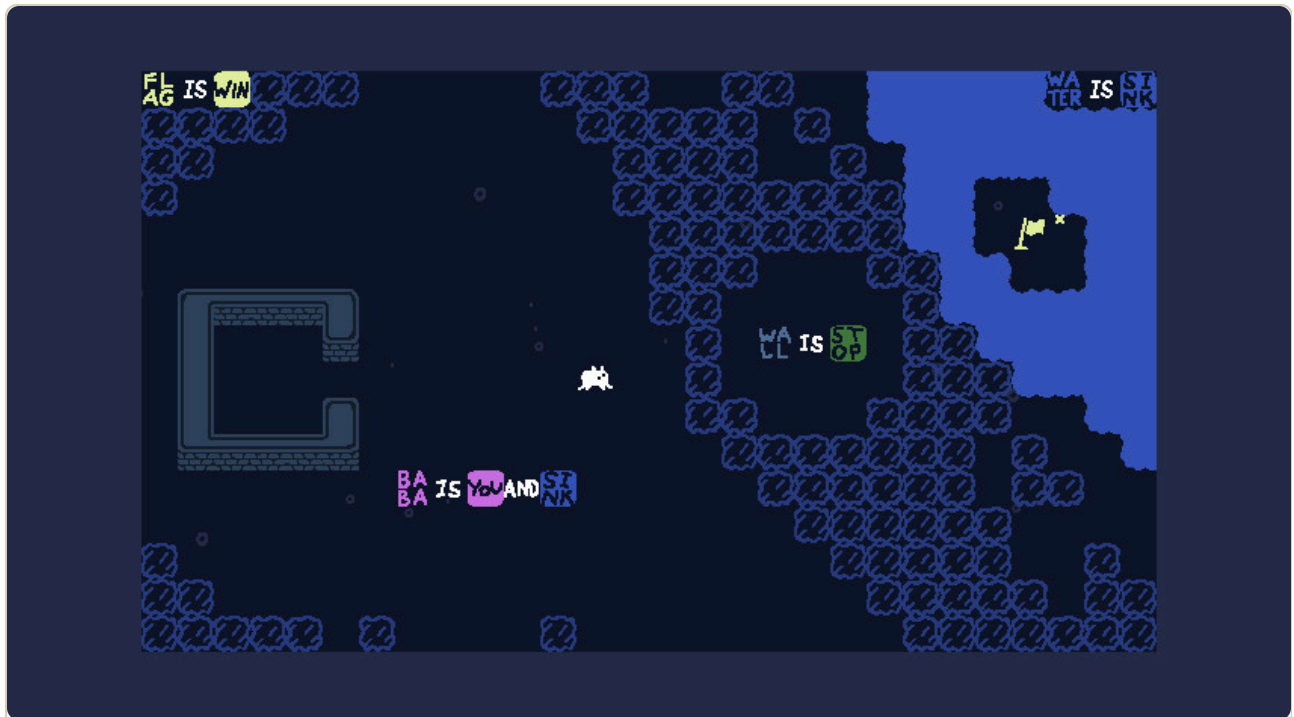
- How much debugging literacy is enough for designers? They don't need to be engineers. Where's the line?
- Does debugging literacy develop specific cognitive skills? The claim that "coding teaches thinking" is contested.
- How do we assess debugging? It's process-oriented, hard to grade.

Related

- [Code as Material](#) - code as design material
- [Code Bank](#) - scaffolds designed for reading
- [Play → Name → Make → Reflect](#) - debugging in the "make" phase

Code as Material

Code is a material. Like clay or paint, it has properties, resistances, and possibilities. Treating code as material - rather than as instruction or logic - changes how designers relate to implementation.



Baba Is You - Rules as material, rearranged to change the game itself.

Courtesy of Hempuli Oy

The Conventional View

In traditional education, code is treated as **instruction**: you tell the computer what to do, it does it. Code is a means to an end - the end being the running program.

This frames code as transparent. You think about what you want; code makes it happen. The code itself is irrelevant; only the result matters.

The Material View

In the material view, code has its own properties - like clay, wood, or paint. You work *with* it, not just through it. It pushes back. It suggests possibilities. It resists certain things and enables others.

Properties

Code has inherent properties:

- **It runs:** Unlike a sketch, code executes. You can feel it.
- **It's exact:** No ambiguity. 0.5 is not "about half."
- **It's repeatable:** Same input, same output (usually). You can iterate precisely.
- **It's modular:** Pieces can be combined, separated, reused.
- **It's fragile:** Small changes can break everything. Syntax matters.

Resistances

Code resists certain things:

- Vagueness ("make it feel good" isn't code)
- Simultaneity (things happen in sequence, not at once)
- Continuous values (you pick a number; the material doesn't)
- Inconsistency (it does exactly what you said, even if you didn't mean it)

Possibilities

Code enables things other materials can't:

- Instant variation (change a number, see different behavior)

- Parametric design (expose values, make them tunable)
- Interactivity (the material responds to input)
- Reproduction (copy infinitely, share globally)
- Combination (stack systems on systems)

What This Changes

Design Through Implementation

If code is material, you design *through* implementation, not before it. You discover possibilities by working with the material. A painter doesn't fully plan the painting before touching canvas; a designer doesn't fully plan the feel before touching code.

Prototyping as Sketching

Code prototypes are sketches in material. Rough, exploratory, meant to be thrown away. Like a sculptor's maquette, the prototype reveals what the material can do.

Feel Is Found

[Gesture](#) emerges from the code. You can imagine how a jump should feel, but the actual feel comes from gravity values, jump velocities, frame timing. The code gives you the feel; you refine it.

Parameters Are Handles

Good code exposes parameters - handles you can grab to change the behavior. The Code Bank scaffolds are designed this way: exposed variables at the top, tunable without understanding the implementation.

The Code Bank as Material Library

The [Code Bank](#) provides pre-shaped materials. Like buying lumber instead of felling trees, or using premixed paint instead of grinding pigments.

You're not cheating by using scaffolds. You're starting with refined material so you can focus on design decisions, not implementation details.

Use scaffolds to:

- Get something running fast

- Feel the behavior before deciding if it's right
- Tune parameters to find the feel you want
- Learn by reading working code

Knowing vs. Feeling

Knowing what you want is not the same as feeling it. "I want a floaty jump" is knowledge. The actual floaty jump - the gravity, the hang time, the landing - is feel. Code makes the jump real. Then you react to the real thing.

This is why prototyping matters. You don't know if your idea works until you feel it in material form.

Exercise: Material Exploration

Take any Code Bank scaffold. Don't try to make anything specific. Instead:

1. Change every exposed variable to see what it does
2. Push values to extremes (gravity = 10000? gravity = 0.1?)
3. Note which changes produce interesting behaviors
4. Find a feel you didn't expect but like

This is material exploration - discovery through manipulation.

Why Teach This

The "code as instruction" framing makes students afraid of code. It's abstract, logical, demanding. You have to know exactly what you want before you start.

The "code as material" framing invites exploration. You don't need to know what you're making - you need to start making and see what happens. This is how artists work. It's less intimidating.

Introducing the Concept

We introduce "code as material" in Week 1, before any coding. We talk about other materials: clay, paint, wood, fabric. What are their properties? What do they resist? What do they enable?

Then: code is the same. It has properties. We're going to learn them by working with the material.

Scaffolds as Entry Point

The Code Bank scaffolds let students work with code-as-material without writing code from scratch. They start by tuning, then modifying, then eventually building.

This is how apprentices learn crafts: start with prepared materials, focus on one skill at a time, gradually take on more of the process.

Common Challenges

"I don't know what values to use":

Good. That's the point. You find them by trying. Start somewhere. See what it feels like. Adjust. The material tells you when you're close.

"I want to do something the scaffold doesn't support":

Perfect. This is when you start modifying the scaffold - or writing new code. The material is telling you it wants to grow. Follow it.

"The code broke and I don't know why":

This is debugging. The material pushed back. Now you read it to understand what happened.

Assessment

Assess process, not just product. Did the student explore the material? Did they discover something unexpected? Can they articulate what the code does and why?

The material exploration exercise above works as an assessment: document what you found by manipulating the material.

Theoretical Background

Key Influences

Donald Schon: *The Reflective Practitioner*. Designing as a conversation with materials. The designer proposes; the material responds; the designer adjusts.

Seymour Papert: *Mindstorms*. Constructionism - learning by building. The computer as a material for thinking.

Richard Sennett: *The Craftsman*. Material engagement as a form of thought. The hand and the head are not separate.

The Phenomenology of Materials

Every material has a phenomenology - a way it presents itself to the person working with it. Wood has grain. Clay has plasticity. Paint has viscosity.

Code's phenomenology includes: instantness (changes appear immediately), precision (exact values), fragility (syntax errors break everything), and depth (layers of abstraction).

Learning a material means learning its phenomenology - how it feels to work with, what it wants to do, where it resists.

What We're Taking

The insight that design happens *in dialogue with material*, not before touching it. This licenses exploration, reduces the pressure to "know what you want" in advance, and positions prototyping as central to design.

What We're Resisting

The "code is just a tool" view: Tools are extensions of intention. Materials have their own agency. Code shapes design, not just executes it.

The "design documents first" approach: Documents have their place, but they're not design. The design is in the material. Docs are plans; material is reality.

Unresolved Questions

- How much should designers know about the material's internal structure? You can work with clay without knowing chemistry - but is code the same?
- At what point does "material exploration" become "random messing around"? How do we scaffold productive exploration?
- Does the material metaphor work for non-programmers? Or does it require enough code literacy to "feel" the material?

Related

- [Debugging as Literacy](#) - reading the material when it pushes back
- [Code Bank](#) - pre-shaped materials for design
- [Play → Name → Make → Reflect](#) - the "make" phase is material work
- [Gesture](#) - what emerges from the material

Why Unity

Unity isn't in this course "because industry." Unity is in this course because videogames are a medium of implementation.

Your Idea Is Not a Videogame

A design document is not a videogame. A concept is not a videogame. A mood board is not a videogame.

Your idea only becomes a videogame when you commit to **constraints**. Until then, it's potential. Implementation makes it real.

The Constraints

When you build in an engine, you must decide:

- **Input** - What buttons? What timing? What feels responsive?
- **Timing** - How many frames? How fast? What duration?
- **State** - What can happen when? What prevents what?
- **Feedback** - What does the player see, hear, feel?
- **Camera** - What's visible? What's framed? What's hidden?
- **Audio** - What sounds? When? How loud? How layered?
- **Iteration** - What do you change? How do you test?

These aren't implementation details - they ARE the design. A jump with 0.2 seconds of hang time is a different videogame than a jump with 0.4 seconds. The engine is where you discover which one is yours.

The Workshop

Unity is the workshop where those constraints become real. The engine teaches you what your videogame design actually is.

You can imagine a jump. You can describe a jump. But until you build a jump - set the gravity, the velocity, the animation frames, the landing sound - you don't know what your jump *is*.

This is why we use an engine in a design course, not a prototyping tool. Prototyping tools let you sketch. Engines let you commit.

Why Unity Specifically

Unity is not the only engine. It's not necessarily the "best" engine. We use it because:

- **2D and 3D:** Handles both without switching tools
- **C# scripting:** Real programming language, transferable skills
- **Documentation:** Extensive, searchable, beginner-friendly
- **Community:** Large enough that most problems are solved somewhere
- **Free tier:** No cost barrier to entry
- **Industry presence:** Skills transfer to jobs (this matters, even if it's not the reason)

Godot, Unreal, GameMaker - all valid choices. The pedagogy transfers. The point is having a real engine, not which real engine.

What Unity Is Not

Unity is not the course.

This is not "learn Unity." This is "learn videogame design, using Unity as the workshop." We don't cover every Unity feature. We cover what you need to implement your designs.

The goal is not Unity proficiency. The goal is design thinking that happens to be implemented in Unity.

Why Teach This

Students often resist implementation. They want to stay in the idea phase, where everything is possible and nothing is tested.

Explaining *why* we use an engine - not just *how* - helps students understand that implementation is not a chore after design. Implementation IS design.

The "I'm Not a Programmer" Problem

Many design students identify as "not programmers." They see code as someone else's job.

Our response:

- You don't need to be a programmer. You need to commit to constraints.
- The [Code Bank](#) scaffolds let you design without writing from scratch.
- [Debugging](#) is the skill, not authoring.
- Every parameter you tune is a design decision. That's the work.

Common Misconceptions

"We use Unity because that's what the industry uses"

Industry relevance is a side effect, not the reason. We use an engine because videogames are implemented things. The industry connection is bonus, not justification.

"I can design without an engine"

You can concept without an engine. You can document without an engine. You cannot design a [Gesture](#) without feeling it, and you cannot feel it without building it.

Assessment

We assess the design, not the technical implementation. A buggy prototype with clear design intent passes. A polished prototype with no design thinking fails.

Implementation as Epistemology

There are things you can only know by building. The feel of a jump. The timing of feedback. The camera distance that makes a space readable. These are not specifiable in advance.

This is not anti-intellectual. It's recognizing that videogames are a medium where knowledge lives in the implementation. The engine is not just a tool - it's where you learn what your design actually is.

The Sketch vs. the Commitment

Paper prototypes, Figma mockups, design documents - these are sketches. Valuable, but uncommitted. They preserve optionality.

Engine implementation is commitment. You must choose: this gravity, this frame count, this sound. The constraint is the design. The sketch proposes; the engine disposes.

Material Engagement

This connects to [Code as Material](#). The engine is the workshop where you work with the material. You discover what the material can do by working with it.

Unresolved Questions

- How much engine proficiency is enough? Where's the line between "can implement designs" and "is an engineer"?
- Do visual scripting tools (Bolt, Blueprints) change this equation? Is text code necessary?
- What do we lose when students only know one engine? Is engine-agnostic thinking possible or desirable?

Related

- Code as Material - code has properties you work with
- Debugging as Literacy - the skill that makes engine work possible
- Code Bank - scaffolds for implementation
- The Final Project - what implementation leads to

The Final Project

The final is not "make a big videogame."

Make one action feel undeniable.

The Requirement

One or two verbs, maximum. Strong feedback. Clear [Arc](#). A small slice with a real identity.

The final is proof that you understand videogames as a medium, not as a mood board.

What "Undeniable" Means

When you play the final, the action should feel **right**. Not "good enough." Not "it works."
Right.

The jump lands exactly when it should. The dash has exactly the punch it needs. The feedback tells you exactly what happened. The [Gesture](#) is complete.

You'll know it when you feel it. So will we.

What the Final Is NOT

It's not a complete videogame.

No one expects levels, progression, menus, or polish. Those are production concerns. This is a design course. We're looking for design clarity, not shipping readiness.

It's not a tech demo.

Implementing five mechanics poorly is worse than implementing one mechanic well.

Restraint is the skill. Saying "this is what my videogame IS" requires saying "this is what my videogame ISN'T."

It's not a concept.

"I wanted it to feel like..." is not the final. The final is what it actually feels like. Intent without execution is not design.

The Components

The Build

A playable prototype. Ideally 30 seconds to 2 minutes of focused experience. Long enough to feel the action. Short enough to be tight.

The Articulation

A brief written or verbal explanation of:

- What is the core Gesture?
- How do the 4 A's work together?
- What Aesthetic Heritage are you drawing from?
- What Permissions did you design?

The articulation proves you can name what you made - that you understand it, not just that you stumbled into it.

Examples of Scope

A platformer final might be:

- One character
- Walk, jump, maybe one more ability
- A single room or short sequence
- Clear feedback on every action
- 30 seconds of intentional experience

An action final might be:

- One attack, one enemy type
- A single arena
- Hitstop, screen shake, sound design
- Clear consequences for hitting and being hit

A puzzle final might be:

- One mechanic
- Three puzzles that teach and test it

- Clear feedback when the solution clicks

Notice: all of these are small. Small is the point.

The Question We're Asking

When we play your final, we're asking one question:

Does this person understand what a videogame is?

Not "is this marketable?" Not "is this finished?" Not even "is this fun?"

Does this person understand that videogames are a medium of action? That the design lives in the Gesture? That the feel is the form?

If yes, you've succeeded. If no, we talk about why.

Why This Format

Students come in wanting to make "their dream game." That's beautiful, but it's not learnable in one course. The dream game is a multi-year, multi-person project.

The "one undeniable action" constraint forces focus. It's achievable in the time available. It requires all the skills we teach. And it produces something that can actually be felt.

The Scope Conversation

You will have scope conversations. Students will want to add more.

The answer is always: "Is the core action undeniable yet? If not, adding more won't help. If yes, you're done."

This is hard for students who equate "more" with "better." The final teaches that "complete" is better than "more."

Assessment Criteria

Does the action feel intentional?

Not "does it work" but "does it feel designed." There's a difference between functional and intentional. We're looking for intentional.

Can the student articulate their choices?

Using the framework vocabulary. If they can name it, they understand it. If they can't, they may have stumbled into something without learning from it.

Is the scope appropriate?

Did they resist the urge to add more? Did they focus? Restraint is a skill we're assessing.

Common Pitfalls

"It's almost done, I just need to add..."

If the core isn't there, adding won't save it. Redirect to the core.

"I know it doesn't feel right, but the idea is..."

The idea is not the final. The feel is the final. Help them close the gap or reduce scope.

"I want to show range by doing multiple things"

Range is demonstrated by doing one thing deeply, not many things shallowly. One undeniable action shows more skill than five mediocre ones.

Why "Undeniable"

The word matters. Not "good." Not "fun." Not "polished." *Undeniable*.

Undeniable means: when you experience it, you can't deny what it is. The action has identity. It's not generic. It's not "a jump" - it's *this* jump.

This is the threshold for design. Below it, you have implementation. At it and above, you have something that could only be what it is.

The Proof of Understanding

The final is not about making something great. It's about proving understanding.

A student who makes a small, clear, intentional thing understands videogames. A student who makes a sprawling, unfocused, accidental thing does not - yet.

The final reveals where the student is. That's its pedagogical purpose.

Medium Specificity

The final is specifically a *videogame* final. Not a document about a videogame. Not a presentation about a videogame. A videogame.

This is because videogames are a medium of action. The proof of understanding must be in the medium itself. You can't prove you understand painting by writing about painting. You paint.

Unresolved Questions

- How do we assess students with different technical starting points fairly? The constraint helps, but gaps remain.
- Is "undeniable" too subjective? How do we calibrate across assessors?

- What about students whose strength is systems, not feel? Is this final format biased toward action videogames?

Related

- [Gesture](#) - what the final demonstrates
- [Why Unity](#) - why implementation matters
- [Play → Name → Make → Reflect](#) - the process that leads here
- [Code Bank](#) - scaffolds for building the final

PART VII

Case Studies

Close readings of individual Gestures from published videogames.

Case Studies

Deep dives into specific videogames, analyzing how they implement the concepts in this knowledge base.

Available Case Studies

Celeste

- [Dash](#) - The dash as perfect [Gesture](#)
- [Assist Mode](#) - Accessibility as design, not compromise

Dark Souls

- [Bonfire](#) - The bonfire as transformed [Aesthetic Heritage](#)

Hades

- [God Mode](#) - Automatic accessibility through escalating resistance

Portal

- [Level One](#) - Implicit teaching through spatial constraint

Undertale

- [Genocide Route](#) - [Permissions](#) as moral architecture

Planned Case Studies

Future case studies will examine:

Celeste (continued)

- Chapter Structure - Pacing and progression

Dark Souls (continued)

- Interconnected World - Spatial communication masterclass
- Difficulty As Meaning - [Permissions](#) and meaning

Other Videogames

- **Portal:** Momentum (Gesture and physics feel)
- **Undertale:** Mercy Mechanic (Permissions as moral architecture)
- **Breath of the Wild:** Physics Systems (designed emergence), Great Plateau (tutorial through world)
- **Hades:** Narrative Repetition (story through roguelike structure)
- **Noita:** Emergent Behavior (systemic interaction)

By Concept

Concept	Case Studies
<u>Gesture</u>	Celeste/Dash
<u>Aesthetic Heritage</u>	Dark Souls/Bonfire
<u>Permissions</u>	Undertale/Genocide Route
Accessibility	Celeste/Assist Mode, Hades/God Mode
Emergence	(Planned: Breath of the Wild/Physics, Noita)
Tutorial Design	Portal/Level One

How to Read Case Studies

Each case study follows a consistent structure:

1. **Describes** the specific design element
2. **Analyzes** how it implements concepts from this KB
3. **Connects** to broader principles
4. **Questions** what's unresolved or debatable

Case studies aren't celebrations. They're examinations. Even great videogames have questionable choices.

Using Case Studies in Teaching

Case studies work best when students have already played the videogame. If that's not possible, video footage can substitute, but the haptic experience is lost.

Teaching sequence:

1. Students play the relevant section (or watch footage)
2. Discuss: What did you notice? What did you feel?
3. Read the case study together
4. Discuss: Did the analysis match your experience? What did it miss?
5. Connect to framework concepts ([Gesture](#), [The 4 A's](#), etc.)

Writing Case Studies

If you want to add a case study:

1. Pick a specific element (not "Dark Souls is good" but "the bonfire as checkpoint")
2. Use concepts from this KB - especially [Gesture](#), [The 4 A's](#), [Aesthetic Heritage](#), and [Permissions](#)
3. Include your interpretation but also acknowledge other readings

4. Connect to at least 2-3 other entries

THEORY

- deeper grounding

Why Case Studies

Videogame design knowledge is often tacit - designers know what works but struggle to articulate why. Case studies make tacit knowledge explicit by examining specific decisions in context.

The case study method comes from business and law education, where students learn through detailed analysis of real situations rather than abstract principles. For videogames, this means examining actual design decisions rather than hypothetical scenarios.

The Specificity Principle

Case studies must be specific. "Dark Souls has good level design" teaches nothing. "The bonfire creates a checkpoint with meaningful cost" teaches a principle.

Specificity enables transfer: once you understand how the bonfire transforms checkpoint heritage, you can apply that thinking to your own work.

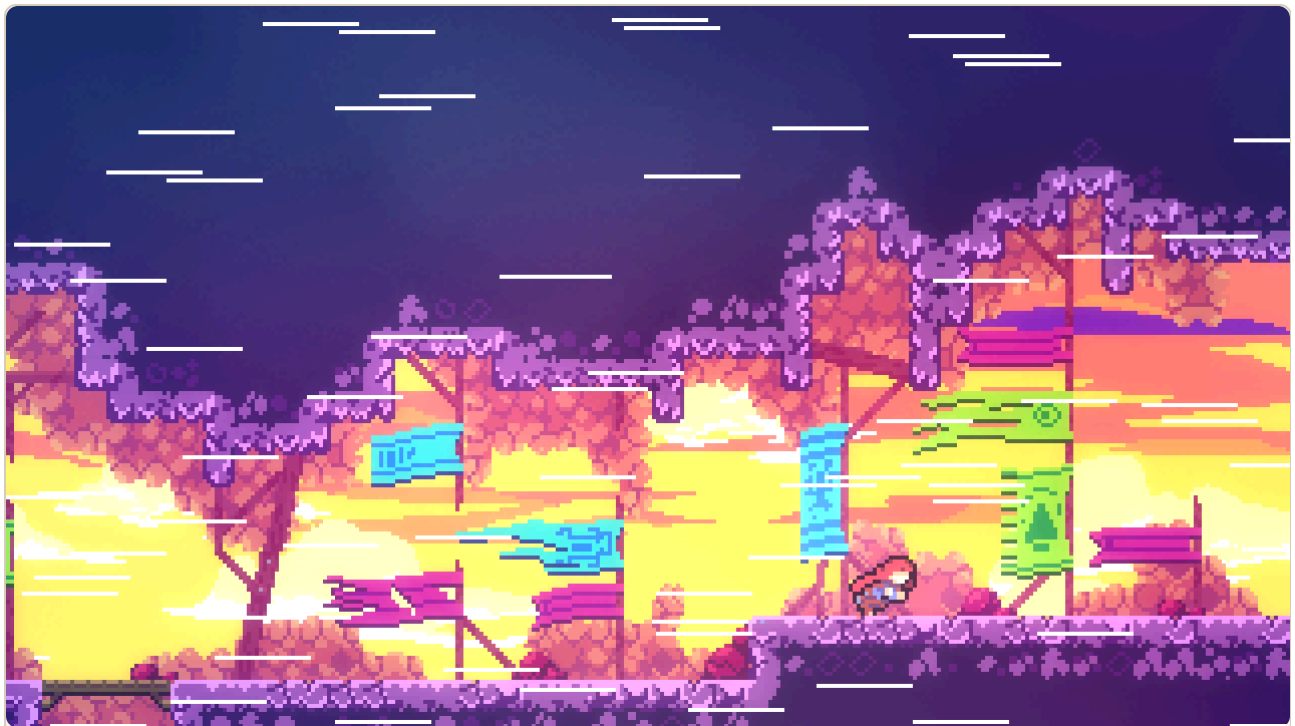
Limitations

Case studies are retrospective. They explain what worked, not how to discover what will work. They're tools for analysis and vocabulary, not prediction.

Case studies also risk canonization - treating analyzed videogames as "correct" examples rather than one set of choices among many. Every case study implicitly argues for the importance of its subject.

Celeste: The Dash

The dash in Celeste is one of the most celebrated Gestures in modern platforming. It's simple - press a button, move in a direction - and perfect. Understanding why it's perfect teaches us what Gesture design can achieve.



Celeste - The dash's visual feedback: sprite blur, trail, hair color change.

Courtesy of Maddy Makes Games

The Gesture

Input: Press dash button + direction

Action: Burst of movement in 8 directions (including diagonals), briefly invulnerable

Duration: ~0.15 seconds of dash, brief recovery

Limitation: One dash per airborne period, refreshes on ground/wall/certain objects

What Makes It Perfect

Clarity

You always know if you have your dash. Visual indicator, audio cue, and kinesthetic memory all align. No ambiguity.

Expressiveness

Eight directions means player choice. The dash isn't a fixed escape - it's a tool you aim. Skill expression happens in the aiming.

Commitment

Once you dash, you're committed. No canceling mid-dash. This creates the tension: you have to be right.

Resource Management

One dash per air time. You're always asking: "Do I spend it now?" This transforms platforming into resource decisions.

Feel

The combination of:

- Instant start (no wind-up)
- Consistent duration

- Clear end state
- Visual trail
- Sound design
- Screen effects

All elements reinforce: this is a *burst*, a *thrust*, a *commitment*.

The 4 A's of the Dash

Analyzing through the 4 A's:

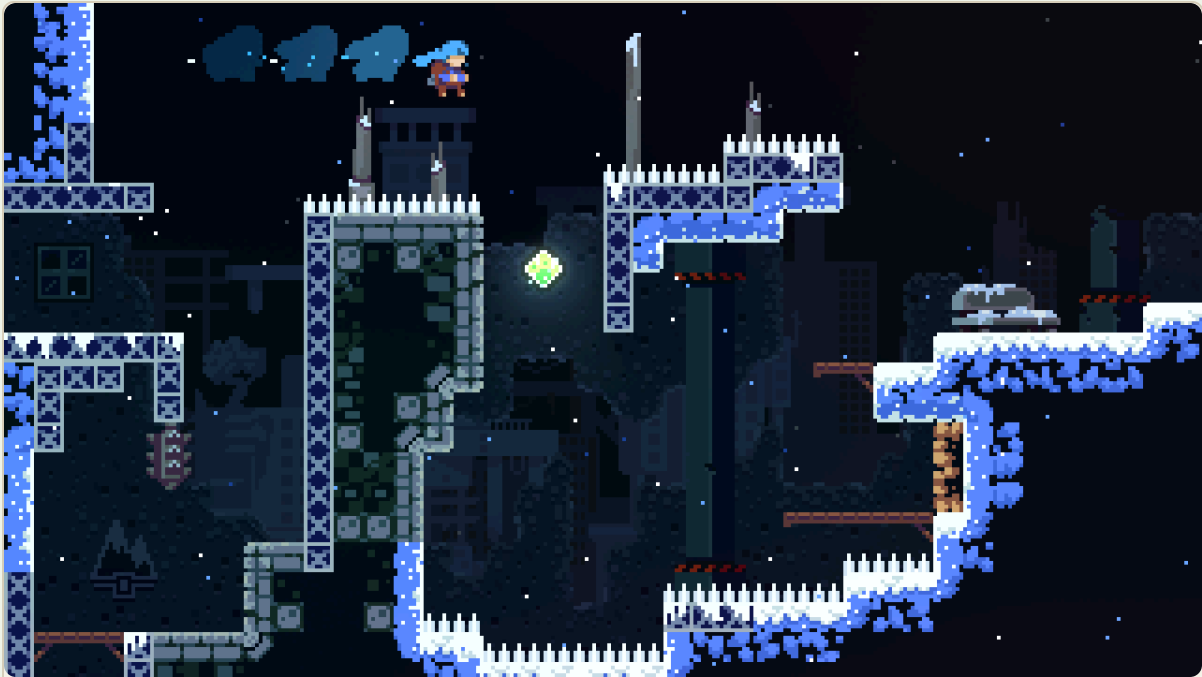
- **Action:** Directional burst, committed, one-per-air. The player chooses direction and timing.
- **Art:** Sprite blur, hair color change, particle trail, whoosh sound. Every sensory channel confirms the dash.
- **Arc:** ~0.15 seconds - a micro-moment. But that micro-moment punctuates longer arcs of traversal.
- **Atmosphere:** Urgency, precision, the feeling of barely-making-it. The dash feels athletic.

Feedback Design

The dash is a feedback symphony:

Channel	Element
Visual	Sprite blur, hair color change, particle trail
Audio	Distinct dash sound (whoosh + impact)
Animation	Squash/stretch, body angle
Screen	Brief shake, possible freeze frame on direction change
UI	Dash indicator (hair color shows availability)

Every channel confirms: you dashed. The layering creates *certainty* without being overwhelming.



Celeste - Level design built around the dash as a single-use resource.
Courtesy of Maddy Makes Games

The Dash and Difficulty

Celeste is hard. The dash is why it can be hard without being frustrating.

Because the dash is:

- **Reliable** - same every time
- **Readable** - you know what happened
- **Fair** - you had the tool; you used it wrong

Death feels earned. "I should have dashed left." Not: "the videogame cheated me."

Try This

Recreate the Dash

Using the Code Bank [Dash](#) scaffold, recreate the *Celeste* dash feel. What variables matter most?

Remove One Element

Take a working dash. Remove the sound. Or the trail. Or the recovery. What changes?

One Dash vs. Infinite

Play a level with one dash. Then with infinite. How does the experience change? What do you lose?

Why Use This Case Study

The *Celeste* dash is ideal for teaching [Gesture](#) because:

- It's short enough to analyze frame-by-frame
- Students can feel the difference when elements are removed
- The videogame has Assist Mode, allowing direct experimentation
- Matt Thorson has discussed the design publicly

Teaching Sequence

1. Play *Celeste* Chapter 1 together (15-20 min)
2. Identify the dash as a [Gesture](#)
3. Break down the [4 A's](#)
4. Discuss: Why does this feel good?
5. Use Assist Mode to add infinite dashes - how does experience change?
6. Connect to constraint and decision-making

Discussion Prompts

- What would happen if the dash had a wind-up (delay before activation)?
- What if you could cancel the dash mid-movement?
- How does the one-dash limit create "questions" for the player?
- Compare to dashes in other videogames. What's different?

Common Misconceptions

"The dash is just fast movement"

The dash isn't speed - it's commitment and resource. The speed matters less than the decision to use it.

"More feedback is always better"

Celeste's feedback is layered but restrained. Too much would overwhelm. Each element serves a purpose - clarity, not spectacle.

Exercise: Feedback Audit

Have students audit the feedback channels in a [Gesture](#) from a videogame they're making:

1. List every feedback element (visual, audio, haptic, etc.)
2. For each, ask: What information does this convey?
3. Identify gaps: What's unclear? What's redundant?
4. Prototype one improvement

Aesthetic Heritage

The dash inherits from:

Mega Man X's dash - The ground dash that changed platformers. *Celeste* extends it into air.

Air-dash fighting videogames - Aerial mobility as skill expression. The eight directions echo fighting videogame motion.

Super Meat Boy's run - Tight, committed movement. *Celeste* adds the dash as discrete resource.

But *Celeste* transforms the inheritance:

- Dash is **limited** (once per air), creating decision
- Dash is **directional** (8-way), creating expression
- Dash is **tied to narrative** (Madeline's abilities reflect her emotional state)

The Dash and Accessibility

Celeste's Assist Mode lets you:

- Add extra dashes (infinite if desired)
- Slow videogame speed
- Grant invincibility

This doesn't "break" the dash. It changes the **Permission structure**. The dash is still the dash. Players who need it get more of it.

The dash design is robust enough to survive different permission settings. That's good **Gesture** design.

What *Celeste* Teaches

1. **Gestures are designed wholes.** Input, action, feedback, consequence - all aligned.
2. **Constraint creates meaning.** One dash = decisions. Infinite dash = no decisions.

3. [Heritage](#) can be transformed. Take the Mega Man dash, add limitation, add verticality, add narrative. New Gesture.
4. **Feel is layered feedback.** The dash "feel" is a dozen elements working together.
5. **Good design survives accessibility.** If your Gesture breaks when made easier, it was fragile.

Unresolved Questions

- How much of the dash's success comes from the level design built around it?
- Is the dash's "perfection" a matter of tuning, or fundamental design choices?
- Could the dash work in 3D, or is it inherently a 2D Gesture?

Related

- [Gesture](#) - the dash as canonical Gesture
- [Aesthetic Heritage](#) - where the dash comes from
- [The 4 A's](#) - analyzing the dash's components
- [Celeste: Assist Mode](#) - accessibility implications
- [Code Bank: Dash](#) - scaffold for dash implementation

Celeste: Assist Mode

Celeste's Assist Mode lets players customize difficulty with granular options. It's become a model for accessibility in challenging videogames. A solid proving out that difficulty and accessibility are not actually opposites.



Celeste - Granular accessibility options that let players shape their own experience, their own customized gestures.

Courtesy of Maddy Makes Games

The Options

Assist Mode provides granular control over difficulty:

Option	What it changes
Game Speed	Slow the videogame to 50%, 60%, 70%, 80%, 90%
Invincibility	Hazards don't kill you
Infinite Stamina	Climbing doesn't drain stamina
Air Dashes	Add extra dashes (2, 3, or infinite)
Dash Assist	Skip precise inputs, hold direction

Critically: these can be mixed. Want normal speed but two dashes? That's possible. Want invincibility but no extra dashes? Also possible.

How It's Presented

When entering Assist Mode, the videogame shows this message:

"Celeste was designed to be a challenging and rewarding experience. We believe that its difficulty is essential to the experience. However, we understand that everyone's skills and needs are different. So we've provided these tools to help you play the game in a way that works for you. We hope that you'll find the right balance between challenge and accessibility."

Key elements:

- Acknowledges difficulty is intentional
- Affirms player autonomy ("works for you")
- Framed as tools, not concessions

What Doesn't Change

Assist Mode doesn't:

- Skip content
- Change the narrative
- Reduce visual fidelity
- Lock achievements
- Mark your save file as "assisted"

The experience is yours. Nobody else knows how you played. Nobody else deserves to have a space in that conversation between you and a game.

The 4 A's Perspective

Assist Mode primarily modifies Action and Arc:

- **Action:** More dashes = more options. Invincibility = removed failure state.
- **Arc:** Slower speed = longer duration for the same content.
- **Art:** Unchanged. The videogame looks and sounds the same.
- **Atmosphere:** Changed - but intentionally. Less stress, more exploration.

The core Gestures remain intact. The dash is still the dash. The levels are still the levels.

Try This

Play the same room twice

Play Chapter 2's hotel section normally, then with 70% speed. How does the experience differ? What's preserved?

Audit your own videogame

What could you make optional without breaking your core experience? What can't be changed without losing the point?

Why Use This Case Study

Celeste's Assist Mode is the clearest modern example of accessibility done right in a challenging videogame. It proves:

- Difficulty and accessibility can coexist
- Granular options beat binary "easy/hard"
- Framing matters (tools vs. cheats)
- Player trust is possible

Teaching Sequence

1. Have students play *Celeste* without Assist Mode (experience the challenge)
2. Introduce Assist Mode; read the message together
3. Discuss: How is this different from traditional "easy mode"?
4. Play with different assist combinations - what feels right?
5. Design exercise: Create an Assist Mode for a hypothetical videogame

Discussion Prompts

- Why doesn't *Celeste* call it "Easy Mode"?
- What's the difference between "lowering difficulty" and "providing tools"?
- Should achievements be disabled when using Assist Mode? Why or why not?
- How does the framing message affect how players feel about using it?

The Curb Cut Principle

Curb cuts (ramps at sidewalk edges) were designed for wheelchairs but help everyone - parents with strollers, delivery workers with carts, travelers with luggage.

Celeste's Assist Mode is a curb cut:

- Designed for players with disabilities

- Also helps: players with limited time, players who want story, players learning, players streaming
- Makes the videogame more accessible to all without removing the core challenge for those who want it

This is the [Accessibility as Craft](#) principle in action.

Common Misconceptions

"Assist Mode waters down the experience"

The experience changes, but the core remains. A player using Assist Mode still climbs the mountain. The metaphor holds.

"Players will always take the easy way"

Studies and anecdotes suggest otherwise. Many players use Assist Mode briefly, then turn it off. Others use specific options (like slower speed) while keeping others default. Trust works.

Permissions and Accessibility

From a Permissions perspective, Assist Mode expands what's **allowed** without changing what's **required**:

- **Required:** Still must traverse each room. Still must reach the summit.
- **Allowed:** More tools to do so. More attempts. More recovery options.
- **Forbidden:** Nothing new is forbidden. No punishment for using assistance.

This is key: accessibility expands permissions. It doesn't shrink requirements (you still play the videogame) or forbid anything (no shame, no marks).

The Difficulty Discourse

Celeste launched during heated debates about difficulty in videogames (often centered on *Dark Souls* and "should it have easy mode"). Assist Mode offers a response:

- Difficulty can be optional without being absent
- The "intended" experience can coexist with alternatives
- Designer intent and player need aren't opposites

What Makes Assist Mode Work

1. **Granularity:** Options are specific, not binary. This respects that accessibility needs vary.
2. **Framing:** "Tools" not "cheats." Language matters.
3. **No punishment:** No achievement locks, no save file marks, no public shaming.
4. **Preserved core:** The Gestures remain. The level design remains. Only constraints change.
5. **Trust:** Players are trusted to find their own balance.

Compare: Hades God Mode

Hades takes a different approach: God Mode grants increasing damage resistance with each death. It's automatic escalation rather than manual tuning.

Both work. *Celeste*'s approach requires more player agency. *Hades*'s approach requires less menu navigation. Different videogames, different solutions.

Unresolved Questions

- Where's the line between "accessibility option" and "different videogame"? Does infinite dashes fundamentally change *Celeste*?
- Should competitive videogames have Assist Mode? What about multiplayer?
- Is there a risk that Assist Mode becomes expected, pressuring developers who can't implement it?

Related

- [Celeste: Dash](#) - the core mechanic that Assist Mode modifies
- [Hades: God Mode](#) - another accessibility approach
- [Accessibility as Craft](#) - the philosophy behind this
- [Permissions](#) - how accessibility changes permission structures

Dark Souls: The Bonfire

The bonfire is Dark Souls' most iconic design element - a checkpoint system that became a symbol. It transforms inherited mechanics into something new: a checkpoint you might not want to use.



Dark Souls - Warmth in a hostile world, the bonfire's visual contrast is its atmosphere.

Courtesy of FromSoftware / Bandai Namco

What the Bonfire Does

- Restores health and Estus (healing) charges
- Serves as respawn point on death
- Allows leveling up, attuning spells, fast travel (in some entries)
- **Respawns all non-boss enemies**

That last point changes everything.

The Inherited Problem

Checkpoints in most videogames:

- Save progress
- No downside
- Use as soon as possible

The design problem: checkpoints remove tension. Once you hit a checkpoint, you're safe. Everything before it doesn't matter anymore.

Horror videogames solved this with limited saves (*Resident Evil* typewriters, ink ribbons). But survival horror creates different tension than action RPGs.

The Transformation

Dark Souls keeps checkpoints as healing/respawn points but adds **cost**: enemies return.

This creates decisions:

"Should I use this bonfire?"

- Pro: I'll respawn here, and I have full heals
- Con: Every enemy between here and my goal is back

"Should I push forward?"

- Pro: I might find another bonfire, skip these enemies
- Con: If I die, I lose souls and start further back

The checkpoint becomes **strategic choice**, not automatic relief.

The 4 A's of the Bonfire

Analyzing the bonfire through the 4 A's:

- **Action:** Sit, rest, make decisions (level up, attune, etc.). The action is deliberately slow - you choose to stop.
- **Art:** Warm light in a dark world. Crackling fire, ambient shift to peaceful. The visual and audio contrast with the hostile environment.
- **Arc:** A pause in the action. The bonfire creates rhythm - explore, fight, rest, repeat. The arc of a play session is structured around bonfire visits.
- **Atmosphere:** Safety, respite, but with awareness of what waits outside. The bonfire *feels* like refuge because the mechanics and aesthetics align.

Permission Structure

The bonfire grants permission to:

- Rest safely
- Refill resources
- Save progress

The bonfire *requires*:

- Accepting enemy respawn
- Committing to a new respawn point

The bonfire *forbids*:

- Nothing (you can always use it)

But "can" and "should" differ. You *can* always rest. You *should* think first. This is Permissions in action: the gap between technical possibility and wise action.

Spatial Design

Bonfire placement is level design:

- **Before boss:** Classic. Reduces repetition of easy content.
- **After tough section:** Reward for completion.
- **In hidden area:** Reward for exploration.
- **Seemingly close but far:** Bonfires visible but unreachable create longing.
- **Multiple in same area:** Choice about which to commit to.

The interconnected world means bonfires anchor your **mental map**. You navigate by bonfire.

Try This

Play Without Resting

Play a section of *Dark Souls* without using a bonfire. How does this change your decision-making? Your tension level?

Analyze Placement

Map the bonfire locations in one area. Why are they where they are? What choices do they create?

Design a Checkpoint System

Create a checkpoint system with a cost. What cost? How does it change player behavior?

Teaching Applications

The bonfire is an excellent case study for several reasons:

1. **Clear transformation:** Students can compare to traditional checkpoints
2. **Visible trade-off:** The cost (enemy respawn) is immediately understandable
3. **Multi-layered:** Works as mechanic, atmosphere, spatial design, and narrative

Discussion Prompts

- What if enemies didn't respawn? How would that change the videogame?
- What if you could only use each bonfire once? What would that create?
- Compare to checkpoints in a videogame students know. What does each system prioritize?

Common Misconceptions

"The bonfire is just difficulty"

Students often reduce the bonfire to "makes the videogame harder." Redirect: the bonfire makes the videogame *more thoughtful*. It turns automatic actions (hit checkpoint) into decisions (should I rest?).

"Dark Souls is about punishment"

The bonfire actually reduces punishment - you always have a safe place. The videogame is about meaningful consequence, not arbitrary difficulty.

Exercise: Redesign a Checkpoint

Have students redesign a checkpoint system from a videogame they know:

1. Identify the current checkpoint behavior
2. Add a meaningful cost (not arbitrary difficulty)
3. Consider how this changes player psychology

4. Prototype in paper or describe in detail

What *Dark Souls* Teaches

1. **Heritage** can be transformed by cost. Checkpoints are old. Checkpoints with enemy respawn are new. One change, new meaning.
2. **Mechanics and aesthetics align.** The bonfire feels like safety because it is safety - and because it looks/sounds like warmth in a cold world.
3. **Decisions beat convenience.** A checkpoint that's always-use is convenience. A checkpoint that's sometimes-use is design.
4. **Spatial meaning:** Where you put mechanical elements matters. Bonfire placement *is* level design.
5. **Symbols emerge from systems.** Nobody designed "the bonfire as iconic symbol." It emerged from the mechanics being good.

The Dark Souls Legacy

Post-*Dark Souls*, many videogames use the bonfire pattern:

- *Hollow Knight*: Benches (similar enemy respawn logic)
- *Sekiro*: Sculptor's Idols
- *Elden Ring*: Sites of Grace (expanded systems)
- Many "Souls-likes"

The bonfire became its own **Aesthetic Heritage**. Designers now inherit from *Dark Souls*.

Unresolved Questions

- Is the bonfire system exclusionary? Does meaningful checkpoint cost inherently conflict with accessibility?
- How much of the bonfire's impact comes from the interconnected world vs. the mechanic itself?
- Could a "cost-based checkpoint" work in genres other than action RPGs?

Related

- [Aesthetic Heritage](#) - bonfire as transformed inheritance
- [Permissions](#) - can vs. should
- [The 4 A's](#) - analyzing the bonfire's components
- [Environmental Storytelling](#) - bonfire as spatial meaning

Hades: God Mode

Hades' God Mode grants escalating damage resistance with each death. Unlike Celeste's granular options, it's a single toggle that adapts automatically - proving there's more than one way to design accessibility.



Hades - A single toggle that adapts, damage resistance increases with each death.
Courtesy of Supergiant Games

How It Works

God Mode is simple:

- Toggle it on in settings
- You start with 20% damage resistance
- Each time you die, resistance increases by 2%
- Maximum resistance caps at 80%

That's it. No sliders, no options, no configuration. Just one toggle and an automatic escalation system.

The Math

Deaths	Resistance	Effective Health
0	20%	1.25x
10	40%	1.67x
20	60%	2.5x
30	80% (max)	5x

The resistance never goes back down. Once earned, it's permanent (unless you toggle God Mode off entirely).

How It's Presented

When enabling God Mode, the videogame shows:

"God Mode gives you a permanent +20% damage resistance, growing by +2% each time you die (up to 80%). You can turn God Mode off at any time. It doesn't prevent you from unlocking anything or doing anything in the game."

Key elements:

- Clear mechanical explanation
- Emphasizes it's reversible
- Explicitly states no content is locked
- Framed as a feature, not a concession

What God Mode Changes

God Mode affects:

- Damage taken (reduced)

That's all. Everything else remains identical:

- Enemy patterns unchanged
- Damage you deal unchanged
- Boss mechanics unchanged
- Story, unlocks, achievements unchanged
- Visual indicator (subtle glow) appears on health bar

The 4 A's Perspective

God Mode primarily modifies Action:

- **Action:** Combat becomes more forgiving. Mistakes cost less.
- **Arc:** Largely unchanged - runs take similar time. You might attempt harder Heat levels sooner.
- **Art:** Minimal visual change (health bar glow).
- **Atmosphere:** Less punishing, potentially less tense. But still demanding.

The Gesture of combat remains intact. You're still dodging, dashing, and timing attacks. You just survive longer when you mess up.

Try This

Compare two runs

Play a run without God Mode, then enable it for the next run. What feels different? What stays the same?

Notice the psychology

With God Mode, how does your relationship with death change? Do you play more aggressively? Take more risks?

Why Use This Case Study

Hades' God Mode offers a different accessibility philosophy than *Celeste*. Comparing them illuminates tradeoffs:

Aspect	Celeste Assist Mode	Hades God Mode
Configuration	Granular (5+ options)	Binary (on/off)
Adaptation	Manual (player chooses)	Automatic (system responds)
Direction	Static	Escalating
Cognitive load	Higher (what to enable?)	Lower (just toggle)

Neither is "better." They solve different problems for different videogames.

Teaching Sequence

1. Have students play *Hades* normally (experience the roguelike loop)
2. Enable God Mode; play multiple runs
3. Discuss: How does the automatic escalation feel compared to manual options?
4. Compare with *Celeste's* Assist Mode
5. Design exercise: Which approach would you use for your videogame, and why?

Discussion Prompts

- Why might *Hades* have chosen automatic escalation over manual configuration?
- Does the 2% incremental increase feel different from immediately having 80% resistance?
- What does the cap at 80% communicate about design intent?
- Would God Mode work for *Celeste*? Would Assist Mode work for *Hades*?

The Roguelike Factor

Hades' God Mode works because dying is already part of the videogame. In a roguelike:

- Death is expected and frequent
- Progress persists across deaths (meta-progression)
- The story frames death as canonical (Zagreus returns from the Styx)

God Mode leverages what's already there. Death counts up resistance automatically because death is already being counted. The system is elegant because it builds on existing mechanics.

Common Misconceptions

"God Mode makes the videogame too easy"

At 80% resistance, you still take damage. Bosses still have complex patterns. The videogame is easier, not trivial. And players can turn it off anytime.

"The escalation is just training wheels"

Some players keep God Mode on permanently. That's valid. Others use it temporarily. The design accommodates both without judgment.

Dynamic Difficulty Adjustment

God Mode is a form of **transparent DDA** (Dynamic Difficulty Adjustment). Unlike hidden DDA systems (which secretly make videogames easier), God Mode:

- Requires explicit opt-in
- Shows its numbers clearly
- Never activates without consent

This transparency matters. Hidden DDA can feel patronizing ("the videogame thinks I need help"). God Mode says: "Here's a tool. Use it if you want."

Permissions and Thresholds

From a [Permissions](#) perspective, God Mode shifts what's **required**:

- **Without God Mode:** Perfect play required for certain challenges. High skill floor.
- **With God Mode:** Imperfect play becomes viable. Lower skill floor.

The videogame still requires combat engagement. It doesn't require mastery.

Why 80% Cap?

The 80% cap is deliberate. At 100% resistance, combat would become meaningless - you could ignore all mechanics. At 80%, you're still mortal:

- Boss attacks still hurt
- Attention still required
- Death still possible (just less likely)

This preserves what makes *Hades* feel like *Hades*. The cap protects the core experience while expanding access.

The Earned Escalation

God Mode's resistance is "earned" through death. This creates an interesting psychological framing:

- You're not given easy mode - you build toward it
- Each death contributes to future success
- The videogame acknowledges your struggles by responding to them

Whether this framing is helpful or just narrative sugar depends on the player. Some find it empowering. Others just want the assistance without the theater.

Roguelike Accessibility

Roguelikes present unique accessibility challenges:

- Long play sessions before checkpoints
- Loss of progress on death
- Compound difficulty from randomness
- Skill checks that gate all content

God Mode addresses the skill check problem without touching the roguelike structure. You still do full runs, still face randomness, still build meta-progress. The videogame just hurts less.

Unresolved Questions

- Should God Mode affect damage dealt, or just received? What would change?
- Is the escalation necessary, or would a fixed resistance work equally well?
- Does the 2% increment feel like "earning" assistance or just "grinding" it?
- Would other roguelikes benefit from this approach, or is it specific to *Hades*' design?

Related

- Celeste: Assist Mode - contrasting accessibility approach
- Accessibility as Craft - the philosophy behind this
- Permissions - how difficulty shapes permission structures
- Player Psychology - motivation and frustration

Portal: Level One

Portal's first test chamber teaches the core mechanic without a single line of tutorial text. It's a masterclass in implicit instruction - showing instead of telling, constraining instead of explaining.



Portal - Test Chamber 00 teaches portals through spatial constraint, not text.

Courtesy of Valve Corporation

The Setup

You wake up in a glass box. There's a radio, a toilet, and a bed. Outside the glass, you see a room with a door. A timer counts down. When it hits zero, an orange portal opens on the wall outside. Through it, you can see... yourself. From behind.

What You Learn (Without Being Told)

Lesson	How It's Taught
Portals connect two places	You see yourself through the portal - it's showing you from another angle
You can walk through portals	The only exit is through the portal. No other option.
Portals preserve orientation	Walking through doesn't disorient you. You exit the way you entered.
The blue portal exists too	After exiting, you see the blue portal on the chamber wall
Orange and blue are linked	Looking back through blue, you see where you came from

What's NOT In Level One

Critically, the first chamber does NOT include:

- Text instructions ("Press E to interact")
- Button prompts on screen
- A portal gun (you don't get it until later)
- Multiple paths or choices
- Enemies or hazards
- Time pressure (after the initial countdown)

The level is stripped to essentials. One concept. One path. No distractions.

The 4 A's Perspective

- **Action:** Minimal - walk, look. The simplicity focuses attention on the portal.
- **Art:** Clean, clinical environment. The orange portal glows against sterile white. Visual hierarchy is unmistakable.
- **Arc:** Brief. Maybe 30 seconds for most players. Long enough to understand, short enough to maintain curiosity.
- **Atmosphere:** Curious and slightly unsettling. You're being watched. Tested. The vibe is established immediately.

Try This

Time yourself

How long does it take a first-time player to complete Level One? Watch someone who's never played. Where do they pause? What do they examine?

Reverse engineer it

List everything Level One teaches. Now list what it deliberately excludes. What gets introduced in Level Two? Level Three?

Why Use This Case Study

Portal's first chamber is the canonical example of implicit tutorial design. Valve documented their process extensively in developer commentary, making it perfect for teaching:

- The problem: How do you teach a mechanic that doesn't exist in other videogames?
- The solution: Constrain the space until only the desired action is possible
- The principle: Show, don't tell. Better yet, make the player do.

Teaching Sequence

1. Have students play through first three chambers blind
2. Discuss: What did you learn? When did you learn it?
3. Replay with developer commentary enabled
4. Map each chamber to its teaching goal
5. Design exercise: Create a tutorial for a novel mechanic using only space and constraints

The Valve Method

Valve's tutorial philosophy, refined across *Half-Life*, *Portal*, and *Left 4 Dead*:

1. **Isolate the concept** - Remove everything except what you're teaching
2. **Constrain the space** - Make the correct action the only obvious action
3. **Let them discover** - Player agency feels better than being told
4. **Repeat with variation** - Same concept, different context, increasing complexity

Level One exemplifies all four. The glass box isolates. The single exit constrains. Walking through is discovery. Later chambers repeat with variation.

Discussion Prompts

- Why does Level One let you see yourself through the portal? What would change if you couldn't?

- What's the purpose of the countdown timer?
- Why include the radio and toilet? (Hint: it's about making the space feel real)
- How does *Portal*'s implicit approach compare to explicit tutorial text?

Common Student Mistakes

Over-explaining: Students often want to add text "just to be safe." Push them to trust spatial design.

Adding too much: The temptation is to teach multiple things at once. *Portal*'s discipline is one thing per chamber.

Forgetting repetition: One exposure isn't enough. Players need to apply concepts multiple times with variation.

Developer Commentary

Portal includes built-in developer commentary. Enable it for educational use - Valve's designers explain their decisions at each moment. It's a free masterclass.

Implicit vs Explicit Instruction

Tutorial design falls on a spectrum:

Implicit	Explicit
Player discovers	Videogame explains
Spatial constraints	Text/voice instructions
Feels like playing	Feels like learning
Easy to miss	Hard to miss
Memorable through doing	Memorable through reading

Neither is universally better. *Portal* uses implicit instruction because its mechanic is spatial - the best way to understand portals is to walk through one.

Affordance and Constraint

Level One works through **constraint** more than **affordance**:

- **Affordance:** The portal looks like something you can walk through
- **Constraint:** The glass box prevents any other action

The affordance invites. The constraint requires. Together, they teach.

The Permissions Angle

From a Permissions perspective:

- **Allowed:** Walk, look, exist in the space
- **Required:** Walk through the portal to progress
- **Forbidden:** Everything else (glass walls, no other exits)

By narrowing permissions, Level One makes the desired action inevitable. This is **permission design as tutorial**.

Aesthetic Heritage: The Test Chamber

The test chamber format draws from scientific experimentation aesthetics - clean, controlled, observable. This serves multiple purposes:

- Justifies the artificiality of isolated teaching spaces
- Establishes the Aperture Science fiction
- Creates expectation for increasing complexity (you're being tested)
- Makes the behind-the-scenes sections more impactful by contrast

The [Aesthetic Heritage](#) of the laboratory isn't just visual - it's structural.

Unresolved Questions

- How much does implicit instruction rely on player patience? What about players who want to "get to the good part"?
- Can implicit instruction work for abstract or non-spatial mechanics?
- Is there a genre bias in implicit design? (Puzzle videogames expect it; action videogames often don't)
- What's lost when players skip tutorial content? Is that their right?

Related

- [Level Design](#) - spatial teaching principles
- [Permissions](#) - how constraints create meaning
- [Player Psychology](#) - learning and discovery

Undertale: Genocide Route

*Undertale's Genocide Route is what happens when a videogame says "you can do this" and means it as a warning, not an invitation. It's a case study in Permissions that carry weight - where what's **allowed** becomes deeply uncomfortable.*



Long ago, two races
ruled over Earth:
HUMANS and MONSTERS.

Undertale - *The aftermath of choice, what the videogame allows and what it costs.*
Courtesy of Toby Fox

The Three Routes

Undertale has three primary paths:

Route	How to Trigger	Tone
Neutral	Default - kill some, spare some	Bittersweet
Pacifist	Kill nobody, befriend everyone	Hopeful, earned happiness
Genocide	Kill everything. Hunt until areas are empty.	Horror, emptiness

What Genocide Requires

The Genocide Route isn't just "kill everyone you meet." It requires:

- **Active hunting:** You must seek out every enemy in every area
- **Grinding:** Random encounters don't stop until you've killed everyone
- **Persistence:** The videogame tells you how many are left ("But nobody came.")
- **No exceptions:** Sparing anyone ends the route

This is tedious by design. You can't stumble into Genocide. You have to want it.

What Changes

On the Genocide Route:

- Music becomes slower, darker, eventually silent
- NPCs flee or hide from you
- Save points change text ("Determination." becomes numbers)
- Boss fights are replaced with one-sided slaughters - or nightmare difficulty
- The final boss is one of the hardest fights in any videogame
- Dialogue reflects what you've become

The Permanent Consequence

Completing Genocide permanently alters your save data. Even after a full reset, the game "remembers." Future Pacifist endings are tainted. The videogame sold your soul - and the transaction persists outside any in-game reset.

This consequence extends to your actual computer's save files, not just in-game state.

The 4 A's Perspective

- **Action:** Combat becomes mandatory grinding. The [Gesture](#) shifts from choice-based encounters to repetitive elimination.
- **Art:** Visual and audio design degrades. Music slows. Colors drain. The videogame becomes hostile to look at.
- **Arc:** Much longer than other routes due to required grinding. Intentionally tedious.
- **Atmosphere:** Horror. Not jump-scare horror - existential horror. You're the monster.

Try This

Watch, don't play

The Genocide Route is designed to be unpleasant to complete. Consider watching a playthrough first. Notice how the videogame makes you work for every kill.

Compare the fights

Watch the Papyrus fight on Neutral/Pacifist vs. Genocide. Same character, same location. Radically different experience.

Why Use This Case Study

The Genocide Route is the clearest example of **Permissions as moral architecture**. Most videogames that let you be evil either:

- Reward it (power fantasy, more loot)
- Ignore it (NPCs respawn, no consequences)
- Prevent it (invincible children, locked-out options)

Undertale does something different: it **allows** evil, but makes you earn it through labor, and makes the consequences permanent. This is permissions design at its most deliberate.

Teaching Sequence

1. Have students play *Undertale* blind (most will get Neutral)
2. Discuss the Mercy mechanic and what it represents
3. Introduce the existence of the Genocide Route - without spoiling details
4. Watch key Genocide moments together (boss fights, ending)
5. Discuss: Why did Toby Fox include this? What does allowing it say?

Discussion Prompts

- Why is Genocide tedious? Is that a design flaw or intentional?
- What's the difference between forbidding evil actions and allowing them with consequences?
- How does the permanent save alteration change the stakes?
- Can a videogame judge the player? Should it?
- What does it mean that most players who start Genocide don't finish it?

The Labor of Evil

Genocide requires grinding - walking around empty areas waiting for encounters that become less frequent. This is intentional friction:

- You can't kill casually - you must commit
- The videogame gives you time to reconsider
- Boredom becomes part of the horror
- The effort involved makes completion a choice, not an accident

This is the opposite of how most videogames handle player agency. Usually, evil is easy. Here, evil is work.

Common Misconceptions

"The Genocide Route is the 'true' or 'complete' experience"

It's one of three routes. Many players (and the creator) consider Pacifist the intended "good" ending. Genocide exists to make a point, not to be played by everyone.

"You have to play Genocide to understand Undertale"

You don't. The route exists for those who choose it. Watching is valid. Not engaging is valid. The videogame respects that choice.

Permissions as Moral Framework

From a Permissions perspective, the Genocide Route reveals that what a videogame **allows** carries meaning:

- **Allowed:** Killing everything. The videogame doesn't stop you.
- **Required:** Active hunting. Passive killing won't trigger the route.
- **Forbidden:** Mercy. Any mercy ends Genocide.

The permission to commit genocide exists specifically so the videogame can respond to it. By allowing it, the videogame can judge it.

Complicity and Player Agency

Undertale makes the player complicit:

- You weren't forced - you chose
- You weren't tricked - the videogame warned you
- You weren't efficient - you worked for this

This complicity is what makes the permanent consequences feel earned rather than arbitrary. The videogame didn't punish you - you punished yourself.

The Fourth Wall Break

Genocide breaks the fourth wall by:

- Acknowledging your save/load behavior
- Persisting consequences in actual files
- Addressing "you" directly, not the character
- Making the final boss explicitly aware of meta-game behavior

This transforms Genocide from "playing a villain" to "being judged for playing." The locus of moral evaluation shifts from character to player.

Aesthetic Heritage: RPG Violence

The Genocide Route critiques RPG conventions:

- Grinding is standard in JRPGs - here it's horrifying
- Killing for XP is normal - here it's examined
- Power growth is rewarding - here it's corrupting
- The "kill everything" player is a common archetype - here, that's the villain

Undertale transforms its [Aesthetic Heritage](#) from celebration to critique. It uses familiar mechanics to ask: why do we do this?

The Sans Fight

Sans, the final Genocide boss, is one of the hardest fights in any videogame. This serves multiple purposes:

- **Mechanical punishment:** You wanted power? Earn it.
- **Narrative coherence:** Someone finally stops you
- **Player psychology:** Most Genocide runs end here
- **Meaning:** The videogame is literally fighting back

The difficulty isn't arbitrary - it's the videogame's last attempt to stop you.

Unresolved Questions

- Is the permanent save alteration a violation of player trust, or the ultimate consequence?
- Can a videogame ethically judge players for engaging with content it provides?
- Does watching Genocide (via YouTube) carry the same moral weight as playing it?
- What does it mean that Genocide is the most-discussed route despite being least-played?

Related

- [Permissions](#) - the framework this case study exemplifies
- [Dark Souls: Bonfire](#) - another videogame where difficulty carries meaning
- [Player Psychology](#) - motivation and moral reasoning

PART VIII

Code Bank

Small, readable implementations of the mechanics discussed throughout the text.

Code Bank

*A library of **Unity script scaffolds** for recreating and experimenting with videogame gestures. Each scaffold exposes the variables that matter for feel without requiring you to write code from scratch.*

The goal: **tune, don't code**. You're learning to shape gestures, not learning to program. The scaffolds handle the engineering; you handle the design.

Multi-perspective support: Each scaffold includes variants for **2D**, **3D Third-Person**, **3D First-Person**, and **Isometric** perspectives where applicable. All scripts use the same [InputReader pattern](#) for seamless gamepad and keyboard+mouse support.

How to Use

1. **Find the scaffold** that matches the gesture you're recreating
2. **Drop it into Unity** (drag the prefab or attach the script)
3. **Tune the exposed variables** in the Inspector until it *feels* right
4. **Document what you changed** and why - this is the learning

Every variable has a tooltip explaining what it does and what changing it affects.

Available Scaffolds

Setup & Foundations

Scaffold	What it does	Key components
Input Setup	Unified input system for all perspectives	InputReader , New Input System, device detection

Movement

Scaffold	What it does	Key variables
Basic Jump	Single jump with tunable arc	<code>jumpForce</code> , <code>gravity</code> , <code>fallGravityMultiplier</code>
Variable Jump	Jump height based on button hold	<code>minJumpForce</code> , <code>maxJumpForce</code> , <code>holdTime</code>
Double Jump	Extra jump in mid-air	<code>extraJumps</code> , <code>airJumpForce</code> , <code>resetVelocity</code>
Coyote Time	Grace period after leaving ledge	<code>coyoteTime</code>
Dash	Burst movement with recovery	<code>dashSpeed</code> , <code>dashDuration</code> , <code>cooldown</code>

Feedback & Feel

Scaffold	What it does	Key variables
Screen Shake	Camera shake on impact	<code>intensity</code> , <code>duration</code> , <code>decay</code>
Hitstop	Freeze frame on hit	<code>hitstopDuration</code> , <code>timeScale</code>
Squash & Stretch	Deform sprite based on motion	<code>stretchAmount</code> , <code>squashAmount</code> , <code>returnSpeed</code>
Knockback	Push away on hit	<code>knockbackForce</code> , <code>knockbackUp</code> , <code>hitstunDuration</code>

Timing & Input

Scaffold	What it does	Key variables
Input Buffer	Remember inputs for grace window	<code>bufferTime</code>

Scaffolds In Development

Future scaffolds will include:

- **Movement:** Ground Movement, Air Control, Wall Jump
- **Feedback:** Trail Effect, Landing Impact
- **Camera:** Follow Camera, Camera Bounds, Focus Point
- **Timing:** Cooldown Timer, Charge Mechanic
- **Combat:** Melee Attack, Health System

Starting Points by Exercise

Gesture Recreation (Week 2)

Start with: [Basic Jump](#)

Add: Screen Shake for landing feedback

Heritage Translation (Week 3)

Start with: the scaffold closest to your source gesture

Modify: tune until it matches, then deliberately transform one variable set

The 30-Minute Action (Week 5)

Combine: [Dash](#) + [Screen Shake](#)

Constraint: only tune what's exposed, no new code

Feel Experiment (Week 7)

Pick any scaffold. Change one variable dramatically. Document:

- What changed in the feel?
- What *didn't* change that you expected to?
- What does this tell you about how the components interact?

Why the Code Bank Exists

The Code Bank exists because **code is a barrier to feel exploration**. Students who have to write their own jump code spend all their time debugging, not designing. The scaffolds remove that barrier.

Key Principles

Expose the design variables, hide the engineering variables. Students don't need to know how the physics integration works; they need to know what `hangTime` does to the jump arc.

Tooltip discipline: Every exposed variable has a tooltip. This is non-negotiable. Students should be able to hover and understand without leaving Unity.

Scaffold Philosophy

- **Modular:** Each scaffold does one thing. Combine them for complex behaviors.
- **Readable:** Variable names should be self-explanatory.
- **Tunable:** Default values should produce a "neutral" feel. Students move from there.

Common Issue

Students want to dive into the code. Redirect: "First get it feeling right by tuning. *Then* you can look under the hood." The sequence matters - feel before implementation.

Assessment

Can they articulate *why* they changed a variable, not just *that* they changed it? "I increased coyote time because the original felt punishing" is good. "I set it to 0.15" is insufficient.

Code as Material

The Code Bank embodies the idea that **code is material, not prerequisite**.

Traditional videogame education often teaches programming first, design second. Students spend semesters learning to make things move before they can ask whether the movement *feels* right. This gets it backwards. Designers need to develop feel intuition early, even if they can't yet implement from scratch.

The scaffolds are like **woodworking jigs** - tools that let you focus on the craft decision (the cut, the joint) without rebuilding the saw each time. Eventually you learn how the saw works, but not before you've made some furniture.

What This Approach Resists

The "**real programmers write from scratch**" **mentality** that pervades CS-adjacent education. Using scaffolds isn't cheating; it's working at the right level of abstraction for the learning goal.

The **black-box tool critique** - yes, scaffolds hide complexity. That's the point. You can always open the box later. But if you never get to the design layer because you're stuck in the implementation layer, the box might as well not exist.

Unresolved Tension

When do students graduate from scaffolds to raw implementation? We don't have a clean answer. Current approach: scaffolds for the intro course, increasing implementation in intermediate courses, full implementation in advanced. But some students never need full implementation - they're designers, not engineers. Is that okay? We think yes, but the industry often disagrees.

Contributing

The Code Bank is a living library. If you build a scaffold that others might use:

1. Follow the naming conventions

2. Add tooltips to every exposed variable
3. Set sensible defaults
4. Document in the same format as existing scaffolds
5. Tag with relevant concepts

Related

- [Gesture](#) - what you're recreating with these scaffolds
- [Aesthetic Heritage](#) - using scaffolds to trace and transform inherited gestures
- [Code as Material](#) - the philosophy behind this approach

Input Setup

*The foundation for all Code Bank scripts. Set this up once, and all other scaffolds will work with **both gamepad and keyboard+mouse** automatically.*

Do this first! All Code Bank scripts expect this input setup. Complete this page before using any other scaffold.

Step 1: Create the Input Actions Asset

1. In Unity: **Assets** → **Create** → **Input Actions**
2. Name it `GameInputActions`
3. Double-click to open the Input Actions editor

Step 2: Create Action Maps

We'll create two action maps: **Gameplay** (movement, actions) and **UI** (menus).

1. Click + next to "Action Maps"
2. Name it `Gameplay`
3. Click + again, name it `UI`

Step 3: Add Actions to Gameplay Map

Select the **Gameplay** action map, then add these actions:

Action Name	Action Type	Control Type	Purpose
<code>Move</code>	Value	Vector2	WASD / Left Stick movement
<code>Look</code>	Value	Vector2	Mouse / Right Stick camera
<code>Jump</code>	Button	-	Space / South Button (A/X)
<code>Dash</code>	Button	-	Shift / Right Trigger
<code>Attack</code>	Button	-	Left Click / West Button (X/Square)
<code>Interact</code>	Button	-	E / East Button (B/Circle)

Step 4: Add Bindings

For each action, add bindings for keyboard+mouse AND gamepad:

Move Action Bindings:

1. Select `Move` action
2. Click + → Add 2D Vector Composite → name it "WASD"
3. Bind: Up=W, Down=S, Left=A, Right=D
4. Click + → Add Binding → path: `Gamepad/leftStick`

Look Action Bindings:

1. Select `Look` action
2. Click + → Add Binding → path: `Mouse/delta`
3. Click + → Add Binding → path: `Gamepad/rightStick`

Jump Action Bindings:

1. Select `Jump` action
2. Click + → Add Binding → path: `Keyboard/space`
3. Click + → Add Binding → path: `Gamepad/buttonSouth`

Dash Action Bindings:

1. Select `Dash` action
2. Click + → Add Binding → path: `Keyboard/leftShift`
3. Click + → Add Binding → path: `Gamepad/rightTrigger`

Attack Action Bindings:

1. Select `Attack` action
2. Click + → Add Binding → path: `Mouse/leftButton`
3. Click + → Add Binding → path: `Gamepad/buttonWest`

Interact Action Bindings:

1. Select `Interact` action
2. Click + → Add Binding → path: `Keyboard/e`
3. Click + → Add Binding → path: `Gamepad/buttonEast`

Step 5: Save and Generate C# Class

1. Click **Save Asset** in the Input Actions window
2. Select your `GameInputActions` asset in the Project window
3. In the Inspector, check **Generate C# Class**
4. Click **Apply**

Unity generates `GameInputActions.cs` - you won't edit this directly.

Step 6: Create the InputReader Script

This is the **bridge** between Unity's Input System and your gameplay scripts. All Code Bank scripts read from this.

Create a new C# script called `InputReader.cs` :

▼ InputReader.cs

```

// =====
// InputReader.cs
// Central input hub - all gameplay scripts read from this.
// Handles gamepad, keyboard, and mouse seamlessly.
// Unity 6.x + New Input System
// =====

using UnityEngine;
using UnityEngine.InputSystem;
using System;

/// <summary>
/// Reads input from the New Input System and exposes it
/// in a clean, device-agnostic way. Attach to a GameObject
/// and reference from other scripts.
/// </summary>
public class InputReader : MonoBehaviour
{
    // =====
    // SINGLETON (optional - convenient for prototyping)
    // =====

    public static InputReader Instance { get; private set; }

    [Header("Settings")]
    [Tooltip("Use singleton pattern for easy global access")]
    [SerializeField] private bool useSingleton = true;

    [Tooltip("Mouse look sensitivity multiplier")]
    [SerializeField] [Range(0.1f, 5f)] private float mouseSensitivity =
1f;

    [Tooltip("Gamepad look sensitivity multiplier")]
    [SerializeField] [Range(0.1f, 10f)] private float
gamepadLookSensitivity = 3f;

```

```
// =====  
// INPUT VALUES - Read these from other scripts  
// =====  
  
/// <summary>  
/// Movement input as Vector2 (-1 to 1 on each axis)  
/// WASD or Left Stick  
/// </summary>  
public Vector2 MoveInput { get; private set; }  
  
/// <summary>  
/// Look/camera input as Vector2  
/// Mouse delta or Right Stick (sensitivity-adjusted)  
/// </summary>  
public Vector2 LookInput { get; private set; }  
  
/// <summary>  
/// True while jump button is held  
/// </summary>  
public bool JumpHeld { get; private set; }  
  
/// <summary>  
/// True while dash button is held  
/// </summary>  
public bool DashHeld { get; private set; }  
  
/// <summary>  
/// True while attack button is held  
/// </summary>  
public bool AttackHeld { get; private set; }  
  
/// <summary>  
/// True while interact button is held  
/// </summary>  
public bool InteractHeld { get; private set; }  
  
// =====
```

```

// EVENTS - Subscribe for button press/release
// =====

/// <summary>Fired when jump is pressed</summary>
public event Action OnJumpPressed;

/// <summary>Fired when jump is released</summary>
public event Action OnJumpReleased;

/// <summary>Fired when dash is pressed</summary>
public event Action OnDashPressed;

/// <summary>Fired when dash is released</summary>
public event Action OnDashReleased;

/// <summary>Fired when attack is pressed</summary>
public event Action OnAttackPressed;

/// <summary>Fired when attack is released</summary>
public event Action OnAttackReleased;

/// <summary>Fired when interact is pressed</summary>
public event Action OnInteractPressed;

// =====
// INTERNAL
// =====

private GameInputActions inputActions;
private bool isGamepad = false;

// =====
// UNITY LIFECYCLE
// =====

void Awake()
{

```

```
// Singleton setup
if (useSingleton)
{
    if (Instance != null && Instance != this)
    {
        Destroy(gameObject);
        return;
    }
    Instance = this;
    DontDestroyOnLoad(gameObject);
}

// Create input actions
inputActions = new GameInputActions();
}

void OnEnable()
{
    inputActions.Gameplay.Enable();

    // Subscribe to actions
    inputActions.Gameplay.Move.performed += OnMove;
    inputActions.Gameplay.Move.canceled += OnMove;

    inputActions.Gameplay.Look.performed += OnLook;
    inputActions.Gameplay.Look.canceled += OnLook;

    inputActions.Gameplay.Jump.performed += OnJump;
    inputActions.Gameplay.Jump.canceled += OnJumpCancel;

    inputActions.Gameplay.Dash.performed += OnDash;
    inputActions.Gameplay.Dash.canceled += OnDashCancel;

    inputActions.Gameplay.Attack.performed += OnAttack;
    inputActions.Gameplay.Attack.canceled += OnAttackCancel;

    inputActions.Gameplay.Interact.performed += OnInteract;
```

```

        // Track device changes
        InputSystem.onActionChange += OnActionChange;
    }

    void OnDisable()
    {
        inputActions.Gameplay.Disable();

        inputActions.Gameplay.Move.performed -= OnMove;
        inputActions.Gameplay.Move.canceled -= OnMove;

        inputActions.Gameplay.Look.performed -= OnLook;
        inputActions.Gameplay.Look.canceled -= OnLook;

        inputActions.Gameplay.Jump.performed -= OnJump;
        inputActions.Gameplay.Jump.canceled -= OnJumpCancel;

        inputActions.Gameplay.Dash.performed -= OnDash;
        inputActions.Gameplay.Dash.canceled -= OnDashCancel;

        inputActions.Gameplay.Attack.performed -= OnAttack;
        inputActions.Gameplay.Attack.canceled -= OnAttackCancel;

        inputActions.Gameplay.Interact.performed -= OnInteract;

        InputSystem.onActionChange -= OnActionChange;
    }

    // =====
    // INPUT HANDLERS
    // =====

    private void OnMove(InputAction.CallbackContext ctx)
    {
        MoveInput = ctx.ReadValue<Vector2>();
    }

```

```
private void OnLook(InputAction.CallbackContext ctx)
{
    Vector2 raw = ctx.ReadValue<Vector2>();

    // Apply appropriate sensitivity based on device
    float sensitivity = isGamepad ? gamepadLookSensitivity :
mouseSensitivity;
    LookInput = raw * sensitivity;
}

private void OnJump(InputAction.CallbackContext ctx)
{
    JumpHeld = true;
    OnJumpPressed?.Invoke();
}

private void OnJumpCancel(InputAction.CallbackContext ctx)
{
    JumpHeld = false;
    OnJumpReleased?.Invoke();
}

private void OnDash(InputAction.CallbackContext ctx)
{
    DashHeld = true;
    OnDashPressed?.Invoke();
}

private void OnDashCancel(InputAction.CallbackContext ctx)
{
    DashHeld = false;
    OnDashReleased?.Invoke();
}

private void OnAttack(InputAction.CallbackContext ctx)
{

```

```

        AttackHeld = true;
        OnAttackPressed?.Invoke();
    }

    private void OnAttackCancel(InputAction.CallbackContext ctx)
    {
        AttackHeld = false;
        OnAttackReleased?.Invoke();
    }

    private void OnInteract(InputAction.CallbackContext ctx)
    {
        InteractHeld = true;
        OnInteractPressed?.Invoke();
        // Auto-release interact after one frame
        Invoke(nameof(ReleaseInteract), 0.1f);
    }

    private void ReleaseInteract()
    {
        InteractHeld = false;
    }

    // =====
    // DEVICE DETECTION
    // =====

    private void OnActionChange(object obj, InputActionChange change)
    {
        if (change == InputActionChange.ActionPerformed)
        {
            var action = obj as InputAction;
            if (action?.activeControl?.device != null)
            {
                isGamepad = action.activeControl.device is Gamepad;
            }
        }
    }

```



```

    }

    /// <summary>
    /// Returns true if current input device is a gamepad
    /// </summary>
    public bool IsUsingGamepad => isGamepad;

    // =====
    // PUBLIC METHODS
    // =====

    /// <summary>
    /// Enable gameplay input (call when unpausing)
    /// </summary>
    public void EnableGameplayInput()
    {
        inputActions.Gameplay.Enable();
    }

    /// <summary>
    /// Disable gameplay input (call when pausing/in menus)
    /// </summary>
    public void DisableGameplayInput()
    {
        inputActions.Gameplay.Disable();
        MoveInput = Vector2.zero;
        LookInput = Vector2.zero;
    }
}

// =====
// USAGE:
//
// 1. Create empty GameObject, add InputReader component
// 2. Other scripts read input like this:
//
//     // Option A: Using singleton (quick prototyping)

```

```
// Vector2 move = InputReader.Instance.MoveInput;
// if (InputReader.Instance.JumpHeld) { ... }
//
// // Option B: Using reference (better for production)
// [SerializeField] private InputReader input;
// Vector2 move = input.MoveInput;
//
// // Option C: Subscribe to events
// void OnEnable() {
//     InputReader.Instance.OnJumpPressed += HandleJump;
// }
// void OnDisable() {
//     InputReader.Instance.OnJumpPressed -= HandleJump;
// }
//
// 3. Works automatically with both keyboard+mouse and gamepad!
// =====
```

Step 7: Add InputReader to Your Scene

1. Create empty GameObject: **GameObject** → **Create Empty**
2. Name it `InputReader`
3. Add the `InputReader` component
4. It will persist across scenes if `useSingleton` is checked

Done! All Code Bank scripts can now read from `InputReader.Instance` and will automatically work with gamepad or keyboard+mouse.

Quick Reference: Reading Input

▼ Quick Reference

```
// Movement (WASD or Left Stick)
Vector2 moveInput = InputReader.Instance.MoveInput;
float horizontal = moveInput.x; // -1 to 1
float vertical = moveInput.y;   // -1 to 1

// Camera (Mouse or Right Stick)
Vector2 lookInput = InputReader.Instance.LookInput;

// Buttons - check if held
if (InputReader.Instance.JumpHeld) { }
if (InputReader.Instance.DashHeld) { }
if (InputReader.Instance.AttackHeld) { }

// Buttons - subscribe to press events
InputReader.Instance.OnJumpPressed += MyJumpHandler;
InputReader.Instance.OnJumpReleased += MyJumpReleaseHandler;
```

Perspective-Specific Notes

2D Platformer

- Use `MoveInput.x` for horizontal movement
- Ignore `MoveInput.y` (or use for ladder climbing)
- `LookInput` typically unused

3D Third-Person

- Use `MoveInput` relative to camera forward
- Use `LookInput` to orbit camera around player

3D First-Person

- Use `MoveInput` relative to player forward
- Use `LookInput` to rotate player view directly

- Consider locking and hiding cursor

Isometric / Top-Down

- Use `MoveInput` directly as world-space direction
- `LookInput` can aim independently of movement

Why This Architecture

The InputReader pattern serves several pedagogical goals:

- **Device-agnostic:** Students don't write separate gamepad/keyboard code
- **Single source of truth:** All input flows through one place
- **Event-driven option:** Supports both polling and event patterns
- **Easily mockable:** Can simulate input for testing

Common Student Issues

"My input isn't working"

1. Check InputReader exists in scene
2. Check GameInputActions asset has bindings saved
3. Check "Generate C# Class" is enabled and applied
4. Check Gameplay action map is enabled

"Gamepad works but keyboard doesn't (or vice versa)"

Check bindings in the Input Actions asset. Each action needs BOTH keyboard and gamepad bindings.

Extending for New Actions

To add a new action (e.g., "Crouch"):

1. Add action to GameInputActions asset
2. Add bindings for both keyboard and gamepad
3. Save and Apply (regenerates C# class)
4. Add property and handler in InputReader.cs

Input Abstraction Philosophy

The gap between "player intent" and "hardware signal" is larger than it seems:

- **Hardware:** "Left stick at (0.7, 0.3)"
- **Intent:** "Move forward-right at moderate speed"

Good input architecture bridges this gap cleanly. The InputReader transforms hardware signals into semantic intent that gameplay code can understand without caring about the source device.

Why Events AND Polling

Different systems need input differently:

- **Polling** (reading MoveInput each frame): Movement, aiming - continuous actions
- **Events** (OnJumpPressed): Jump, attack - discrete actions with timing that matters

Offering both lets each system use the appropriate pattern.

The Sensitivity Problem

Mouse delta and stick position are fundamentally different:

- **Mouse:** "Moved 50 pixels since last frame" (velocity)
- **Stick:** "Currently at 0.8 right" (position)

The sensitivity multipliers help normalize these, but perfect parity is impossible. This is why many videogames have separate mouse and gamepad sensitivity settings - and why we expose both in the InputReader.

Related

- [Basic Jump](#) - uses this input setup
- [Dash](#) - uses this input setup
- [Code Bank Overview](#)

Basic Jump

A simple, fixed-height jump. Press button, go up, come down. The foundation of character movement across all perspectives.

The Feel

Basic jump is **committed and consistent**. Same height every time. The player knows exactly what they'll get.

Compare to [Variable Jump](#): less expressive, but more predictable. Some videogames want that predictability.

Exposed Variables

Variable	Type	Default	What it does
<code>jumpForce</code>	float	12.0	Initial upward velocity
<code>gravity</code>	float	40.0	Downward acceleration
<code>fallGravityMultiplier</code>	float	1.5	Extra gravity when falling
<code>groundCheckRadius</code>	float	0.2	Size of ground detection sphere/circle

Tuning Guide

For a floaty feel:

- Lower `gravity` (20-30)
- Lower `fallGravityMultiplier` (1.0-1.2)
- Higher `jumpForce`

Reference: Kirby, early Mario

For a snappy feel:

- Higher `gravity` (50-70)
- Higher `fallGravityMultiplier` (2.0-3.0)
- Moderate `jumpForce`

Reference: Super Meat Boy, modern precision platformers

For a weighty feel:

- High `gravity`
- Low `jumpForce`
- Minimal `fallGravityMultiplier` (1.0)

Reference: Castlevania, simulation-leaning videogames

Perspective Scripts

Prerequisite: All scripts below require the [InputReader](#) to be set up in your scene. Complete the Input Setup first.

○○○○

2D Platformer 3D Third-Person 3D First-Person Isometric

BasicJump2D.cs

Classic 2D platformer jump. Uses `Rigidbody2D` and `Physics2D.OverlapCircle` for ground detection.

Setup:

1. Add `Rigidbody2D` to your player (Freeze Rotation Z)
2. Add a `Collider2D` (BoxCollider2D or CapsuleCollider2D)
3. Create an empty child GameObject at the feet, name it "GroundCheck"
4. Add this script and configure in Inspector

▼ BasicJump2D.cs

```

using UnityEngine;

/// <summary>
/// BASIC JUMP 2D - Fixed height jump for 2D platformers
/// VG101 Code Bank - Requires InputReader from Input Setup
/// </summary>
[RequireComponent(typeof(Rigidbody2D))]
public class BasicJump2D : MonoBehaviour
{
    [Header("Jump Feel")]
    [Tooltip("How hard we push off the ground. Higher = taller jump.")]
    [Range(5f, 25f)]
    [SerializeField] private float jumpForce = 12f;

    [Tooltip("How fast we fall. Higher = snappier, less floaty.")]
    [Range(20f, 80f)]
    [SerializeField] private float gravity = 40f;

    [Tooltip("Extra gravity when falling. Makes descent faster than ascent.")]
    [Range(1f, 4f)]
    [SerializeField] private float fallGravityMultiplier = 1.5f;

    [Header("Ground Detection")]
    [Tooltip("Empty GameObject at character's feet")]
    [SerializeField] private Transform groundCheckPoint;

    [Tooltip("Size of the ground check circle")]
    [Range(0.05f, 0.5f)]
    [SerializeField] private float groundCheckRadius = 0.2f;

    [Tooltip("Which layers count as ground")]
    [SerializeField] private LayerMask groundLayer;

    [Header("Debug")]
    [SerializeField] private bool showGroundCheck = true;

```

```
private Rigidbody2D rb;
private float verticalVelocity;
private bool isGrounded;

private void Awake()
{
    rb = GetComponent<Rigidbody2D>();
    rb.gravityScale = 0f; // We handle gravity ourselves
}

private void OnEnable()
{
    // Subscribe to InputReader jump event
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed += HandleJump;
    }
}

private void OnDisable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed -= HandleJump;
    }
}

private void FixedUpdate()
{
    CheckGrounded();
    ApplyGravity();
    ApplyVelocity();
}

private void CheckGrounded()
{

```

```

        isGrounded = Physics2D.OverlapCircle(
            groundCheckPoint.position,
            groundCheckRadius,
            groundLayer
        );

        if (isGrounded && verticalVelocity < 0)
        {
            verticalVelocity = 0f;
        }
    }

    private void ApplyGravity()
    {
        if (!isGrounded)
        {
            float gravityThisFrame = gravity;
            if (verticalVelocity < 0)
            {
                gravityThisFrame *= fallGravityMultiplier;
            }
            verticalVelocity -= gravityThisFrame * Time.fixedDeltaTime;
        }
    }

    private void HandleJump()
    {
        if (isGrounded)
        {
            verticalVelocity = jumpForce;
        }
    }

    private void ApplyVelocity()
    {
        Vector2 velocity = rb.linearVelocity;
        velocity.y = verticalVelocity;
    }

```

```
        rb.linearVelocity = velocity;
    }

    private void OnDrawGizmos()
    {
        if (showGroundCheck && groundCheckPoint != null)
        {
            Gizmos.color = isGrounded ? Color.green : Color.red;
            Gizmos.DrawWireSphere(groundCheckPoint.position,
groundCheckRadius);
        }
    }
}
```

BasicJump3D.cs

Third-person 3D jump. Uses `CharacterController` for smooth movement integration. Works with camera-relative movement.

Setup:

1. Add `CharacterController` to your player
2. Add this script and configure in Inspector
3. Set Ground Layer to your environment's layer

▼ BasicJump3DThirdPerson.cs


```

using UnityEngine;

/// <summary>
/// BASIC JUMP 3D - Fixed height jump for third-person 3D games
/// VG101 Code Bank - Requires InputReader from Input Setup
/// Uses CharacterController for movement integration
/// </summary>
[RequireComponent(typeof(CharacterController))]
public class BasicJump3D : MonoBehaviour
{
    [Header("Jump Feel")]
    [Tooltip("How hard we push off the ground. Higher = taller jump.")]
    [Range(5f, 25f)]
    [SerializeField] private float jumpForce = 10f;

    [Tooltip("How fast we fall. Higher = snappier, less floaty.")]
    [Range(20f, 80f)]
    [SerializeField] private float gravity = 30f;

    [Tooltip("Extra gravity when falling. Makes descent faster than ascent.")]
    [Range(1f, 4f)]
    [SerializeField] private float fallGravityMultiplier = 2f;

    [Header("Ground Detection")]
    [Tooltip("Offset from center for ground check sphere")]
    [SerializeField] private Vector3 groundCheckOffset = new Vector3(0, -0.9f, 0);

    [Tooltip("Size of the ground check sphere")]
    [Range(0.1f, 0.5f)]
    [SerializeField] private float groundCheckRadius = 0.3f;

    [Tooltip("Which layers count as ground")]
    [SerializeField] private LayerMask groundLayer;

```

```
[Header("Movement (Optional)")]
[Tooltip("Movement speed on ground")]
[SerializeField] private float moveSpeed = 8f;

[Tooltip("How much control in the air (0-1)")]
[Range(0f, 1f)]
[SerializeField] private float airControl = 0.5f;

[Header("Debug")]
[SerializeField] private bool showGroundCheck = true;

private CharacterController controller;
private Vector3 velocity;
private bool isGrounded;
private Transform cameraTransform;

private void Awake()
{
    controller = GetComponent<CharacterController>();
    cameraTransform = Camera.main?.transform;
}

private void OnEnable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed += HandleJump;
    }
}

private void OnDisable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed -= HandleJump;
    }
}
```

```

private void Update()
{
    CheckGrounded();
    HandleMovement();
    ApplyGravity();
    ApplyVelocity();
}

private void CheckGrounded()
{
    Vector3 checkPosition = transform.position + groundCheckOffset;
    isGrounded = Physics.CheckSphere(checkPosition,
groundCheckRadius, groundLayer);

    if (isGrounded && velocity.y < 0)
    {
        velocity.y = -2f; // Small downward force to stay grounded
    }
}

private void HandleMovement()
{
    Vector2 input = InputReader.Instance != null ?
InputReader.Instance.MoveInput : Vector2.zero;

    // Camera-relative movement
    Vector3 forward = cameraTransform != null ?
cameraTransform.forward : transform.forward;
    Vector3 right = cameraTransform != null ? cameraTransform.right :
transform.right;
    forward.y = 0f;
    right.y = 0f;
    forward.Normalize();
    right.Normalize();

    Vector3 moveDirection = (forward * input.y + right * input.x);

```

```

        float currentSpeed = moveSpeed;
        if (!isGrounded)
        {
            currentSpeed *= airControl;
        }

        velocity.x = moveDirection.x * currentSpeed;
        velocity.z = moveDirection.z * currentSpeed;

        // Rotate to face movement direction
        if (moveDirection.magnitude > 0.1f)
        {
            Quaternion targetRotation =
Quaternion.LookRotation(moveDirection);
            transform.rotation = Quaternion.Slerp(transform.rotation,
targetRotation, 10f * Time.deltaTime);
        }
    }

    private void ApplyGravity()
    {
        if (!isGrounded)
        {
            float gravityThisFrame = gravity;
            if (velocity.y < 0)
            {
                gravityThisFrame *= fallGravityMultiplier;
            }
            velocity.y -= gravityThisFrame * Time.deltaTime;
        }
    }

    private void HandleJump()
    {
        if (isGrounded)
        {

```

```

        velocity.y = jumpForce;
    }
}

private void ApplyVelocity()
{
    controller.Move(velocity * Time.deltaTime);
}

private void OnDrawGizmos()
{
    if (showGroundCheck)
    {
        Vector3 checkPosition = transform.position +
groundCheckOffset;
        Gizmos.color = isGrounded ? Color.green : Color.red;
        Gizmos.DrawWireSphere(checkPosition, groundCheckRadius);
    }
}
}

```

BasicJumpFPS.cs

First-person jump with classic FPS movement. Includes mouse look integration. The camera is the player's viewpoint.

Setup:

1. Add `CharacterController` to your player
2. Create a child Camera (or use Main Camera as child)
3. Add this script to the player root
4. Assign the Camera reference in Inspector

▼ BasicJumpFPS.cs

```

using UnityEngine;

/// <summary>
/// BASIC JUMP FPS - First-person jump with integrated mouselook
/// VG101 Code Bank - Requires InputReader from Input Setup
/// Complete FPS controller with jumping
/// </summary>
[RequireComponent(typeof(CharacterController))]
public class BasicJumpFPS : MonoBehaviour
{
    [Header("Jump Feel")]
    [Tooltip("How hard we push off the ground.")]
    [Range(5f, 25f)]
    [SerializeField] private float jumpForce = 8f;

    [Tooltip("How fast we fall.")]
    [Range(20f, 80f)]
    [SerializeField] private float gravity = 25f;

    [Tooltip("Extra gravity when falling.")]
    [Range(1f, 4f)]
    [SerializeField] private float fallGravityMultiplier = 2f;

    [Header("Ground Detection")]
    [SerializeField] private Vector3 groundCheckOffset = new Vector3(0,
-0.9f, 0);
    [SerializeField] private float groundCheckRadius = 0.3f;
    [SerializeField] private LayerMask groundLayer;

    [Header("Movement")]
    [Tooltip("Walking speed")]
    [SerializeField] private float moveSpeed = 6f;

    [Tooltip("Air control multiplier")]
    [Range(0f, 1f)]
    [SerializeField] private float airControl = 0.3f;

```

```
[Header("Mouse Look")]
[Tooltip("The camera to rotate for vertical look")]
[SerializeField] private Transform playerCamera;

[Tooltip("Mouse sensitivity for gamepad")]
[SerializeField] private float gamepadLookSpeed = 150f;

[Tooltip("Mouse sensitivity multiplier")]
[SerializeField] private float mouseSensitivity = 0.15f;

[Tooltip("Maximum up/down look angle")]
[Range(45f, 90f)]
[SerializeField] private float maxLookAngle = 85f;

[Header("Debug")]
[SerializeField] private bool showGroundCheck = true;

private CharacterController controller;
private Vector3 velocity;
private bool isGrounded;
private float cameraPitch;

private void Awake()
{
    controller = GetComponent<CharacterController>();

    // Lock and hide cursor for FPS
    Cursor.lockState = CursorLockMode.Locked;
    Cursor.visible = false;
}

private void OnEnable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed += HandleJump;
    }
}
```



```

    }
}

private void OnDisable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed -= HandleJump;
    }
}

private void Update()
{
    CheckGrounded();
    HandleLook();
    HandleMovement();
    ApplyGravity();
    ApplyVelocity();
}

private void CheckGrounded()
{
    Vector3 checkPosition = transform.position + groundCheckOffset;
    isGrounded = Physics.CheckSphere(checkPosition,
groundCheckRadius, groundLayer);

    if (isGrounded && velocity.y < 0)
    {
        velocity.y = -2f;
    }
}

private void HandleLook()
{
    if (InputReader.Instance == null) return;

    Vector2 lookInput = InputReader.Instance.LookInput;

```

```

// Apply different sensitivity for mouse vs gamepad
float lookSpeedX, lookSpeedY;
if (InputReader.Instance.IsUsingGamepad)
{
    lookSpeedX = gamepadLookSpeed * Time.deltaTime;
    lookSpeedY = gamepadLookSpeed * Time.deltaTime;
}
else
{
    // Mouse input is already deltaTime-independent
    lookSpeedX = mouseSensitivity;
    lookSpeedY = mouseSensitivity;
}

// Horizontal rotation - rotate the whole player
transform.Rotate(Vector3.up * lookInput.x * lookSpeedX);

// Vertical rotation - rotate just the camera
cameraPitch -= lookInput.y * lookSpeedY;
cameraPitch = Mathf.Clamp(cameraPitch, -maxLookAngle,
maxLookAngle);

if (playerCamera != null)
{
    playerCamera.localRotation = Quaternion.Euler(cameraPitch,
0f, 0f);
}

private void HandleMovement()
{
    Vector2 input = InputReader.Instance != null ?
InputReader.Instance.MoveInput : Vector2.zero;

    // Movement relative to player facing direction
    Vector3 moveDirection = transform.forward * input.y +

```

```
transform.right * input.x;

    float currentSpeed = moveSpeed;
    if (!isGrounded)
    {
        currentSpeed *= airControl;
    }

    velocity.x = moveDirection.x * currentSpeed;
    velocity.z = moveDirection.z * currentSpeed;
}

private void ApplyGravity()
{
    if (!isGrounded)
    {
        float gravityThisFrame = gravity;
        if (velocity.y < 0)
        {
            gravityThisFrame *= fallGravityMultiplier;
        }
        velocity.y -= gravityThisFrame * Time.deltaTime;
    }
}

private void HandleJump()
{
    if (isGrounded)
    {
        velocity.y = jumpForce;
    }
}

private void ApplyVelocity()
{
    controller.Move(velocity * Time.deltaTime);
}
```

```

private void OnDrawGizmos()
{
    if (showGroundCheck)
    {
        Vector3 checkPosition = transform.position +
groundCheckOffset;

        Gizmos.color = isGrounded ? Color.green : Color.red;
        Gizmos.DrawWireSphere(checkPosition, groundCheckRadius);
    }
}
}

```

BasicJumpIsometric.cs

Isometric/top-down jump. Movement is world-axis aligned (or rotated 45 degrees for true isometric). Jump adds vertical motion separate from XZ movement.

Setup:

1. Add `CharacterController` to your player
2. Set up isometric camera (typically 45-degree Y rotation, 30-45 degree X tilt)
3. Add this script and configure in Inspector
4. Choose rotation mode: World Axis or Isometric (45-degree rotated)

▼ BasicJumpIsometric.cs

```

using UnityEngine;

/// <summary>
/// BASIC JUMP ISOMETRIC - Jump for isometric/top-down games
/// VG101 Code Bank - Requires InputReader from Input Setup
/// Movement on XZ plane with jump on Y axis
/// </summary>
[RequireComponent(typeof(CharacterController))]
public class BasicJumpIsometric : MonoBehaviour
{
    public enum MovementMode
    {
        WorldAxis,      // Up = +Z, Right = +X (standard top-down)
        Isometric45      // Up = +Z+X, Right = +X-Z (true isometric)
    }

    [Header("Jump Feel")]
    [Tooltip("How hard we push off the ground.")]
    [Range(5f, 25f)]
    [SerializeField] private float jumpForce = 10f;

    [Tooltip("How fast we fall.")]
    [Range(20f, 80f)]
    [SerializeField] private float gravity = 35f;

    [Tooltip("Extra gravity when falling.")]
    [Range(1f, 4f)]
    [SerializeField] private float fallGravityMultiplier = 1.5f;

    [Header("Ground Detection")]
    [SerializeField] private Vector3 groundCheckOffset = new Vector3(0,
-0.5f, 0);
    [SerializeField] private float groundCheckRadius = 0.3f;
    [SerializeField] private LayerMask groundLayer;

    [Header("Movement")]

```

```

[Tooltip("How movement input maps to world space")]
[SerializeField] private MovementMode movementMode =
MovementMode.Isometric45;

[Tooltip("Movement speed on ground")]
[SerializeField] private float moveSpeed = 7f;

[Tooltip("Air control multiplier")]
[Range(0f, 1f)]
[SerializeField] private float airControl = 0.5f;

[Tooltip("Rotate character to face movement direction")]
[SerializeField] private bool rotateToMovement = true;

[Tooltip("How fast character rotates to face movement")]
[SerializeField] private float rotationSpeed = 720f;

[Header("Debug")]
[SerializeField] private bool showGroundCheck = true;

private CharacterController controller;
private Vector3 velocity;
private bool isGrounded;

// Isometric conversion matrix (45-degree rotation)
private static readonly Matrix4x4 isoMatrix =
Matrix4x4.Rotate(Quaternion.Euler(0, 45, 0));

private void Awake()
{
    controller = GetComponent<CharacterController>();
}

private void OnEnable()
{
    if (InputReader.Instance != null)
    {

```

```

        InputReader.Instance.OnJumpPressed += HandleJump;
    }
}

private void OnDisable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed -= HandleJump;
    }
}

private void Update()
{
    CheckGrounded();
    HandleMovement();
    ApplyGravity();
    ApplyVelocity();
}

private void CheckGrounded()
{
    Vector3 checkPosition = transform.position + groundCheckOffset;
    isGrounded = Physics.CheckSphere(checkPosition,
groundCheckRadius, groundLayer);

    if (isGrounded && velocity.y < 0)
    {
        velocity.y = -2f;
    }
}

private void HandleMovement()
{
    Vector2 input = InputReader.Instance != null ?
InputReader.Instance.MoveInput : Vector2.zero;

```



```

// Convert 2D input to 3D movement direction
Vector3 inputDirection = new Vector3(input.x, 0, input.y);

// Apply isometric rotation if needed
Vector3 moveDirection;
if (movementMode == MovementMode.Isometric45)
{
    moveDirection = isoMatrix.MultiplyVector(inputDirection);
}
else
{
    moveDirection = inputDirection;
}

float currentSpeed = moveSpeed;
if (!isGrounded)
{
    currentSpeed *= airControl;
}

velocity.x = moveDirection.x * currentSpeed;
velocity.z = moveDirection.z * currentSpeed;

// Rotate to face movement direction
if (rotateToMovement && moveDirection.magnitude > 0.1f)
{
    Quaternion targetRotation =
Quaternion.LookRotation(moveDirection);
    transform.rotation = Quaternion.RotateTowards(
        transform.rotation,
        targetRotation,
        rotationSpeed * Time.deltaTime
    );
}
}

private void ApplyGravity()

```

```

{
    if (!isGrounded)
    {
        float gravityThisFrame = gravity;
        if (velocity.y < 0)
        {
            gravityThisFrame *= fallGravityMultiplier;
        }
        velocity.y -= gravityThisFrame * Time.deltaTime;
    }
}

private void HandleJump()
{
    if (isGrounded)
    {
        velocity.y = jumpForce;
    }
}

private void ApplyVelocity()
{
    controller.Move(velocity * Time.deltaTime);
}

private void OnDrawGizmos()
{
    if (showGroundCheck)
    {
        Vector3 checkPosition = transform.position +
groundCheckOffset;
        Gizmos.color = isGrounded ? Color.green : Color.red;
        Gizmos.DrawWireSphere(checkPosition, groundCheckRadius);
    }
}
}

```

Common Issues

"It feels floaty"

Increase `fallGravityMultiplier`. Most good-feeling jumps have faster descent than ascent.

"It's too short/tall"

Adjust `jumpForce` and `gravity` together. Higher force + higher gravity = same height, different speed.

"I can jump in the air"

Ground check isn't working. Check `groundCheckRadius` and your collision layers.

"InputReader.Instance is null"

Make sure you have the InputReader prefab in your scene. See [Input Setup](#).

Combine With

- [Coyote Time](#) - grace period makes basic jump more forgiving
- [Input Buffer](#) - catches jumps pressed slightly before landing
- [Squash Stretch](#) - visual feedback for the jump (2D)
- [Variable Jump](#) - height control alternative

When to Use Basic vs. Variable

Use Basic Jump when...	Use Variable Jump when...
Predictability matters (puzzle platformers)	Expression matters (action platformers)
Jump height is carefully designed per obstacle	Player should adapt jump to situation
Simplicity is a virtue	Depth is a virtue
Retro aesthetic	Modern platformer aesthetic

Why Start Here

Basic jump is the ideal first scaffold because it's **simple enough to understand completely** but **expressive enough to teach tuning**. Four variables create hundreds of distinct feels.

Teaching Sequence

1. Have students play with defaults first - establish baseline
2. Change ONE variable dramatically - notice the difference
3. Try to match a reference videogame's jump feel
4. Document what they changed and why

Perspective Choice

Let students pick their perspective based on the videogame they're studying:

- **2D:** Celeste, Hollow Knight, Mario
- **3D Third-Person:** Dark Souls, Zelda, platforming collectathons
- **3D First-Person:** Half-Life, Titanfall, classic FPS
- **Isometric:** Hades, Diablo, action RPGs with verticality

Common Student Mistakes

Changing too many variables at once: They can't isolate what's causing the feel. Enforce single-variable changes first.

Only adjusting jumpForce: Students often miss that gravity is equally important. Same height can feel completely different with different gravity/force ratios.

Ignoring fallGravityMultiplier: This is the secret ingredient that makes platformer jumps feel good. Pure physics feels wrong.

Assessment Questions

- Why does increasing both jumpForce and gravity maintain the same height but change the feel?
- What real-world physics does fallGravityMultiplier violate, and why is that violation important?
- When would you choose basic jump over variable jump for a design?

Heritage Notes

The basic jump predates videogames - it's how jumping works in reality. But videogame jumps have always been exaggerated:

- *Donkey Kong* (1981): Fixed arc jump, limited air control
- *Super Mario Bros.* (1985): Introduced variable jump, but basic enemies still use fixed jump
- Modern videogames: Variable is default, but basic persists in certain contexts

The Physics Lie

Real physics would produce symmetrical parabolas. Videogame jumps almost universally use asymmetrical arcs - faster falling than rising. This started as a technical necessity (faster gameplay on limited hardware) and became an aesthetic expectation.

When students ask "why doesn't it use real physics?" the common answer is: real physics feels wrong because we've been trained by decades of videogame physics. The heritage has shaped our expectations.

But there's another answer: the more reality-related a videogame experience or feel is, the less surprise and exploration of that videogame's feel and systems is necessary. It's just not surprising, interesting, maybe even fun.

The Jump as Gesture

A basic jump is a complete Gesture - it has Action (the input and resulting movement), Art (the visual arc, any animation), Arc (the duration of the jump), and Atmosphere (the feel of weightiness or lightness). Even this "simple" mechanic contains all four A's.

Aesthetic Heritage of Jump Parameters

When you choose a floaty jump, you're inheriting from Kirby's tradition. When you choose a snappy jump, you're inheriting from Super Meat Boy. These aren't just technical choices - they're [Aesthetic Heritage](#) choices that connect your videogame to lineages of feel.

Related

- [Variable Jump](#) - more expressive alternative
- [Coyote Time](#) - makes any jump more forgiving
- [Input Setup](#) - required InputReader system
- [Gesture](#) - the jump as gesture

Glossary Terms

- [Coyote Time](#) - grace period for jump input
- [Input Buffer](#) - remembering inputs for grace
- [Juice](#) - feedback and feel techniques

Variable Jump

*A jump where **height** depends on how long you hold the button. Tap for a short hop, hold for a full jump. This is the standard for modern platformers - it gives players expressive control over their arc.*

The Feel

Variable jump creates a sense of **responsive precision**. The player feels like the character is listening to their intention, not just executing a canned animation.

Compare:

- **Fixed jump:** "I pressed jump and it happened"
- **Variable jump:** "I *shaped* that jump"

The difference is ownership.

Exposed Variables

Variable	Type	Default	What it does
minJumpForce	float	8.0	Jump height if you tap (minimum)
maxJumpForce	float	16.0	Jump height if you hold (maximum)
jumpCutMultiplier	float	0.5	Velocity multiplier when releasing early
gravity	float	40.0	Downward acceleration
fallGravityMultiplier	float	1.5	Extra gravity when falling

Tuning Guide

For a floaty, forgiving feel:

- Lower `gravity` (25-30)
- Lower `fallGravityMultiplier` (1.0-1.2)
- Higher `jumpCutMultiplier` (0.7-0.8)

Reference: Kirby, Little Big Planet

For a tight, precise feel:

- Higher `gravity` (45-60)
- Higher `fallGravityMultiplier` (1.8-2.5)
- Lower `jumpCutMultiplier` (0.3-0.5)

Reference: Celeste, Super Meat Boy

Perspective Scripts

Prerequisite: All scripts below require the [InputReader](#) to be set up in your scene.

Complete the Input Setup first.



2D Platformer 3D Third-Person 3D First-Person Isometric

VariableJump2D.cs

Classic 2D variable jump. Hold to go higher, release early to cut the jump short.

Setup:

1. Add `Rigidbody2D` to your player (Freeze Rotation Z)
2. Add a `Collider2D`
3. Create an empty child GameObject at the feet, name it "GroundCheck"
4. Add this script and configure in Inspector

▼ VariableJump2D.cs

```

using UnityEngine;

/// <summary>
/// VARIABLE JUMP 2D - Height depends on hold duration
/// VG101 Code Bank - Requires InputReader from Input Setup
/// </summary>
[RequireComponent(typeof(Rigidbody2D))]
public class VariableJump2D : MonoBehaviour
{
    [Header("Jump Feel")]
    [Tooltip("Jump force if you tap (minimum height)")]
    [Range(5f, 15f)]
    [SerializeField] private float minJumpForce = 8f;

    [Tooltip("Jump force if you hold (maximum height)")]
    [Range(10f, 25f)]
    [SerializeField] private float maxJumpForce = 16f;

    [Tooltip("Velocity multiplier when releasing early")]
    [Range(0.1f, 1f)]
    [SerializeField] private float jumpCutMultiplier = 0.5f;

    [Tooltip("Gravity while rising and falling")]
    [Range(20f, 80f)]
    [SerializeField] private float gravity = 40f;

    [Tooltip("Extra gravity when falling")]
    [Range(1f, 4f)]
    [SerializeField] private float fallGravityMultiplier = 1.5f;

    [Header("Ground Detection")]
    [SerializeField] private Transform groundCheckPoint;
    [Range(0.05f, 0.5f)]
    [SerializeField] private float groundCheckRadius = 0.2f;
    [SerializeField] private LayerMask groundLayer;

```

```
[Header("Debug")]
[SerializeField] private bool showGroundCheck = true;

private Rigidbody2D rb;
private float verticalVelocity;
private bool isGrounded;
private bool isJumping;
private bool jumpHeld;

private void Awake()
{
    rb = GetComponent<Rigidbody2D>();
    rb.gravityScale = 0f;
}

private void OnEnable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed += HandleJumpPressed;
        InputReader.Instance.OnJumpReleased += HandleJumpReleased;
    }
}

private void OnDisable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed -= HandleJumpPressed;
        InputReader.Instance.OnJumpReleased -= HandleJumpReleased;
    }
}

private void FixedUpdate()
{
    CheckGrounded();
    HandleJumpHold();
}
```

```
        ApplyGravity();
        ApplyVelocity();
    }

    private void CheckGrounded()
    {
        isGrounded = Physics2D.OverlapCircle(
            groundCheckPoint.position,
            groundCheckRadius,
            groundLayer
        );

        if (isGrounded && verticalVelocity < 0)
        {
            verticalVelocity = 0f;
            isJumping = false;
        }
    }

    private void HandleJumpPressed()
    {
        if (isGrounded)
        {
            verticalVelocity = minJumpForce;
            isJumping = true;
            jumpHeld = true;
        }
    }

    private void HandleJumpReleased()
    {
        jumpHeld = false;

        // Cut the jump if still rising
        if (isJumping && verticalVelocity > 0)
        {
            verticalVelocity *= jumpCutMultiplier;
        }
    }
}
```

```

    }
}

private void HandleJumpHold()
{
    // While holding and rising, boost toward max
    if (isJumping && jumpHeld && verticalVelocity > 0)
    {
        // Smoothly increase force while holding
        verticalVelocity = Mathf.MoveTowards(
            verticalVelocity,
            maxJumpForce,
            (maxJumpForce - minJumpForce) * Time.fixedDeltaTime * 10f
        );
    }
}

private void ApplyGravity()
{
    if (!isGrounded)
    {
        float gravityThisFrame = gravity;
        if (verticalVelocity < 0)
        {
            gravityThisFrame *= fallGravityMultiplier;
        }
        verticalVelocity -= gravityThisFrame * Time.fixedDeltaTime;
    }
}

private void ApplyVelocity()
{
    Vector2 velocity = rb.linearVelocity;
    velocity.y = verticalVelocity;
    rb.linearVelocity = velocity;
}

```

```
private void OnDrawGizmos()
{
    if (showGroundCheck && groundCheckPoint != null)
    {
        Gizmos.color = isGrounded ? Color.green : Color.red;
        Gizmos.DrawWireSphere(groundCheckPoint.position,
groundCheckRadius);
    }
}
```

VariableJump3D.cs

Third-person variable jump with camera-relative movement. Hold for higher jumps.

Setup:

1. Add `CharacterController` to your player
2. Add this script and configure in Inspector
3. Set Ground Layer to your environment's layer

▼ VariableJump3D.cs

```

using UnityEngine;

/// <summary>
/// VARIABLE JUMP 3D - Height depends on hold duration
/// VG101 Code Bank - Requires InputReader from Input Setup
/// </summary>
[RequireComponent(typeof(CharacterController))]
public class VariableJump3D : MonoBehaviour
{
    [Header("Jump Feel")]
    [Range(5f, 15f)]
    [SerializeField] private float minJumpForce = 6f;

    [Range(10f, 25f)]
    [SerializeField] private float maxJumpForce = 12f;

    [Range(0.1f, 1f)]
    [SerializeField] private float jumpCutMultiplier = 0.5f;

    [Range(20f, 80f)]
    [SerializeField] private float gravity = 30f;

    [Range(1f, 4f)]
    [SerializeField] private float fallGravityMultiplier = 2f;

    [Header("Ground Detection")]
    [SerializeField] private Vector3 groundCheckOffset = new Vector3(0,
-0.9f, 0);
    [SerializeField] private float groundCheckRadius = 0.3f;
    [SerializeField] private LayerMask groundLayer;

    [Header("Movement")]
    [SerializeField] private float moveSpeed = 8f;
    [Range(0f, 1f)]
    [SerializeField] private float airControl = 0.5f;

```

```
[Header("Debug")]
[SerializeField] private bool showGroundCheck = true;

private CharacterController controller;
private Vector3 velocity;
private bool isGrounded;
private bool isJumping;
private bool jumpHeld;
private Transform cameraTransform;

private void Awake()
{
    controller = GetComponent<CharacterController>();
    cameraTransform = Camera.main?.transform;
}

private void OnEnable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed += HandleJumpPressed;
        InputReader.Instance.OnJumpReleased += HandleJumpReleased;
    }
}

private void OnDisable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed -= HandleJumpPressed;
        InputReader.Instance.OnJumpReleased -= HandleJumpReleased;
    }
}

private void Update()
{
    CheckGrounded();
}
```

```

        HandleMovement();
        HandleJumpHold();
        ApplyGravity();
        ApplyVelocity();
    }

    private void CheckGrounded()
    {
        Vector3 checkPosition = transform.position + groundCheckOffset;
        isGrounded = Physics.CheckSphere(checkPosition,
groundCheckRadius, groundLayer);

        if (isGrounded && velocity.y < 0)
        {
            velocity.y = -2f;
            isJumping = false;
        }
    }

    private void HandleMovement()
    {
        Vector2 input = InputReader.Instance != null ?
InputReader.Instance.MoveInput : Vector2.zero;

        Vector3 forward = cameraTransform != null ?
cameraTransform.forward : transform.forward;
        Vector3 right = cameraTransform != null ? cameraTransform.right :
transform.right;
        forward.y = 0f;
        right.y = 0f;
        forward.Normalize();
        right.Normalize();

        Vector3 moveDirection = (forward * input.y + right * input.x);

        float currentSpeed = isGrounded ? moveSpeed : moveSpeed *
airControl;

```

```

        velocity.x = moveDirection.x * currentSpeed;
        velocity.z = moveDirection.z * currentSpeed;

        if (moveDirection.magnitude > 0.1f)
        {
            Quaternion targetRotation =
Quaternion.LookRotation(moveDirection);
            transform.rotation = Quaternion.Slerp(transform.rotation,
targetRotation, 10f * Time.deltaTime);
        }
    }

    private void HandleJumpPressed()
    {
        if (isGrounded)
        {
            velocity.y = minJumpForce;
            isJumping = true;
            jumpHeld = true;
        }
    }

    private void HandleJumpReleased()
    {
        jumpHeld = false;

        if (isJumping && velocity.y > 0)
        {
            velocity.y *= jumpCutMultiplier;
        }
    }

    private void HandleJumpHold()
    {
        if (isJumping && jumpHeld && velocity.y > 0)
        {
            velocity.y = Mathf.MoveTowards(

```

```

        velocity.y,
        maxJumpForce,
        (maxJumpForce - minJumpForce) * Time.deltaTime * 10f
    );
}
}

private void ApplyGravity()
{
    if (!isGrounded)
    {
        float gravityThisFrame = gravity;
        if (velocity.y < 0)
        {
            gravityThisFrame *= fallGravityMultiplier;
        }
        velocity.y -= gravityThisFrame * Time.deltaTime;
    }
}

private void ApplyVelocity()
{
    controller.Move(velocity * Time.deltaTime);
}

private void OnDrawGizmos()
{
    if (showGroundCheck)
    {
        Vector3 checkPosition = transform.position +
groundCheckOffset;
        Gizmos.color = isGrounded ? Color.green : Color.red;
        Gizmos.DrawWireSphere(checkPosition, groundCheckRadius);
    }
}
}

```

VariableJumpFPS.cs

First-person variable jump. Less common in FPS but useful for platforming-focused games like Titanfall.

Setup:

1. Add `CharacterController` to your player
2. Create a child Camera
3. Add this script and assign the Camera reference

▼ VariableJumpFPS.cs


```

using UnityEngine;

/// <summary>
/// VARIABLE JUMP FPS - First-person variable jump
/// VG101 Code Bank - Requires InputReader from Input Setup
/// </summary>
[RequireComponent(typeof(CharacterController))]
public class VariableJumpFPS : MonoBehaviour
{
    [Header("Jump Feel")]
    [Range(5f, 15f)]
    [SerializeField] private float minJumpForce = 6f;

    [Range(10f, 25f)]
    [SerializeField] private float maxJumpForce = 10f;

    [Range(0.1f, 1f)]
    [SerializeField] private float jumpCutMultiplier = 0.5f;

    [Range(20f, 80f)]
    [SerializeField] private float gravity = 25f;

    [Range(1f, 4f)]
    [SerializeField] private float fallGravityMultiplier = 2f;

    [Header("Ground Detection")]
    [SerializeField] private Vector3 groundCheckOffset = new Vector3(0,
-0.9f, 0);
    [SerializeField] private float groundCheckRadius = 0.3f;
    [SerializeField] private LayerMask groundLayer;

    [Header("Movement")]
    [SerializeField] private float moveSpeed = 6f;
    [Range(0f, 1f)]
    [SerializeField] private float airControl = 0.3f;

```

```
[Header("Mouse Look")]
[SerializeField] private Transform playerCamera;
[SerializeField] private float gamepadLookSpeed = 150f;
[SerializeField] private float mouseSensitivity = 0.15f;
[Range(45f, 90f)]
[SerializeField] private float maxLookAngle = 85f;

[Header("Debug")]
[SerializeField] private bool showGroundCheck = true;

private CharacterController controller;
private Vector3 velocity;
private bool isGrounded;
private bool isJumping;
private bool jumpHeld;
private float cameraPitch;

private void Awake()
{
    controller = GetComponent<CharacterController>();
    Cursor.lockState = CursorLockMode.Locked;
    Cursor.visible = false;
}

private void OnEnable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed += HandleJumpPressed;
        InputReader.Instance.OnJumpReleased += HandleJumpReleased;
    }
}

private void OnDisable()
{
    if (InputReader.Instance != null)
    {
```

```

        InputReader.Instance.OnJumpPressed -= HandleJumpPressed;
        InputReader.Instance.OnJumpReleased -= HandleJumpReleased;
    }
}

private void Update()
{
    CheckGrounded();
    HandleLook();
    HandleMovement();
    HandleJumpHold();
    ApplyGravity();
    ApplyVelocity();
}

private void CheckGrounded()
{
    Vector3 checkPosition = transform.position + groundCheckOffset;
    isGrounded = Physics.CheckSphere(checkPosition,
groundCheckRadius, groundLayer);

    if (isGrounded && velocity.y < 0)
    {
        velocity.y = -2f;
        isJumping = false;
    }
}

private void HandleLook()
{
    if (InputReader.Instance == null) return;

    Vector2 lookInput = InputReader.Instance.LookInput;

    float lookSpeedX, lookSpeedY;
    if (InputReader.Instance.IsUsingGamepad)
    {

```

```

        lookSpeedX = gamepadLookSpeed * Time.deltaTime;
        lookSpeedY = gamepadLookSpeed * Time.deltaTime;
    }
    else
    {
        lookSpeedX = mouseSensitivity;
        lookSpeedY = mouseSensitivity;
    }

    transform.Rotate(Vector3.up * lookInput.x * lookSpeedX);

    cameraPitch -= lookInput.y * lookSpeedY;
    cameraPitch = Mathf.Clamp(cameraPitch, -maxLookAngle,
maxLookAngle);

    if (playerCamera != null)
    {
        playerCamera.localRotation = Quaternion.Euler(cameraPitch,
0f, 0f);
    }
}

private void HandleMovement()
{
    Vector2 input = InputReader.Instance != null ?
InputReader.Instance.MoveInput : Vector2.zero;
    Vector3 moveDirection = transform.forward * input.y +
transform.right * input.x;

    float currentSpeed = isGrounded ? moveSpeed : moveSpeed *
airControl;
    velocity.x = moveDirection.x * currentSpeed;
    velocity.z = moveDirection.z * currentSpeed;
}

private void HandleJumpPressed()
{

```

```
        if (isGrounded)
        {
            velocity.y = minJumpForce;
            isJumping = true;
            jumpHeld = true;
        }
    }

    private void HandleJumpReleased()
    {
        jumpHeld = false;

        if (isJumping && velocity.y > 0)
        {
            velocity.y *= jumpCutMultiplier;
        }
    }

    private void HandleJumpHold()
    {
        if (isJumping && jumpHeld && velocity.y > 0)
        {
            velocity.y = Mathf.MoveTowards(
                velocity.y,
                maxJumpForce,
                (maxJumpForce - minJumpForce) * Time.deltaTime * 10f
            );
        }
    }

    private void ApplyGravity()
    {
        if (!isGrounded)
        {
            float gravityThisFrame = gravity;
            if (velocity.y < 0)
            {
```

```

        gravityThisFrame *= fallGravityMultiplier;
    }
    velocity.y -= gravityThisFrame * Time.deltaTime;
}

private void ApplyVelocity()
{
    controller.Move(velocity * Time.deltaTime);
}

private void OnDrawGizmos()
{
    if (showGroundCheck)
    {
        Vector3 checkPosition = transform.position +
groundCheckOffset;
        Gizmos.color = isGrounded ? Color.green : Color.red;
        Gizmos.DrawWireSphere(checkPosition, groundCheckRadius);
    }
}
}

```

VariableJumpIsometric.cs

Isometric variable jump. Useful for action games like Hades where vertical movement adds depth.

Setup:

1. Add `CharacterController` to your player
2. Set up isometric camera
3. Add this script and configure movement mode

▼ VariableJumpIsometric.cs

```

using UnityEngine;

/// <summary>
/// VARIABLE JUMP ISOMETRIC - Isometric variable jump
/// VG101 Code Bank - Requires InputReader from Input Setup
/// </summary>
[RequireComponent(typeof(CharacterController))]
public class VariableJumpIsometric : MonoBehaviour
{
    public enum MovementMode { WorldAxis, Isometric45 }

    [Header("Jump Feel")]
    [Range(5f, 15f)]
    [SerializeField] private float minJumpForce = 8f;

    [Range(10f, 25f)]
    [SerializeField] private float maxJumpForce = 14f;

    [Range(0.1f, 1f)]
    [SerializeField] private float jumpCutMultiplier = 0.5f;

    [Range(20f, 80f)]
    [SerializeField] private float gravity = 35f;

    [Range(1f, 4f)]
    [SerializeField] private float fallGravityMultiplier = 1.5f;

    [Header("Ground Detection")]
    [SerializeField] private Vector3 groundCheckOffset = new Vector3(0,
-0.5f, 0);
    [SerializeField] private float groundCheckRadius = 0.3f;
    [SerializeField] private LayerMask groundLayer;

    [Header("Movement")]
    [SerializeField] private MovementMode movementMode =
MovementMode.Isometric45;

```



```
[SerializeField] private float moveSpeed = 7f;
[Range(0f, 1f)]
[SerializeField] private float airControl = 0.5f;
[SerializeField] private bool rotateToMovement = true;
[SerializeField] private float rotationSpeed = 720f;

[Header("Debug")]
[SerializeField] private bool showGroundCheck = true;

private CharacterController controller;
private Vector3 velocity;
private bool isGrounded;
private bool isJumping;
private bool jumpHeld;

private static readonly Matrix4x4 isoMatrix =
Matrix4x4.Rotate(Quaternion.Euler(0, 45, 0));

private void Awake()
{
    controller = GetComponent<CharacterController>();
}

private void OnEnable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed += HandleJumpPressed;
        InputReader.Instance.OnJumpReleased += HandleJumpReleased;
    }
}

private void OnDisable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed -= HandleJumpPressed;
    }
}
```

```

        InputReader.Instance.OnJumpReleased -= HandleJumpReleased;
    }
}

private void Update()
{
    CheckGrounded();
    HandleMovement();
    HandleJumpHold();
    ApplyGravity();
    ApplyVelocity();
}

private void CheckGrounded()
{
    Vector3 checkPosition = transform.position + groundCheckOffset;
    isGrounded = Physics.CheckSphere(checkPosition,
groundCheckRadius, groundLayer);

    if (isGrounded && velocity.y < 0)
    {
        velocity.y = -2f;
        isJumping = false;
    }
}

private void HandleMovement()
{
    Vector2 input = InputReader.Instance != null ?
InputReader.Instance.MoveInput : Vector2.zero;

    Vector3 inputDirection = new Vector3(input.x, 0, input.y);
    Vector3 moveDirection = movementMode == MovementMode.Isometric45
        ? isoMatrix.MultiplyVector(inputDirection)
        : inputDirection;

    float currentSpeed = isGrounded ? moveSpeed : moveSpeed *

```

```

airControl;

    velocity.x = moveDirection.x * currentSpeed;
    velocity.z = moveDirection.z * currentSpeed;

    if (rotateToMovement && moveDirection.magnitude > 0.1f)
    {
        Quaternion targetRotation =
Quaternion.LookRotation(moveDirection);
        transform.rotation = Quaternion.RotateTowards(
            transform.rotation,
            targetRotation,
            rotationSpeed * Time.deltaTime
        );
    }
}

private void HandleJumpPressed()
{
    if (isGrounded)
    {
        velocity.y = minJumpForce;
        isJumping = true;
        jumpHeld = true;
    }
}

private void HandleJumpReleased()
{
    jumpHeld = false;

    if (isJumping && velocity.y > 0)
    {
        velocity.y *= jumpCutMultiplier;
    }
}

private void HandleJumpHold()

```

```

{
    if (isJumping && jumpHeld && velocity.y > 0)
    {
        velocity.y = Mathf.MoveTowards(
            velocity.y,
            maxJumpForce,
            (maxJumpForce - minJumpForce) * Time.deltaTime * 10f
        );
    }
}

private void ApplyGravity()
{
    if (!isGrounded)
    {
        float gravityThisFrame = gravity;
        if (velocity.y < 0)
        {
            gravityThisFrame *= fallGravityMultiplier;
        }
        velocity.y -= gravityThisFrame * Time.deltaTime;
    }
}

private void ApplyVelocity()
{
    controller.Move(velocity * Time.deltaTime);
}

private void OnDrawGizmos()
{
    if (showGroundCheck)
    {
        Vector3 checkPosition = transform.position +
groundCheckOffset;
        Gizmos.color = isGrounded ? Color.green : Color.red;
        Gizmos.DrawWireSphere(checkPosition, groundCheckRadius);
    }
}

```

```
}  
}  
}
```

Key Differences from Basic Jump

- **Two force values:** `minJumpForce` and `maxJumpForce` define the range
- **Hold tracking:** We track whether the button is held
- **Jump cut:** Releasing early multiplies velocity by `jumpCutMultiplier`
- **Real-time shaping:** The arc is determined during the jump, not at the start

Common Issues

"The jump feels mushy"

Increase `fallGravityMultiplier`. The descent should be snappier than the ascent.

"I can't do small hops"

Lower `minJumpForce` or decrease `jumpCutMultiplier`.

"InputReader.Instance is null"

Make sure you have the `InputReader` in your scene. See [Input Setup](#).

Combine With

- [Coyote Time](#) - grace period makes variable jump more forgiving
- [Input Buffer](#) - catch jump presses just before landing
- [Squash Stretch](#) - visual feedback that matches the jump's force
- [Double Jump](#) - extend the vocabulary mid-air

Why This Matters

Variable jump is where students first encounter **input as expression**. The same button press can mean different things based on how it's performed. This is fundamental to understanding how videogames create nuanced control.

Teaching Sequence

1. Start with Basic Jump - establish baseline
2. Switch to Variable Jump with same base parameters
3. Have students *feel* the difference before explaining
4. Then explain the mechanism

Common Student Misconceptions

"Variable jump is just two jump heights": Students sometimes implement it as binary (tap = short, hold = tall). The real technique is continuous shaping.

"The player decides at jump start": No - the player decides *during* the jump. The arc is shaped in real-time. This is what makes it feel responsive.

Assessment

- Can they explain why `jumpCutMultiplier` matters for responsiveness?
- Can they tune for a specific feel (floaty vs. snappy) deliberately?
- Can they articulate when basic jump would be better than variable jump?

Heritage Notes

Variable jump emerged in the **early 1980s** as designers realized fixed jumps felt unresponsive. *Super Mario Bros.* (1985) is the canonical example, but the technique appeared earlier in various forms.

The problem it solved: **fixed jumps force level design to accommodate the jump, not the player.** With variable jump, the player adapts to the level.

The Expression Problem

Variable jump raises a design question: **how much should the player be able to express through a single input?**

A single button press can produce many different outcomes based on:

- How long it's held
- When it's pressed (coyote time)
- What direction is held simultaneously
- What state the character is in

This is the beginning of [Gesture](#) as a concept - the player's input and the videogame's response form a complete expressive unit.

The Celeste Refinement

Celeste (2018) represents the current state of the art for variable jump. It combines variable height, generous coyote time, input buffering, distinct fall gravity, and air dash integration. The result feels **precise but forgiving**.

Related

- [Basic Jump](#) - simpler version without variable height
- [Double Jump](#) - extending the variable jump vocabulary
- [Input Setup](#) - required InputReader system
- [Gesture](#) - the variable jump as a complete gesture

Glossary Terms

- [Coyote Time](#) - grace period for jump input
- [Input Buffer](#) - remembering inputs for grace
- [Game Feel](#) - the experience of controlling a videogame

Double Jump

An extra jump in mid-air. Press jump again before landing to get additional height or change direction. The most common platformer upgrade.

The Feel

Double jump adds **aerial control and recovery**. It lets players correct mistakes, reach higher platforms, and feel more expressive in the air.

It's also a psychological safety net - players feel less committed to their first jump because they have a second chance.

Exposed Variables

Variable	Type	Default	What it does
<code>extraJumps</code>	int	1	Number of air jumps allowed
<code>airJumpForce</code>	float	10.0	Force of air jump (often less than ground)
<code>resetVelocity</code>	bool	true	Cancel downward velocity before air jump

Tuning Guide

For same-height double jump:

- `airJumpForce` = same as `jumpForce`
- `resetVelocity` = true

For diminishing jumps:

- `airJumpForce` = 70-80% of `jumpForce`
- Encourages using first jump wisely

Reference: Hollow Knight, Metroidvanias

For boosted air jump:

- `airJumpForce` > `jumpForce`
- First jump is setup, air jump is powerful

Perspective Scripts

Prerequisite: All scripts below require the [InputReader](#) to be set up in your scene.



2D Platformer 3D (All Perspectives)

DoubleJump2D.cs

2D double jump with configurable air jumps and distinct feedback for ground vs air jumps.

Setup:

1. Add `Rigidbody2D` to your player
2. Create a child "GroundCheck" at the feet
3. Add this script and configure in Inspector
4. Optional: assign airJumpEffect particle prefab

▼ DoubleJump2D.cs

```

using UnityEngine;

/// <summary>
/// DOUBLE JUMP 2D - Air jump with resource management
/// VG101 Code Bank - Requires InputReader from Input Setup
/// </summary>
[RequireComponent(typeof(Rigidbody2D))]
public class DoubleJump2D : MonoBehaviour
{
    [Header("Ground Jump")]
    [Range(5f, 25f)]
    [SerializeField] private float jumpForce = 12f;

    [Header("Air Jump")]
    [Tooltip("Number of air jumps (1 = double jump, 2 = triple)")]
    [Range(1, 5)]
    [SerializeField] private int extraJumps = 1;

    [Range(5f, 25f)]
    [SerializeField] private float airJumpForce = 10f;

    [Tooltip("Cancel downward velocity before air jump")]
    [SerializeField] private bool resetVelocity = true;

    [Header("Gravity")]
    [Range(20f, 80f)]
    [SerializeField] private float gravity = 40f;

    [Range(1f, 4f)]
    [SerializeField] private float fallGravityMultiplier = 1.5f;

    [Header("Ground Detection")]
    [SerializeField] private Transform groundCheckPoint;
    [Range(0.05f, 0.5f)]
    [SerializeField] private float groundCheckRadius = 0.2f;
    [SerializeField] private LayerMask groundLayer;

```

```
[Header("Air Jump Effect (Optional)")]
[SerializeField] private GameObject airJumpEffect;

[Header("Debug")]
[SerializeField] private bool showGroundCheck = true;
[SerializeField] private bool debugMode = false;

private Rigidbody2D rb;
private float verticalVelocity;
private bool isGrounded;
private bool wasGrounded;
private int jumpsRemaining;

public int JumpsRemaining => jumpsRemaining;
public bool CanJump => isGrounded || jumpsRemaining > 0;

private void Awake()
{
    rb = GetComponent<Rigidbody2D>();
    rb.gravityScale = 0f;
    jumpsRemaining = extraJumps;
}

private void OnEnable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed += HandleJump;
    }
}

private void OnDisable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed -= HandleJump;
    }
}
```

```

    }
}

private void FixedUpdate()
{
    CheckGrounded();
    ApplyGravity();
    ApplyVelocity();
}

private void CheckGrounded()
{
    wasGrounded = isGrounded;

    isGrounded = Physics2D.OverlapCircle(
        groundCheckPoint.position,
        groundCheckRadius,
        groundLayer
    );

    if (isGrounded && verticalVelocity < 0)
    {
        verticalVelocity = 0f;
    }

    // Refresh jumps on landing
    if (isGrounded && !wasGrounded)
    {
        jumpsRemaining = extraJumps;
        if (debugMode) Debug.Log($"Landed! Air jumps:
{jumpsRemaining}");
    }
}

private void HandleJump()
{
    if (isGrounded)

```

```

    {
        // Ground jump
        verticalVelocity = jumpForce;
        if (debugMode) Debug.Log("Ground jump!");
    }
    else if (jumpsRemaining > 0)
    {
        // Air jump
        if (resetVelocity)
        {
            verticalVelocity = 0f;
        }
        verticalVelocity = airJumpForce;
        jumpsRemaining--;

        SpawnAirJumpEffect();
        if (debugMode) Debug.Log($"Air jump! Remaining:
{jumpsRemaining}");
    }
}

private void SpawnAirJumpEffect()
{
    if (airJumpEffect != null)
    {
        {
            GameObject effect = Instantiate(airJumpEffect,
transform.position, Quaternion.identity);
            Destroy(effect, 2f);
        }
    }
}

private void ApplyGravity()
{
    {
        if (!isGrounded)
        {
            float gravityThisFrame = gravity;
            if (verticalVelocity < 0)

```



```

        {
            gravityThisFrame *= fallGravityMultiplier;
        }
        verticalVelocity -= gravityThisFrame * Time.fixedDeltaTime;
    }
}

private void ApplyVelocity()
{
    Vector2 velocity = rb.linearVelocity;
    velocity.y = verticalVelocity;
    rb.linearVelocity = velocity;
}

// Call from external scripts (e.g., wall touch)
public void RefreshAirJumps()
{
    jumpsRemaining = extraJumps;
}

private void OnDrawGizmos()
{
    if (showGroundCheck && groundCheckPoint != null)
    {
        Gizmos.color = isGrounded ? Color.green : Color.red;
        Gizmos.DrawWireSphere(groundCheckPoint.position,
groundCheckRadius);
    }
}
}

```

DoubleJump3D.cs

3D double jump for third-person, first-person, and isometric. Works with CharacterController.

Setup:

1. Add `CharacterController` to your player
2. Add this script and configure in Inspector
3. Optional: assign `airJumpEffect` particle prefab

▼ DoubleJump3D.cs

```

using UnityEngine;

/// <summary>
/// DOUBLE JUMP 3D - Air jump for all 3D perspectives
/// VG101 Code Bank - Requires InputReader from Input Setup
/// </summary>
[RequireComponent(typeof(CharacterController))]
public class DoubleJump3D : MonoBehaviour
{
    [Header("Ground Jump")]
    [Range(5f, 25f)]
    [SerializeField] private float jumpForce = 10f;

    [Header("Air Jump")]
    [Range(1, 5)]
    [SerializeField] private int extraJumps = 1;

    [Range(5f, 25f)]
    [SerializeField] private float airJumpForce = 8f;

    [SerializeField] private bool resetVelocity = true;

    [Header("Gravity")]
    [Range(20f, 80f)]
    [SerializeField] private float gravity = 30f;

    [Range(1f, 4f)]
    [SerializeField] private float fallGravityMultiplier = 2f;

    [Header("Ground Detection")]
    [SerializeField] private Vector3 groundCheckOffset = new Vector3(0,
-0.9f, 0);
    [SerializeField] private float groundCheckRadius = 0.3f;
    [SerializeField] private LayerMask groundLayer;

    [Header("Movement (Optional)")]

```

```

[SerializeField] private float moveSpeed = 8f;
[Range(0f, 1f)]
[SerializeField] private float airControl = 0.5f;
[SerializeField] private bool useBuiltInMovement = true;

[Header("Air Jump Effect (Optional)")]
[SerializeField] private GameObject airJumpEffect;

[Header("Debug")]
[SerializeField] private bool showGroundCheck = true;
[SerializeField] private bool debugMode = false;

private CharacterController controller;
private Vector3 velocity;
private bool isGrounded;
private bool wasGrounded;
private int jumpsRemaining;
private Transform cameraTransform;

public int JumpsRemaining => jumpsRemaining;
public bool CanJump => isGrounded || jumpsRemaining > 0;

private void Awake()
{
    controller = GetComponent<CharacterController>();
    cameraTransform = Camera.main?.transform;
    jumpsRemaining = extraJumps;
}

private void OnEnable()
{
    {
        if (InputReader.Instance != null)
        {
            InputReader.Instance.OnJumpPressed += HandleJump;
        }
    }
}

```

```
private void OnDisable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed -= HandleJump;
    }
}

private void Update()
{
    CheckGrounded();

    if (useBuiltInMovement)
    {
        HandleMovement();
    }

    ApplyGravity();
    ApplyVelocity();
}

private void CheckGrounded()
{
    wasGrounded = isGrounded;

    Vector3 checkPosition = transform.position + groundCheckOffset;
    isGrounded = Physics.CheckSphere(checkPosition,
groundCheckRadius, groundLayer);

    if (isGrounded && velocity.y < 0)
    {
        velocity.y = -2f;
    }

    // Refresh jumps on landing
    if (isGrounded && !wasGrounded)
    {

```

```

        jumpsRemaining = extraJumps;
        if (debugMode) Debug.Log($"Landed! Air jumps:
{jumpsRemaining}");
    }
}

private void HandleMovement()
{
    Vector2 input = InputReader.Instance != null ?
InputReader.Instance.MoveInput : Vector2.zero;

    Vector3 forward = cameraTransform != null ?
cameraTransform.forward : transform.forward;
    Vector3 right = cameraTransform != null ? cameraTransform.right :
transform.right;
    forward.y = 0f;
    right.y = 0f;
    forward.Normalize();
    right.Normalize();

    Vector3 moveDirection = (forward * input.y + right * input.x);

    float currentSpeed = isGrounded ? moveSpeed : moveSpeed *
airControl;
    velocity.x = moveDirection.x * currentSpeed;
    velocity.z = moveDirection.z * currentSpeed;

    if (moveDirection.magnitude > 0.1f)
    {
        Quaternion targetRotation =
Quaternion.LookRotation(moveDirection);
        transform.rotation = Quaternion.Slerp(transform.rotation,
targetRotation, 10f * Time.deltaTime);
    }
}

private void HandleJump()

```

```

{
    if (isGrounded)
    {
        velocity.y = jumpForce;
        if (debugMode) Debug.Log("Ground jump!");
    }
    else if (jumpsRemaining > 0)
    {
        if (resetVelocity)
        {
            velocity.y = 0f;
        }
        velocity.y = airJumpForce;
        jumpsRemaining--;

        SpawnAirJumpEffect();
        if (debugMode) Debug.Log($"Air jump! Remaining:
{jumpsRemaining}");
    }
}

private void SpawnAirJumpEffect()
{
    if (airJumpEffect != null)
    {
        GameObject effect = Instantiate(airJumpEffect,
transform.position, Quaternion.identity);
        Destroy(effect, 2f);
    }
}

private void ApplyGravity()
{
    if (!isGrounded)
    {
        float gravityThisFrame = gravity;
        if (velocity.y < 0)

```



```

        {
            gravityThisFrame *= fallGravityMultiplier;
        }
        velocity.y -= gravityThisFrame * Time.deltaTime;
    }
}

private void ApplyVelocity()
{
    controller.Move(velocity * Time.deltaTime);
}

public void SetHorizontalVelocity(Vector3 horizontalVelocity)
{
    velocity.x = horizontalVelocity.x;
    velocity.z = horizontalVelocity.z;
}

public void RefreshAirJumps()
{
    jumpsRemaining = extraJumps;
}

private void OnDrawGizmos()
{
    if (showGroundCheck)
    {
        Vector3 checkPosition = transform.position +
groundCheckOffset;
        Gizmos.color = isGrounded ? Color.green : Color.red;
        Gizmos.DrawWireSphere(checkPosition, groundCheckRadius);
    }
}
}

```

Key Implementation Details

- **Counter management:** Track `jumpsRemaining` , decrement on air jump, reset on landing
- **Velocity reset:** Optional - makes air jump height consistent regardless of fall speed
- **Distinct feedback:** Air jump MUST look/sound different from ground jump
- **RefreshAirJumps():** Call from wall touch or other refresh triggers

Common Issues

"Double jump doesn't feel different"

Add distinct feedback: particle effect, different sound, animation change. Players need to know they used their resource.

"Infinite jumps"

Make sure you only reset `jumpsRemaining` on landing (`wasGrounded` check), not every frame.

"Air jump barely helps when falling"

Set `resetVelocity` to true for consistent air jump height.

Design Decisions

Question	Options
Coyote jump uses air jump?	Most videogames: no. Coyote is grace, not air jump.
Reset on wall touch?	Yes for wall-jump videogames (enables infinite climbing)
Reset on taking damage?	Generous design - prevents unfair deaths

Combine With

- Variable Jump - both jumps can be variable height
- Coyote Time - coyote shouldn't consume air jump
- Dash - dash + double jump for maximum air mobility

Why Teach This

Double jump teaches **ability management**: tracking state (jumps remaining), reset conditions (landing), and feedback (air jump effects). More complex than basic jump but still approachable.

Teaching Sequence

1. Start with basic jump working
2. Add counter for remaining jumps
3. Allow jump when jumpsRemaining > 0 and in air
4. Add distinct visual/audio feedback
5. Tune the feel (force, velocity reset)

The Feedback Lesson

Air jump needs different feedback than ground jump so players know they've used their resource:

- Different sound effect
- Different particle effect (wings, sparkles)
- Different animation

Assessment Questions

- Why might air jump be weaker than ground jump?
- What feedback tells the player they've used double jump?
- How does double jump change level design?

Heritage Notes

Double jump has murky origins but became codified in the 1980s-90s. Dragon Buster (1984) is an early example. It spread through action-platformers and Metroidvanias.

Now so common that players expect it. Many videogames start without it and grant it as an upgrade - the Metroidvania tradition of ability-gated progression.

The Abstraction Question

Double jump is **physically impossible**. You can't push off nothing. But videogames aren't physics simulations. They need to be internally consistent, not realistic.

Permission and Progression

From a [Permissions](#) perspective, double jump expands what's **allowed**. Before: certain heights forbidden. After: reachable. New abilities = new permissions = new areas.

Triple Jump and Beyond

Why stop at two? Diminishing returns:

- More jumps = more to track mentally
- More jumps = harder level design
- More jumps = less meaningful each jump becomes

Double is the sweet spot for most videogames.

Related

- [Basic Jump](#) - the foundation
- [Variable Jump](#) - can combine with double jump

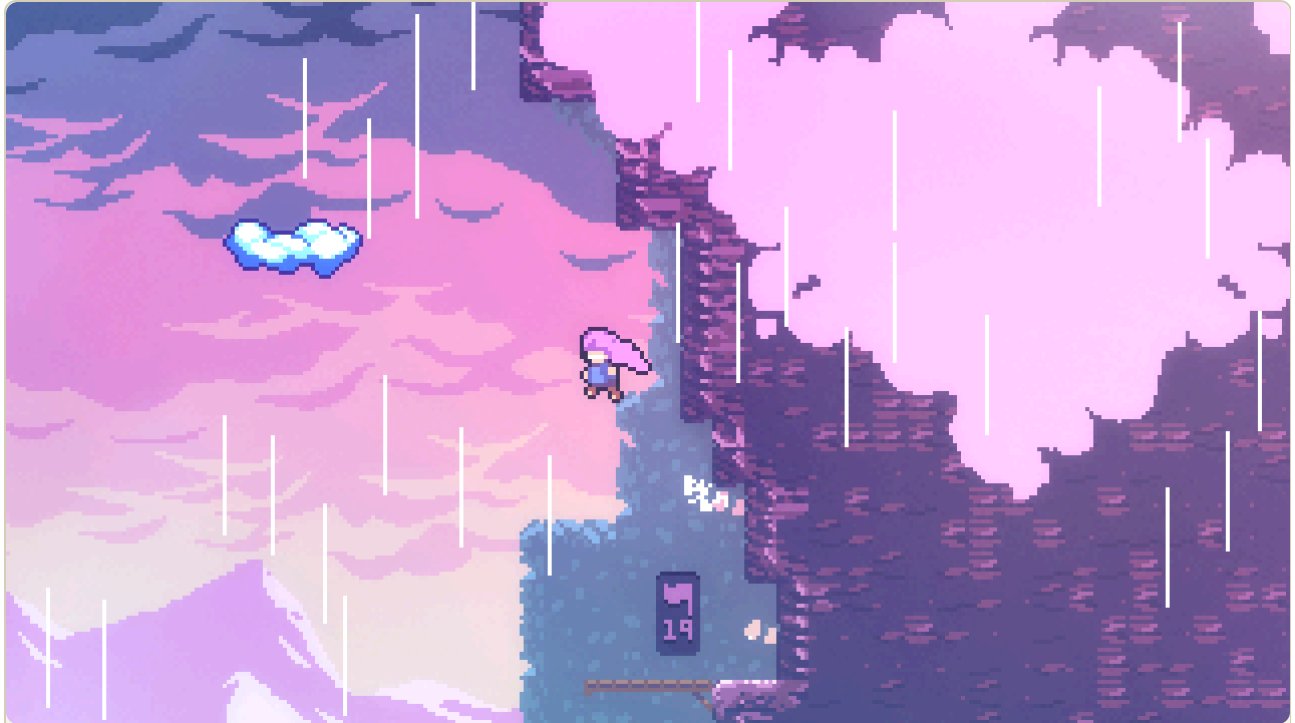
- [Input Setup](#) - required InputReader system
- [Dash](#) - another air mobility option

Glossary Terms

- [Coyote Time](#)
- [Metroidvania](#)

Coyote Time

A brief grace period after walking off a ledge where you can still jump. Named for Wile E. Coyote, who doesn't fall until he looks down.



Celeste - *The invisible grace period, player-fair design in action.*

Courtesy of Maddy Makes Games

The Feel

Coyote time is **invisible forgiveness**. Players don't notice when it helps them - they just feel like the videogame is fair. Without it, players constantly "miss" jumps they thought they made.

This is "player fair" vs. "computer fair." The computer knows exactly when you left the ground. The player doesn't. Coyote time bridges that gap.

Exposed Variables

Variable	Type	Default	What it does
<code>coyoteTime</code>	float	0.1	Seconds after leaving ground when jump still works

Tuning Guide

For tight precision (hardcore):

- `coyoteTime` = 0.05 - 0.08

Reference: Super Meat Boy

For accessible play:

- `coyoteTime` = 0.1 - 0.15

Reference: Celeste

For casual/beginner-friendly:

- `coyoteTime` = 0.15 - 0.2

Perspective Scripts

Prerequisite: All scripts below require the [InputReader](#) to be set up in your scene.



2D Platformer 3D (All Perspectives)

CoyoteJump2D.cs

2D jump with built-in coyote time. Allows jumping briefly after walking off a ledge.

Setup:

1. Add `Rigidbody2D` to your player
2. Create a child "GroundCheck" at the feet
3. Add this script and configure in Inspector

▼ CoyoteJump2D.cs

```

using UnityEngine;

/// <summary>
/// COYOTE JUMP 2D - Jump with grace period after leaving ledge
/// VG101 Code Bank - Requires InputReader from Input Setup
/// </summary>
[RequireComponent(typeof(Rigidbody2D))]
public class CoyoteJump2D : MonoBehaviour
{
    [Header("Jump Feel")]
    [Range(5f, 25f)]
    [SerializeField] private float jumpForce = 12f;

    [Range(20f, 80f)]
    [SerializeField] private float gravity = 40f;

    [Range(1f, 4f)]
    [SerializeField] private float fallGravityMultiplier = 1.5f;

    [Header("Coyote Time")]
    [Tooltip("Seconds after leaving ground when jump still works")]
    [Range(0.01f, 0.3f)]
    [SerializeField] private float coyoteTime = 0.1f;

    [Header("Ground Detection")]
    [SerializeField] private Transform groundCheckPoint;
    [Range(0.05f, 0.5f)]
    [SerializeField] private float groundCheckRadius = 0.2f;
    [SerializeField] private LayerMask groundLayer;

    [Header("Debug")]
    [SerializeField] private bool showGroundCheck = true;
    [SerializeField] private bool debugCoyote = false;

    private Rigidbody2D rb;
    private float verticalVelocity;

```

```
private bool isGrounded;
private bool wasGrounded;
private float coyoteTimer;
private bool usedCoyoteJump;

private void Awake()
{
    rb = GetComponent<Rigidbody2D>();
    rb.gravityScale = 0f;
}

private void OnEnable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed += HandleJump;
    }
}

private void OnDisable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed -= HandleJump;
    }
}

private void FixedUpdate()
{
    wasGrounded = isGrounded;
    CheckGrounded();
    HandleCoyoteTime();
    ApplyGravity();
    ApplyVelocity();
}

private void CheckGrounded()
```

```

{
    isGrounded = Physics2D.OverlapCircle(
        groundCheckPoint.position,
        groundCheckRadius,
        groundLayer
    );

    if (isGrounded && verticalVelocity < 0)
    {
        verticalVelocity = 0f;
        usedCoyoteJump = false;
    }
}

private void HandleCoyoteTime()
{
    // Just walked/fell off edge (not jumped off)
    if (wasGrounded && !isGrounded && verticalVelocity <= 0)
    {
        coyoteTimer = coyoteTime;
        if (debugCoyote) Debug.Log("Coyote time started!");
    }

    if (coyoteTimer > 0)
    {
        coyoteTimer -= Time.fixedDeltaTime;
    }
}

private bool CanJump()
{
    if (isGrounded) return true;
    if (coyoteTimer > 0 && !usedCoyoteJump) return true;
    return false;
}

private void HandleJump()

```

```

{
    if (CanJump())
    {
        verticalVelocity = jumpForce;

        if (!isGrounded)
        {
            usedCoyoteJump = true;
            coyoteTimer = 0;
            if (debugCoyote) Debug.Log("COYOTE JUMP!");
        }
    }
}

private void ApplyGravity()
{
    if (!isGrounded)
    {
        float gravityThisFrame = gravity;
        if (verticalVelocity < 0)
        {
            gravityThisFrame *= fallGravityMultiplier;
        }
        verticalVelocity -= gravityThisFrame * Time.fixedDeltaTime;
    }
}

private void ApplyVelocity()
{
    Vector2 velocity = rb.linearVelocity;
    velocity.y = verticalVelocity;
    rb.linearVelocity = velocity;
}

private void OnDrawGizmos()
{
    if (showGroundCheck && groundCheckPoint != null)

```

```
        {  
            Gizmos.color = isGrounded ? Color.green : Color.red;  
            Gizmos.DrawWireSphere(groundCheckPoint.position,  
groundCheckRadius);  
        }  
    }  
}
```

CoyoteJump3D.cs

3D jump with coyote time. Works for third-person, first-person, and isometric - just integrate with your camera/movement system.

Setup:

1. Add `CharacterController` to your player
2. Add this script and configure in Inspector
3. Integrate with your existing movement script or use the built-in movement

▼ CoyoteJump3D.cs

```

using UnityEngine;

/// <summary>
/// COYOTE JUMP 3D - Jump with grace period for all 3D perspectives
/// VG101 Code Bank - Requires InputReader from Input Setup
/// Works with CharacterController for third-person, FPS, or isometric
/// </summary>
[RequireComponent(typeof(CharacterController))]
public class CoyoteJump3D : MonoBehaviour
{
    [Header("Jump Feel")]
    [Range(5f, 25f)]
    [SerializeField] private float jumpForce = 10f;

    [Range(20f, 80f)]
    [SerializeField] private float gravity = 30f;

    [Range(1f, 4f)]
    [SerializeField] private float fallGravityMultiplier = 2f;

    [Header("Coyote Time")]
    [Tooltip("Seconds after leaving ground when jump still works")]
    [Range(0.01f, 0.3f)]
    [SerializeField] private float coyoteTime = 0.1f;

    [Header("Ground Detection")]
    [SerializeField] private Vector3 groundCheckOffset = new Vector3(0,
-0.9f, 0);
    [SerializeField] private float groundCheckRadius = 0.3f;
    [SerializeField] private LayerMask groundLayer;

    [Header("Movement (Optional)")]
    [SerializeField] private float moveSpeed = 8f;
    [Range(0f, 1f)]
    [SerializeField] private float airControl = 0.5f;
    [SerializeField] private bool useBuiltInMovement = true;

```



```
[Header("Debug")]
[SerializeField] private bool showGroundCheck = true;
[SerializeField] private bool debugCoyote = false;

private CharacterController controller;
private Vector3 velocity;
private bool isGrounded;
private bool wasGrounded;
private float coyoteTimer;
private bool usedCoyoteJump;
private Transform cameraTransform;

private void Awake()
{
    controller = GetComponent<CharacterController>();
    cameraTransform = Camera.main?.transform;
}

private void OnEnable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed += HandleJump;
    }
}

private void OnDisable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed -= HandleJump;
    }
}

private void Update()
{

```

```

        wasGrounded = isGrounded;
        CheckGrounded();
        HandleCoyoteTime();

        if (useBuiltInMovement)
        {
            HandleMovement();
        }

        ApplyGravity();
        ApplyVelocity();
    }

    private void CheckGrounded()
    {
        Vector3 checkPosition = transform.position + groundCheckOffset;
        isGrounded = Physics.CheckSphere(checkPosition,
groundCheckRadius, groundLayer);

        if (isGrounded && velocity.y < 0)
        {
            velocity.y = -2f;
            usedCoyoteJump = false;
        }
    }

    private void HandleCoyoteTime()
    {
        // Just walked/fell off edge (not jumped off)
        if (wasGrounded && !isGrounded && velocity.y <= 0)
        {
            coyoteTimer = coyoteTime;
            if (debugCoyote) Debug.Log("Coyote time started!");
        }

        if (coyoteTimer > 0)
        {

```

```

        coyoteTimer -= Time.deltaTime;
    }
}

private void HandleMovement()
{
    Vector2 input = InputReader.Instance != null ?
InputReader.Instance.MoveInput : Vector2.zero;

    // Camera-relative movement
    Vector3 forward = cameraTransform != null ?
cameraTransform.forward : transform.forward;
    Vector3 right = cameraTransform != null ? cameraTransform.right :
transform.right;
    forward.y = 0f;
    right.y = 0f;
    forward.Normalize();
    right.Normalize();

    Vector3 moveDirection = (forward * input.y + right * input.x);

    float currentSpeed = isGrounded ? moveSpeed : moveSpeed *
airControl;
    velocity.x = moveDirection.x * currentSpeed;
    velocity.z = moveDirection.z * currentSpeed;

    if (moveDirection.magnitude > 0.1f)
    {
        Quaternion targetRotation =
Quaternion.LookRotation(moveDirection);
        transform.rotation = Quaternion.Slerp(transform.rotation,
targetRotation, 10f * Time.deltaTime);
    }
}

private bool CanJump()
{

```

```

        if (isGrounded) return true;
        if (coyoteTimer > 0 && !usedCoyoteJump) return true;
        return false;
    }

    private void HandleJump()
    {
        if (CanJump())
        {
            velocity.y = jumpForce;

            if (!isGrounded)
            {
                usedCoyoteJump = true;
                coyoteTimer = 0;
                if (debugCoyote) Debug.Log("COYOTE JUMP!");
            }
        }
    }

    private void ApplyGravity()
    {
        if (!isGrounded)
        {
            float gravityThisFrame = gravity;
            if (velocity.y < 0)
            {
                gravityThisFrame *= fallGravityMultiplier;
            }
            velocity.y -= gravityThisFrame * Time.deltaTime;
        }
    }

    private void ApplyVelocity()
    {
        controller.Move(velocity * Time.deltaTime);
    }

```

```

        // Call this from external movement scripts if not using built-in
movement
        public void SetHorizontalVelocity(Vector3 horizontalVelocity)
        {
            velocity.x = horizontalVelocity.x;
            velocity.z = horizontalVelocity.z;
        }

        private void OnDrawGizmos()
        {
            if (showGroundCheck)
            {
                Vector3 checkPosition = transform.position +
groundCheckOffset;
                Gizmos.color = isGrounded ? Color.green : Color.red;
                Gizmos.DrawWireSphere(checkPosition, groundCheckRadius);
            }
        }
    }
}

```

Integration Notes

If you have your own movement script:

1. Set `useBuiltInMovement` to false
2. Call `SetHorizontalVelocity()` from your movement script each frame
3. The coyote time and vertical velocity are handled automatically

Key Implementation Details

- **wasGrounded tracking:** Compare current vs. last frame to detect leaving ground
- **Only on passive exit:** Coyote only starts if velocity ≤ 0 (walking off, not jumping off)
- **usedCoyoteJump flag:** Prevents using coyote time twice in one airtime
- **Timer reset on jump:** Once used, it's gone

The Complete Forgiveness Package

Technique	What it forgives
Coyote Time	Pressing jump slightly AFTER leaving edge
Input Buffer	Pressing jump slightly BEFORE landing
Both together	Nearly all "unfair" missed jumps

Common Issues

"Player can double jump with coyote time"

You're not resetting the timer after jump or triggering coyote on jump. Only trigger when leaving ground passively.

"It feels like I can jump in mid-air"

coyoteTime is too long. Keep it under 0.15 for most videogames.

Combine With

- [Input Buffer](#) - catches jumps pressed early; coyote catches jumps pressed late
- [Variable Jump](#) - coyote time works great with variable height

Why Teach This

Coyote time is the perfect introduction to **player-fair design**. It's simple to implement but teaches a profound lesson: what the computer knows and what the player experiences are different things.

Teaching Sequence

1. Have students play a platformer without coyote time - note frustrations
2. Add coyote time - notice how complaints disappear
3. Make coyote time too long (0.5s) - notice how it breaks immersion
4. Find the sweet spot where it helps without being noticed

The Key Insight

The best player-fair techniques are **invisible**. Players don't thank you for coyote time - they just don't complain about unfair deaths.

Assessment Questions

- Why is the best coyote time invisible to players?
- What's the difference between "player fair" and "computer fair"?
- Why does coyote time pair well with input buffering?

Heritage Notes

Coyote time wasn't named until the 2010s, but the technique is much older. Early platformers often had generous ground detection that produced the same effect accidentally.

The name comes from the Looney Tunes trope where Wile E. Coyote runs off a cliff and hangs in the air until he looks down.

Player Fair vs. Computer Fair

- **Computer fair:** Rules applied exactly. You left the ground at frame 47; you cannot jump after frame 47.
- **Player fair:** Rules feel fair. If they intended to jump at the edge, respect that intent.

Coyote time is the videogame **lying to be kind**.

The Permissions Angle

From a [Permissions](#) perspective, coyote time expands what's **allowed**. It says: "You're allowed to jump for 0.1 seconds after leaving the ground." This permission is hidden but shapes their experience of fairness.

Related

- [Input Buffer](#) - the other half of jump forgiveness
- [Variable Jump](#) - pairs well with coyote time
- [Input Setup](#) - required InputReader system

Glossary Terms

- Coyote Time
- Input Buffer

Input Buffer

Remember button presses for a short window, executing them when possible. If you press jump just before landing, the jump happens when you land.

The Feel

Input buffering makes actions feel **responsive and forgiving**. Without it, players need frame-perfect timing. With it, they can queue up their next action and trust it will happen.

Exposed Variables

Variable	Type	Default	What it does
<code>bufferTime</code>	float	0.1	How long to remember an input

Tuning Guide

For precision gameplay:

- `bufferTime` = 0.05 - 0.08

For accessible/casual:

- `bufferTime` = 0.1 - 0.15

For fighting games:

- `bufferTime` = 0.1 - 0.2 (longer for combo inputs)

Perspective Scripts

Prerequisite: All scripts below require the [InputReader](#) to be set up in your scene.



2D Platformer 3D (All Perspectives)

BufferedJump2D.cs

2D jump with input buffering. Press jump before landing and it executes on landing.

Setup:

1. Add `Rigidbody2D` to your player

2. Create a child "GroundCheck" at the feet
3. Add this script and configure in Inspector

▼ BufferedJump2D.cs

```

using UnityEngine;

/// <summary>
/// BUFFERED JUMP 2D - Remember jump input and execute when possible
/// VG101 Code Bank - Requires InputReader from Input Setup
/// </summary>
[RequireComponent(typeof(Rigidbody2D))]
public class BufferedJump2D : MonoBehaviour
{
    [Header("Jump Feel")]
    [Range(5f, 25f)]
    [SerializeField] private float jumpForce = 12f;

    [Range(20f, 80f)]
    [SerializeField] private float gravity = 40f;

    [Range(1f, 4f)]
    [SerializeField] private float fallGravityMultiplier = 1.5f;

    [Header("Input Buffer")]
    [Tooltip("How long to remember a jump press")]
    [Range(0.01f, 0.3f)]
    [SerializeField] private float bufferTime = 0.1f;

    [Header("Ground Detection")]
    [SerializeField] private Transform groundCheckPoint;
    [Range(0.05f, 0.5f)]
    [SerializeField] private float groundCheckRadius = 0.2f;
    [SerializeField] private LayerMask groundLayer;

    [Header("Debug")]
    [SerializeField] private bool showGroundCheck = true;
    [SerializeField] private bool debugBuffer = false;

    private Rigidbody2D rb;
    private float verticalVelocity;

```

```
private bool isGrounded;
private float bufferTimer;
private bool jumpBuffered;

private void Awake()
{
    rb = GetComponent<Rigidbody2D>();
    rb.gravityScale = 0f;
}

private void OnEnable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed += HandleJumpInput;
    }
}

private void OnDisable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed -= HandleJumpInput;
    }
}

private void FixedUpdate()
{
    CheckGrounded();
    HandleBuffer();
    ApplyGravity();
    ApplyVelocity();
}

private void CheckGrounded()
{
    isGrounded = Physics2D.OverlapCircle(
```

```

        groundCheckPoint.position,
        groundCheckRadius,
        groundLayer
    );

    if (isGrounded && verticalVelocity < 0)
    {
        verticalVelocity = 0f;
    }
}

private void HandleBuffer()
{
    // Count down buffer timer
    if (bufferTimer > 0)
    {
        bufferTimer -= Time.fixedDeltaTime;
    }
    else
    {
        jumpBuffered = false;
    }

    // Execute buffered jump when grounded
    if (jumpBuffered && isGrounded)
    {
        ExecuteJump();
        jumpBuffered = false;
        bufferTimer = 0;
        if (debugBuffer) Debug.Log("BUFFERED JUMP executed!");
    }
}

private void HandleJumpInput()
{
    if (isGrounded)
    {

```



```

        // Jump immediately
        ExecuteJump();
    }
    else
    {
        // Buffer the input
        jumpBuffered = true;
        bufferTimer = bufferTime;
        if (debugBuffer) Debug.Log("Jump BUFFERED...");
    }
}

private void ExecuteJump()
{
    verticalVelocity = jumpForce;
}

private void ApplyGravity()
{
    if (!isGrounded)
    {
        float gravityThisFrame = gravity;
        if (verticalVelocity < 0)
        {
            gravityThisFrame *= fallGravityMultiplier;
        }
        verticalVelocity -= gravityThisFrame * Time.fixedDeltaTime;
    }
}

private void ApplyVelocity()
{
    Vector2 velocity = rb.linearVelocity;
    velocity.y = verticalVelocity;
    rb.linearVelocity = velocity;
}

```

```
private void OnDrawGizmos()
{
    if (showGroundCheck && groundCheckPoint != null)
    {
        Gizmos.color = isGrounded ? Color.green : Color.red;
        Gizmos.DrawWireSphere(groundCheckPoint.position,
groundCheckRadius);
    }
}
```

BufferedJump3D.cs

3D jump with input buffering. Works for third-person, first-person, and isometric perspectives.

Setup:

1. Add `CharacterController` to your player
2. Add this script and configure in Inspector
3. Integrate with your movement script or use built-in movement

▼ BufferedJump3D.cs

```

using UnityEngine;

/// <summary>
/// BUFFERED JUMP 3D - Input buffer for all 3D perspectives
/// VG101 Code Bank - Requires InputReader from Input Setup
/// </summary>
[RequireComponent(typeof(CharacterController))]
public class BufferedJump3D : MonoBehaviour
{
    [Header("Jump Feel")]
    [Range(5f, 25f)]
    [SerializeField] private float jumpForce = 10f;

    [Range(20f, 80f)]
    [SerializeField] private float gravity = 30f;

    [Range(1f, 4f)]
    [SerializeField] private float fallGravityMultiplier = 2f;

    [Header("Input Buffer")]
    [Tooltip("How long to remember a jump press")]
    [Range(0.01f, 0.3f)]
    [SerializeField] private float bufferTime = 0.1f;

    [Header("Ground Detection")]
    [SerializeField] private Vector3 groundCheckOffset = new Vector3(0,
-0.9f, 0);
    [SerializeField] private float groundCheckRadius = 0.3f;
    [SerializeField] private LayerMask groundLayer;

    [Header("Movement (Optional)")]
    [SerializeField] private float moveSpeed = 8f;
    [Range(0f, 1f)]
    [SerializeField] private float airControl = 0.5f;
    [SerializeField] private bool useBuiltInMovement = true;

```

```
[Header("Debug")]
[SerializeField] private bool showGroundCheck = true;
[SerializeField] private bool debugBuffer = false;

private CharacterController controller;
private Vector3 velocity;
private bool isGrounded;
private float bufferTimer;
private bool jumpBuffered;
private Transform cameraTransform;

private void Awake()
{
    controller = GetComponent<CharacterController>();
    cameraTransform = Camera.main?.transform;
}

private void OnEnable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed += HandleJumpInput;
    }
}

private void OnDisable()
{
    if (InputReader.Instance != null)
    {
        InputReader.Instance.OnJumpPressed -= HandleJumpInput;
    }
}

private void Update()
{
    CheckGrounded();
    HandleBuffer();
}
```

```
        if (useBuiltInMovement)
        {
            HandleMovement();
        }

        ApplyGravity();
        ApplyVelocity();
    }

    private void CheckGrounded()
    {
        Vector3 checkPosition = transform.position + groundCheckOffset;
        isGrounded = Physics.CheckSphere(checkPosition,
groundCheckRadius, groundLayer);

        if (isGrounded && velocity.y < 0)
        {
            velocity.y = -2f;
        }
    }

    private void HandleBuffer()
    {
        if (bufferTimer > 0)
        {
            bufferTimer -= Time.deltaTime;
        }
        else
        {
            jumpBuffered = false;
        }

        if (jumpBuffered && isGrounded)
        {
            ExecuteJump();
            jumpBuffered = false;
        }
    }
}
```

```

        bufferTimer = 0;
        if (debugBuffer) Debug.Log("BUFFERED JUMP executed!");
    }
}

private void HandleMovement()
{
    Vector2 input = InputReader.Instance != null ?
InputReader.Instance.MoveInput : Vector2.zero;

    Vector3 forward = cameraTransform != null ?
cameraTransform.forward : transform.forward;
    Vector3 right = cameraTransform != null ? cameraTransform.right :
transform.right;
    forward.y = 0f;
    right.y = 0f;
    forward.Normalize();
    right.Normalize();

    Vector3 moveDirection = (forward * input.y + right * input.x);

    float currentSpeed = isGrounded ? moveSpeed : moveSpeed *
airControl;
    velocity.x = moveDirection.x * currentSpeed;
    velocity.z = moveDirection.z * currentSpeed;

    if (moveDirection.magnitude > 0.1f)
    {
        Quaternion targetRotation =
Quaternion.LookRotation(moveDirection);
        transform.rotation = Quaternion.Slerp(transform.rotation,
targetRotation, 10f * Time.deltaTime);
    }
}

private void HandleJumpInput()
{

```

```

        if (isGrounded)
        {
            ExecuteJump();
        }
        else
        {
            jumpBuffered = true;
            bufferTimer = bufferTime;
            if (debugBuffer) Debug.Log("Jump BUFFERED...");
        }
    }

    private void ExecuteJump()
    {
        velocity.y = jumpForce;
    }

    private void ApplyGravity()
    {
        if (!isGrounded)
        {
            float gravityThisFrame = gravity;
            if (velocity.y < 0)
            {
                gravityThisFrame *= fallGravityMultiplier;
            }
            velocity.y -= gravityThisFrame * Time.deltaTime;
        }
    }

    private void ApplyVelocity()
    {
        controller.Move(velocity * Time.deltaTime);
    }

    public void SetHorizontalVelocity(Vector3 horizontalVelocity)
    {

```



```

        velocity.x = horizontalVelocity.x;
        velocity.z = horizontalVelocity.z;
    }

    private void OnDrawGizmos()
    {
        if (showGroundCheck)
        {
            Vector3 checkPosition = transform.position +
groundCheckOffset;
            Gizmos.color = isGrounded ? Color.green : Color.red;
            Gizmos.DrawWireSphere(checkPosition, groundCheckRadius);
        }
    }
}

```

Key Implementation Details

- **Immediate when possible:** If grounded, jump happens instantly
- **Buffer when not possible:** If airborne, store the input with a timer
- **Check every frame:** Once grounded + buffered, execute the jump
- **Clear after use:** Prevents accidental double activation

The Complete Forgiveness Package

Technique	What it forgives
<u>Coyote Time</u>	Pressing jump slightly AFTER leaving edge
Input Buffer	Pressing jump slightly BEFORE landing
Both together	Nearly all "unfair" missed jumps

Beyond Jump: What Else to Buffer

Action	Why Buffer
Jump	Press before landing, executes on land
Attack	Queue next attack during current one
Dash	Press during jump, executes when able
Dodge	Press during hitstun, executes on recovery

Common Issues

"Buffered input executes multiple times"

You're not clearing the buffer after execution. Set `jumpBuffered = false` after `ExecuteJump()`.

"Buffer doesn't seem to work"

Make sure you're recording input even when grounded. Test with `debugBuffer` enabled.

Combine With

- [Coyote Time](#) - together they cover both timing error directions
- [Hitstop](#) - buffer inputs during freeze frames
- [Dash](#) - buffer dash during other actions

Why Teach This

Input buffering teaches that **player intent matters more than precise timing**. The videogame should try to understand what the player wanted to do.

Teaching Sequence

1. Have students play and note when jumps "don't work"
2. Add input buffering - watch failures disappear
3. Extend to other actions (attacks, dashes)
4. Discuss: where would buffering be unwanted?

The Complementary Pair

Teach with coyote time. Together they illustrate complete timing forgiveness:

- **Coyote Time:** Forgives pressing AFTER leaving platform
- **Input Buffer:** Forgives pressing BEFORE being able to

Assessment Questions

- When would you NOT want to buffer an input?
- How does buffer time affect combat rhythm?
- Why must immediate execution still work?

Heritage Notes

Input buffering became essential with fighting games in the 1990s. Street Fighter II's combo system required inputting moves during other moves' animations. Without buffering, combos would be nearly impossible.

The Intent Problem

Input buffering attempts to solve the **intent problem**: the player knows what they want to do, but the videogame only sees button presses. Timing errors don't mean the player changed their mind.

A generous buffer says: "I trust that you meant to do this, so I'll do it when I can."

When Buffering Is Wrong

- **Defensive actions:** Blocking/dodging often requires precise timing for skill expression
- **Commitment actions:** If commitment matters, buffering undermines it
- **Context-sensitive:** Same button, different actions - buffering can cause wrong action

Related

- [Coyote Time](#) - the complementary forgiveness technique
- [Hitstop](#) - buffer inputs during freeze
- [Input Setup](#) - required InputReader system

Glossary Terms

- [Input Buffer](#)
- [Coyote Time](#)

Dash

*A burst of movement in a direction. Quick, committed, often with invincibility.
The defining gesture of modern action platformers.*

The Feel

Dash is **explosive and committed**. You press, you move, no take-backs. The commitment creates tension; the burst creates power.

Exposed Variables

Variable	Type	Default	What it does
<code>dashSpeed</code>	float	25.0	How fast you move during dash
<code>dashDuration</code>	float	0.15	How long the dash lasts (seconds)
<code>dashCooldown</code>	float	0.0	Time before you can dash again
<code>dashDirection</code>	enum	EightWay	Which directions are valid
<code>dashCount</code>	int	1	Dashes available per air period
<code>refreshOnGround</code>	bool	true	Does landing restore dash?
<code>invincibleDuringDash</code>	bool	false	Ignore damage while dashing?
<code>cancelWindow</code>	float	0.0	When can you cancel into other actions?
<code>momentumTransfer</code>	float	0.0	How much dash speed carries into normal movement

Tuning Guide

Celeste-style (precise, resource-based):

- `dashSpeed` : 25
- `dashDuration` : 0.12-0.15
- `dashCount` : 1
- `dashDirection` : EightWay

- `refreshOnGround : true`
- `invincibleDuringDash : true` (brief)
- `momentumTransfer : 0`

One dash per airtime, eight directions, full commitment

Hades-style (combat, spammable):

- `dashSpeed : 18`
- `dashDuration : 0.1`
- `dashCooldown : 0.3`
- `dashCount : unlimited` (via cooldown)
- `dashDirection : EightWay`
- `invincibleDuringDash : true`
- `cancelWindow : 0.05`

Faster, shorter dashes you can chain

Mega Man X-style (ground only, forward):

- `dashSpeed : 15`
- `dashDuration : 0.3`
- `dashDirection : Horizontal`
- `dashCooldown : 0`
- `groundOnly : true` (separate variable)

Longer duration, ground-based

Pseudocode

On Dash Press (if dashCount > 0):

- Record dash direction from input
- Set dash state
- Start dash timer
- If invincible: enable invincibility frames

During Dash:

- Override normal movement with dashSpeed in dashDirection
- Ignore gravity (optional)
- Check for cancel window inputs

On Dash End:

- Return to normal movement
- Apply momentum transfer if any
- Start cooldown timer if applicable
- Decrement dashCount

On Ground Contact:

- If refreshOnGround: restore dashCount

Prerequisite: These scripts use the [InputReader](#) pattern for device-agnostic input. Set that up first.



2D Platformer 3D (All Perspectives)

Unity Setup (2D)

1. **InputReader Setup:** See [Input Setup](#) page - ensure you have a "Dash" action (Button type) bound to Left Shift and Right Trigger

2. **Player Setup:**

- o Add `Rigidbody2D` (Gravity Scale: 3, Freeze Rotation Z: checked)
- o Add `Collider2D` (BoxCollider2D or CapsuleCollider2D)
- o Add the `Dash2D.cs` script below

3. **Ground Check:** Create empty child object at player's feet, assign to `groundCheck`

4. **Layers:** Create "Ground" layer, assign to ground objects, set in script's Inspector

The Script (2D)


```

// =====
// Dash2D.cs
// 2D dash with direction options, resource management,
// invincibility, and momentum transfer.
// Uses InputReader for device-agnostic input (gamepad + KB/M).
// Unity 6.x + New Input System
// =====

using UnityEngine;

public enum DashDirectionMode2D
{
    EightWay,          // All 8 cardinal + diagonal directions
    FourWay,           // Up, Down, Left, Right only
    Horizontal,        // Left and Right only
    FacingDirection    // Dash the way sprite is facing
}

[RequireComponent(typeof(Rigidbody2D))]
public class Dash2D : MonoBehaviour
{
    [Header("Dash Settings")]
    [Tooltip("Speed during dash (units/second)")]
    [SerializeField] private float dashSpeed = 25f;

    [Tooltip("Duration of the dash (seconds)")]
    [SerializeField] private float dashDuration = 0.15f;

    [Tooltip("Cooldown before next dash (0 = none)")]
    [SerializeField] private float dashCooldown = 0f;

    [Header("Direction")]
    [Tooltip("Which directions are valid")]
    [SerializeField] private DashDirectionMode2D directionMode =
DashDirectionMode2D.EightWay;

```

```

[Tooltip("Default direction if no input (1=right, -1=left)")]
[SerializeField] private int facingDirection = 1;

[Header("Resource Management")]
[Tooltip("Dashes available per air period")]
[SerializeField] private int maxDashCount = 1;

[Tooltip("Landing restores dashes?")]
[SerializeField] private bool refreshOnGround = true;

[Header("Combat")]
[Tooltip("Invincible while dashing?")]
[SerializeField] private bool invincibleDuringDash = false;

[Tooltip("Momentum carried after dash (0-1)")]
[SerializeField] [Range(0f, 1f)] private float momentumTransfer = 0f;

[Header("Ground Detection")]
[SerializeField] private Transform groundCheck;
[SerializeField] private float groundCheckRadius = 0.1f;
[SerializeField] private LayerMask groundLayer;

// Components
private Rigidbody2D rb;

// Dash state
private bool isDashing;
private float dashTimer;
private float cooldownTimer;
private int dashesRemaining;
private Vector2 dashDirection;
private float storedGravityScale;

// Ground state
private bool isGrounded;
private bool wasGrounded;

```

```

// Public properties
public bool IsDashing => isDashing;
public bool IsInvincible => isDashing && invincibleDuringDash;
public int DashesRemaining => dashesRemaining;
public bool CanDash => !isDashing && dashesRemaining > 0 &&
cooldownTimer <= 0f;

void Awake()
{
    rb = GetComponent<Rigidbody2D>();
    storedGravityScale = rb.gravityScale;
    dashesRemaining = maxDashCount;
}

void OnEnable()
{
    InputReader.Instance.OnDashPressed += TryDash;
}

void OnDisable()
{
    InputReader.Instance.OnDashPressed -= TryDash;
}

void Update()
{
    if (cooldownTimer > 0f)
        cooldownTimer -= Time.deltaTime;

    CheckGroundState();
}

void FixedUpdate()
{
    if (isDashing)
        ProcessDash();
}

```

```
private void TryDash()
{
    if (!CanDash) return;

    dashDirection = CalculateDashDirection();
    if (dashDirection == Vector2.zero) return;

    StartDash();
}

private Vector2 CalculateDashDirection()
{
    Vector2 input = InputReader.Instance.MoveInput;

    switch (directionMode)
    {
        case DashDirectionMode2D.EightWay:
            if (input.magnitude > 0.1f)
                return input.normalized;
            return new Vector2(facingDirection, 0);

        case DashDirectionMode2D.FourWay:
            if (Mathf.Abs(input.x) > Mathf.Abs(input.y))
                return new Vector2(Mathf.Sign(input.x), 0);
            else if (Mathf.Abs(input.y) > 0.1f)
                return new Vector2(0, Mathf.Sign(input.y));
            return new Vector2(facingDirection, 0);

        case DashDirectionMode2D.Horizontal:
            if (Mathf.Abs(input.x) > 0.1f)
                return new Vector2(Mathf.Sign(input.x), 0);
            return new Vector2(facingDirection, 0);

        case DashDirectionMode2D.FacingDirection:
            return new Vector2(facingDirection, 0);
    }
}
```

```

        default:
            return new Vector2(facingDirection, 0);
        }
    }

    private void StartDash()
    {
        isDashing = true;
        dashTimer = dashDuration;
        dashesRemaining--;

        storedGravityScale = rb.gravityScale;
        rb.gravityScale = 0f;
        rb.linearVelocity = dashDirection * dashSpeed;
    }

    private void ProcessDash()
    {
        dashTimer -= Time.fixedDeltaTime;
        rb.linearVelocity = dashDirection * dashSpeed;

        if (dashTimer <= 0f)
            EndDash();
    }

    private void EndDash()
    {
        isDashing = false;
        rb.gravityScale = storedGravityScale;
        rb.linearVelocity *= momentumTransfer;
        cooldownTimer = dashCooldown;
    }

    private void CheckGroundState()
    {
        wasGrounded = isGrounded;
    }

```

```

        if (groundCheck != null)
        {
            isGrounded = Physics2D.OverlapCircle(
                groundCheck.position,
                groundCheckRadius,
                groundLayer
            );
        }

        if (refreshOnGround && isGrounded && !wasGrounded)
            dashesRemaining = maxDashCount;
    }

    // Public API
    public void RefreshDashes() => dashesRemaining = maxDashCount;
    public void SetFacingDirection(int dir) => facingDirection = dir > 0
    ? 1 : -1;
    public void CancelDash() { if (isDashing) EndDash(); }

    void OnDrawGizmosSelected()
    {
        if (groundCheck != null)
        {
            Gizmos.color = Color.green;
            Gizmos.DrawWireSphere(groundCheck.position,
groundCheckRadius);
        }
    }
}

```

Unity Setup (3D - Third Person, First Person, or Isometric)

1. **InputReader Setup:** See [Input Setup](#) page - ensure you have a "Dash" action (Button type)
2. **Player Setup:**

- Add `CharacterController` component
- Add the `Dash3D.cs` script below

3. **Optional - Assign Camera:** For camera-relative dashing, assign your main camera to the `playerCamera` field

The Script (3D)


```
// =====
// Dash3D.cs
// 3D dash for third-person, first-person, or isometric.
// Uses CharacterController and InputReader for device-agnostic
// input (gamepad + KB/M). Works with camera-relative movement.
// Unity 6.x + New Input System
// =====

using UnityEngine;

public enum DashDirectionMode3D
{
    EightWay,      // 8 directions on XZ plane
    FourWay,       // Cardinal directions only
    Forward,       // Always forward
    CameraRelative // Based on camera facing
}

[RequireComponent(typeof(CharacterController))]
public class Dash3D : MonoBehaviour
{
    [Header("Dash Settings")]
    [Tooltip("Speed during dash (units/second)")]
    [SerializeField] private float dashSpeed = 25f;

    [Tooltip("Duration of the dash (seconds)")]
    [SerializeField] private float dashDuration = 0.15f;

    [Tooltip("Cooldown before next dash (0 = none)")]
    [SerializeField] private float dashCooldown = 0f;

    [Header("Direction")]
    [Tooltip("Which directions are valid")]
    [SerializeField] private DashDirectionMode3D directionMode =
DashDirectionMode3D.CameraRelative;

```

```
[Tooltip("Camera for relative movement (optional)")]
[SerializeField] private Transform playerCamera;

[Header("Resource Management")]
[Tooltip("Dashes available per air period")]
[SerializeField] private int maxDashCount = 1;

[Tooltip("Landing restores dashes?")]
[SerializeField] private bool refreshOnGround = true;

[Header("Combat")]
[Tooltip("Invincible while dashing?")]
[SerializeField] private bool invincibleDuringDash = false;

[Tooltip("Momentum carried after dash (0-1)")]
[SerializeField] [Range(0f, 1f)] private float momentumTransfer = 0f;

[Header("Vertical Behavior")]
[Tooltip("Ignore gravity during dash?")]
[SerializeField] private bool suspendGravity = true;

[Tooltip("Allow vertical dashing (up/down)?")]
[SerializeField] private bool allowVerticalDash = false;

// Components
private CharacterController controller;

// Dash state
private bool isDashing;
private float dashTimer;
private float cooldownTimer;
private int dashesRemaining;
private Vector3 dashDirection;
private Vector3 dashVelocity;

// Ground state
private bool wasGrounded;
```

```

// Public properties
public bool IsDashing => isDashing;
public bool IsInvincible => isDashing && invincibleDuringDash;
public int DashesRemaining => dashesRemaining;
public bool CanDash => !isDashing && dashesRemaining > 0 &&
cooldownTimer <= 0f;

// For external movement scripts to read
public Vector3 DashVelocity => isDashing ? dashVelocity :
Vector3.zero;

void Awake()
{
    controller = GetComponent<CharacterController>();
    dashesRemaining = maxDashCount;

    if (playerCamera == null)
        playerCamera = Camera.main?.transform;
}

void OnEnable()
{
    InputReader.Instance.OnDashPressed += TryDash;
}

void OnDisable()
{
    InputReader.Instance.OnDashPressed -= TryDash;
}

void Update()
{
    if (cooldownTimer > 0f)
        cooldownTimer -= Time.deltaTime;

    CheckGroundState();
}

```

```

        if (isDashing)
            ProcessDash();
    }

    private void TryDash()
    {
        if (!CanDash) return;

        dashDirection = CalculateDashDirection();
        if (dashDirection == Vector3.zero) return;

        StartDash();
    }

    private Vector3 CalculateDashDirection()
    {
        Vector2 input = InputReader.Instance.MoveInput;
        Vector3 direction = Vector3.zero;

        switch (directionMode)
        {
            case DashDirectionMode3D.CameraRelative:
                if (playerCamera != null && input.magnitude > 0.1f)
                {
                    Vector3 camForward = playerCamera.forward;
                    Vector3 camRight = playerCamera.right;

                    if (!allowVerticalDash)
                    {
                        camForward.y = 0;
                        camRight.y = 0;
                        camForward.Normalize();
                        camRight.Normalize();
                    }

                    direction = (camForward * input.y + camRight *

```

```

        input.x).normalized;
    }
    else
    {
        direction = transform.forward;
    }
    break;

    case DashDirectionMode3D.EightWay:
        if (input.magnitude > 0.1f)
            direction = new Vector3(input.x, 0,
input.y).normalized;
        else
            direction = transform.forward;
        break;

    case DashDirectionMode3D.FourWay:
        if (Mathf.Abs(input.x) > Mathf.Abs(input.y))
            direction = new Vector3(Mathf.Sign(input.x), 0, 0);
        else if (Mathf.Abs(input.y) > 0.1f)
            direction = new Vector3(0, 0, Mathf.Sign(input.y));
        else
            direction = transform.forward;
        break;

    case DashDirectionMode3D.Forward:
        direction = transform.forward;
        break;
    }

    return direction;
}

private void StartDash()
{
    isDashing = true;
    dashTimer = dashDuration;
}

```

```
        dashesRemaining--;  
        dashVelocity = dashDirection * dashSpeed;  
    }  
  
    private void ProcessDash()  
    {  
        dashTimer -= Time.deltaTime;  
  
        Vector3 movement = dashVelocity * Time.deltaTime;  
  
        if (!suspendGravity && !controller.isGrounded)  
            movement.y -= 9.81f * Time.deltaTime;  
  
        controller.Move(movement);  
  
        if (dashTimer <= 0f)  
            EndDash();  
    }  
  
    private void EndDash()  
    {  
        isDashing = false;  
        dashVelocity *= momentumTransfer;  
        cooldownTimer = dashCooldown;  
    }  
  
    private void CheckGroundState()  
    {  
        bool isGrounded = controller.isGrounded;  
  
        if (refreshOnGround && isGrounded && !wasGrounded)  
            dashesRemaining = maxDashCount;  
  
        wasGrounded = isGrounded;  
    }  
  
    // Public API
```



```

public void RefreshDashes() => dashesRemaining = maxDashCount;
public void CancelDash() { if (isDashing) EndDash(); }

void OnDrawGizmosSelected()
{
    if (isDashing)
    {
        Gizmos.color = Color.cyan;
        Gizmos.DrawRay(transform.position, dashDirection * 2f);
    }
}

// =====
// INTEGRATION WITH YOUR MOVEMENT SCRIPT:
//
// If you have a separate movement script using CharacterController,
// check for IsDashing before applying your own movement:
//
// void Update()
// {
//     if (dash.IsDashing) return; // Let dash handle movement
//
//     // Your normal movement code here...
// }
//
// Or combine velocities by reading DashVelocity:
//
// Vector3 totalVelocity = myVelocity + dash.DashVelocity;
// controller.Move(totalVelocity * Time.deltaTime);
// =====

```

Common Issues

"The dash feels weak"

Increase `dashSpeed`. Dash should feel noticeably faster than running.

"I can't control where I'm going"

Check `dashDirection` . If set to four-way, you might expect eight. Also check input handling for diagonals.

"The dash is too short/long"

Adjust `dashDuration` . 0.1-0.2 is typical. Shorter = snappier; longer = more distance.

"It feels like I have infinite dashes"

Check `refreshOnGround` and `dashCount` . If grounded resets dash instantly, add a brief window.

"I'm taking damage during dash"

`invincibleDuringDash` needs to be true, and your damage system needs to check for invincibility frames.

Direction Systems

System	Directions	Use Case
FourWay	Up, Down, Left, Right	Simple, classic
EightWay	+ Diagonals	Precise, modern platformers
Horizontal	Left, Right only	Ground-focused videogames
Aim-Based	Follows mouse/stick direction	Twin-stick, shooters
Movement-Based	Follows current velocity	Momentum-focused

Combine With

- [Coyote Time](#) - brief dash window after leaving platform
- [Double Jump](#) - dash extends air mobility
- [Hitstop](#) - freeze when dash-attack connects
- [Screen Shake](#) - on dash-attack impact
- [Squash & Stretch](#) - visual stretch during dash

Why Teach Dash

Dash introduces students to **resource management in movement**. Unlike jump (which is almost always available), dash often has a count or cooldown. This creates decisions:

"Should I dash now or save it?"

Teaching Sequence

1. Start with unlimited dashes, no cooldown - feel the basic mechanic
2. Add `dashCount = 1` with `refreshOnGround` - introduces resource thinking
3. Add invincibility - changes how dash is used (evasion vs. movement)
4. Experiment with direction systems

Common Student Mistakes

Making dash too similar to running: If dash speed is only 1.5x run speed, it doesn't feel special. It needs to feel like a distinct mode.

Forgetting cancel windows: Students implement dash that can't be interrupted. This feels unresponsive. Even committed dashes usually have late cancel windows.

Ignoring the resource: When `dashCount` is treated as infinite, students miss the design opportunity. One dash per airtime creates meaningful choices.

Assessment

- Can they articulate why Celeste uses one dash per airtime?
- Can they explain how invincibility changes the dash's role?
- Can they design a dash that fits a specific videogame genre?

Exercise: Dash Role Analysis

Have students play three videogames with dashes (Celeste, Hades, Mega Man X). For each:

- What role does the dash serve? (Traversal? Evasion? Combat?)

- How does the resource system support that role?
- What would change if you removed dash invincibility?

Heritage Notes

The dash evolved from several sources:

Mega Man X (1993) - Ground dash that changed the pace of action platformers. Made movement itself feel aggressive.

Fighting games - Air dashes in *Marvel vs. Capcom*, *Guilty Gear* created aerial mobility that felt distinct from jumping.

Modern platformers - *Celeste* refined the single-dash-per-air model into a precise tool.

The dash is now so standard that its *absence* is notable. A platformer without dash feels deliberately retro.

Dash as Commitment

Dash represents a design philosophy: **powerful but costly**. Unlike jump (low commitment, low power), dash trades control for speed. You're locked into a direction.

This connects to [Permissions](#): dash *allows* burst movement but *requires* commitment. It *forbids* changing direction mid-dash. The permissions shape the feel.

The Invincibility Question

Adding invincibility to dash transforms it from *movement tool* to *survival tool*. This changes everything:

- Players dash *through* attacks, not around them
- Dash timing becomes a core skill
- Enemy design must account for i-frames

The decision to include invincibility is one of the most impactful in action videogame design.

The Celeste Model

Celeste made the dash-per-airtime model canonical for precision platformers. Key insights:

- One dash per air period creates meaningful scarcity
- Eight directions enables precise positioning
- Refreshing on ground makes failure recoverable
- Brief invincibility forgives close calls

This model is now the default expectation. Deviating from it is a deliberate choice.

Unresolved Questions

- How much should momentum carry after dash? Full stop vs. smooth transition creates very different feels.
- Should dash refresh on other surfaces (walls, enemies)? Each refresh point changes level design possibilities.
- Is the Celeste model becoming too dominant? Are we losing variety in dash design?

Related

- [Gesture](#) - dash as canonical gesture
- [Aesthetic Heritage](#) - dash lineage
- [Trail Effect](#) - common visual pairing

Glossary Terms

- [Invincibility Frames](#) - damage immunity period
- [Cancel Window](#) - when you can interrupt an action
- [Juice](#) - feedback techniques that make dash feel powerful

Squash & Stretch

Deform the character sprite or model to emphasize motion. Stretch when moving fast, squash on impact. The oldest animation principle, applied to videogames.

The Feel

Squash and stretch adds **life and weight**. A rigid character feels robotic. A squashing/stretching character feels organic, even when it's a geometric shape.

It communicates physics through shape: "I'm compressing because I hit something" or "I'm elongating because I'm moving fast."

Exposed Variables

Variable	Type	Default	What it does
<code>stretchAmount</code>	float	0.2	Maximum stretch (1.2 = 20% taller)
<code>squashAmount</code>	float	0.2	Maximum squash (0.8 = 20% shorter)
<code>returnSpeed</code>	float	10.0	How fast to return to normal
<code>velocityThreshold</code>	float	5.0	Velocity needed to trigger stretch

Tuning Guide

For subtle realism:

- `stretchAmount` = 0.05 - 0.1
- `squashAmount` = 0.05 - 0.1
- Barely noticeable but adds life

Reference: Celeste (subtle character deformation)

For cartoony feel:

- `stretchAmount` = 0.2 - 0.4
- `squashAmount` = 0.2 - 0.4
- Obvious, playful, bouncy

Reference: Hollow Knight, Ori

For extreme stylization:

- `stretchAmount = 0.5+`
- `squashAmount = 0.4+`
- Very exaggerated, full cartoon

Reference: Cuphead, classic Disney

Pseudocode

▼ Pseudocode

Every Frame:

- Get vertical velocity
- If falling fast: stretch vertically, compress horizontally
- If rising fast: stretch vertically, compress horizontally
- If grounded: return to normal scale

On Land:

- Apply squash (compress vertically, expand horizontally)
- Lerp back to normal over time

Key: Preserve volume - if you stretch Y, compress X proportionally

Unity Setup (2D)

1. Setup Visual Hierarchy:

- Your player object has Rigidbody2D and Collider2D
- Create a child object called "Visual" or "Sprite"
- Move your SpriteRenderer to this child
- The script scales the child, not the physics object

2. Add the Script:

- Add `SquashStretch2D.cs` to the player (parent object)
- Assign the visual child to `visualTransform`
- The physics collider stays unaffected

3. Create Ground Check (optional for landing squash):

- Create empty child "GroundCheck" at player's feet
- Assign to script's `groundCheck` field
- Configure ground layer mask

4. Tune the Feel:

- Subtle: $\text{stretchAmount} / \text{squashAmount} = 0.05\text{-}0.1$
- Cartoony: $\text{stretchAmount} / \text{squashAmount} = 0.2\text{-}0.4$
- Extreme: $\text{stretchAmount} / \text{squashAmount} = 0.5+$

The Script (2D)

▼ SquashStretch2D.cs

```

// =====
// SquashStretch2D.cs
// Deforms a visual sprite based on velocity and landing.
// Attach to player, assign child visual transform.
// Physics collider is NOT affected - only visuals.
// Unity 6.x
// =====

using UnityEngine;

[RequireComponent(typeof(Rigidbody2D))]
public class SquashStretch2D : MonoBehaviour
{
    // =====
    // REFERENCES - Assign in Inspector
    // =====

    [Header("Visual Transform")]
    [Tooltip("The child transform to scale (NOT the physics object!)")]
    [SerializeField] private Transform visualTransform;

    [Header("Ground Detection (for landing squash)")]
    [Tooltip("Empty transform at player's feet")]
    [SerializeField] private Transform groundCheck;

    [Tooltip("Radius of ground check")]
    [SerializeField] [Range(0.05f, 0.5f)] private float groundCheckRadius
= 0.1f;

    [Tooltip("Which layers count as ground")]
    [SerializeField] private LayerMask groundLayer;

    // =====
    // SQUASH & STRETCH PARAMETERS
    // =====

```

```
[Header("Stretch (during fast vertical movement)")]
[Tooltip("Maximum vertical stretch (0.2 = 20% taller at max
velocity)")]
[SerializeField] [Range(0f, 1f)] private float stretchAmount = 0.2f;

[Tooltip("Velocity needed to reach full stretch")]
[SerializeField] [Range(1f, 30f)] private float stretchVelocityMax =
15f;

[Header("Squash (on landing)")]
[Tooltip("Maximum squash on landing (0.3 = 30% shorter)")]
[SerializeField] [Range(0f, 0.5f)] private float squashAmount =
0.25f;

[Tooltip("Fall velocity needed for full squash")]
[SerializeField] [Range(1f, 30f)] private float squashVelocityMax =
12f;

[Header("Recovery")]
[Tooltip("How fast to return to normal scale")]
[SerializeField] [Range(1f, 30f)] private float returnSpeed = 12f;

[Header("Horizontal Stretch (for dashing)")]
[Tooltip("Enable horizontal stretch based on horizontal velocity")]
[SerializeField] private bool enableHorizontalStretch = true;

[Tooltip("Maximum horizontal stretch")]
[SerializeField] [Range(0f, 0.5f)] private float
horizontalStretchAmount = 0.15f;

[Tooltip("Horizontal velocity for full stretch")]
[SerializeField] [Range(1f, 50f)] private float
horizontalStretchVelocityMax = 20f;

[Header("Debug")]
[Tooltip("Show debug info")]
[SerializeField] private bool debugMode = false;
```

```

// =====
// INTERNAL STATE
// =====

private Rigidbody2D rb;
private Vector3 targetScale = Vector3.one;
private Vector3 baseScale = Vector3.one;
private bool isGrounded = false;
private bool wasGrounded = false;
private float lastFallVelocity = 0f;

// =====
// UNITY LIFECYCLE
// =====

void Awake()
{
    rb = GetComponent<Rigidbody2D>();

    // Store the original scale of the visual
    if (visualTransform != null)
    {
        baseScale = visualTransform.localScale;
    }
}

void Update()
{
    if (visualTransform == null) return;

    // Check ground state
    CheckGroundState();

    // Calculate target scale based on current state
    CalculateTargetScale();
}

```

```

        // Smoothly interpolate to target scale
        visualTransform.localScale = Vector3.Lerp(
            visualTransform.localScale,
            targetScale,
            returnSpeed * Time.deltaTime
        );
    }

    // =====
    // SCALE CALCULATION
    // =====

    /// <summary>
    /// Calculate what the scale should be based on velocity
    /// </summary>
    private void CalculateTargetScale()
    {
        float scaleX = baseScale.x;
        float scaleY = baseScale.y;

        float velocityY = rb.linearVelocity.y;
        float velocityX = Mathf.Abs(rb.linearVelocity.x);

        // -----
        // VERTICAL STRETCH (rising or falling fast)
        // -----
        if (!isGrounded && Mathf.Abs(velocityY) > 0.5f)
        {
            // Normalize velocity to 0-1 range
            float stretchFactor = Mathf.Clamp01(
                Mathf.Abs(velocityY) / stretchVelocityMax
            );

            // Apply stretch: taller vertically
            float stretchY = 1f + (stretchAmount * stretchFactor);

            // Preserve volume: compress horizontally

```



```

        // If Y grows by 20%, X shrinks to 1/1.2 = 0.833
        float stretchX = 1f / stretchY;

        scaleY = baseScale.y * stretchY;
        scaleX = baseScale.x * stretchX;

        if (debugMode && stretchFactor > 0.1f)
        {
            Debug.Log($"Vertical stretch: {stretchFactor:F2} (vel: {velocityY:F1})");
        }
    }

    // -----
    // HORIZONTAL STRETCH (dashing/running fast)
    // -----
    if (enableHorizontalStretch && velocityX > 1f)
    {
        float hStretchFactor = Mathf.Clamp01(
            velocityX / horizontalStretchVelocityMax
        );

        // Apply horizontal stretch: wider horizontally
        float hStretchX = 1f + (horizontalStretchAmount *
hStretchFactor);

        // Preserve volume: compress vertically
        float hStretchY = 1f / hStretchX;

        // Blend with vertical stretch (take the more extreme)
        scaleX = Mathf.Max(scaleX, baseScale.x * hStretchX);
        scaleY = Mathf.Min(scaleY, baseScale.y * hStretchY);
    }

    // Track fall velocity for landing squash
    if (velocityY < 0)
    {

```

```

        lastFallVelocity = Mathf.Min(lastFallVelocity, velocityY);
    }

    targetScale = new Vector3(scaleX, scaleY, baseScale.z);
}

// =====
// GROUND DETECTION + LANDING SQUASH
// =====

/// <summary>
/// Check ground and trigger squash on landing
/// </summary>
private void CheckGroundState()
{
    wasGrounded = isGrounded;

    if (groundCheck != null)
    {
        isGrounded = Physics2D.OverlapCircle(
            groundCheck.position,
            groundCheckRadius,
            groundLayer
        );
    }

    // Just landed - trigger squash!
    if (isGrounded && !wasGrounded)
    {
        ApplyLandingSquash();
    }

    // Reset fall velocity tracking when grounded
    if (isGrounded)
    {
        lastFallVelocity = 0f;
    }
}

```

```

}

/// <summary>
/// Apply instant squash when landing
/// </summary>
private void ApplyLandingSquash()
{
    // Calculate squash based on fall velocity
    float squashFactor = Mathf.Clamp01(
        Mathf.Abs(lastFallVelocity) / squashVelocityMax
    );

    // Apply squash: shorter vertically, wider horizontally
    float squashY = 1f - (squashAmount * squashFactor);
    float squashX = 1f / squashY; // Preserve volume

    // Apply immediately (then returnSpeed lerps back to normal)
    visualTransform.localScale = new Vector3(
        baseScale.x * squashX,
        baseScale.y * squashY,
        baseScale.z
    );

    if (debugMode)
    {
        Debug.Log($"Landing squash: {squashFactor:F2} (fall vel: {lastFallVelocity:F1})");
    }
}

// =====
// PUBLIC METHODS - Call from other scripts
// =====

/// <summary>
/// Manually trigger a squash (e.g., on hit, on jump anticipation)
/// </summary>

```

```

/// <param name="intensity">0-1, how much to squash</param>
public void TriggerSquash(float intensity = 1f)
{
    float squashY = 1f - (squashAmount * intensity);
    float squashX = 1f / squashY;

    visualTransform.localScale = new Vector3(
        baseScale.x * squashX,
        baseScale.y * squashY,
        baseScale.z
    );
}

/// <summary>
/// Manually trigger a vertical stretch (e.g., on jump launch)
/// </summary>
/// <param name="intensity">0-1, how much to stretch</param>
public void TriggerStretch(float intensity = 1f)
{
    float stretchY = 1f + (stretchAmount * intensity);
    float stretchX = 1f / stretchY;

    visualTransform.localScale = new Vector3(
        baseScale.x * stretchX,
        baseScale.y * stretchY,
        baseScale.z
    );
}

/// <summary>
/// Immediately reset to normal scale
/// </summary>
public void ResetScale()
{
    if (visualTransform != null)
    {
        visualTransform.localScale = baseScale;
    }
}

```

```

        targetScale = baseScale;
    }
}

// =====
// DEBUG VISUALIZATION
// =====

void OnDrawGizmosSelected()
{
    if (groundCheck != null)
    {
        Gizmos.color = Color.yellow;
        Gizmos.DrawWireSphere(groundCheck.position,
groundCheckRadius);
    }
}

// =====
// USAGE NOTES:
//
// 1. IMPORTANT: Only scale the VISUAL, not the physics object!
//    Structure your player like this:
//    - Player (has Rigidbody2D, Collider2D, SquashStretch2D)
//    - Visual (child with SpriteRenderer - this gets scaled)
//
// 2. Volume preservation formula:
//    If scaleY = 1.2 (20% taller), then scaleX = 1/1.2 = 0.833
//    This keeps the visual "area" roughly constant
//
// 3. Call TriggerSquash() manually for:
//    - Jump anticipation (brief squash before launch)
//    - Taking damage
//    - Charging up an attack
//
// 4. Call TriggerStretch() manually for:

```

```
// - Jump launch moment
// - Dash start
// - Attack follow-through
//
// 5. Combine with other juice:
// void OnJump()
// {
//     squashStretch.TriggerSquash(0.5f); // Anticipation
//     // Small delay, then...
//     squashStretch.TriggerStretch(0.8f); // Launch
// }
//
// 6. For 3D: Same principles apply, but scale on local Y axis
//     and compensate on X and Z to preserve volume
// =====
```

Common Issues

"It looks wrong/distorted"

You're probably not preserving volume. When you stretch on Y, compress on X by the inverse amount. Total area should stay roughly constant.

"It happens too suddenly"

Use lerp/smoothing for the return to normal. The squash can be instant, but recovery should ease out.

"It doesn't match the movement"

Make sure stretch is based on velocity, not just "jumping" state. A slow rise shouldn't stretch as much as a fast dash.

"Colliders are getting messed up"

Only deform the visual, not the physics collider. Keep your hitbox at constant size.

When to Apply

Moment	Deformation
Fast falling	Stretch vertically
Landing	Squash on impact
Jump anticipation	Brief squash before launch
At jump apex	Return to normal briefly
Dash/fast horizontal	Stretch horizontally
Hit/damage	Squash in hit direction

Combine With

- [Basic Jump](#) / [Variable Jump](#) - squash/stretch the jump arc
- [Dash](#) - stretch during dash
- [Hitstop](#) - squash at moment of impact

Why Teach This

Squash and stretch is the first of Disney's 12 Principles of Animation. It's been making things look alive since the 1930s. Teaching it connects videogame development to a century of animation craft.

The Volume Principle

The key insight: **preserve volume**. A ball that squashes down must expand outward. This mimics how real soft objects deform and makes the deformation feel physically plausible even when exaggerated.

Mathematically: if you scale Y by 0.8, scale X by $1/0.8 = 1.25$

Teaching Sequence

1. Show animation references (bouncing ball tutorial)
2. Implement basic squash on landing
3. Add stretch based on velocity
4. Tune for different aesthetic goals (realistic vs. cartoony)

The Aesthetic Choice

Amount of squash/stretch is a **style choice**, not a technical one. A realistic military shooter might use almost none. A cartoon platformer might use extreme amounts. Students should calibrate to their intended aesthetic.

Assessment Questions

- Why does preserving volume matter?
- How would you adjust squash/stretch for a realistic vs. cartoony videogame?
- What other animation principles apply to videogame characters?

Heritage Notes

Squash and stretch was codified by Disney animators in the 1930s. The bouncing ball exercise - still taught to animators today - is pure squash and stretch training. The principle predates videogames by decades.

Videogames inherited this from animation, but with a difference: in videogames, the deformation responds to **player input**, not predetermined keyframes. This makes it feel reactive and alive in a new way.

Why It Works

Our eyes track motion through shape change. A rigid object moving looks mechanical. A deforming object looks organic because real living things (and soft materials) deform under force.

Even geometric shapes like circles seem "alive" when they squash and stretch. This is why the bouncing ball is such an effective demonstration - a circle becomes a character just through deformation.

The 4 A's Application

Squash and stretch is pure Art - it doesn't change the Action or the Arc, only how things look. But by changing the Art, it transforms the Atmosphere. The same jump feels playful or weighty depending on deformation style.

The 12 Principles in Videogames

Disney's 12 Principles of Animation all apply to videogames:

1. Squash and Stretch (this page)
2. Anticipation (wind-up before jump)
3. Staging (camera, composition)
4. Straight Ahead / Pose to Pose (animation approach)
5. Follow Through / Overlapping Action (secondary motion)

6. Slow In / Slow Out (easing)
7. Arcs (movement paths)
8. Secondary Action (hair, clothes)
9. Timing (speed of actions)
10. Exaggeration (stylization)
11. Solid Drawing (3D form)
12. Appeal (character design)

Videogame animators learn these principles and apply them to interactive contexts.

Related

- [Basic Jump](#) - where squash/stretch is most visible
- [Hitstop](#) - squash at impact moment
- [The 4 A's](#) - Art component

Glossary Terms

- [Juice](#)

Screen Shake

Camera shake on impact, explosion, or emphasis. One of the most powerful feedback tools - and one of the most overused.



Nuclear Throne - Screenshake as juice, Vlambeer's signature impact.

Courtesy of Vlambeer

The Feel

Screen shake says: **something important happened**. It's emphasis. Exclamation point. Used well, it makes impacts feel weighty. Used poorly, it makes videogames feel noisy.

Exposed Variables

Variable	Type	Default	What it does
<code>intensity</code>	float	5.0	Maximum pixel offset from center
<code>duration</code>	float	0.2	How long shake lasts (seconds)
<code>decayType</code>	enum	Linear	How intensity decreases (Linear, Exponential, None)
<code>frequency</code>	float	30.0	How many shake "hits" per second
<code>rotationIntensity</code>	float	0.0	Camera rotation shake (degrees)
<code>shakePattern</code>	enum	Random	Pattern of offset (Random, Perlin, Directional)
<code>direction</code>	Vector2	null	For Directional pattern: bias direction

Tuning Guide

Subtle impact (footsteps, small hits):

- `intensity` : 1-3
- `duration` : 0.05-0.1
- `decayType` : Exponential
- `rotationIntensity` : 0

Medium impact (melee hits, jumps landing):

- `intensity` : 5-10

- `duration` : 0.1-0.2
- `decayType` : Linear
- `rotationIntensity` : 0-1

Heavy impact (explosions, boss hits):

- `intensity` : 15-25
- `duration` : 0.2-0.4
- `decayType` : Exponential
- `rotationIntensity` : 1-3

Directional impact (knockback, recoil):

- `shakePattern` : Directional
- `direction` : Opposite of force direction
- Shorter `duration` , higher initial `intensity`

Pseudocode

▼ Pseudocode

On Shake Trigger:

- Set shake timer to duration
- Store base intensity

Every Frame (while shaking):

- Calculate current intensity (based on decayType)
- Generate offset based on shakePattern
- Apply offset to camera position
- If rotationIntensity > 0: apply rotation offset
- Decrement timer

On Shake End:

- Smoothly return camera to base position

Unity Setup

1. Add to Camera:

- Add `ScreenShake.cs` to your Main Camera
- Or create an empty "CameraShake" child of Camera
- The script uses singleton pattern - easy access from anywhere

2. Call from Game Events:

- `ScreenShake.Instance.Shake(intensity, duration);`
- Or use presets: `ScreenShake.Instance.ShakeLight();`

3. IMPORTANT - Accessibility:

- Connect the `globalIntensityMultiplier` to your settings menu
- Set to 0 to disable shake entirely
- Expose as a slider (0-100%)
- This is required accessibility, not optional polish

4. Camera Follow Note:

- If using Cinemachine, use their impulse system instead
- If using custom camera follow, apply shake AFTER follow logic

The Script


```

// =====
// ScreenShake.cs
// Camera shake with multiple patterns, decay types, and
// accessibility support. Singleton for easy access.
// Unity 6.x
// =====

using UnityEngine;

public enum ShakeDecayType
{
    None,          // Constant intensity throughout
    Linear,         // Steady decrease
    Exponential    // Fast decrease that slows down
}

public enum ShakePattern
{
    Random,        // Random direction each frame
    Perlin,        // Smooth noise (less jittery)
    Directional    // Biased toward a direction
}

public class ScreenShake : MonoBehaviour
{
    // =====
    // SINGLETON
    // =====

    public static ScreenShake Instance { get; private set; }

    // =====
    // ACCESSIBILITY - CRITICAL!
    // =====

    [Header("Accessibility Settings")]

```



```

[Tooltip("Global shake intensity multiplier (0 = disabled). Connect
to settings menu!")]
[SerializeField] [Range(0f, 1f)] private float
globalIntensityMultiplier = 1f;

// =====
// PRESET INTENSITIES
// =====

[Header("Preset Values")]
[Tooltip("Light shake: intensity")]
[SerializeField] private float lightIntensity = 0.1f;
[Tooltip("Light shake: duration")]
[SerializeField] private float lightDuration = 0.05f;

[Tooltip("Medium shake: intensity")]
[SerializeField] private float mediumIntensity = 0.3f;
[Tooltip("Medium shake: duration")]
[SerializeField] private float mediumDuration = 0.15f;

[Tooltip("Heavy shake: intensity")]
[SerializeField] private float heavyIntensity = 0.6f;
[Tooltip("Heavy shake: duration")]
[SerializeField] private float heavyDuration = 0.25f;

[Header("Default Settings")]
[Tooltip("Default decay type for all shakes")]
[SerializeField] private ShakeDecayType defaultDecayType =
ShakeDecayType.Exponential;

[Tooltip("Default shake pattern")]
[SerializeField] private ShakePattern defaultPattern =
ShakePattern.Perlin;

[Header("Rotation Shake")]
[Tooltip("Enable rotation shake (can cause more motion sickness)")]
[SerializeField] private bool enableRotationShake = false;

```

```

[Tooltip("Maximum rotation in degrees")]
[SerializeField] [Range(0f, 5f)] private float maxRotation = 1f;

[Header("Smoothing")]
[Tooltip("How fast shake returns to zero when done")]
[SerializeField] [Range(1f, 30f)] private float recoverySpeed = 15f;

[Header("Debug")]
[SerializeField] private bool debugMode = false;

// =====
// INTERNAL STATE
// =====

private Vector3 originalPosition;
private Quaternion originalRotation;

// Current shake state
private float currentIntensity = 0f;
private float currentDuration = 0f;
private float shakeTimer = 0f;
private ShakeDecayType currentDecayType;
private ShakePattern currentPattern;
private Vector2 shakeDirection = Vector2.zero;

// Perlin noise seeds (for smooth shake)
private float perlinSeedX;
private float perlinSeedY;

// Current offset (for smooth recovery)
private Vector3 currentOffset = Vector3.zero;
private float currentRotationOffset = 0f;

// =====
// PUBLIC PROPERTIES
// =====

```

```

    /// <summary>
    /// True while shake is active
    /// </summary>
    public bool IsShaking => shakeTimer > 0f;

    /// <summary>
    /// Get/set the global intensity (for settings menu)
    /// </summary>
    public float GlobalIntensity
    {
        get => globalIntensityMultiplier;
        set => globalIntensityMultiplier = Mathf.Clamp01(value);
    }

    // =====
    // UNITY LIFECYCLE
    // =====

    void Awake()
    {
        // Singleton setup
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }
        Instance = this;

        // Store original transform
        originalPosition = transform.localPosition;
        originalRotation = transform.localRotation;

        // Random perlin seeds
        perlinSeedX = Random.Range(0f, 100f);
        perlinSeedY = Random.Range(0f, 100f);
    }

```

```

void LateUpdate()
{
    // LateUpdate so shake happens AFTER camera follow

    if (shakeTimer > 0f)
    {
        ProcessShake();
    }
    else
    {
        // Smooth recovery to original position
        RecoverFromShake();
    }
}

// =====
// SHAKE PROCESSING
// =====

/// <summary>
/// Process active shake
/// </summary>
private void ProcessShake()
{
    shakeTimer -= Time.deltaTime;

    // Calculate current intensity based on decay type
    float progress = 1f - (shakeTimer / currentDuration);
    float intensityMultiplier = CalculateDecay(progress);
    float finalIntensity = currentIntensity * intensityMultiplier *
globalIntensityMultiplier;

    // Skip if accessibility disabled shake
    if (globalIntensityMultiplier <= 0f)
    {
        return;
    }
}

```

```

    }

    // Generate offset based on pattern
    Vector2 offset = CalculateOffset(finalIntensity);
    currentOffset = new Vector3(offset.x, offset.y, 0f);

    // Apply position offset
    transform.localPosition = originalPosition + currentOffset;

    // Apply rotation if enabled
    if (enableRotationShake && maxRotation > 0f)
    {
        currentRotationOffset = CalculateRotation(finalIntensity);
        transform.localRotation = originalRotation *
Quaternion.Euler(0f, 0f, currentRotationOffset);
    }
}

/// <summary>
/// Calculate decay multiplier based on type
/// </summary>
private float CalculateDecay(float progress)
{
    switch (currentDecayType)
    {
        case ShakeDecayType.None:
            return 1f;

        case ShakeDecayType.Linear:
            return 1f - progress;

        case ShakeDecayType.Exponential:
            // Fast falloff that slows down
            return Mathf.Pow(1f - progress, 2f);

        default:
            return 1f - progress;
    }
}

```

```

    }
}

/// <summary>
/// Calculate position offset based on pattern
/// </summary>
private Vector2 CalculateOffset(float intensity)
{
    switch (currentPattern)
    {
        case ShakePattern.Random:
            return new Vector2(
                Random.Range(-1f, 1f) * intensity,
                Random.Range(-1f, 1f) * intensity
            );

        case ShakePattern.Perlin:
            // Smooth noise that changes over time
            float time = Time.time * 20f; // Speed of noise change
            float x = (Mathf.PerlinNoise(perlinSeedX + time, 0f) -
0.5f) * 2f * intensity;
            float y = (Mathf.PerlinNoise(0f, perlinSeedY + time) -
0.5f) * 2f * intensity;
            return new Vector2(x, y);

        case ShakePattern.Direction:
            // Bias toward direction, with some perpendicular noise
            float parallel = Random.Range(0.5f, 1f) * intensity;
            float perpendicular = Random.Range(-0.3f, 0.3f) *
intensity;

            Vector2 perp = new Vector2(-shakeDirection.y,
shakeDirection.x);
            return (shakeDirection * parallel) + (perp *
perpendicular);

        default:
            return Vector2.zero;
    }
}

```

```

    }
}

/// <summary>
/// Calculate rotation offset
/// </summary>
private float CalculateRotation(float intensity)
{
    float normalizedIntensity = intensity / currentIntensity;
    return Random.Range(-1f, 1f) * maxRotation * normalizedIntensity;
}

/// <summary>
/// Smooth return to original position
/// </summary>
private void RecoverFromShake()
{
    // Lerp back to original
    currentOffset = Vector3.Lerp(currentOffset, Vector3.zero,
recoverySpeed * Time.deltaTime);
    transform.localPosition = originalPosition + currentOffset;

    if (enableRotationShake)
    {
        currentRotationOffset = Mathf.Lerp(currentRotationOffset, 0f,
recoverySpeed * Time.deltaTime);
        transform.localRotation = originalRotation *
Quaternion.Euler(0f, 0f, currentRotationOffset);
    }
}

// =====
// PUBLIC METHODS - Call these to trigger shake
// =====

/// <summary>
/// Trigger shake with full control

```

```

/// </summary>
public void Shake(float intensity, float duration,
                 ShakeDecayType decayType =
ShakeDecayType.Exponential,
                 ShakePattern pattern = ShakePattern.Perlin,
                 Vector2? direction = null)
{
    // Don't override stronger shake with weaker one
    if (shakeTimer > 0f && currentIntensity > intensity)
    {
        return;
    }

    currentIntensity = intensity;
    currentDuration = duration;
    shakeTimer = duration;
    currentDecayType = decayType;
    currentPattern = pattern;

    if (direction.HasValue)
    {
        shakeDirection = direction.Value.normalized;
    }
    else
    {
        shakeDirection = Vector2.right;
    }

    // New perlin seeds for variety
    perlinSeedX = Random.Range(0f, 100f);
    perlinSeedY = Random.Range(0f, 100f);

    if (debugMode)
    {
        Debug.Log($"Shake triggered: intensity={intensity}, duration=
{duration}, pattern={pattern}");
    }
}

```



```
}

/// <summary>
/// Simple shake with just intensity and duration (uses defaults)
/// </summary>
public void Shake(float intensity, float duration)
{
    Shake(intensity, duration, defaultDecayType, defaultPattern);
}

/// <summary>
/// Light shake preset
/// </summary>
public void ShakeLight()
{
    Shake(lightIntensity, lightDuration);
}

/// <summary>
/// Medium shake preset
/// </summary>
public void ShakeMedium()
{
    Shake(mediumIntensity, mediumDuration);
}

/// <summary>
/// Heavy shake preset
/// </summary>
public void ShakeHeavy()
{
    Shake(heavyIntensity, heavyDuration);
}

/// <summary>
/// Directional shake (e.g., recoil, knockback)
/// </summary>
```

```

    public void ShakeDirectional(float intensity, float duration, Vector2
direction)
    {
        Shake(intensity, duration, ShakeDecayType.Exponential,
ShakePattern.Direction, direction);
    }

    /// <summary>
    /// Immediately stop shake
    /// </summary>
    public void StopShake()
    {
        shakeTimer = 0f;
    }
}

// =====
// USAGE EXAMPLES:
//
// 1. Basic shake on hit:
//     void OnEnemyHit()
//     {
//         ScreenShake.Instance.ShakeMedium();
//     }
//
// 2. Scaled shake based on damage:
//     void TakeDamage(int damage)
//     {
//         float intensity = Mathf.Clamp(damage * 0.1f, 0.1f, 1f);
//         ScreenShake.Instance.Shake(intensity, 0.15f);
//     }
//
// 3. Directional shake for recoil:
//     void FireWeapon(Vector2 fireDirection)
//     {
//         // Shake opposite to fire direction
//         ScreenShake.Instance.ShakeDirectional(0.2f, 0.1f, -

```

```

fireDirection);
//    }
//
// 4. Combine with hitstop:
//    void OnBigHit()
//    {
//        HitstopManager.Instance.TriggerHeavyHit();
//        ScreenShake.Instance.ShakeHeavy();
//    }
//
// 5. In your settings menu (REQUIRED!):
//    void OnShakeSliderChanged(float value)
//    {
//        ScreenShake.Instance.GlobalIntensity = value;
//        PlayerPrefs.SetFloat("ScreenShakeIntensity", value);
//    }
//
// 6. For 3D: Same script works, just uses XY offset by default.
//    For full 3D shake, add Z offset in CalculateOffset.
// =====

```

Common Issues

"The shake is nauseating"

Reduce `intensity` and `duration`. Add `decayType : Exponential` so it calms quickly.

"Everything shakes and nothing feels special"

Shake less often. Reserve for meaningful moments. Create intensity hierarchy.

"The shake feels jittery/wrong"

Try `shakePattern : Perlin` for smoother noise. Or reduce `frequency`.

"Players with motion sensitivity can't play"

Always include a shake disable option. This is accessibility, not preference.

Shake Hierarchy

Not all shakes should be equal. Create a hierarchy:

Event Type	Intensity	Duration
Ambient (footsteps)	1-2	0.03
Minor impact	3-5	0.1
Standard impact	8-12	0.15
Major impact	15-20	0.2
Boss/explosion	25+	0.3+

If everything is at "standard impact" level, nothing stands out.

When NOT to Shake

- Every jump landing (unless you want that)
- Background events player didn't cause
- Menu interactions
- Constant effects (rain, wind) - use subtle camera drift instead
- If the player has shake disabled

Combine With

- [Hitstop](#) - freeze THEN shake for maximum impact
- [Squash Stretch](#) - visual deformation matches shake
- [Landing Impact](#) - shake as part of landing feedback
- Sound - shake without audio feels hollow

Accessibility Note

Screen shake can cause motion sickness, nausea, and disorientation for many players.

Always provide:

- Shake intensity slider (0-100%)
- Or disable option
- Ideally both

This isn't optional. This is basic accessibility.

Why Teach Screen Shake

Screen shake is the canonical example of **"juice"** - feedback that makes actions feel impactful. It's easy to implement, immediately visible, and demonstrates the power (and danger) of feedback systems.

Teaching Sequence

1. Start with NO shake - establish baseline
2. Add moderate shake to one event - feel the difference
3. Add shake to EVERYTHING - experience the problem
4. Create a hierarchy - learn restraint
5. Add accessibility options - learn responsibility

Common Student Mistakes

Shake addiction: Students discover shake feels good and add it everywhere. The lesson is restraint: if everything is important, nothing is.

Ignoring accessibility: Students don't think about motion sensitivity until prompted. Make accessibility options a requirement, not an afterthought.

Using wrong pattern: Random shake for directional impacts feels wrong. Teach pattern selection based on the event type.

Assessment

- Can they explain why shake hierarchy matters?
- Can they identify overuse in published videogames?
- Did they implement accessibility options?

Exercise: Shake Audit

Have students play a "juicy" videogame (Nuclear Throne, Vlambeer titles) and document:

- Every event that triggers shake
- Estimated intensity and duration
- What the shake communicates
- What would be lost without it

Heritage Notes

Screen shake entered videogame design from multiple sources:

Film - Handheld camera shake for intensity (Saving Private Ryan, documentary style). Shake communicates chaos, impact, presence.

Arcade games - Physical cabinet shake on certain events. The screen shake simulates what the cabinet did physically.

Modern "juicing" - Vlambeer popularized aggressive shake in indie games. Their GDC talk "The Art of Screenshake" became canonical.

Post-2010, screen shake became standard. Post-2015, the backlash reminded designers to use it judiciously.

Shake as Art Direction

Screen shake is part of the [Art](#) of a Gesture. It's not just feedback - it's aesthetic choice. A videogame with heavy shake has a different vibe than one with none.

Consider:

- *Nuclear Throne* - Heavy shake is part of the chaotic, overwhelming aesthetic
- *Celeste* - Minimal shake preserves precision platforming clarity
- *Dark Souls* - Reserved shake emphasizes significant moments

The Vlambeer Legacy

Vlambeer's "Art of Screenshake" talk (GDC 2013) codified what was intuitive knowledge.

Key insights:

- Shake makes things feel impactful
- Different shakes for different intensities
- Combine with other feedback (sound, particles, hitstop)

But the talk also triggered overuse. The post-Vlambeer era saw shake everywhere, leading to the current understanding: shake is powerful, but restraint is wisdom.

The Accessibility Imperative

Motion sensitivity affects a significant portion of players. Vestibular disorders, migraines, and other conditions can make screen shake physically painful or nauseating.

This isn't preference - it's access. A videogame without shake options excludes players. The industry consensus now: **shake options are not optional.**

Unresolved Questions

- How do you balance artistic intent (shake as aesthetic) with accessibility (shake as barrier)?
- Can shake be replaced with other feedback for accessibility without losing impact?
- Is there a "right amount" of shake, or is it purely contextual?

Related

- [Hitstop](#) - common pairing
- [Accessibility as Craft](#) - shake accessibility
- [The 4 A's](#) - shake as part of Art

Glossary Terms

- [Juice](#) - shake as juice technique
- [Feedback](#) - shake as feedback mechanism
- [Game Feel](#) - the experience shake contributes to

Hitstop

A brief freeze when an attack connects, giving visual weight to the impact. The world pauses for a split second to emphasize the hit.

The Feel

Hitstop creates **weight and impact**. Without it, attacks feel like they pass through enemies. With it, you feel the collision. The pause says: "This mattered."

It's counterintuitive - you're making the videogame slower to make it feel more powerful. But the pause creates contrast that makes the surrounding motion feel faster.

Exposed Variables

Variable	Type	Default	What it does
<code>hitstopDuration</code>	float	0.05	How long to freeze (seconds)
<code>timeScale</code>	float	0.0	What to set Time.timeScale to during hitstop
<code>affectBoth</code>	bool	true	Freeze both attacker and target (or just target)

Tuning Guide

For light/fast attacks:

- `hitstopDuration` = 0.02 - 0.04
- Brief stutter, maintains speed

Reference: Devil May Cry (light attacks)

For medium attacks:

- `hitstopDuration` = 0.05 - 0.08
- Noticeable pause, solid impact

Reference: Most action games' standard attacks

For heavy/finishing attacks:

- `hitstopDuration` = 0.1 - 0.2

- Dramatic pause, emphasizes power

Reference: God of War finishers, Smash Bros. killing blows

Pseudocode

▼ Pseudocode

On Hit:

- Store original `Time.timeScale`
- Set `Time.timeScale` to 0 (or near-zero)
- Start `hitstopTimer`

Every Frame (using `unscaledDeltaTime`):

- If in hitstop:
 - Decrease `hitstopTimer`
 - If `hitstopTimer <= 0`:
 - Restore original `Time.timeScale`

Critical: Use `Time.unscaledDeltaTime` for the hitstop timer, so it counts down even when time is frozen.

Unity Setup

1. Add the HitstopManager:

- Create empty GameObject named "HitstopManager"
- Add the `HitstopManager.cs` script below
- This is a singleton - only need one in the scene

2. Call from Attack Scripts:

- When an attack connects, call
`HitstopManager.Instance.TriggerHitstop(duration)`
- Pass different durations for light (0.03), medium (0.06), heavy (0.12) attacks

3. Optional - Fixed Timestep:

- Edit → Project Settings → Time
- Consider adjusting Fixed Timestep if physics behaves oddly during slow-mo

4. Audio Note:

- Audio pitch may scale with timeScale
- Set `minTimeScale` to 0.01 instead of 0 if audio sounds wrong
- Or use `AudioSource.ignoreListenerPause = true`

The Script


```

// =====
// HitstopManager.cs
// A global hitstop system using Time.timeScale.
// Singleton pattern - add to one empty GameObject in scene.
// Unity 6.x
// =====

using UnityEngine;

public class HitstopManager : MonoBehaviour
{
    // =====
    // SINGLETON - Access from anywhere via HitstopManager.Instance
    // =====

    public static HitstopManager Instance { get; private set; }

    // =====
    // HITSTOP PARAMETERS - Tune these in the Inspector
    // =====

    [Header("Hitstop Settings")]
    [Tooltip("Time scale during hitstop (0 = full freeze, 0.01 = near-freeze)")]
    [SerializeField] [Range(0f, 0.1f)] private float minTimeScale = 0f;

    [Tooltip("Default hitstop duration if not specified")]
    [SerializeField] [Range(0.01f, 0.5f)] private float defaultDuration = 0.05f;

    [Header("Preset Durations (for convenience)")]
    [Tooltip("Duration for light/fast attacks")]
    [SerializeField] [Range(0.01f, 0.1f)] private float lightHitDuration = 0.03f;

    [Tooltip("Duration for medium attacks")]

```

```

[SerializeField] [Range(0.03f, 0.15f)] private float
mediumHitDuration = 0.06f;

[Tooltip("Duration for heavy/finishing attacks")]
[SerializeField] [Range(0.05f, 0.3f)] private float heavyHitDuration
= 0.12f;

[Header("Debug")]
[Tooltip("Show debug messages in console")]
[SerializeField] private bool debugMode = false;

// =====
// INTERNAL STATE
// =====

private float hitstopTimer = 0f;
private float originalTimeScale = 1f;
private bool inHitstop = false;

// =====
// PUBLIC PROPERTIES
// =====

/// <summary>
/// True while hitstop is active
/// </summary>
public bool InHitstop => inHitstop;

/// <summary>
/// Remaining hitstop time in seconds
/// </summary>
public float RemainingTime => hitstopTimer;

// =====
// UNITY LIFECYCLE
// =====

```



```

void Awake()
{
    // Singleton setup - destroy duplicates
    if (Instance != null && Instance != this)
    {
        Destroy(gameObject);
        return;
    }
    Instance = this;

    // Optional: persist across scenes
    // DontDestroyOnLoad(gameObject);
}

void Update()
{
    // Process hitstop timer
    // CRITICAL: Use unscaledDeltaTime so timer works during freeze!
    if (inHitstop)
    {
        hitstopTimer -= Time.unscaledDeltaTime;

        if (hitstopTimer <= 0f)
        {
            EndHitstop();
        }
    }
}

// =====
// PUBLIC METHODS - Call these to trigger hitstop
// =====

/// <summary>
/// Trigger hitstop with a custom duration
/// </summary>
/// <param name="duration">How long to freeze (seconds)</param>

```

```

public void TriggerHitstop(float duration)
{
    // Don't interrupt a longer hitstop with a shorter one
    if (inHitstop && hitstopTimer > duration)
    {
        if (debugMode) Debug.Log($"Hitstop ignored: existing
({hitstopTimer:F3}s) > new ({duration:F3}s)");
        return;
    }

    // Store original time scale (only if not already in hitstop)
    if (!inHitstop)
    {
        originalTimeScale = Time.timeScale;
    }

    // Start hitstop
    inHitstop = true;
    hitstopTimer = duration;
    Time.timeScale = minTimeScale;

    if (debugMode)
    {
        Debug.Log($"Hitstop triggered: {duration:F3}s (timeScale:
{minTimeScale})");
    }
}

/// <summary>
/// Trigger hitstop with default duration
/// </summary>
public void TriggerHitstop()
{
    TriggerHitstop(defaultDuration);
}

/// <summary>

```

```
/// Trigger light attack hitstop
/// </summary>
public void TriggerLightHit()
{
    TriggerHitstop(lightHitDuration);
}

/// <summary>
/// Trigger medium attack hitstop
/// </summary>
public void TriggerMediumHit()
{
    TriggerHitstop(mediumHitDuration);
}

/// <summary>
/// Trigger heavy attack hitstop
/// </summary>
public void TriggerHeavyHit()
{
    TriggerHitstop(heavyHitDuration);
}

/// <summary>
/// Immediately end hitstop (e.g., on scene change)
/// </summary>
public void ForceEndHitstop()
{
    if (inHitstop)
    {
        EndHitstop();
    }
}

// =====
// INTERNAL METHODS
// =====
```

```

    /// <summary>
    /// End hitstop and restore normal time
    /// </summary>
    private void EndHitstop()
    {
        inHitstop = false;
        hitstopTimer = 0f;
        Time.timeScale = originalTimeScale;

        if (debugMode)
        {
            Debug.Log($"Hitstop ended. Time restored to {originalTimeScale}");
        }
    }

    // =====
    // CLEANUP
    // =====

    void OnDestroy()
    {
        // Always restore time scale when destroyed
        if (inHitstop)
        {
            Time.timeScale = originalTimeScale;
        }
    }
}

// =====
// USAGE EXAMPLES:
//
// 1. From an attack script when hit connects:
//     void OnHitEnemy(Enemy enemy)
//     {

```

```

//      // Trigger hitstop based on attack type
//      HitstopManager.Instance.TriggerMediumHit();
//
//      // Do damage, knockback, etc.
//      enemy.TakeDamage(damage);
//  }
//
// 2. Custom duration for special attacks:
//  void OnFinalBlow()
//  {
//      // Extra long hitstop for dramatic effect
//      HitstopManager.Instance.TriggerHitstop(0.2f);
//  }
//
// 3. Combine with other effects:
//  void OnCriticalHit()
//  {
//      HitstopManager.Instance.TriggerHeavyHit();
//      ScreenShake.Instance.TriggerShake(0.3f, 0.5f);
//      SpawnHitEffect(hitPosition);
//      PlaySound(criticalHitSound);
//  }
//
// 4. Check if in hitstop (for UI, input buffering):
//  if (HitstopManager.Instance.InHitstop)
//  {
//      // Buffer this input for after hitstop ends
//  }
//
// 5. For objects that should move during hitstop (like UI),
//  use Time.unscaledDeltaTime for their movement:
//  transform.position += direction * speed * Time.unscaledDeltaTime;
//  =====

```

Common Issues

"Nothing happens during hitstop"

Make sure your timer uses `unscaledDeltaTime`. Regular `deltaTime` will be 0 during the freeze.

"Hitstop affects things it shouldn't"

`Time.timeScale` is global. For selective freezing, use per-object pause states instead of `timeScale`.

"It feels choppy, not impactful"

Hitstop alone isn't enough. Combine with screen shake, hit effects, and sound. The pause is one part of a larger impact moment.

"Audio sounds wrong during hitstop"

Audio pitch may scale with `timeScale`. Either use `unscaledTime` for audio or keep `timeScale` at a low value (0.01) instead of 0.

Combine With

- [Screen Shake](#) - visual impact alongside temporal impact
- [Knockback](#) - movement after the pause
- [Input Buffer](#) - catch inputs during freeze
- [Squash & Stretch](#) - deform on impact

The Impact Stack

Great combat feel comes from layering techniques. A satisfying hit often includes:

Layer	What it does
Hitstop	Temporal emphasis - time acknowledges the hit
Screen Shake	Visual emphasis - camera reacts to hit
Hit VFX	Sparks, flashes, particles
Sound	Impact audio, voice
Knockback	Physical consequence

Why Teach This

Hitstop is the perfect example of **counterintuitive design**. Slowing things down makes them feel faster. Students learn that feel doesn't come from literal representation - it comes from emphasis and contrast.

Teaching Sequence

1. Show two versions of the same attack - with and without hitstop
2. Have students describe the difference in feel (not mechanics)
3. Implement basic hitstop - observe the transformation
4. Vary duration per attack type - learn to calibrate impact

The "Juice" Conversation

Hitstop is often grouped under "juice" or "game feel." This is a good entry point for discussing:

- What is juice? (Feedback that makes actions feel satisfying)
- Why does juice matter? (It's the difference between functional and delightful)
- How much is too much? (When it obscures rather than emphasizes)

Assessment Questions

- Why does pausing the game make an attack feel stronger?
- How would you vary hitstop for different attack types?
- What other effects should accompany hitstop for maximum impact?

Heritage Notes

Hitstop originated in fighting games, likely accidentally. Early Street Fighter games had processing delays when calculating hit effects. Players grew to associate the pause with impact. When hardware got faster, developers intentionally preserved the pause because removing it made hits feel weak.

This is Aesthetic Heritage at work: a technical limitation became an aesthetic expectation.

Why Pause = Power

Several psychological mechanisms explain hitstop's effectiveness:

- **Contrast:** The pause makes surrounding motion feel faster by comparison
- **Attention:** A sudden stop captures attention at the moment of impact
- **Anticipation:** The pause creates micro-tension before the consequence (knockback)
- **Film language:** Slow motion in film emphasizes important moments; hitstop is extreme slow motion

The 4 A's of Hitstop

Hitstop is a micro-Gesture with all four A's:

- **Action:** The attack input and its connection
- **Art:** The frozen frame, often accompanied by VFX
- **Arc:** The brief duration of the pause
- **Atmosphere:** The feeling of weight, impact, power

Even this tiny moment contains the full framework.

When Not to Use Hitstop

Hitstop isn't always appropriate:

- **Continuous damage:** Hitstop on every tick of poison would be annoying

- **Environmental hazards:** Walking into spikes might not need dramatic pause
- **Story emphasis:** Sometimes you want the violence to feel quick and brutal, not dramatic

Related

- [Screen Shake](#) - visual impact feedback
- [Knockback](#) - what happens after the pause
- [Gesture](#) - hitstop as micro-gesture

Glossary Terms

- [Hitstop](#)
- [Juice](#)

Knockback

Push an entity away when hit. The physical consequence of damage. Creates space, shows impact, and can create environmental hazards.

The Feel

Knockback creates **physical consequence**. Damage isn't just a number going down - it's being pushed back, losing ground, potentially falling into danger. It makes combat spatial.

Good knockback also creates opportunities: knock an enemy off a ledge, into spikes, or into another enemy.

Exposed Variables

Variable	Type	Default	What it does
<code>knockbackForce</code>	float	8.0	Horizontal push force
<code>knockbackUp</code>	float	4.0	Vertical lift during knockback
<code>hitstunDuration</code>	float	0.2	Time target can't act after hit
<code>knockbackDecay</code>	float	10.0	How fast knockback velocity decays

Tuning Guide

For light hit reactions:

- `knockbackForce` = 3 - 5
- `knockbackUp` = 0 - 2
- `hitstunDuration` = 0.1
- Quick recovery, fast combat

Reference: Hack-and-slash, character action games

For heavy, consequential hits:

- `knockbackForce` = 10 - 15
- `knockbackUp` = 5 - 8
- `hitstunDuration` = 0.3 - 0.5

- Major displacement, time to reposition

Reference: Smash Bros., platform fighters

For environmental kill potential:

- High `knockbackForce`
- Significant `knockbackUp` (launches into air)
- Design levels with hazards at edges

Pseudocode

▼ Pseudocode

On Hit:

- Calculate direction from attacker to target
- Apply velocity: $(\text{direction} * \text{knockbackForce}) + (\text{up} * \text{knockbackUp})$
- Start hitstun timer
- Disable target's movement/actions

Every Frame during hitstun:

- Decay knockback velocity
- Prevent target actions

On hitstun end:

- Restore target control

Unity Setup (2D)

1. Setup the Target (entity that receives knockback):

- Add `Rigidbody2D` (Gravity Scale: 3, Freeze Rotation Z: checked)
- Add `Collider2D` (BoxCollider2D or CapsuleCollider2D)
- Add the `KnockbackReceiver2D.cs` script below

2. From Attack Scripts:

- Get target's KnockbackReceiver2D component
- Call `ApplyKnockback(attackerPosition, force, upForce)`
- Or use the preset methods for light/medium/heavy hits

3. Optional - Movement Integration:

- Your movement script should check `IsInHitstun`
- Disable player input while in hitstun

4. Optional - Weight System:

- Set `weight` in Inspector (1 = normal, 2 = heavy, 0.5 = light)
- Heavier entities receive less knockback

The Script (2D)

▼ KnockbackReceiver2D.cs

```

// =====
// KnockbackReceiver2D.cs
// Attach to any entity that can be knocked back when hit.
// Includes hitstun (action disable) and weight system.
// Unity 6.x
// =====

using UnityEngine;
using System;

[RequireComponent(typeof(Rigidbody2D))]
public class KnockbackReceiver2D : MonoBehaviour
{
    // =====
    // KNOCKBACK PARAMETERS - Tune these in the Inspector
    // =====

    [Header("Weight")]
    [Tooltip("How resistant to knockback (1 = normal, 2 = half knockback, 0.5 = double)")]
    [SerializeField] [Range(0.1f, 5f)] private float weight = 1f;

    [Header("Knockback Decay")]
    [Tooltip("How fast horizontal knockback velocity decays")]
    [SerializeField] [Range(0f, 20f)] private float horizontalDecay = 8f;

    [Tooltip("Should we apply decay or let physics handle it?")]
    [SerializeField] private bool useManualDecay = true;

    [Header("Preset Forces (for ApplyLightHit, etc.)")]
    [Tooltip("Light hit: horizontal force")]
    [SerializeField] private float lightForce = 4f;
    [Tooltip("Light hit: upward force")]
    [SerializeField] private float lightUpForce = 2f;
    [Tooltip("Light hit: hitstun duration")]
    [SerializeField] private float lightHitstun = 0.1f;

```



```

[Tooltip("Medium hit: horizontal force")]
[SerializeField] private float mediumForce = 8f;
[Tooltip("Medium hit: upward force")]
[SerializeField] private float mediumUpForce = 4f;
[Tooltip("Medium hit: hitstun duration")]
[SerializeField] private float mediumHitstun = 0.2f;

[Tooltip("Heavy hit: horizontal force")]
[SerializeField] private float heavyForce = 15f;
[Tooltip("Heavy hit: upward force")]
[SerializeField] private float heavyUpForce = 8f;
[Tooltip("Heavy hit: hitstun duration")]
[SerializeField] private float heavyHitstun = 0.4f;

[Header("Debug")]
[Tooltip("Show debug info in console")]
[SerializeField] private bool debugMode = false;

// =====
// INTERNAL STATE
// =====

private Rigidbody2D rb;
private float hitstunTimer = 0f;
private bool inHitstun = false;
private Vector2 knockbackVelocity = Vector2.zero;

// =====
// EVENTS - Subscribe to react to knockback
// =====

/// <summary>
/// Fired when knockback starts (direction, force)
/// </summary>
public event Action<Vector2, float> OnKnockbackStart;

```

```

    /// <summary>
    /// Fired when hitstun ends
    /// </summary>
    public event Action OnHitstunEnd;

    // =====
    // PUBLIC PROPERTIES
    // =====

    /// <summary>
    /// True while in hitstun (cannot act)
    /// </summary>
    public bool IsInHitstun => inHitstun;

    /// <summary>
    /// Remaining hitstun time
    /// </summary>
    public float HitstunRemaining => hitstunTimer;

    /// <summary>
    /// This entity's weight (affects knockback received)
    /// </summary>
    public float Weight => weight;

    // =====
    // UNITY LIFECYCLE
    // =====

    void Awake()
    {
        rb = GetComponent<Rigidbody2D>();
    }

    void Update()
    {
        // Process hitstun timer
        if (inHitstun)

```

```

    {
        hitstunTimer -= Time.deltaTime;

        if (hitstunTimer <= 0f)
        {
            EndHitstun();
        }
    }
}

void FixedUpdate()
{
    // Apply manual knockback decay
    if (useManualDecay && inHitstun)
    {
        // Decay horizontal velocity toward zero
        float decayAmount = horizontalDecay * Time.fixedDeltaTime;
        float currentX = rb.linearVelocity.x;

        if (Mathf.Abs(currentX) > decayAmount)
        {
            float newX = currentX - Mathf.Sign(currentX) *
decayAmount;
            rb.linearVelocity = new Vector2(newX,
rb.linearVelocity.y);
        }
        else
        {
            rb.linearVelocity = new Vector2(0f, rb.linearVelocity.y);
        }
    }
}

// =====
// PUBLIC METHODS - Call these to apply knockback
// =====

```

```

    /// <summary>
    /// Apply knockback with full control over parameters
    /// </summary>
    /// <param name="attackerPosition">Position of the attacker
(knockback away from this)</param>
    /// <param name="horizontalForce">Force pushing away
horizontally</param>
    /// <param name="upwardForce">Force pushing upward</param>
    /// <param name="hitstunDuration">How long target can't act</param>
    public void ApplyKnockback(Vector2 attackerPosition, float
horizontalForce,
                                float upwardForce, float hitstunDuration)
    {
        // Calculate direction FROM attacker TO this entity
        Vector2 direction = ((Vector2)transform.position -
attackerPosition).normalized;

        // If directly above/below, default to facing away from
attacker's right
        if (Mathf.Abs(direction.x) < 0.1f)
        {
            direction.x = transform.position.x > attackerPosition.x ? 1f
: -1f;
        }

        // Apply weight modifier (heavier = less knockback)
        float weightModifier = 1f / weight;
        float finalHorizontal = horizontalForce * weightModifier;
        float finalUpward = upwardForce * weightModifier;

        // Build knockback velocity
        knockbackVelocity = new Vector2(
            Mathf.Sign(direction.x) * finalHorizontal,
            finalUpward
        );

        // Apply velocity directly

```

```

        rb.linearVelocity = knockbackVelocity;

        // Start hitstun
        inHitstun = true;
        hitstunTimer = hitstunDuration;

        // Fire event
        OnKnockbackStart?.Invoke(direction, horizontalForce);

        if (debugMode)
        {
            Debug.Log($"Knockback applied! Direction: {direction}, " +
                $"Force: ({finalHorizontal}, {finalUpward}), " +
                $"Hitstun: {hitstunDuration}s");
        }
    }

    /// <summary>
    /// Apply knockback in a specific direction (not away from attacker)
    /// </summary>
    public void ApplyKnockbackDirection(Vector2 direction, float
horizontalForce,

                                float upwardForce, float
hitstunDuration)
    {
        float weightModifier = 1f / weight;
        knockbackVelocity = new Vector2(
            direction.x * horizontalForce * weightModifier,
            upwardForce * weightModifier
        );

        rb.linearVelocity = knockbackVelocity;

        inHitstun = true;
        hitstunTimer = hitstunDuration;

        OnKnockbackStart?.Invoke(direction, horizontalForce);
    }

```

```

    }

    /// <summary>
    /// Apply a light hit knockback
    /// </summary>
    public void ApplyLightHit(Vector2 attackerPosition)
    {
        ApplyKnockback(attackerPosition, lightForce, lightUpForce,
lightHitstun);
    }

    /// <summary>
    /// Apply a medium hit knockback
    /// </summary>
    public void ApplyMediumHit(Vector2 attackerPosition)
    {
        ApplyKnockback(attackerPosition, mediumForce, mediumUpForce,
mediumHitstun);
    }

    /// <summary>
    /// Apply a heavy hit knockback
    /// </summary>
    public void ApplyHeavyHit(Vector2 attackerPosition)
    {
        ApplyKnockback(attackerPosition, heavyForce, heavyUpForce,
heavyHitstun);
    }

    /// <summary>
    /// Immediately end hitstun (for special recovery moves)
    /// </summary>
    public void CancelHitstun()
    {
        if (inHitstun)
        {
            EndHitstun();
        }
    }

```

```

    }
}

// =====
// INTERNAL METHODS
// =====

/// <summary>
/// End hitstun and restore control
/// </summary>
private void EndHitstun()
{
    inHitstun = false;
    hitstunTimer = 0f;

    OnHitstunEnd?.Invoke();

    if (debugMode)
    {
        Debug.Log("Hitstun ended - control restored");
    }
}

// =====
// USAGE EXAMPLES:
//
// 1. From an attack script when dealing damage:
// void DealDamage(GameObject target, int damage)
// {
//     var knockback = target.GetComponent<KnockbackReceiver2D>();
//     if (knockback != null)
//     {
//         knockback.ApplyMediumHit(transform.position);
//     }
//     // Also apply damage to health, play effects, etc.
// }

```

```

//
// 2. Custom knockback for special attacks:
//     void SuperPunch(GameObject target)
//     {
//         var knockback = target.GetComponent<KnockbackReceiver2D>();
//         // Massive knockback with long hitstun
//         knockback.ApplyKnockback(transform.position, 20f, 12f, 0.6f);
//     }
//
// 3. In your movement script, disable input during hitstun:
//     void Update()
//     {
//         if (knockbackReceiver.IsInHitstun)
//         {
//             // Don't process movement input
//             return;
//         }
//         // Normal movement code...
//     }
//
// 4. Subscribe to knockback events:
//     void Start()
//     {
//         GetComponent<KnockbackReceiver2D>().OnKnockbackStart += OnHit;
//         GetComponent<KnockbackReceiver2D>().OnHitstunEnd += OnRecover;
//     }
//     void OnHit(Vector2 dir, float force) { PlayHitAnimation(); }
//     void OnRecover() { PlayRecoverAnimation(); }
//
// 5. Combine with HitstopManager for full impact:
//     void OnAttackHit(GameObject target)
//     {
//         HitstopManager.Instance.TriggerMediumHit();
//         target.GetComponent<KnockbackReceiver2D>
//         ().ApplyMediumHit(transform.position);
//         ScreenShake.Instance.TriggerShake(0.2f, 0.3f);
//     }

```



```
//  
// 6. For 3D: Change Rigidbody2D to Rigidbody, Vector2 to Vector3  
// =====
```

Common Issues

"Knockback goes the wrong direction"

Make sure you're calculating direction FROM attacker TO target, not the reverse. Also check if target's facing direction affects the calculation.

"Target gets stuck in walls"

Add collision checks or reduce knockback when near walls. Some videogames "slide" along walls; others cancel knockback.

"Combat feels floaty"

`knockbackUp` might be too high relative to gravity. Also check `knockbackDecay` - slow decay = floaty.

"Player gets knocked into pits unfairly"

Consider invincibility frames or reduced knockback near hazards. Or lean into it - environmental kills can be a feature, not a bug.

Combine With

- [Hitstop](#) - pause before knockback begins
- [Screen Shake](#) - emphasize the impact
- [Squash & Stretch](#) - deform on impact
- [Input Buffer](#) - buffer recovery actions during hitstun

Knockback Variations

Type	Description
Fixed knockback	Same force regardless of damage dealt
Damage-scaled	More damage = more knockback (Smash Bros.)
Weight-modified	Heavier targets fly less far
Directional	Different attacks knock in different directions
Wall bounce	Hitting a wall bounces you back into play

Why Teach This

Knockback teaches **combat as space control**. It connects damage to positioning, making combat about territory, not just health bars. Students learn that good combat systems have physical consequences.

Teaching Sequence

1. Implement basic knockback (force away from attacker)
2. Add hitstun to disable target temporarily
3. Tune for different attack types
4. Add environmental hazards to make knockback consequential

The Smash Bros. Lesson

Super Smash Bros. is the ultimate knockback-focused design. Damage increases knockback instead of reducing health. The only way to lose is to be knocked off the stage. This design makes positioning central to everything.

Even if students aren't making platform fighters, they can learn from this: **what if knockback was the primary threat?**

Assessment Questions

- How does knockback change where combat happens in a level?
- Why pair hitstun with knockback? What goes wrong without it?
- How would you balance knockback near instant-death hazards?

Heritage Notes

Knockback existed in early videogames (Castlevania's infamous knockback into pits) but became a design focus with fighting games. Street Fighter II's corner pressure, where knockback traps opponents at screen edge, showed that positioning could be a resource.

Platform fighters like Smash Bros. made knockback the entire victory condition, replacing health bars entirely.

Knockback as Permissions

From a [Permissions](#) perspective, knockback temporarily revokes permission to act (hitstun) and forcibly repositions the target. It's one of the most aggressive permission changes in action videogames.

This can feel fair (you got hit, you pay the price) or unfair (chain knockback that never lets you act). Tuning knockback is tuning fairness.

The Castlevania Problem

Classic Castlevania is notorious for knockback into pits - one hit sends you flying backwards into instant death. This was partly technical limitation, partly intentional difficulty.

Modern videogames usually provide either:

- Invincibility frames to escape after one hit
- Reduced knockback near hazards
- Ledge grabbing or recovery mechanics

The Castlevania approach is a **heritage choice**: using it quotes that tradition, with all its implications.

Knockback and Atmosphere

Knockback force communicates power. A weak slap might push you slightly; a massive hit launches you across the screen. The [Atmosphere](#) of a weapon is partially defined by its

knockback. A weighty hammer should send enemies flying; a quick dagger shouldn't.

Related

- [Hitstop](#) - pause before the knockback
- [Screen Shake](#) - visual impact feedback
- [Permissions](#) - knockback as permission revocation

Glossary Terms

- [Hitstun](#)
- [Invincibility Frames](#)

APPENDIX

Reference

Glossary of terms and the reading list behind the course.

Glossary

Key terms and definitions, cross-referenced with main entries.

A

Accessibility: Designing videogames to be playable by people with diverse abilities. See [Accessibility as Craft](#).

Action: One of the 4 A's. What the videogame has the player do, including absence of interaction. See [The 4 A's](#).

Aesthetic Heritage: Where the choices in each A come from. The lineage. See [Aesthetic Heritage](#).

Affordance: What an object suggests you can do with it. A button affords pressing; a lever affords pulling.

Agency: The player's capacity to make meaningful choices and affect outcomes.

Assist Mode: Accessibility options that let players customize difficulty. See [Celeste: Assist Mode](#).

Arc: One of the 4 A's. The amount of time. A micro-moment, a long moment, or an entire videogame. See [The 4 A's](#).

Art: One of the 4 A's. Art direction, lighting, staging, camera grammar, composition, sound design, and their heritage. See [The 4 A's](#).

Atmosphere: One of the 4 A's. The feeling and vibe experienced from the videogame. See [The 4 A's](#).

B

Barks: Short, contextual voice lines that respond to videogame events ("Enemy spotted!", "I'm hit!").

Beat: A single unit of pacing. A moment of tension or release.

C

Cancel Window: The time during which one action can be interrupted by another.

Checkpoint: A saved progress point the player returns to on failure.

Coyote Time: Grace period after leaving a platform during which the player can still jump. Example of player-fair design. See [Player Psychology](#).

Curb Cut Effect: Accessibility features that help specific users often help everyone. Named for sidewalk curb cuts. See [Accessibility as Craft](#).

D

Diegetic: Existing within the videogame world (vs. UI that exists outside it).

Dynamic Difficulty Adjustment (DDA): System that automatically adjusts difficulty based on player performance. Can be hidden or transparent. See [Hades: God Mode](#).

Dialogue Tree: Branching conversation structure where player choices lead to different responses. See [Dialogue Systems](#).

Dynamics: Emergent patterns from player-system interaction.

Debugging: Finding and fixing errors in code or design. See [Debugging as Literacy](#).

E

Emergence: Complex, unexpected behaviors arising from simple rule interactions. See [Emergent Behavior](#).

Extrinsic Motivation: Playing for external rewards (XP, unlocks, achievements). See [Player Psychology](#).

F

False Choice: A choice that appears meaningful but leads to the same outcome. See [Branching & Consequence](#).

Feedback: How the videogame responds to player action.

Feedback Loop: When output of a system becomes input to that system. See [Player Psychology](#).

Flow: State of complete absorption where challenge matches skill (Csikszentmihalyi). See [Player Psychology](#).

Foldback Structure: Branching narrative that reconverges at key points before diverging again. See [Branching & Consequence](#).

Found Narrative: Story discovered through exploration rather than delivered through cutscenes. See [Environmental Storytelling](#).

Frame Data: The exact timing of actions measured in animation frames (common in fighting videogames).

G

Gesture: The surface area of contact between player and videogame. Composed of the 4 A's. See [Gesture](#).

God Mode: (1) Classic cheat granting invincibility. (2) *Hades*' accessibility feature with escalating damage resistance. See [Hades: God Mode](#).

H

Hard Currency: Difficult to obtain, premium currency, often monetized. See [Economy & Resources](#).

Hitstop: Brief freeze when attacks connect, emphasizing impact.

Hub and Spoke: Conversation structure where player returns to a central menu after each topic. See [Dialogue Systems](#).

Human Random vs. Computer Random: True randomness feels unfair to humans. Many videogames use weighted randomness that feels more fair. See [Player Psychology](#).

I

Immersive Sim: Genre built around systemic emergence and player choice (Deus Ex, Dishonored, Prey). See [Emergent Behavior](#).

Implicit Instruction: Teaching through spatial design and constraints rather than explicit text. See [Portal: Level One](#).

Inflation: Too many sources, too few sinks in an economy. Resources pile up, value collapses. See [Economy & Resources](#).

Inheritance: Using a solution because it works, without necessarily referencing its origin. See [Aesthetic Heritage](#).

Input Buffer: System that remembers inputs for a grace period, improving responsiveness. See [Player Psychology](#).

Intrinsic Motivation: Playing because the activity itself is rewarding. See [Player Psychology](#).

J

Juice: Extra feedback beyond what's necessary, making actions feel satisfying. Particles, sounds, screen shake. See [Screen Shake](#).

L

Leading Lines: Visual elements (paths, edges, beams) that direct the player's eye. See [Spatial Communication](#).

Ludonarrative Dissonance: When gameplay and story contradict. See [Ludonarrative Harmony](#).

M

Magic Circle: The boundary between play and ordinary life (Huizinga). A theoretical device that doesn't especially matter for videogame analysis.

Mechanic: A rule governing player action.

N

Negative Feedback Loop: Feedback that dampens change, pulling toward equilibrium (e.g., rubber-banding). See [Player Psychology](#).

P

Pacing: Rhythm of intensity over time. See [Pacing & Flow](#).

Permissions: What a videogame allows, requires, and forbids. Discovered through the 4 A's as we play. See [Permissions](#).

Player Fair vs. Computer Fair: What feels fair to the player is not always mathematically fair. Good design often prioritizes player-fair. See [Player Psychology](#).

Positive Feedback Loop: Feedback that amplifies change (rich get richer). See [Player Psychology](#).

Procedural Rhetoric: Arguments made through videogame rules (Bogost). See [Ludonarrative Harmony](#).

Q

Quality-Based Narrative: Tracking player history as accumulating stats rather than branch points (Failbetter Games). See [Branching & Consequence](#).

Quotation: Deliberately referencing a source. See [Aesthetic Heritage](#).

R

Readability: Whether the player can understand what they're looking at. See [Spatial Communication](#).

S

Screen Shake: Camera shake as feedback for impact or intensity. See [Screen Shake](#).

Sight Line: What's visible from a given position. Control of sight lines = control of information. See [Verticality & Sight Lines](#).

Sink: Where resources go in an economy. Drains. See [Economy & Resources](#).

Soft Currency: Easily obtained in-game currency, earned through play. See [Economy & Resources](#).

Source: Where resources come from in an economy. Faucets. See [Economy & Resources](#).

Squash and Stretch: Animation principle: compression on impact, extension during movement.

Systemic Design: Designing systems that interact with each other to produce emergent behavior. See [Emergent Behavior](#).

T

Tetris Effect: When intensive play bleeds into non-play life. See [Player Psychology](#).

The 4 A's: Action, Art, Arc, Atmosphere. The components of a Gesture. See [The 4 A's](#).

Transformation: Taking something and making it new. See [Aesthetic Heritage](#).

V

Verbs: The actions available to players. What the videogame lets you do.

Verticality: Use of height in level design. See [Verticality & Sight Lines](#).

Vista: Dramatic revealing view; moment of seeing far. See [Verticality & Sight Lines](#).

W

Weenie: Disney Imagineering term: tall, visible landmark that draws players forward. See [Verticality & Sight Lines](#).

References

The intellectual lineage behind this knowledge base - key thinkers, texts, and traditions that inform the Gesture + Aesthetic Heritage approach.

Core Videogame-Focused Books

- **Steve Swink** - *Game Feel* (2009). The definitive text on real-time control feel. Essential for understanding [Gesture](#).
 - **Ian Bogost** - *Persuasive Games* (2007). Procedural rhetoric: videogames make arguments through rules.
 - **Brendan Keogh** - *A Play of Bodies* (2018). Phenomenological approach to videogames. The body as site of meaning.
 - **Anna Anthropy & Naomi Clark** - *A Game Design Vocabulary* (2014). Player-centric vocabulary, accessible design language.
 - **Raph Koster** - *A Theory of Fun for Game Design* (2004). Connects learning and pattern recognition to player enjoyment.
 - **Tracy Fullerton** - *A Playful Production Process* (2024). On iterative design and playtesting.
-

Key Thinkers

Game Studies & Design

- **Jesse Schell** - *The Art of Game Design: A Book of Lenses*. The lenses framework. See [The Design Lens](#).

- **Jesper Juul** - *Half-Real*. Rules and fiction in videogames. Foundational videogame studies text.
- **Miguel Sicart** - *Beyond Choices: The Design of Ethical Gameplay* (2013). Ethics in videogame design.

Design Theory

- **Donald Schon** - *The Reflective Practitioner* (1983). Design as conversation with materials. Core to [Code as Material](#).
- **Christopher Alexander** - *A Pattern Language*. Architecture as communication. Influences level design thinking.

Psychology & Learning

- **Mihaly Csikszentmihalyi** - *Flow: The Psychology of Optimal Experience* (1990). The optimal experience that good pacing supports. See [Player Psychology](#).
- **Seymour Papert** - *Mindstorms* (1980). Coding as thinking tool. Influences [Debugging as Literacy](#).

Heritage

- **Harold Bloom** - *The Anxiety of Influence* (1973). Literary influence theory. Informs our approach to [Aesthetic Heritage](#).

Academic Papers

- **Hunicke, LeBlanc, Zubek** - "MDA: A Formal Approach to Game Design" (2004). The Mechanics-Dynamics-Aesthetics framework.

Talks Worth Watching

- Valve's developer commentaries (in-game, *Portal* and *Half-Life* series)
 - Itay Keren - "Scroll Back: The Theory and Practice of Cameras in Side-Scrollers"
 - Jan Willem Nijman - "The Art of Screenshake" (Vlambeer's juicing philosophy)
 - Tim Rogers - Action Button Reviews (long-form analysis)
-

See Also

- [The Design Lens](#) - how to use multiple perspectives
- [Play → Name → Make → Reflect](#) - learning philosophy
- [Glossary](#) - terminology used in this KB